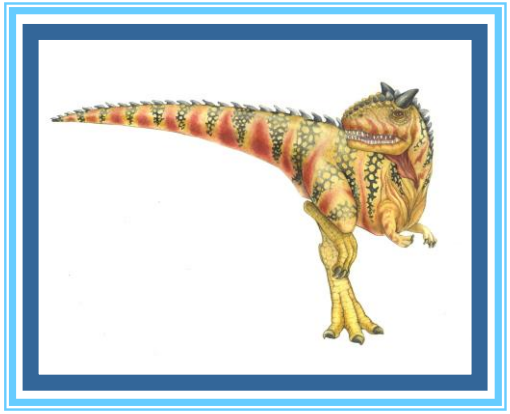
Chapter 1: Introduction



Silberschatz, Galvin and Gagne ©2018



Chapter 1: Introduction

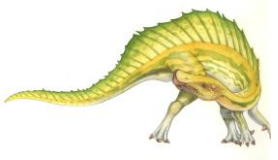
- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Security and Protection
- Virtualization
- Distributed Systems
- Kernel Data Structures
- Computing Environments
- Free/Libre and Open-Source Operating Systems





Objectives

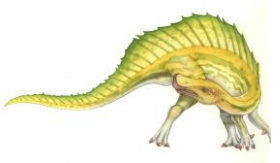
- Describe the general organization of a computer system and the role of interrupts
- Describe the components in a modern, multiprocessor computer system
- Illustrate the transition from user mode to kernel mode
- Discuss how operating systems are used in various computing environments
- Provide examples of free and open-source operating systems





What Does the Term Operating System Mean?

- An operating system is “fill in the blanks”
- What about:
  - Car
  - Airplane
  - Printer
  - Washing Machine
  - Toaster
  - Compiler
  - Etc.





What is an Operating System?

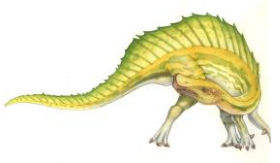
- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner





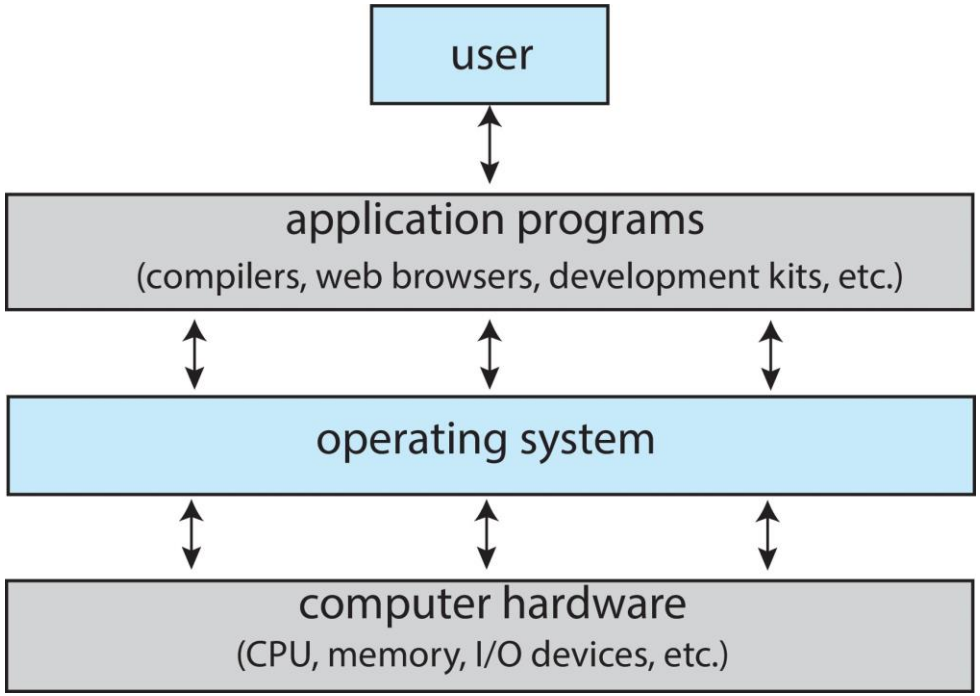
Computer System Structure

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers





Abstract View of Components of Computer







What Operating Systems Do

- Depends on the point of view
- Users want convenience, **ease of use** and **good performance**
  - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
  - Operating system is a **resource allocator** and **control program** making efficient use of HW and managing execution of user programs
- Users of dedicate systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- Mobile devices like smartphones and tables are resource poor, optimized for usability and battery life
  - Mobile user interfaces such as touch screens, voice recognition
- Some computers have little or no user interface, such as embedded computers in devices and automobiles
  - Run primarily without user intervention





Defining Operating Systems

- Term OS covers many roles
  - Because of myriad designs and uses of OSes
  - Present in toasters through ships, spacecraft, game machines, TVs and industrial control systems
  - Born when fixed use computers for military became more general purpose and needed resource management and program control





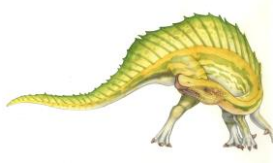
Operating System Definition

- No universally accepted definition
- "Everything a vendor ships when you order an operating system" is a good approximation
  - But varies wildly
- "The one program running at all times on the computer" is the **kernel,** part of the operating system
- Everything else is either
  - A ***system program*** (ships with the operating system, but not part of the kernel) , or
  - An ***application program***, all programs not associated with the operating system
- Today's OSes for general purpose and mobile computing also include ***middleware*** – a set of software frameworks that provide additional services to application developers such as databases, multimedia, graphics





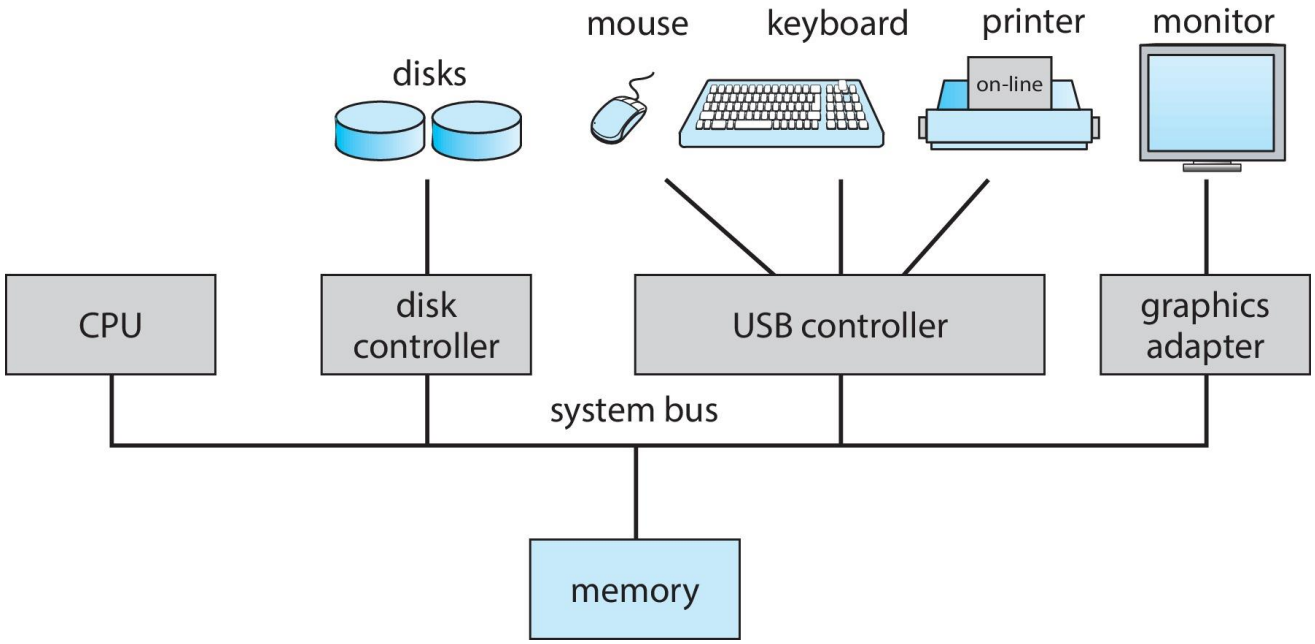
Overview of Computer System Structure





Computer System Organization

- Computer-system operation
  - One or more CPUs, device controllers connect through common **bus** providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles







Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- Each device controller type has an operating system **device driver** to manage it
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an **interrupt**





Common Functions of Interrupts

- Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines
- Interrupt architecture must save the address of the interrupted instruction
- A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request
- An operating system is **interrupt driven**





Exceptii

- evenimente care schimba cursul executiei unui program
- tipuri
  - interne CPU
    - generate de executia programului
    - procesate sincron cu programul
    - subclasificare
      - fault: apare “inainte” de executia instructiunii care genereaza exceptia
      - trap: “dupa”
      - abort: “pe durata”
  - externe CPU
    - intreruperi HW procesate asincron cu executia programului

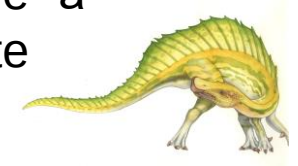




Exceptii externe CPU

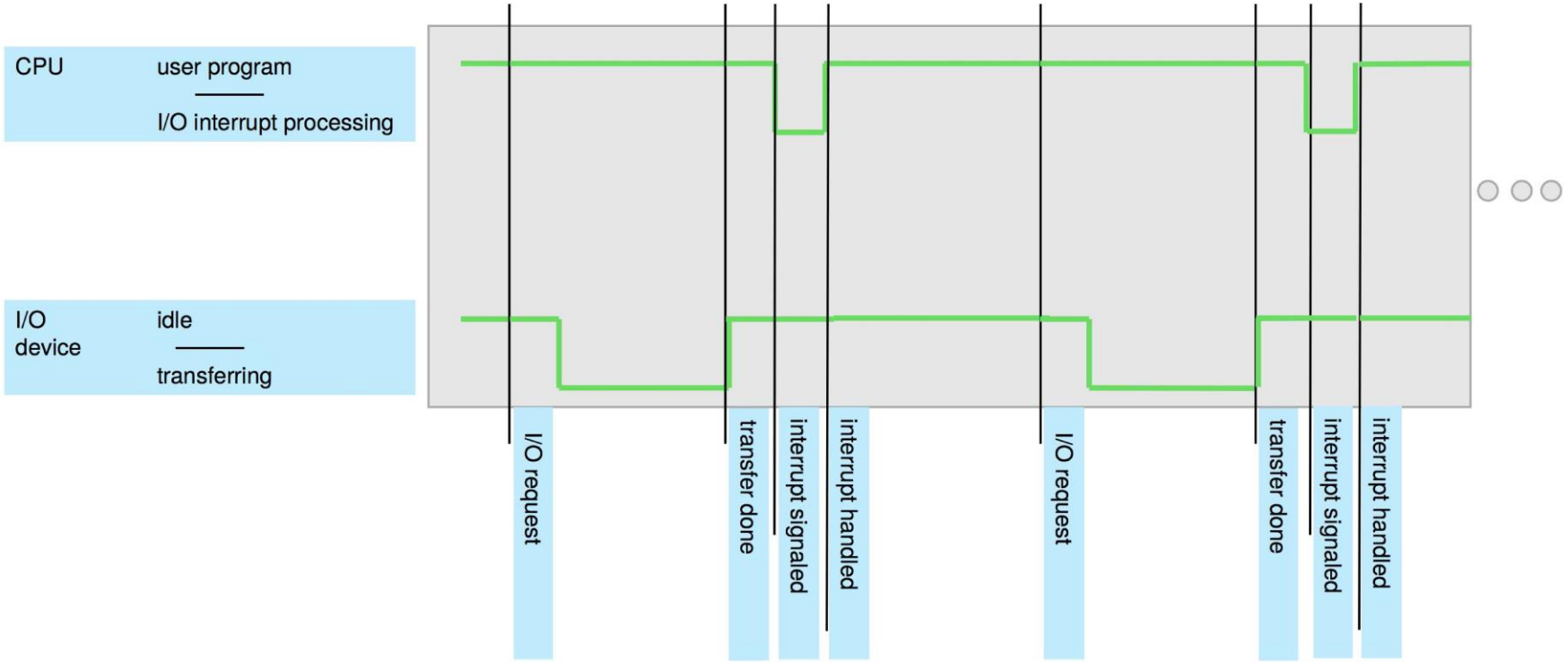
- se pot dezactiva (metoda de protectie a sectiunilor critice in sisteme uniprocesor, induc latenta crescuta a intreruperii)
  - ex: cod driver
- se pot masca in scheme prioritizate
  - intreruperi cu prioritate mare pot intrerupe tratarea intreruperilor de prioritate mica
- NMI (Non Maskable Ints), generate de erori HW nerecuperabile
  - erori de chipset, erori paritate sau ECC memorie, data corruption pe magistrala (bus) de sistem sau periferice (PCI)
- latenta intreruperii
  - aspect critic in proiectarea kernelului
  - ex Linux: - top-half (partea “esentiala” a ISR), inregistrata in vectorul de intreruperi, cod scurt pt. tartare rapida a intreruperii

    - bottom-half (tasklet, executie amanata a partii “neesentiale” a ISR); eg, procesare protocoale de retea pt pachetele primite





Interrupt Timeline







Interrupt Handling

- The operating system preserves the state of the CPU by storing the registers and the program counter
- Determines which type of interrupt has occurred:
- Separate segments of code determine what action should be taken for each type of interrupt





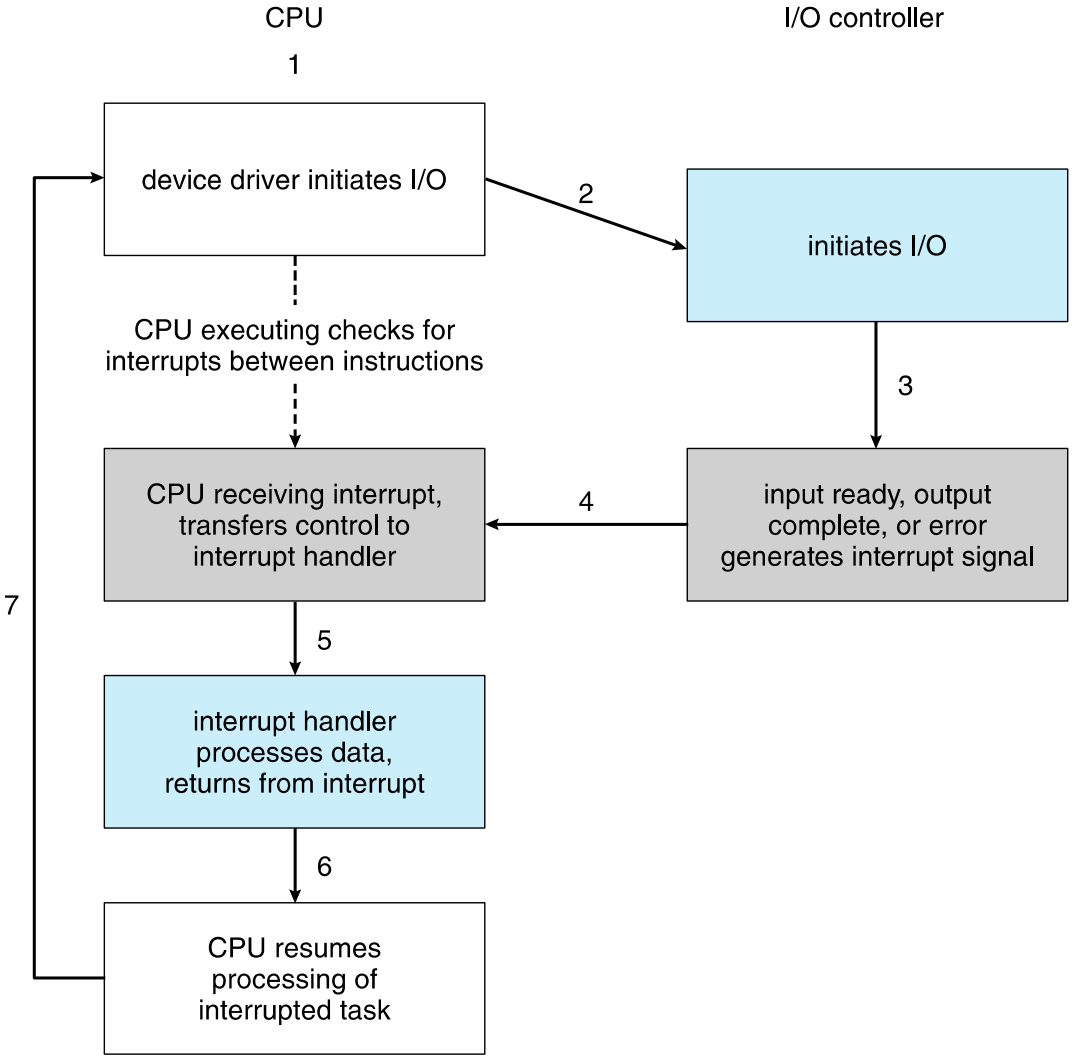
Tratarea intreruperilor

- CPU intra in mod kernel si blocheaza starea sa interna in registrele interne CPU (cu intreruperile blocate)
- se stocheaza tipul exceptiei intr-un registru specific
- se da controlul kernelului la o adresa specifica a unui handler (general) de tratare a intreruperilor
- handlerul localizeaza stiva kernel, salveaza contextul CPU pe stiva si foloseste registrul care stocheaza tipul exceptiei ca index in vectorul de intreruperi pt a gasi handlerul de intrerupere specific
- in acest moment, acest handler poate rula cod C si decide daca exceptia e fatala sau nu
  - pt exceptie fatala, handlerul termina procesul interrupt
  - altfel, inlatura cauza exceptiei si se da controlul handlerului de nivel jos care restaureaza contextul CPU salvat si reincarca adresa instructiunii intrerupte in registrul de instructiuni





Interrupt-drive I/O Cycle

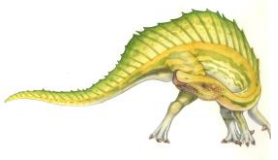






I/O Structure

- Two methods for handling I/O
  - After I/O starts, control returns to user program only upon I/O completion
  - After I/O starts, control returns to user program without waiting for I/O completion





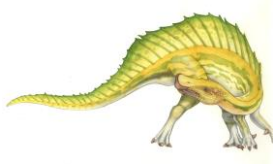
I/O Structure (Cont.)

- After I/O starts, control returns to user program only upon I/O completion
  - Wait instruction idles the CPU until the next interrupt
  - Wait loop (contention for memory access)
  - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
  - **System call** – request to the OS to allow user to wait for I/O completion
  - **Device-status table** contains entry for each I/O device indicating its type, address, and state
  - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt





Storage Structure





Storage Structure

- Main memory – only large storage media that the CPU can access directly
  - **Random access**
  - Typically **volatile**
  - Typically **random-access memory** in the form of **Dynamic Random-access Memory (DRAM)**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity





Storage Structure (Cont.)

- **Hard Disk Drives** (**HDD**) – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
  - The **disk controller** determines the logical interaction between the device and the computer
- **Non-volatile memory** (**NVM**) devices– faster than hard disks, nonvolatile
  - Various technologies
  - Becoming more popular as capacity and performance increases, price drops

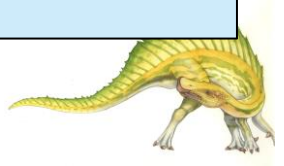




Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

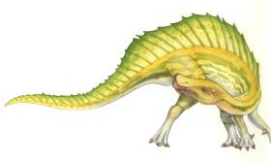
Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes. A **kilobyte**, or KB , is 1,024 bytes; a **megabyte**, or **MB**, is $1,024^2$ bytes; a **gigabyte**, or **GB**, is $1,024^3$ bytes; a **terabyte**, or **TB**, is $1,024^4$ bytes; and a **petabyte**, or **PB**, is $1,024^5$ bytes. Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).





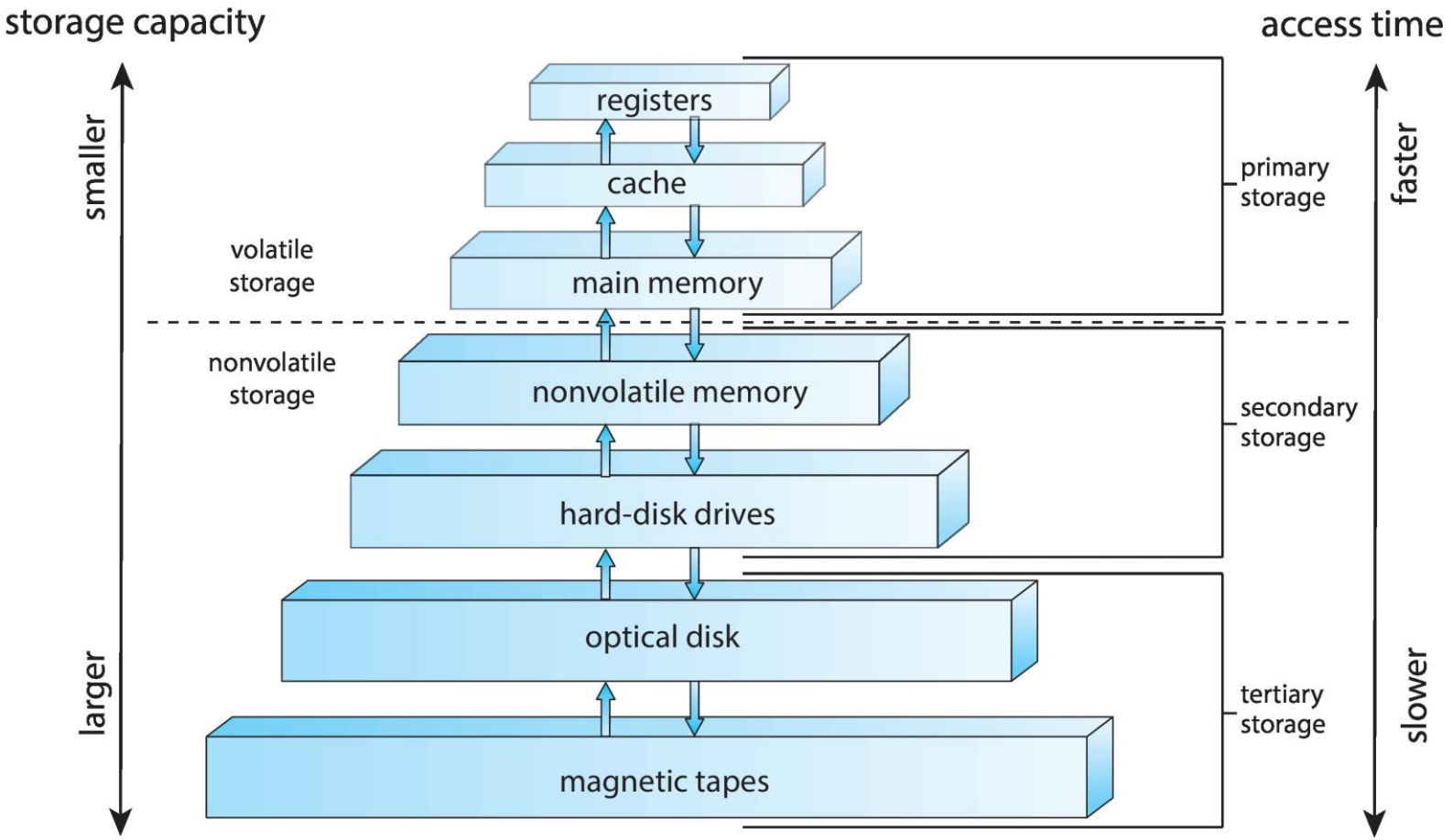
Storage Hierarchy

- Storage systems organized in hierarchy
  - Speed
  - Cost
  - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- **Device Driver** for each device controller to manage I/O
  - Provides uniform interface between controller and kernel





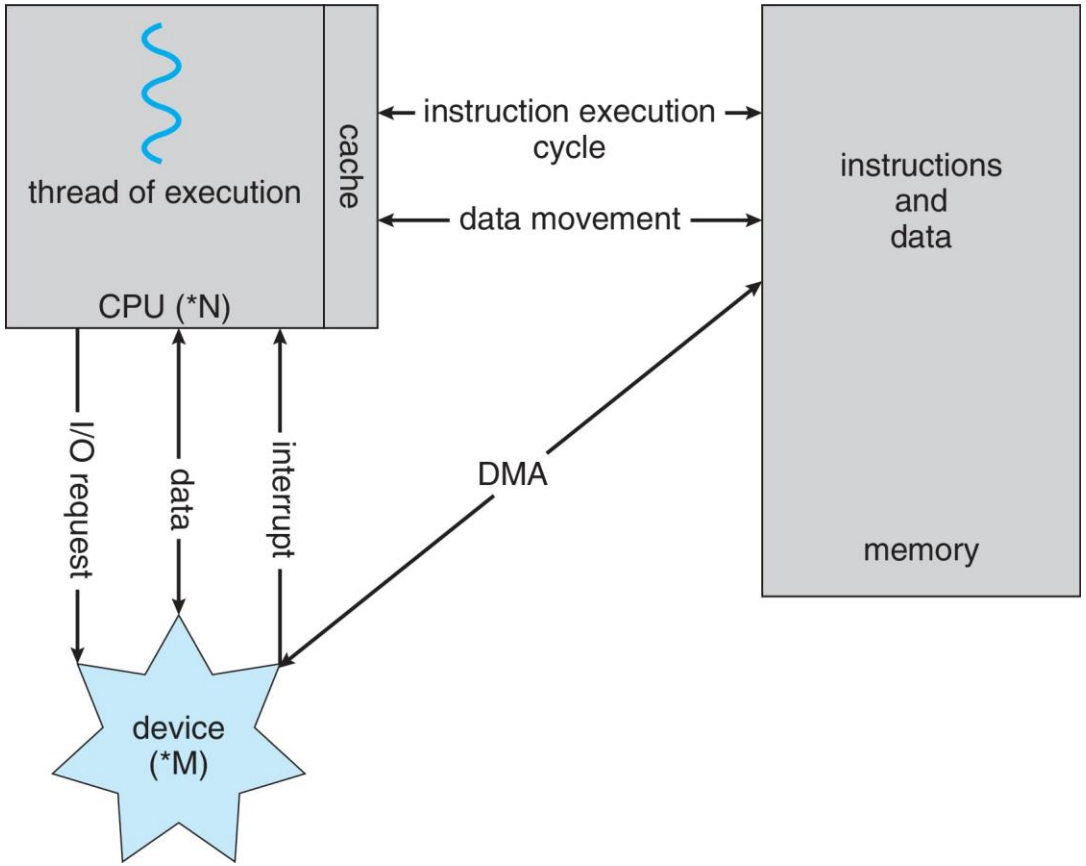
Storage-Device Hierarchy







How a Modern Computer Works



A von Neumann architecture





Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte





Operating-System Operations

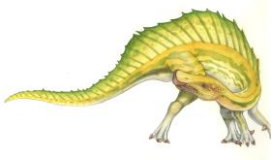
- Bootstrap program – simple code to initialize the system, load the kernel
- Kernel loads
- Starts **system daemons** (services provided outside of the kernel)
- Kernel **interrupt driven** (hardware and software)
  - Hardware interrupt by one of the devices
  - Software interrupt (**exception** or **trap**):
    - Software error (e.g., division by zero)
    - Request for operating system service – **system call**
    - Other process problems include infinite loop, processes modifying each other or the operating system





Multiprogramming (Batch system)

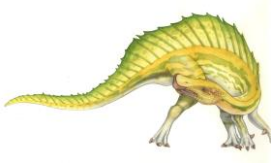
- Single user cannot always keep CPU and I/O devices busy
- Multiprogramming organizes jobs (code and data) so CPU always has one to execute
- A subset of total jobs in system is kept in memory
- One job selected and run via **job scheduling**
- When job has to wait (for I/O for example), OS switches to another job





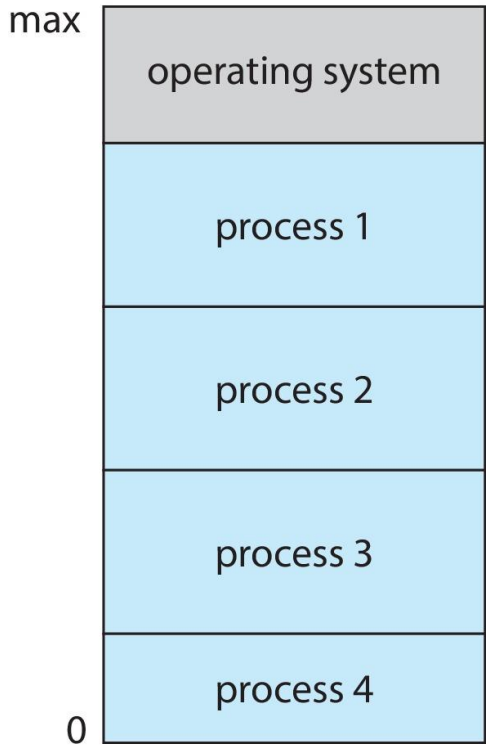
Multitasking (Timesharing)

- A logical extension of Batch systems– the CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing
  - **Response time** should be < 1 second
  - Each user has at least one program executing in memory ⇨ **process**
  - If several jobs ready to run at the same time ⇨ **CPU scheduling**
  - If processes don't fit in memory, **swapping** moves them in and out to run
  - **Virtual memory** allows execution of processes not completely in memory





Memory Layout for Multiprogrammed System







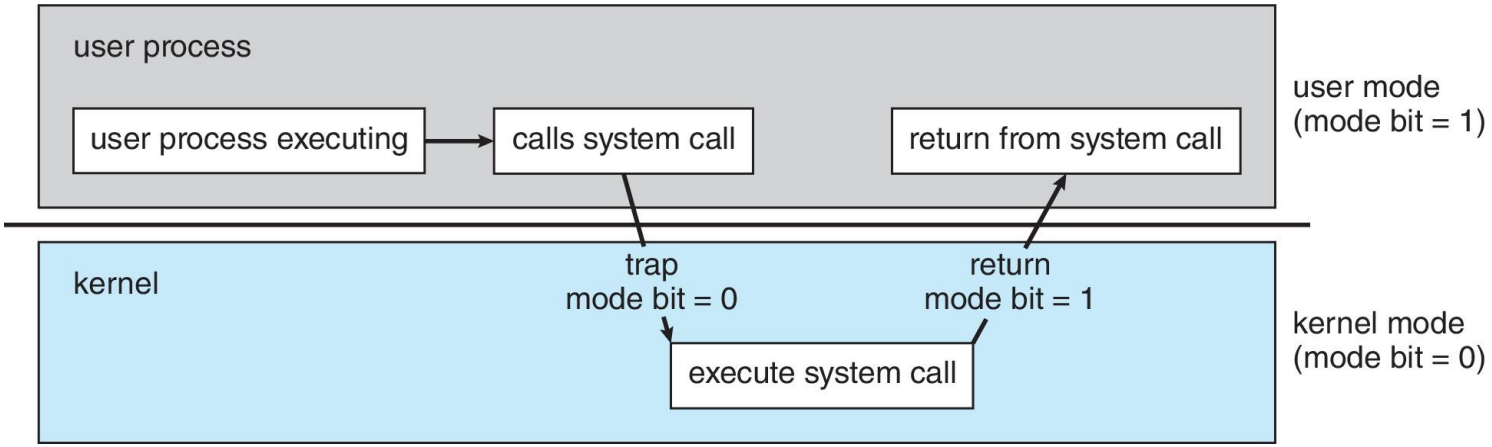
Dual-mode Operation

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
- **Mode bit** provided by hardware
  - Provides ability to distinguish when system is running user code or kernel code.
  - When a user is running ⇨ mode bit is “user”
  - When kernel code is executing ⇨ mode bit is “kernel”
- How do we guarantee that user does not explicitly set the mode bit to “kernel”?
  - System call changes mode to kernel, return from call resets it to user
- Some instructions designated as **privileged**, only executable in kernel mode





Transition from User to Kernel Mode







Timer

- Timer to prevent infinite loop (or process hogging resources)
  - Timer is set to interrupt the computer after some time period
  - Keep a counter that is decremented by the physical clock
  - Operating system set the counter (privileged instruction)
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time





Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a ***passive entity;*** process is an ***active entity***.
- Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
  - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
  - Concurrency by multiplexing the CPUs among the processes / threads





Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling





Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory
- Memory management determines what is in memory and when
  - Optimizing CPU utilization and computer response to users
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed





File-system Management

- OS provides uniform, logical view of information storage
  - Abstracts physical properties to logical storage unit - **file**
  - Each medium is controlled by device (i.e., disk drive, tape drive)
    - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - OS activities include
    - Creating and deleting files and directories
    - Primitives to manipulate files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media





Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
  - Mounting and unmounting
  - Free-space management
  - Storage allocation
  - Disk scheduling
  - Partitioning
  - Protection





Caching

- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
  - If it is, information used directly from the cache (fast)
  - If not, data copied to cache and used there
- Cache smaller than storage being cached
  - Cache management important design problem
  - Cache size and replacement policy





Characteristics of Various Types of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

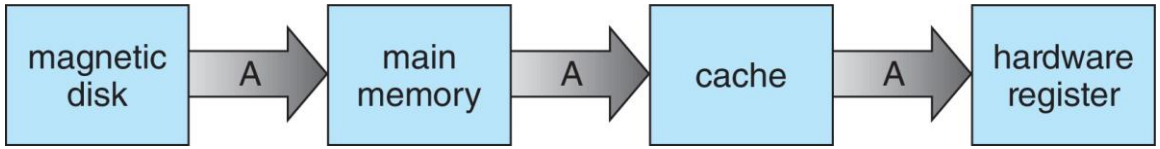
Movement between levels of storage hierarchy can be explicit or implicit





Migration of data “A” from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
  - Several copies of a datum can exist
  - Various solutions covered in Chapter 19





I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
  - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
  - General device-driver interface
  - Drivers for specific hardware devices





Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
  - User identities (**user IDs**, security IDs) include name and associated number, one per user
  - User ID then associated with all files, processes of that user to determine access control
  - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
  - **Privilege escalation** allows user to change to effective ID with more rights





Virtualization

- Allows operating systems to run applications within other OSes
  - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
  - Generally slowest method
  - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSes also natively compiled
  - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
  - **VMM** (virtual machine Manager) provides virtualization services





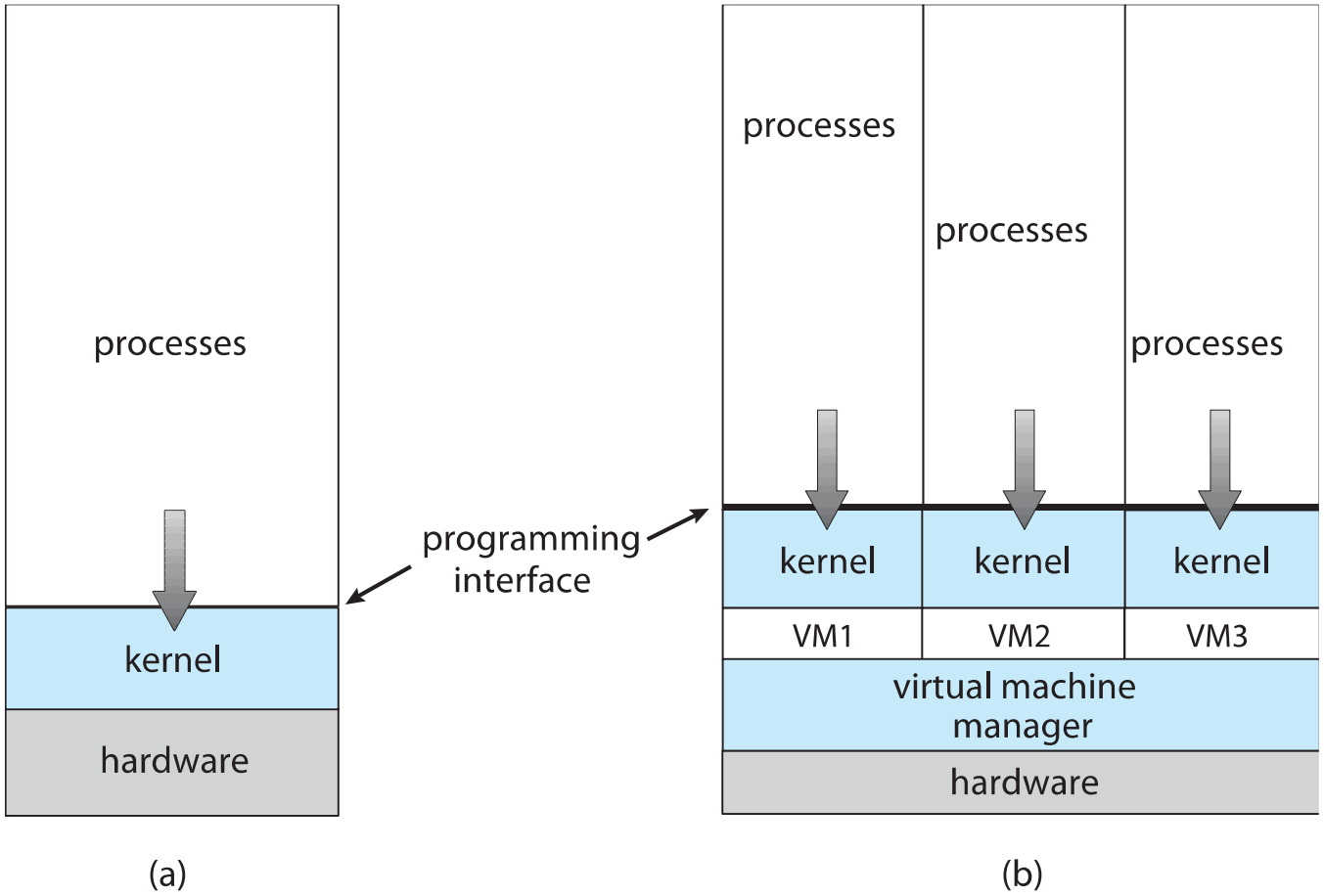
Virtualization (cont.)

- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
  - Apple laptop running Mac OS X host, Windows as a guest
  - Developing apps for multiple OSes without having multiple systems
  - Quality assurance testing applications without having multiple systems
  - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
  - There is no general-purpose host then (VMware ESX and Citrix XenServer)





Computing Environments - Virtualization

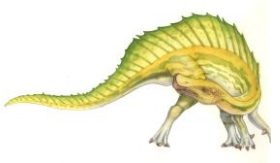






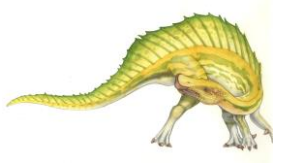
Distributed Systems

- Collection of separate, possibly heterogeneous, systems networked together
  - **Network** is a communications path, **TCP/IP** most common
    - **Local Area Network** (**LAN**)
    - **Wide Area Network** (**WAN**)
    - **Metropolitan Area Network** (**MAN**)
    - **Personal Area Network** (**PAN**)
- **Network Operating System** provides features between systems across network
  - Communication scheme allows systems to exchange messages
  - Illusion of a single system





Computer System Architecture





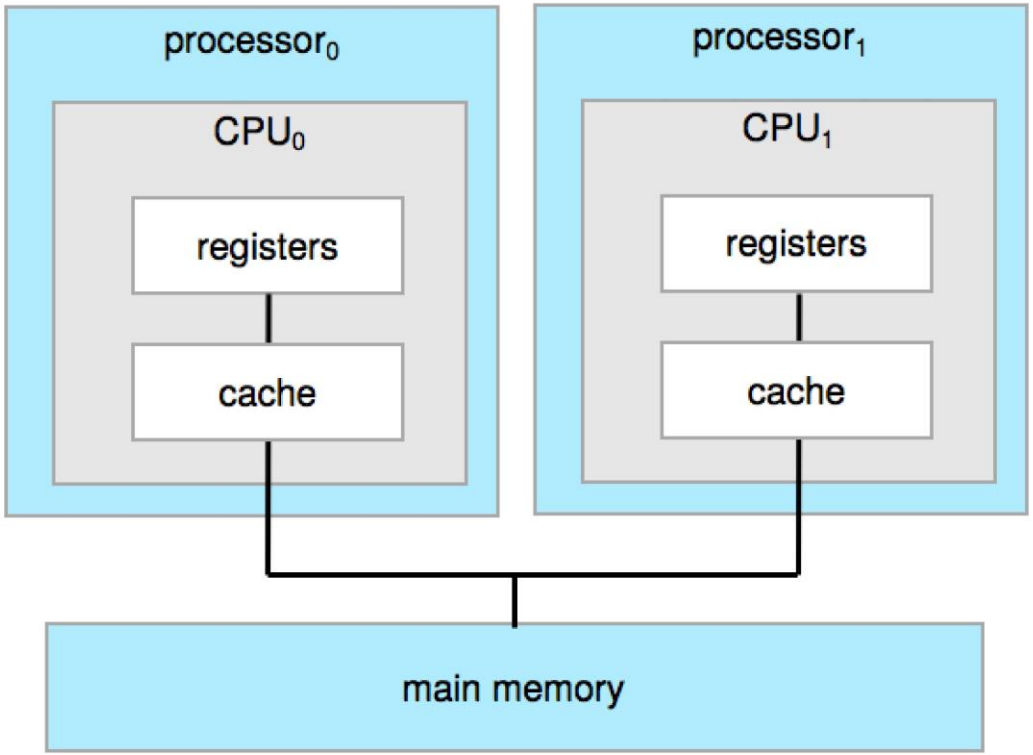
Computer-System Architecture

- Most systems use a single general-purpose processor
  - Most systems have special-purpose processors as well
- **Multiprocessors** systems growing in use and importance
  - Also known as **parallel systems**, **tightly-coupled systems**
  - Advantages include:
    1. **Increased throughput**
    2. **Economy of scale**
    3. **Increased reliability** – graceful degradation or fault tolerance
  - Two types:
    1. **Asymmetric Multiprocessing** – each processor is assigned a specie task.
    2. **Symmetric Multiprocessing** – each processor performs all tasks





Symmetric Multiprocessing Architecture

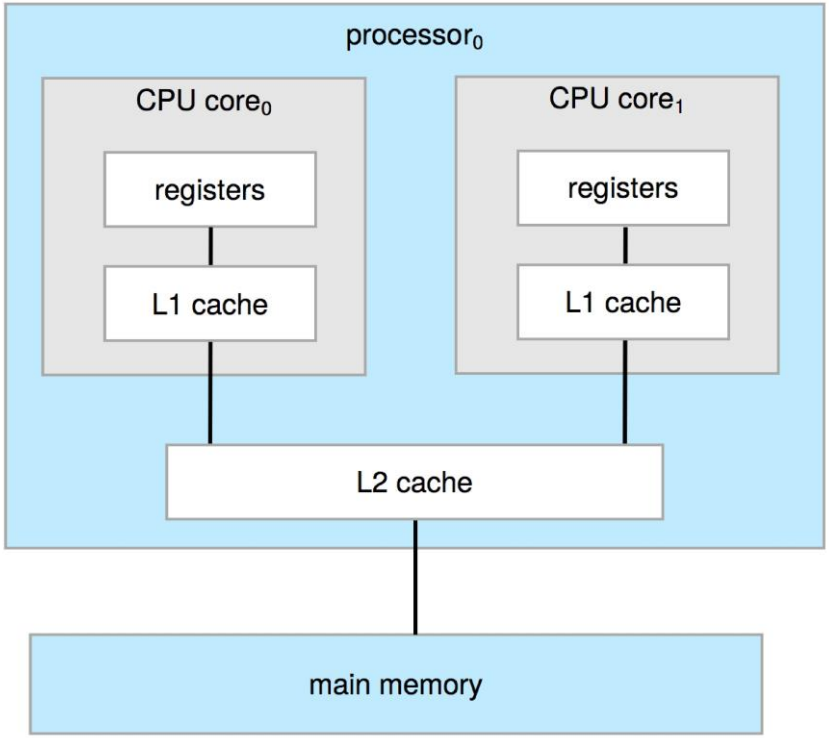






Dual-Core Design

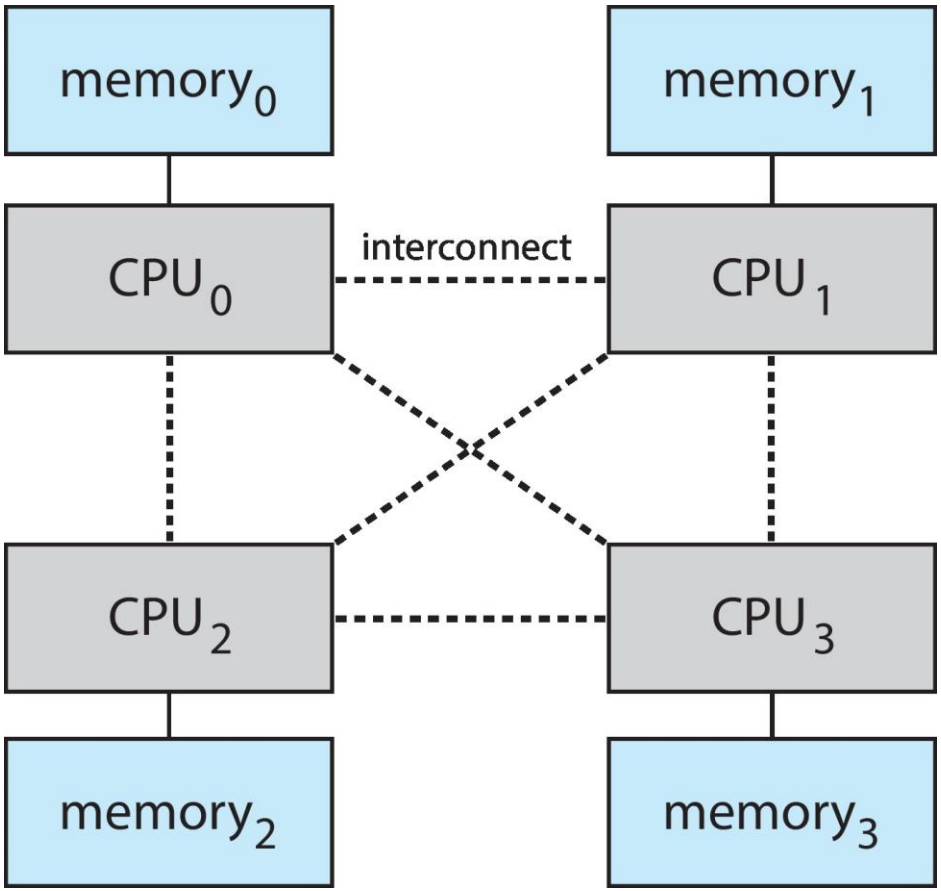
- Multi-chip and **multicore**
- Systems containing all chips
  - Chassis containing multiple separate systems







Non-Uniform Memory Access System

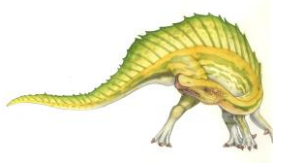






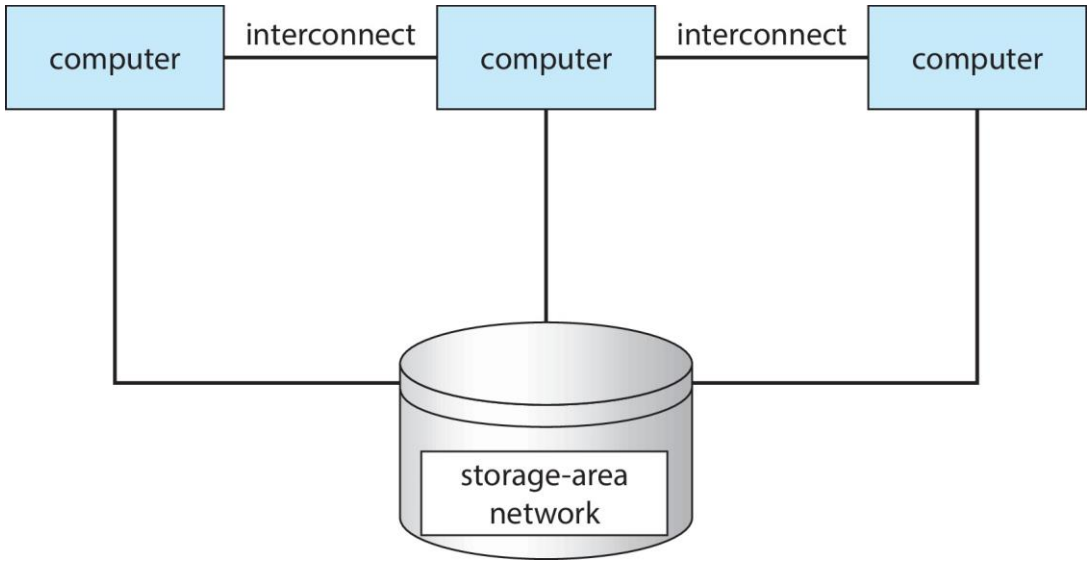
Clustered Systems

- Like multiprocessor systems, but multiple systems working together
  - Usually sharing storage via a **storage-area network** (**SAN**)
  - Provides a **high-availability** service which survives failures
    - **Asymmetric clustering** has one machine in hot-standby mode
    - **Symmetric clustering** has multiple nodes running applications, monitoring each other
  - Some clusters are for **high-performance computing** (**HPC**)
    - Applications must be written to use **parallelization**
  - Some have **distributed lock manager** (**DLM**) to avoid conflicting operations





Clustered Systems

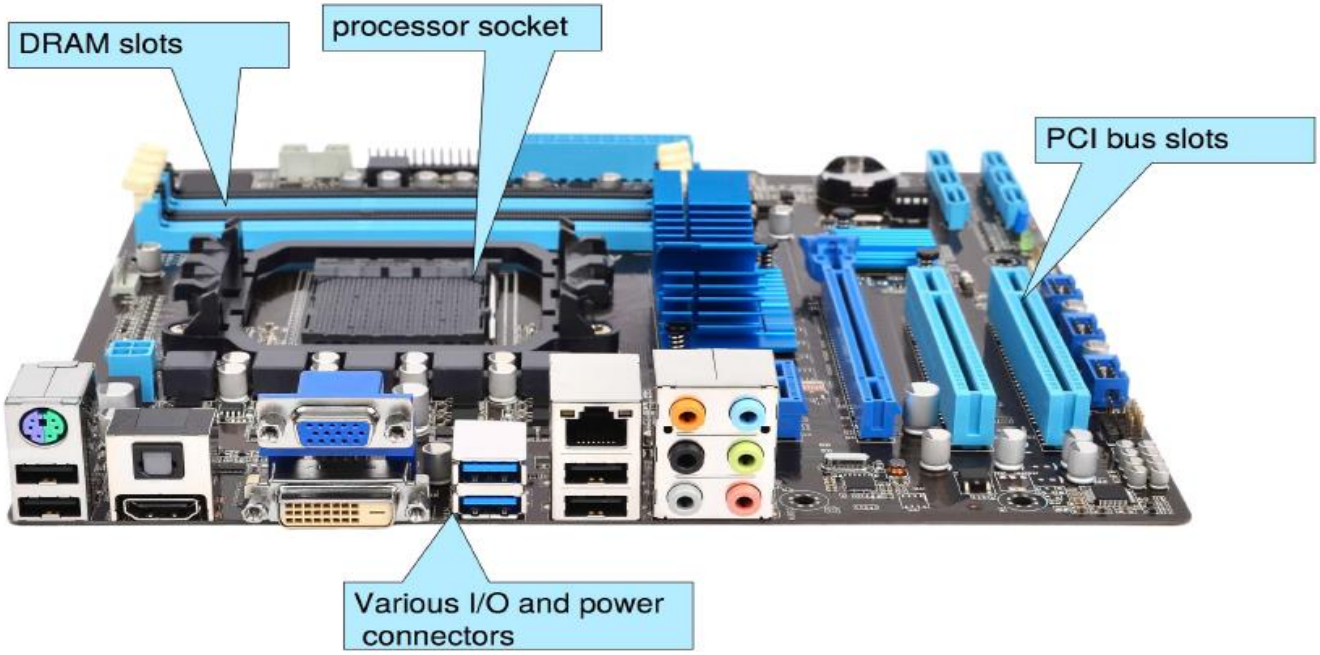




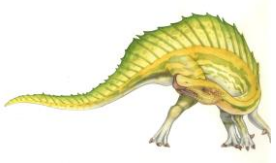


PC Motherboard

Consider the desktop PC motherboard with a processor socket shown below:



This board is a fully-functioning computer, once its slots are populated. It consists of a processor socket containing a CPU, DRAM sockets, PCIe bus slots, and I/O connectors of various types. Even the lowest-cost general-purpose CPU contains multiple cores. Some motherboards contain multiple processor sockets. More advanced computers allow more than one system board, creating NUMA systems.





Computer System Environments





Computing Environments

- Traditional
- Mobile
- Client Server
- Peer-to-Peer
- Cloud computing
- Real-time Embedded





Traditional

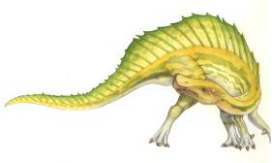
- Stand-alone general-purpose machines
- But blurred as most systems interconnect with others (i.e., the Internet)
- **Portals** provide web access to internal systems
- **Network computers** (**thin clients**) are like Web terminals
- Mobile computers interconnect via **wireless networks**
- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks





Mobile

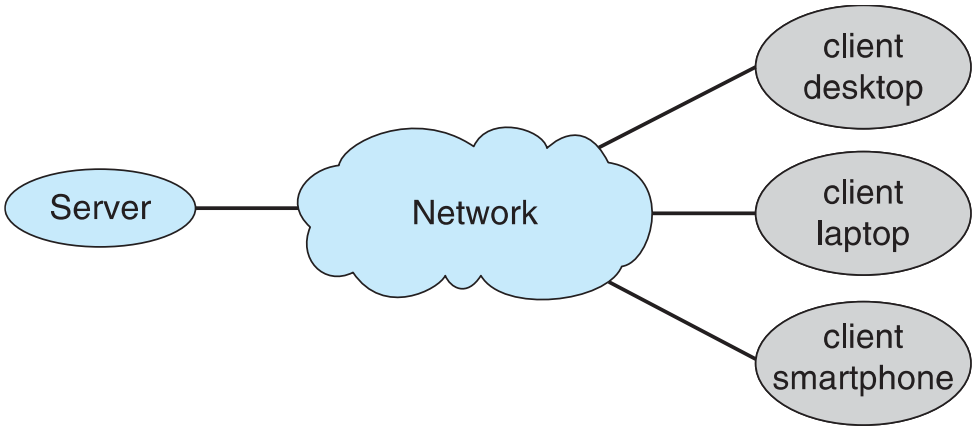
- Handheld smartphones, tablets, etc.
- What is the functional difference between them and a “traditional” laptop?
- Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like ***augmented reality***
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are **Apple iOS** and **Google Android**





Client Server

- Client-Server Computing
  - Dumb terminals supplanted by smart PCs
  - Many systems now **servers**, responding to requests generated by **clients**
    - **Compute-server system** provides an interface to client to request services (i.e., database)
    - **File-server system** provides interface for clients to store and retrieve files

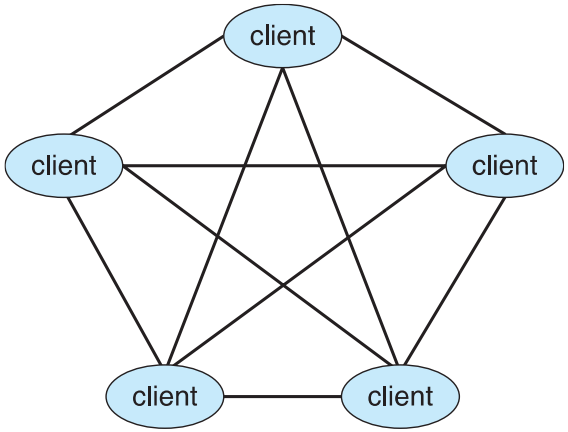






Peer-to-Peer

- Another model of distributed system
- P2P does not distinguish clients and servers
  - Instead all nodes are considered peers
  - May each act as client, server or both
  - Node must join P2P network
    - Registers its service with central lookup service on network, or
    - Broadcast request for service and respond to requests for service via ***discovery protocol***
  - Examples include Napster and Gnutella, **Voice over IP** (**VoIP**) such as Skype







Cloud Computing

- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for it functionality.
  - Amazon **EC2** has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage





Cloud Computing (Cont.)

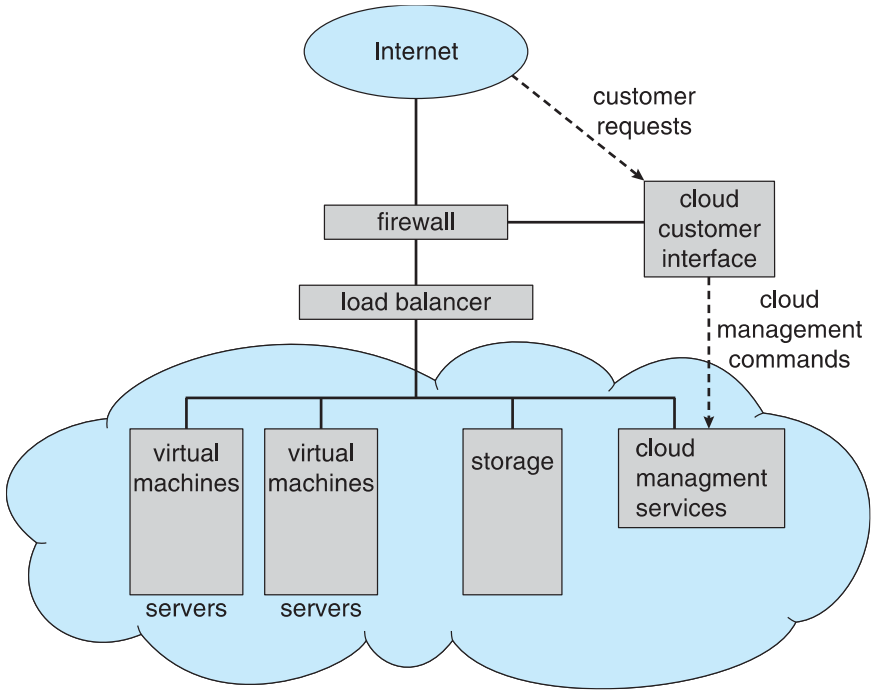
- Many types
  - **Public cloud** – available via Internet to anyone willing to pay
  - **Private cloud** – run by a company for the company's own use
  - **Hybrid cloud** – includes both public and private cloud components
  - Software as a Service (**SaaS**) – one or more applications available via the Internet (i.e., word processor)
  - Platform as a Service (**PaaS**) – software stack ready for application use via the Internet (i.e., a database server)
  - Infrastructure as a Service (**IaaS**) – servers or storage available over Internet (i.e., storage available for backup use)





Cloud Computing (cont.)

- Cloud computing environments composed of traditional OSes, plus VMMs, plus cloud management tools
  - Internet connectivity requires security like firewalls
  - Load balancers spread traffic across multiple applications







Real-Time Embedded Systems

- Real-time embedded systems most prevalent form of computers
  - Vary considerable, special purpose, limited purpose OS, **real-time OS**
  - Use expanding
- Many other special computing environments as well
  - Some have OSes, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
  - Processing ***must*** be done within constraint
  - Correct operation only if constraints met





Free and Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary **closed-source** and **proprietary**
- Counter to the **copy protection** and **Digital Rights Management** (**DRM**) movement
- Started by **Free Software Foundation** (**FSF**), which has “copyleft” **GNU Public License** (**GPL**)
  - Free software and open-source software are two different ideas championed by different groups of people
    - **https://www.gnu.org/philosophy/open-source-misses-the-point.en.html**
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - http://www.virtualbox.com)
  - Use to run guest operating systems for exploration





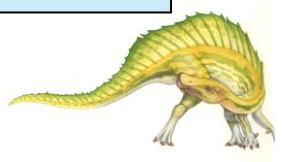
The Study of Operating Systems

There has never been a more interesting time to study operating systems, and it has never been easier. The open-source movement has overtaken operating systems, causing many of them to be made available in both source and binary (executable) format. The list of operating systems available in both formats includes Linux, BUSD UNIX, Solaris, and part of macOS. The availability of source code allows us to study operating systems from the inside out. Questions that we could once answer only by looking at documentation or the behavior of an operating system we can now answer by examining the code itself.

Operating systems that are no longer commercially viable have been open-sourced as well, enabling us to study how systems operated in a time of fewer CPU, memory, and storage resources. An extensive but incomplete list of open-source operating-system projects is available from https://curlie.org/Computers/Software/Operating_Systems/Open_Source/

In addition, the rise of virtualization as a mainstream (and frequently free) computer function makes it possible to run many operating systems on top of one core system. For example, VMware (http://www.vmware.com) provides a free "player" for Windows on which hundreds of free "virtual appliances" can run. Virtualbox (http://www.virtualbox.com) provides a free, open-source virtual machine manager on many operating systems. Using such tools, students can try out hundreds of operating systems without dedicated hardware.

The advent of open-source operating systems has also made it easier to make the move from student to operating-system developer. With some knowledge, some effort, and an Internet connection, a student can even create a new operating-system distribution. Just a few years ago, it was difficult or impossible to get access to source code. Now, such access is limited only by how much interest, time, and disk space a student has.





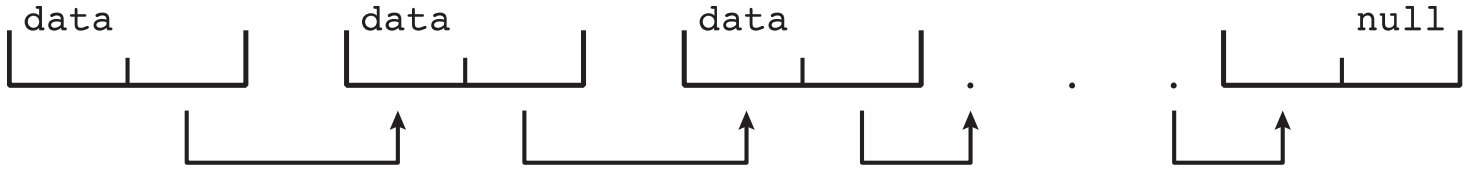
Kernel Data Structure



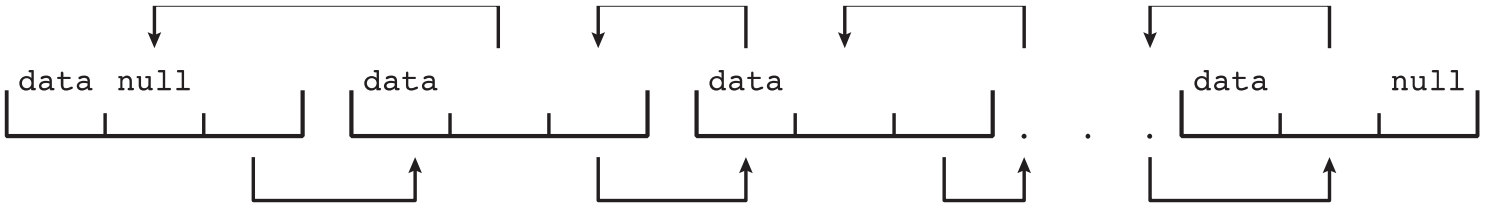


Kernel Data Structures

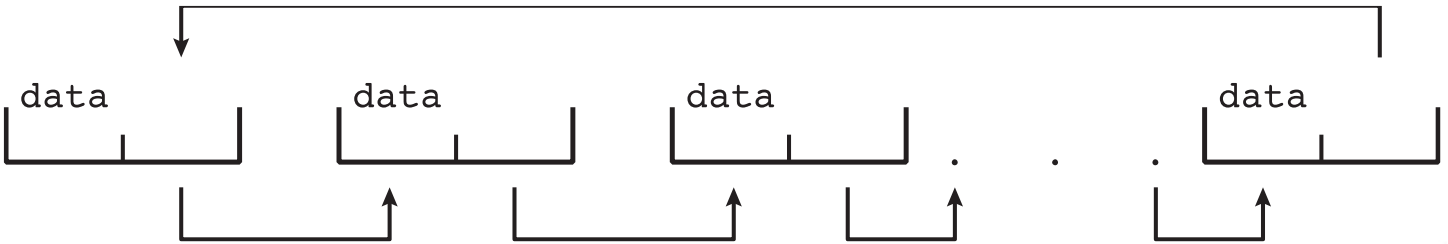
- Many similar to standard programming data structures
- ***Singly linked list***



- ***Doubly linked list***



- ***Circular linked list***

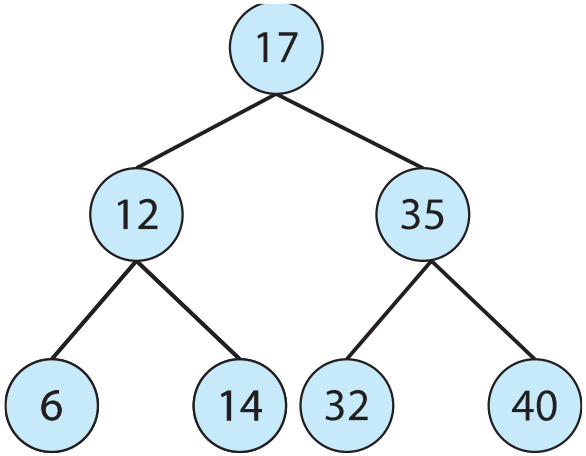






Kernel Data Structures

- **Binary search tree**
  left <= right
  - Search performance is *O(n)*
  - **Balanced binary search tree** is *O(lg n)*

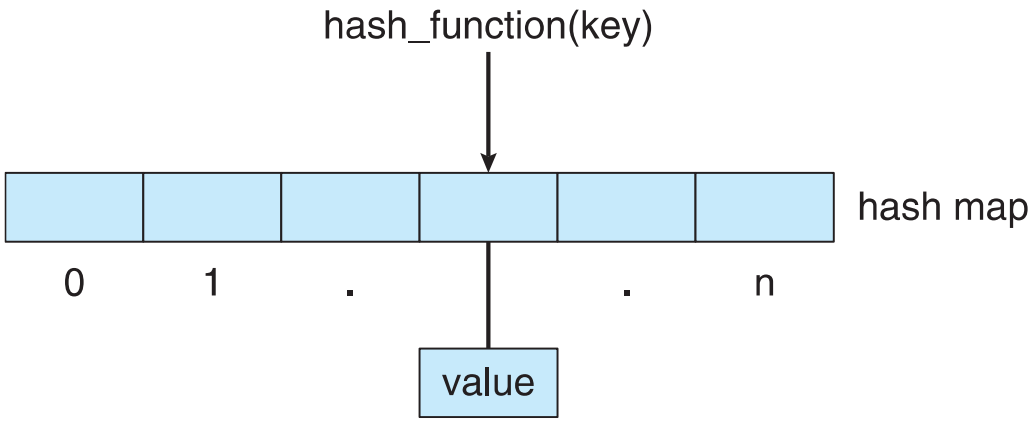






Kernel Data Structures

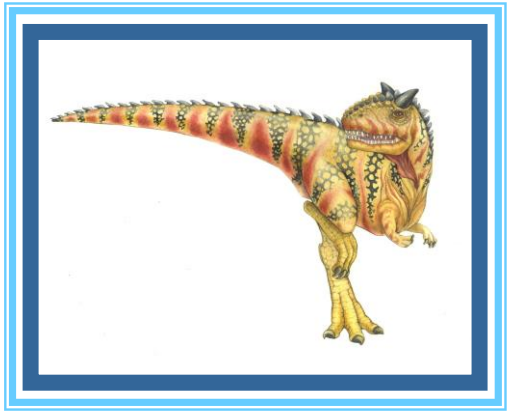
- **Hash function** can create a **hash map**



- **Bitmap** – string of *n* binary digits representing the status of *n* items
- Linux data structures defined in ***include*** files `<linux/list.h>`, `<linux/kfifo.h>`, `<linux/rbtree.h>`

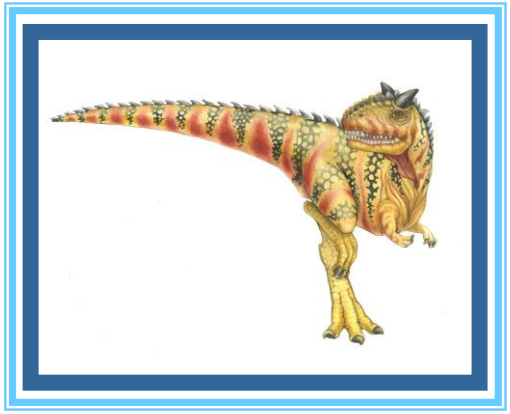


End of Chapter 1



Silberschatz, Galvin and Gagne ©2018

Chapter 2: Operating-System Services



Silberschatz, Galvin and Gagne ©2018



Outline

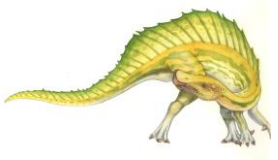
- Operating System Services
- User and Operating System-Interface
- System Calls
- System Services
- Linkers and Loaders
- Why Applications are Operating System Specific
- Design and Implementation
- Operating System Structure
- Building and Booting an Operating System
- Operating System Debugging





Objectives

- Identify services provided by an operating system
- Illustrate how system calls are used to provide operating system services
- Compare and contrast monolithic, layered, microkernel, modular, and hybrid strategies for designing operating systems
- Illustrate the process for booting an operating system
- Apply tools for monitoring operating system performance
- Design and implement kernel modules for interacting with a Linux kernel





Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - Varies between **Command-Line** (**CLI**), **Graphics User Interface** (**GUI**), **touch-screen**, **Batch**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.





Operating System Services (Cont.)

- One set of operating-system services provides functions that are helpful to the user (Cont.):
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - **Error detection** – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system





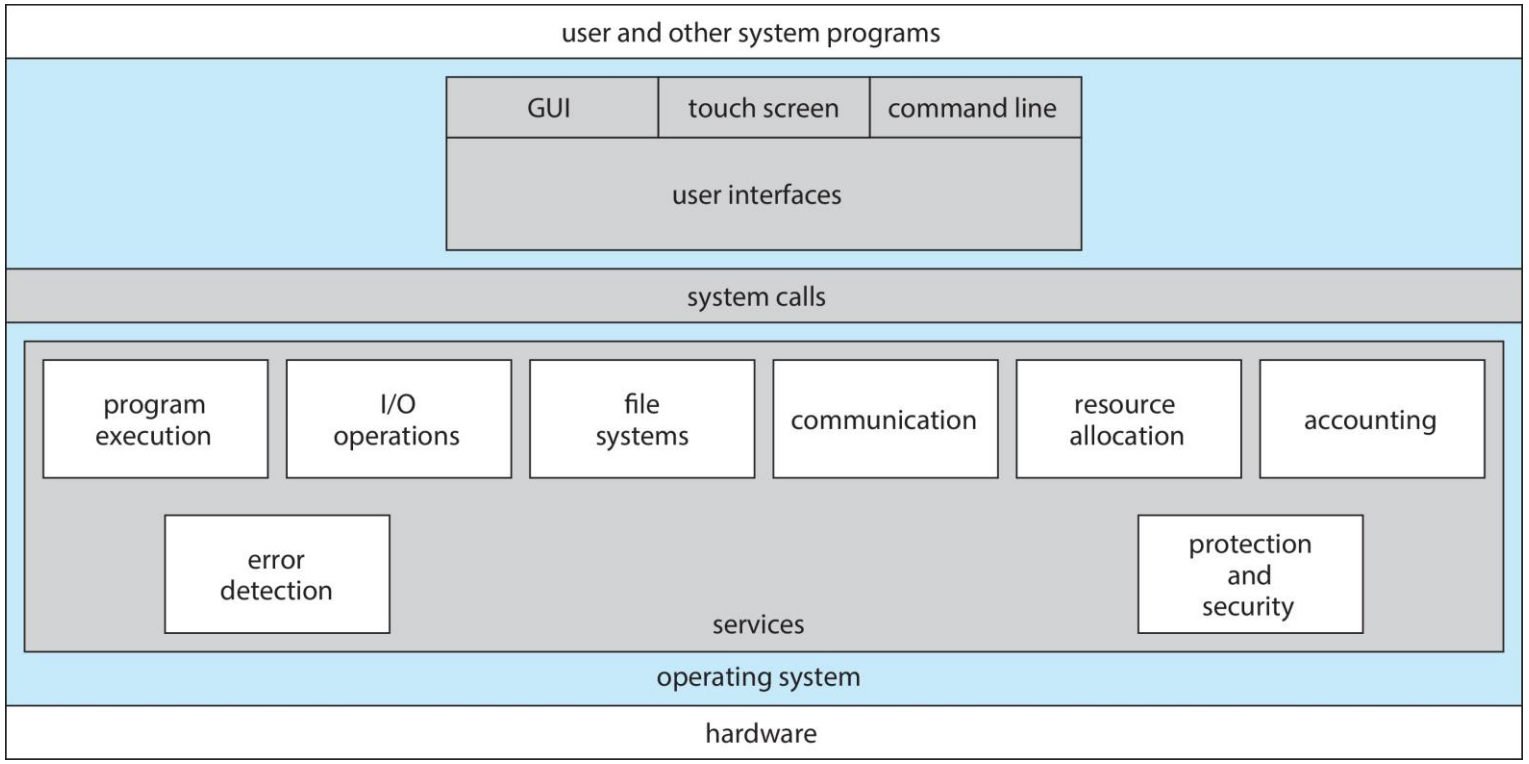
Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Logging -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts





A View of Operating System Services

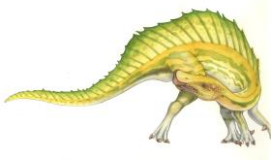






Command Line interpreter

- CLI allows direct command entry
- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – **shells**
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
  - If the latter, adding new features doesn't require shell modification





Bourne Shell Command Interpreter

1. root@r6181-d5-us01:~ (ssh)

root@r6181-d5-u... ⌘1 | ssh ⌘2 | root@r6181-d5-us01... ⌘3

```
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem            Size  Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
                       50G   19G   28G  41% /
tmpfs                 127G  520K  127G   1% /dev/shm
/dev/sda1             477M   71M  381M  16% /boot
/dev/dssd0000         1.0T  480G  545G  47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs
                       12T  5.7T  6.4T  47% /mnt/orangefs
/dev/gpfs-test         23T  1.1T   22T   5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2  6.6 42665344 17520636 ?   S<Ll Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root      69849  6.6  0.0        0        0 ?        S    Jul12 181:54 [vpthread-1-1]
root      69850  6.4  0.0        0        0 ?        S    Jul12 177:42 [vpthread-1-2]
root       3829  3.0  0.0        0        0 ?        S    Jun27 730:04 [rp_thread 7:0]
root       3826  3.0  0.0        0        0 ?        S    Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x------ 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```





User Operating System Interface - GUI

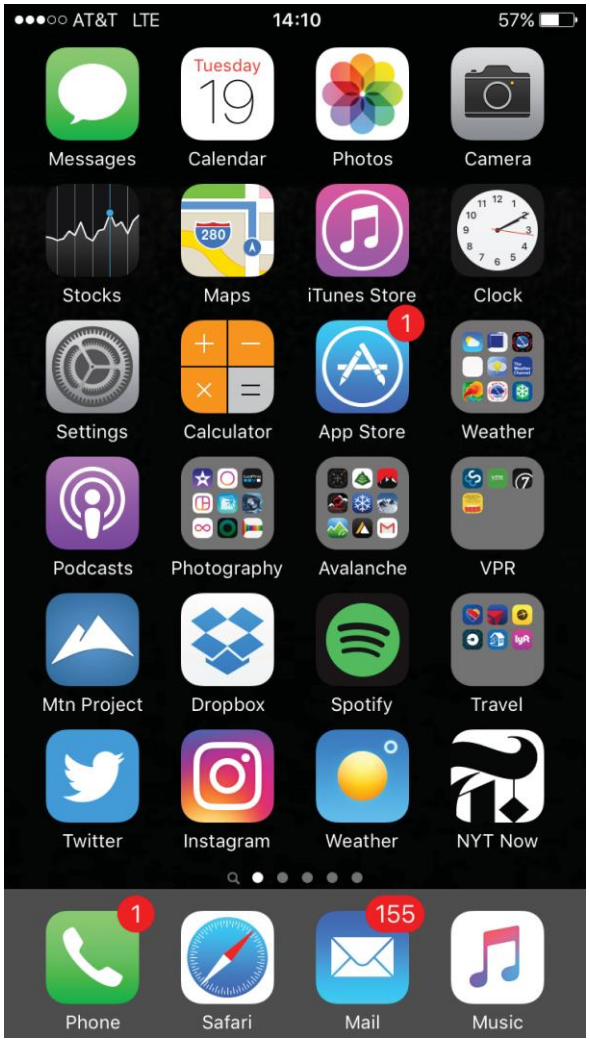
- User-friendly **desktop** metaphor interface
  - Usually mouse, keyboard, and monitor
  - **Icons** represent files, programs, actions, etc
  - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**)
  - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
  - Microsoft Windows is GUI with CLI “command” shell
  - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
  - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

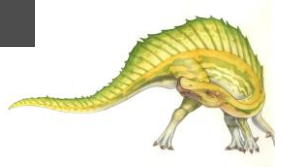




Touchscreen Interfaces

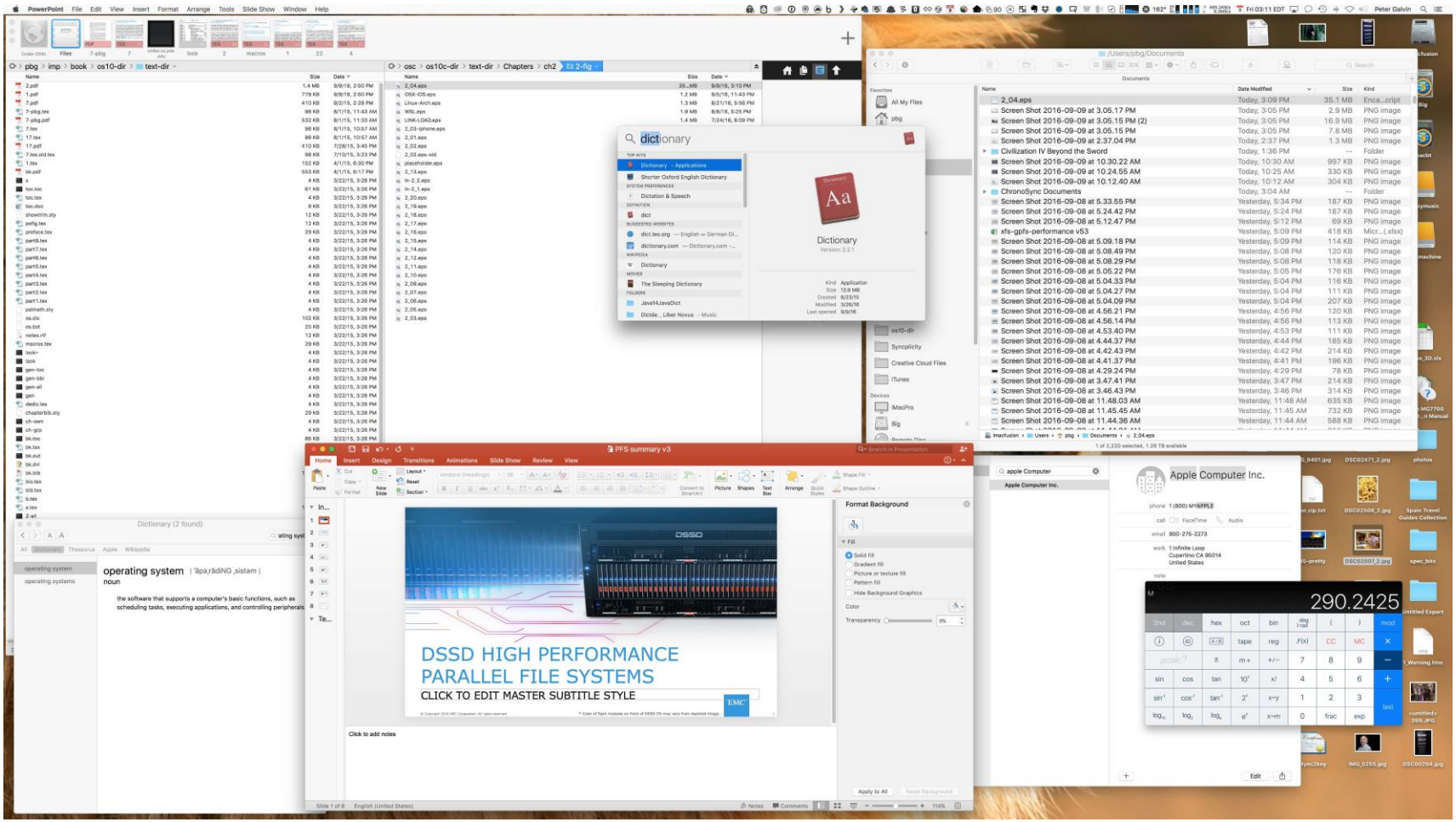
- Touchscreen devices require new interfaces
  - Mouse not possible or not desired
  - Actions and selection based on gestures
  - Virtual keyboard for text entry
- Voice commands

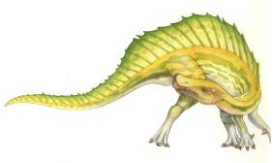






The Mac OS X GUI







System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface** (**API**) rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

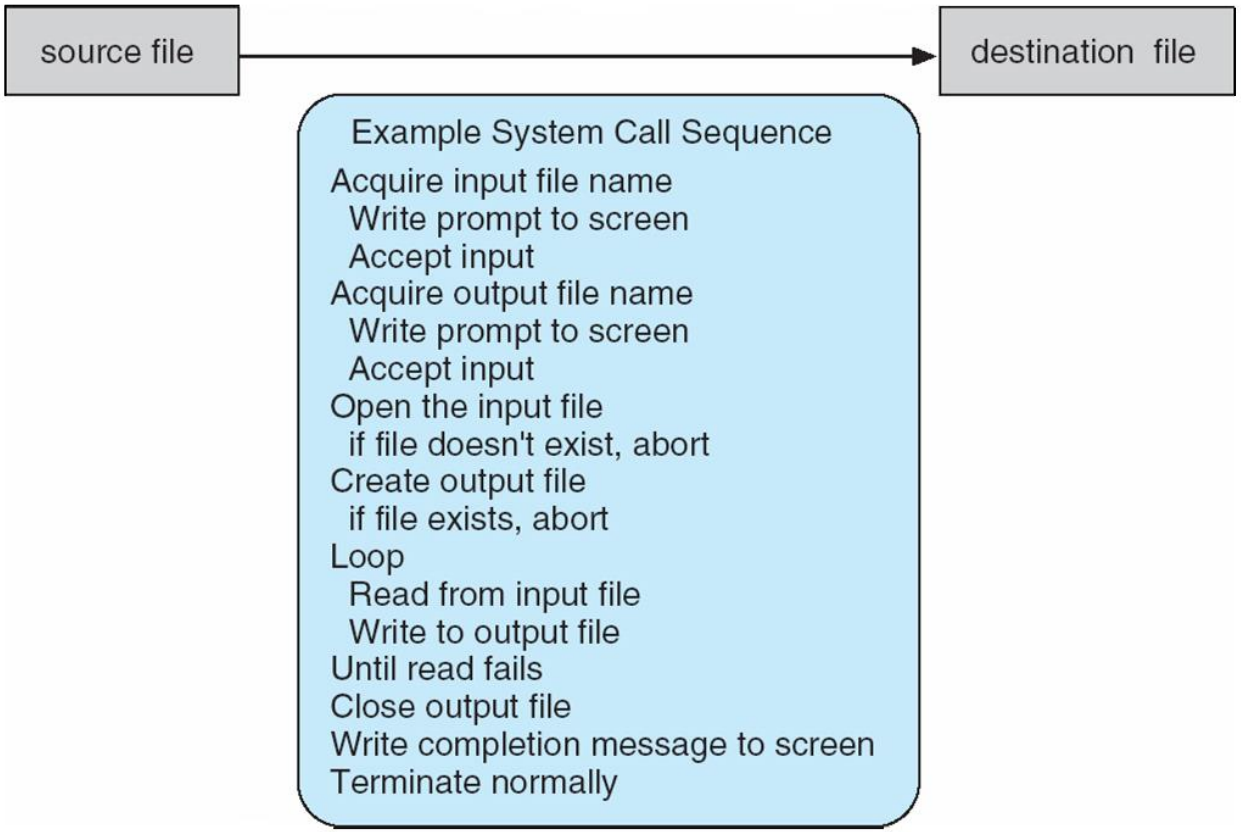
Note that the system-call names used throughout this text are generic





Example of System Calls

- System call sequence to copy the contents of one file to another file







Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

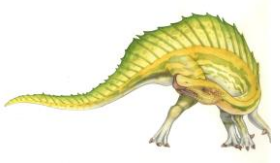
ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.





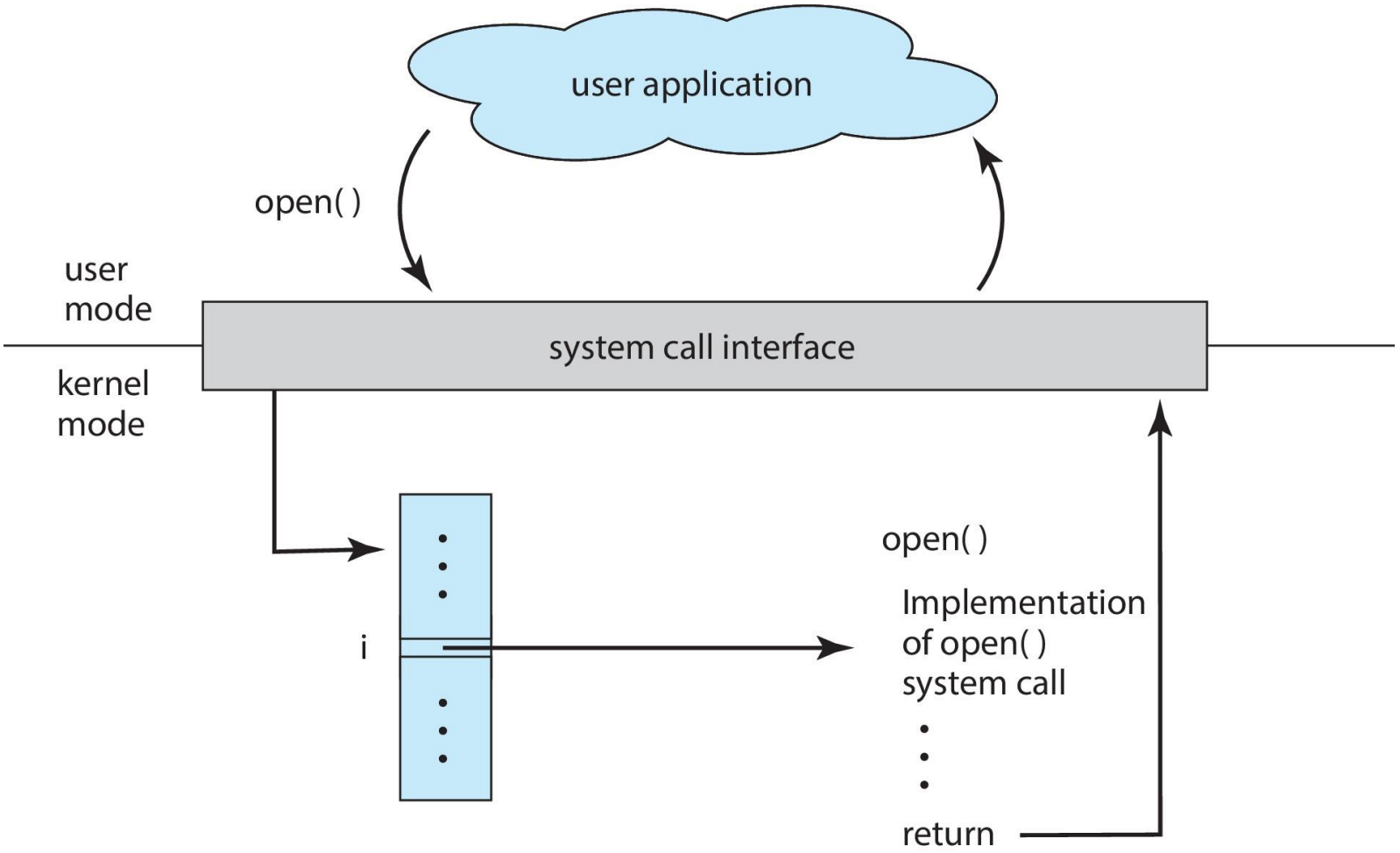
System Call Implementation

- Typically, a number is associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)





API – System Call – OS Relationship







System Call – Example

```
char name[] = "file";
main()
{
        int fd;

        fd = creat(name, 0666);
}
```

Cod dezasamblat (listat partial):

```
addr        instr
====            =====

# main
58:         mov &0x1b6, (%sp)       # pune 0666 pe stiva
5e:         mov &0x204, -(%sp)      # muta pointerul pe stiva si
                                    # pune "name pe stiva
64:         jsr 0x7a                # apeleaza libc pentru creat

.
.
.

# creat (in libc)

7a:         movq &0x8, %d0          # muta 8 in reg de date 0
7c:         trap 0xo            # OS trap
7e:         bcc &0x6 <86>           # salt la adr 86 daca carry bit e
zero
80:         jmp 0x13c           # salt la adr 13c
86:         rts                 # intoarcere din subrutina

.
.
.

# cod de biblioteca pentru erorile de syscall

13c:        mov %d0, &0x20e             # muta reg date 0 la locatia
20e (errno)
142:        movq &-0x1, %d0             # muta const. -1 in reg date 0
144:        mova %d0, %a0
146:        rts                 # intoarcere din subrutina
```





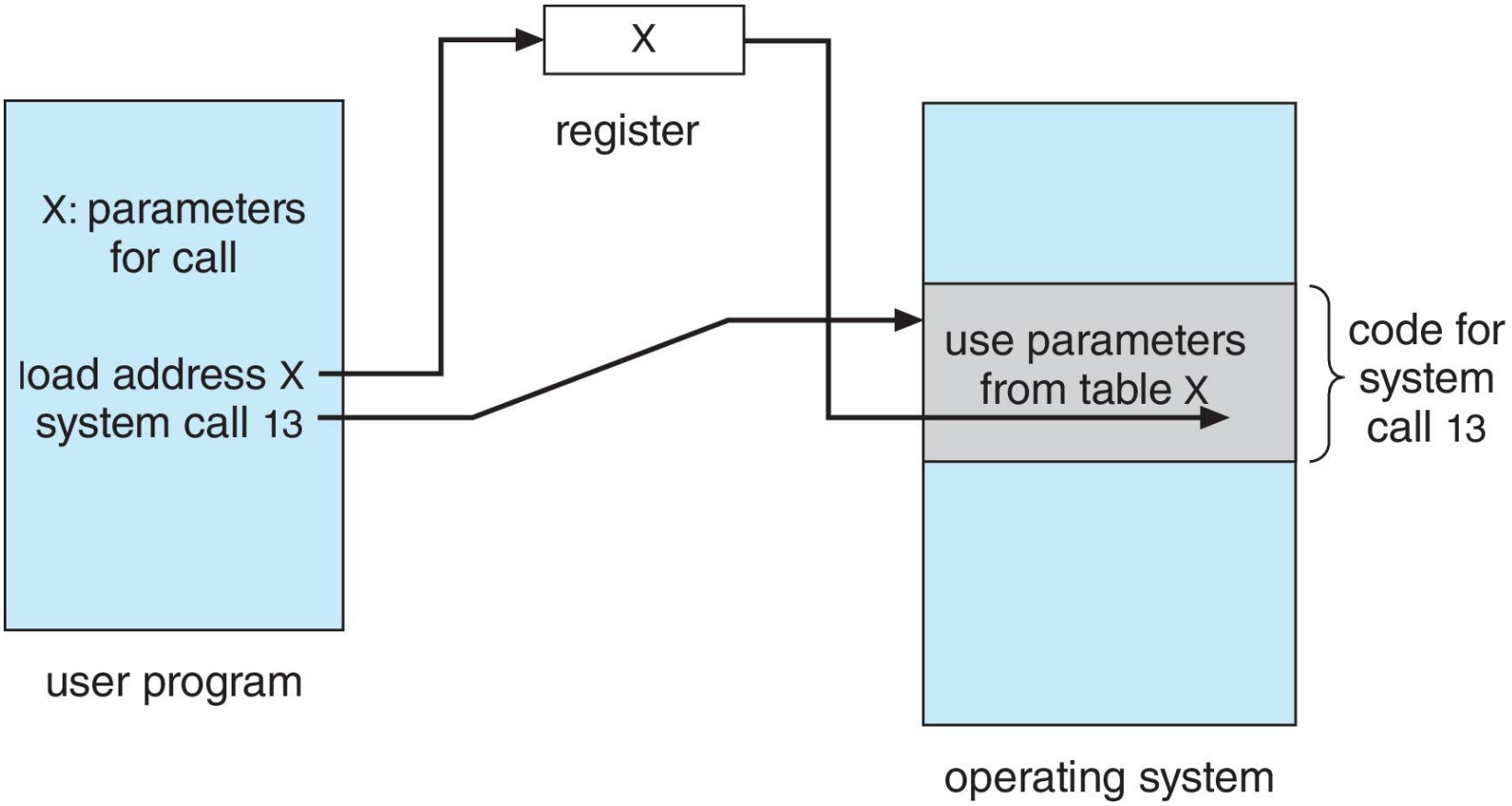
System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
  - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
  - Simplest: pass the parameters in registers
    - In some cases, may be more parameters than registers
  - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
    - This approach taken by Linux and Solaris
  - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
  - Block and stack methods do not limit the number or length of parameters being passed





Parameter Passing via Table

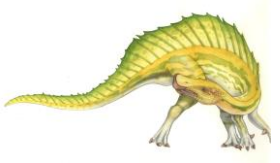






Types of System Calls

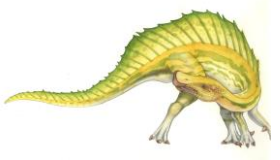
- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes





Types of System Calls (Cont.)

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices





Types of System Calls (Cont.)

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices





Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access



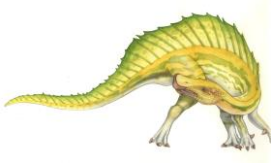


Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`



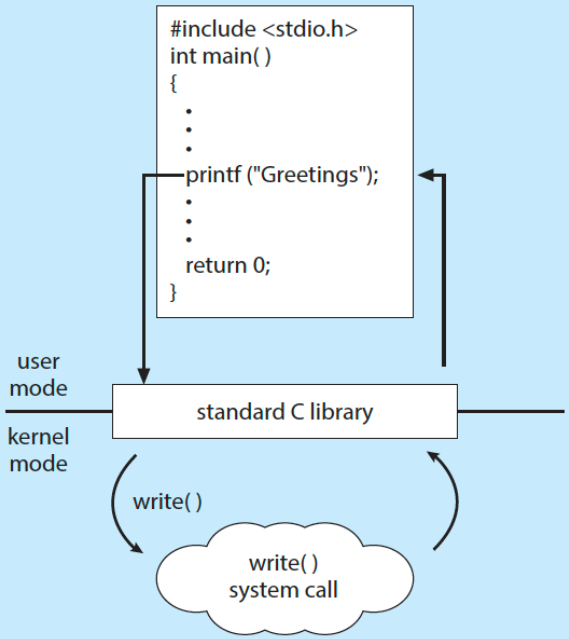


Standard C Library Example

- C program invoking printf() library call, which calls write() system call

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:







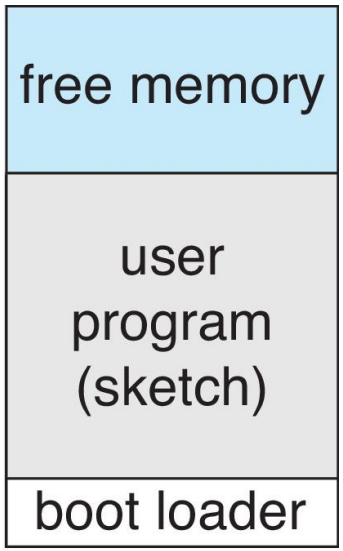
Example: Arduino

- Single-tasking
- No operating system
- Programs (sketch) loaded via USB into flash memory
- Single memory space
- Boot loader loads program
- Program exit -> shell reloaded



(a)

At system startup



(b)

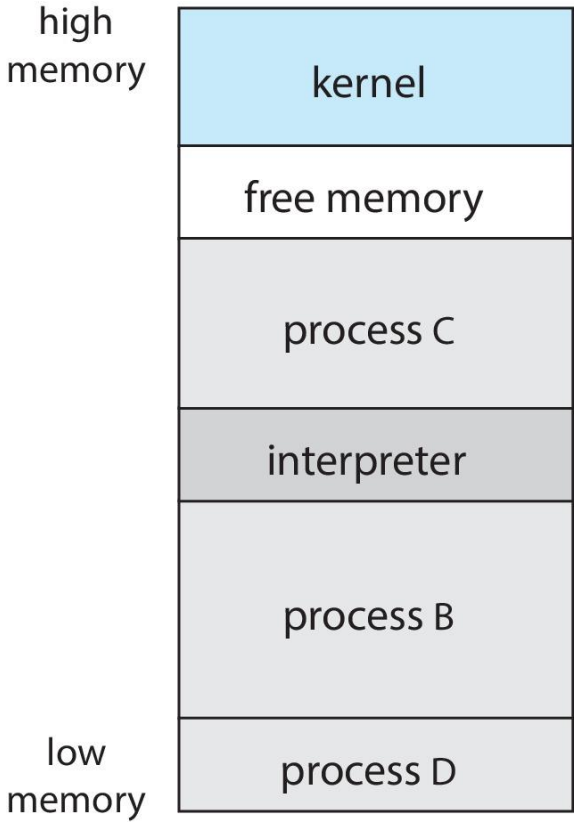
running a program





Example: FreeBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes fork() system call to create process
  - Executes exec() to load program into process
  - Shell waits for process to terminate or continues with user commands
- Process exits with:
  - code = 0 – no error
  - code > 0 – error code







System Services

- System programs provide a convenient environment for program development and execution. They can be divided into:
  - File manipulation
  - Status information sometimes stored in a file
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
  - Application programs
- Most users' view of the operating system is defined by system programs, not the actual system calls





System Services (Cont.)

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
  - Some ask the system for info - date, time, amount of available memory, disk space, number of users
  - Others provide detailed performance, logging, and debugging information
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve configuration information





System Services (Cont.)

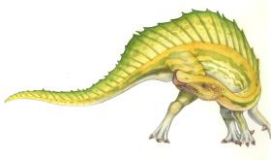
- **File modification**
  - Text editors to create and modify files
  - Special commands to search contents of files or perform transformations of the text
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another





System Services (Cont.)

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as **services**, **subsystems**, **daemons**
- **Application programs**
  - Don't pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke





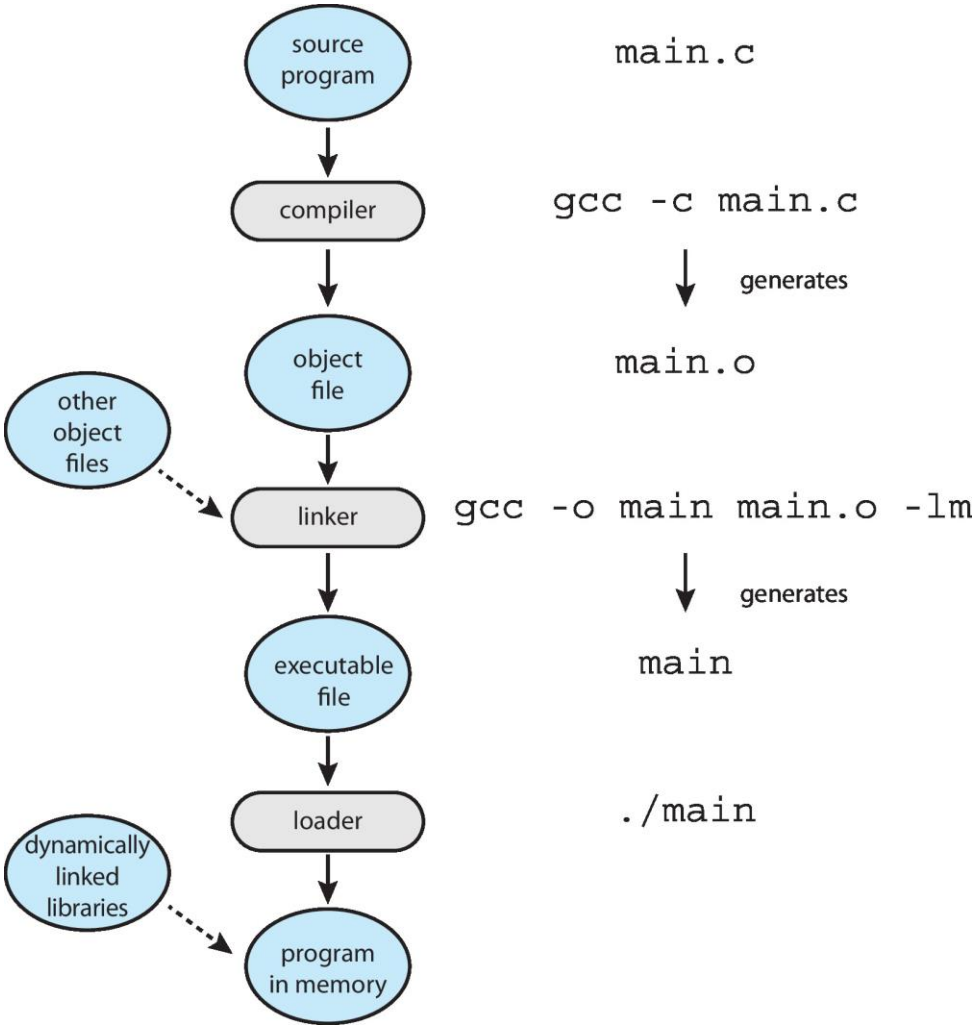
Linkers and Loaders

- Source code compiled into object files designed to be loaded into any physical memory location – **relocatable object file**
- **Linker** combines these into single binary **executable** file
  - Also brings in libraries
- Program resides on secondary storage as binary executable
- Must be brought into memory by **loader** to be executed
  - **Relocation** assigns final addresses to program parts and adjusts code and data in program to match those addresses
- Modern general purpose systems don't link libraries into executables
  - Rather, **dynamically linked libraries** (in Windows, **DLLs**) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them





The Role of the Linker and Loader







Why Applications are Operating System Specific

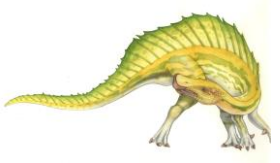
- Apps compiled on one system usually not executable on other operating systems
- Each operating system provides its own unique system calls
  - Own file formats, etc.
- Apps can be multi-operating system
  - Written in interpreted language like Python, Ruby, and interpreter available on multiple operating systems
  - App written in language that includes a VM containing the running app (like Java)
  - Use standard language (like C), compile separately on each operating system to run on each
- **Application Binary Interface** (**ABI**) is architecture equivalent of API, defines how different components of binary code can interface for a given operating system on a given architecture, CPU, etc.





Design and Implementation

- Design and Implementation of OS is not “solvable”, but some approaches have proven successful
- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of hardware, type of system
- **User** goals and **System** goals
  - User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
  - System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient
- Specifying and designing an OS is highly creative task of **software engineering**





Policy and Mechanism

- **Policy**: **What** needs to be done?
  - Example: Interrupt after every 100 seconds
- **Mechanism**: **How** to do something?
  - Example: timer
- Important principle: separate policy from mechanism
- The separation of policy from mechanism is a very important principle, it allows maximum flexibility if policy decisions are to be changed later.
  - Example: change 100 to 200





Implementation

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
  - But slower
- **Emulation** can allow an OS to run on non-native hardware





Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex – UNIX
  - Layered – an abstraction
  - Microkernel – Mach





Monolithic Structure – Original UNIX

- UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring.
- The UNIX OS consists of two separable parts
  - Systems programs
  - The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level





Monolithic Structure – Pros & Cons

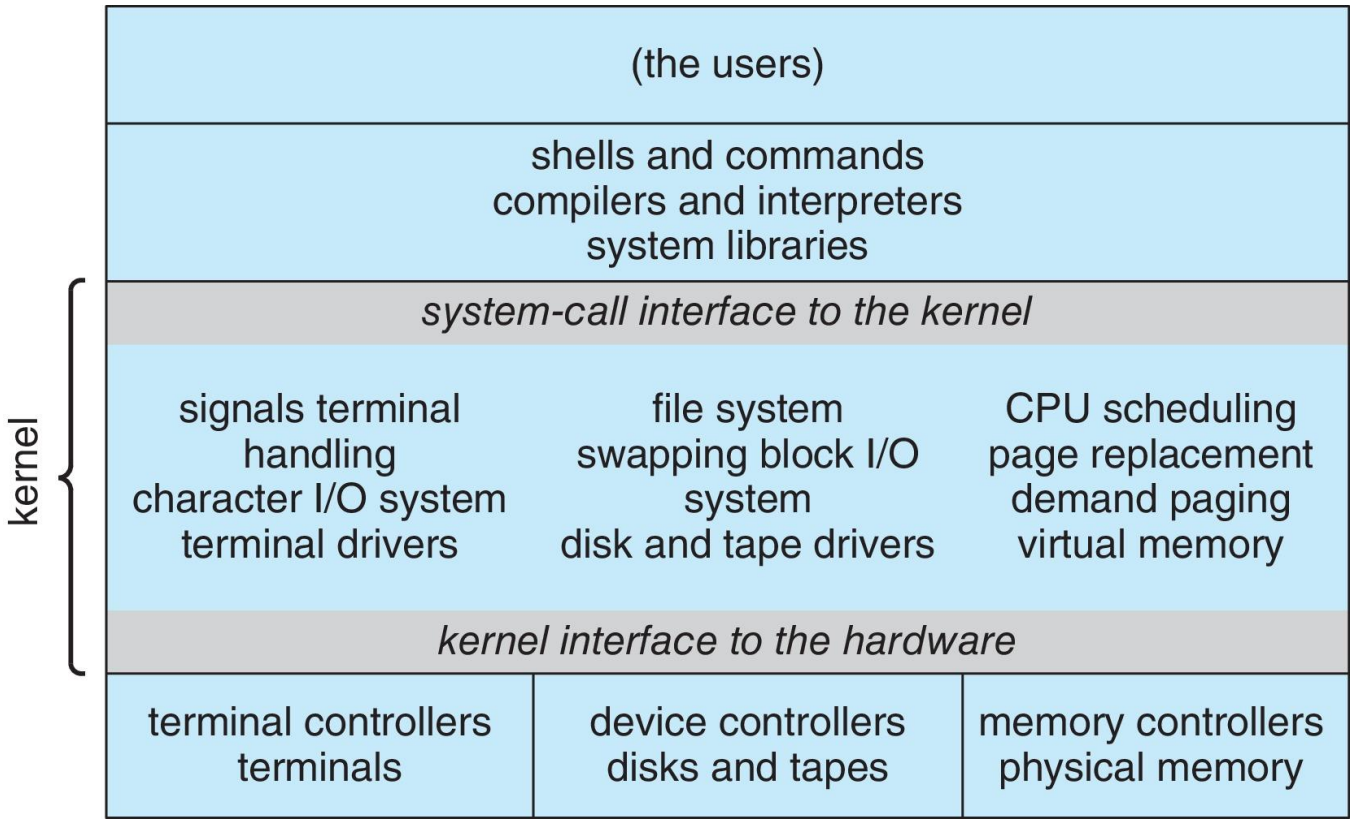
- avantaje: comunicare rapida intre module, overhead mic al apelurilor intre module (costul unui apel de procedura)
- dezavantaje:
  - interfetele intre module se estompeaza in timp dupa mai multe revizii ale kernelului
  - nr mare de syscalls si nu se stie care trebuie optimizat
  - program mare, concurent, dificil de inteles, debugat si intretinut
  - adaugare/inlaturare de drivere complicata care poate rezulta in inghetarea intregului sistem daca driverul are bug-uri





Traditional UNIX System Structure

Beyond simple but not fully layered

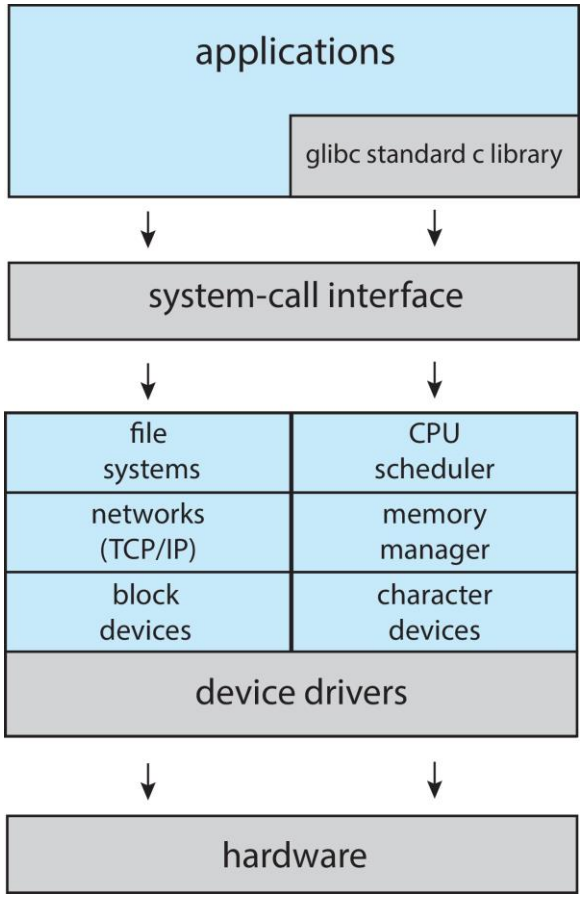






Linux System Structure

Monolithic plus modular design

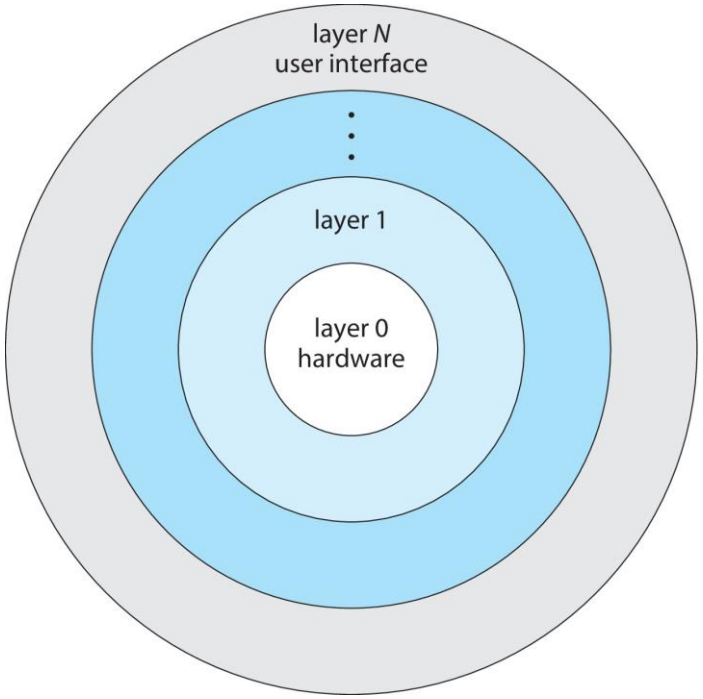


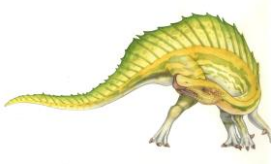




Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers







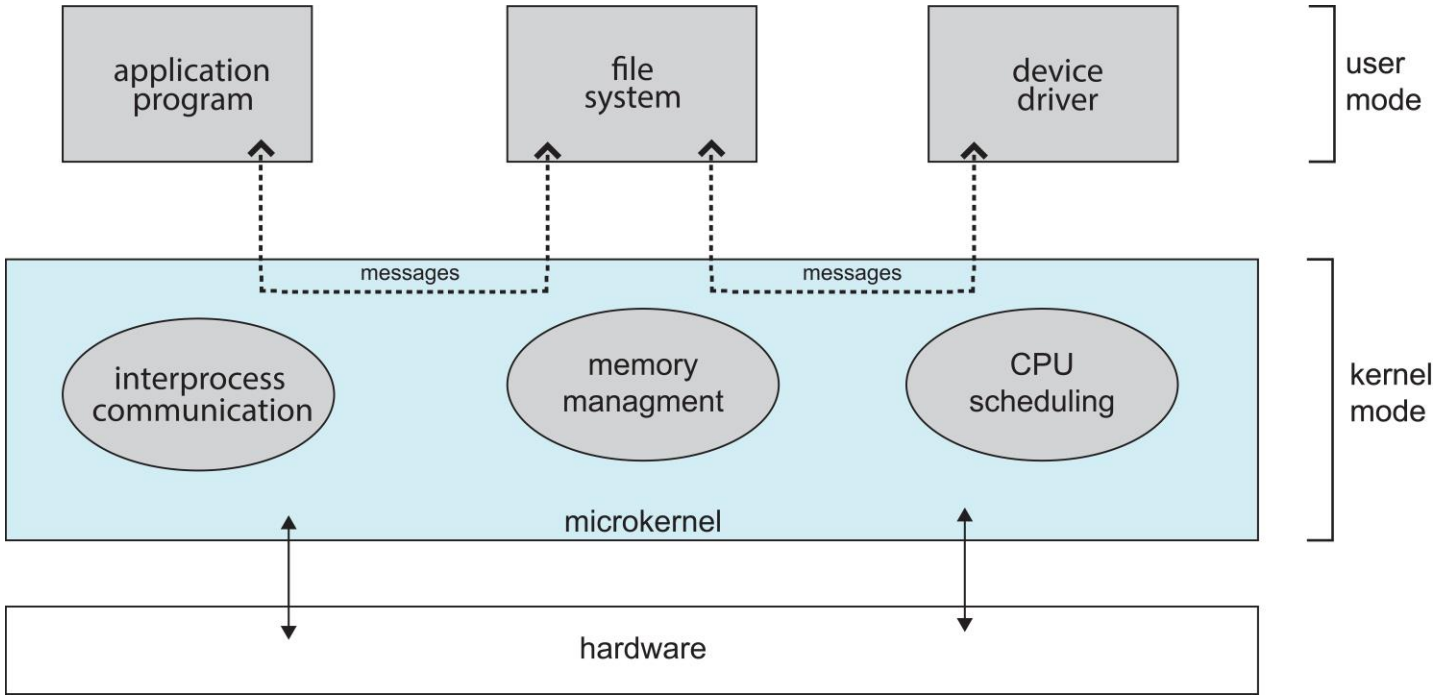
Microkernels

- Moves as much from the kernel into user space
- **Mach** is an example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication





Microkernel System Structure







Microkernels – Pros & Cons

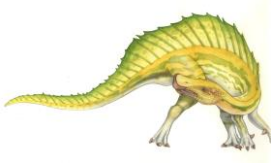
- doar gestiunea proceselor si IPC ruleaza in kernel, restul (filesystems, drivere, protocoale de retea, etc) ruleaza ca procese in user space (dar pot solicita kernelului sa execute operatii privilegiate in numele lor).
- avantaje:
  - kernel mic, usor de inteles si intretinut,
  - nr mic de syscalls (toate folosesc ipc_send/ipc_recv) care permite optimizari usoare,
  - separarea mecanismelor de politici (flexibilitate)
  - IPC peste retea permite implementarea de sisteme eterogene
  - adaugarea/inlaturarea de servicii revine la start/stop de procese in user space, MMU garanteaza granitele dintre module si crash-urile sunt astfel izolate, nu afecteaza tot sistemul
- dezavantaje:
  - comunicarea intre modulele SO este costisitoare (un null RPC necesita dubla trecere a granitei user/kernel si poate rezulta in reschedule)
  - performanta redusa.





Modules

- Many modern operating systems implement **loadable kernel modules** (**LKMs**)
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc.





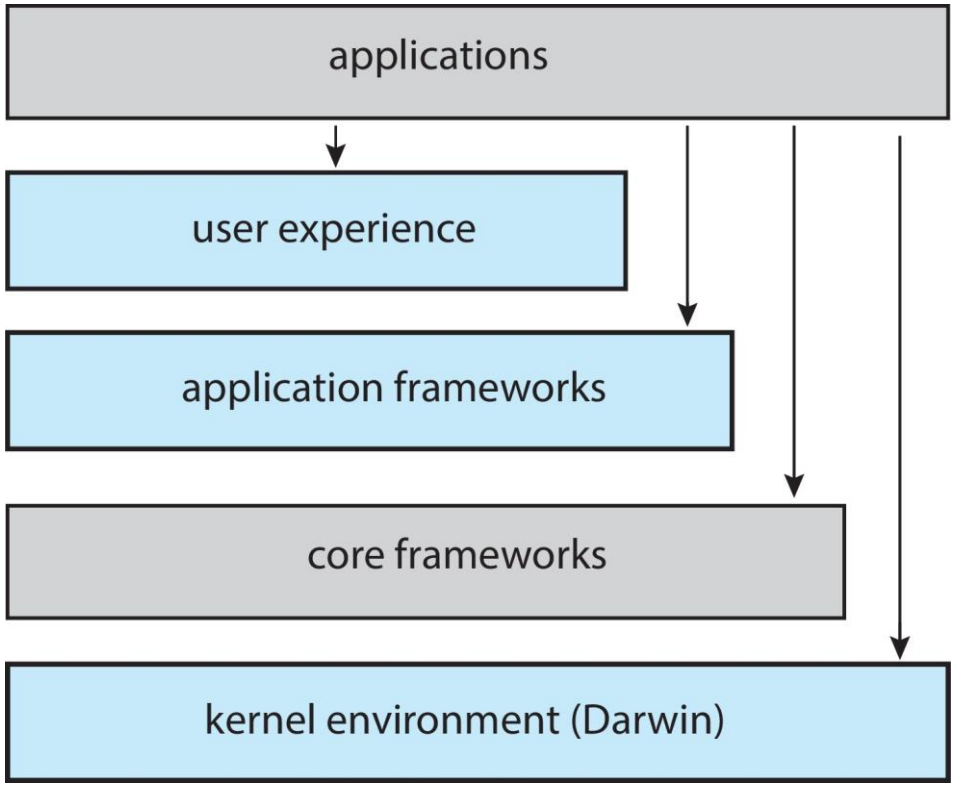
Hybrid Systems

- Most modern operating systems are not one pure model
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
  - Windows mostly monolithic, plus microkernel for different subsystem ***personalities***
- Apple Mac OS X hybrid, layered, **Aqua** UI plus **Cocoa** programming environment
  - Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)





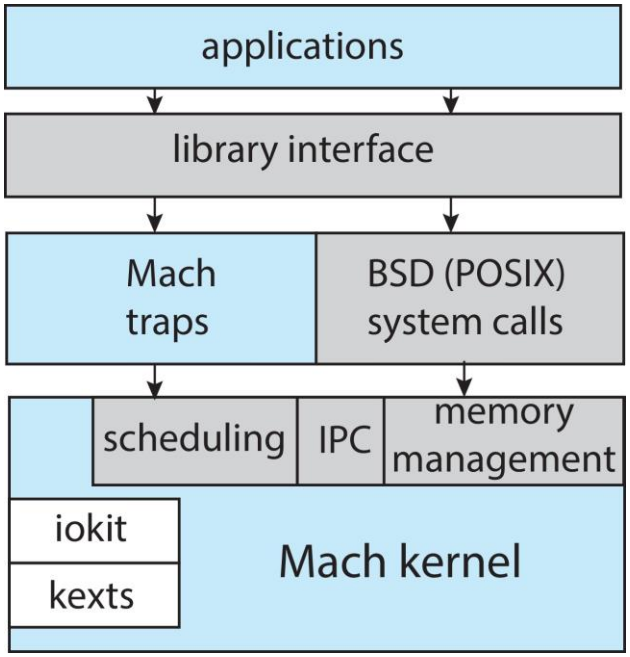
macOS and iOS Structure







Darwin


applications
library interface
Mach traps
BSD (POSIX) system calls
scheduling
IPC
memory management
iokit
kexts
Mach kernel





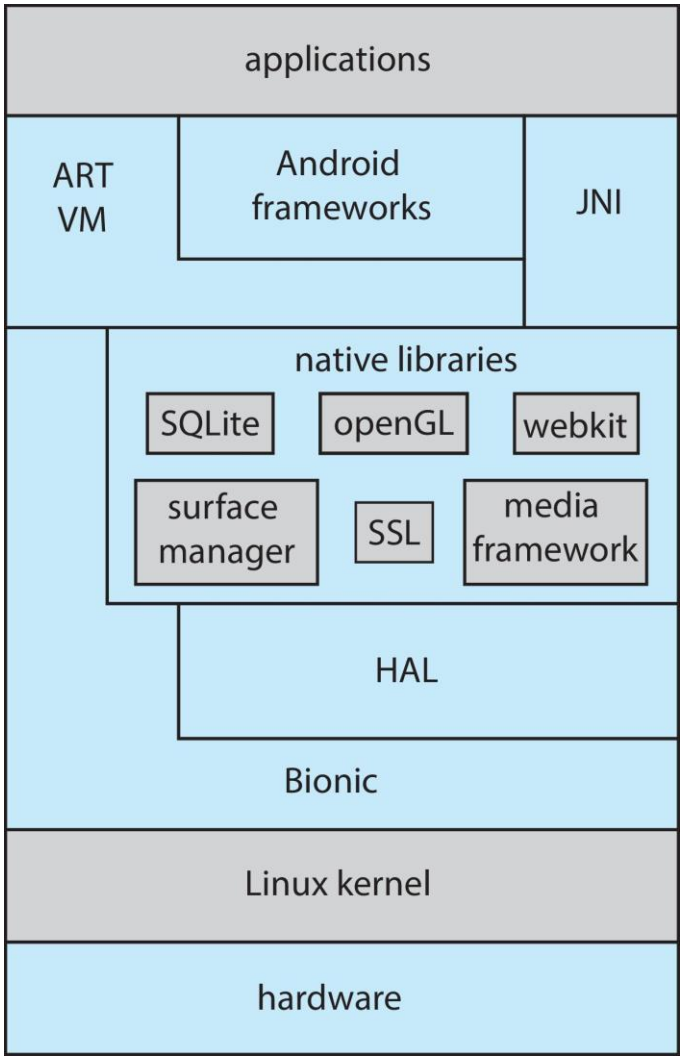
Android

- Developed by Open Handset Alliance (mostly Google)
  - Open Source
- Similar stack to iOS
- Based on Linux kernel but modified
  - Provides process, memory, device-driver management
  - Adds power management
- Runtime environment includes core set of libraries and Dalvik virtual machine
  - Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable thnn runs in Dalvik VM
- Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc





Android Architecture







Building and Booting an Operating System

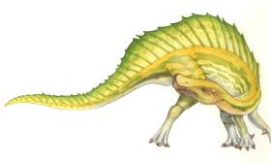
- Operating systems generally designed to run on a class of systems with variety of peripherals
- Commonly, operating system already installed on purchased computer
  - But can build and install some other operating systems
  - If generating an operating system from scratch
    - Write the operating system source code
    - Configure the operating system for the system on which it will run
    - Compile the operating system
    - Install the operating system
    - Boot the computer and its new operating system





Building and Booting Linux

- Download Linux source code (http://www.kernel.org)
- Configure kernel via "`make menuconfig`"
- Compile the kernel using "`make`"
  - Produces `vmlinuz`, the kernel image
  - Compile kernel modules via "`make modules`"
  - Install kernel modules into `vmlinuz` via "`make modules_install`"
  - Install new kernel on the system via "`make install`"





System Boot

- When power initialized on system, execution starts at a fixed memory location
- Operating system must be made available to hardware so hardware can start it
  - Small piece of code – **bootstrap loader**, **BIOS**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
  - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
  - Modern systems replace BIOS with **Unified Extensible Firmware Interface** (**UEFI**)
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Boot loaders frequently allow various boot states, such as single user mode





Operating-System Debugging

- **Debugging** is finding and fixing errors, or **bugs**
- Also **performance tuning**
- OS generate **log files** containing error information
- Failure of an application can generate **core dump** file capturing memory of the process
- Operating system failure can generate **crash dump** file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
  - Sometimes using ***trace listings*** of activities, recorded for analysis
  - **Profiling** is periodic sampling of instruction pointer to look for statistical trends

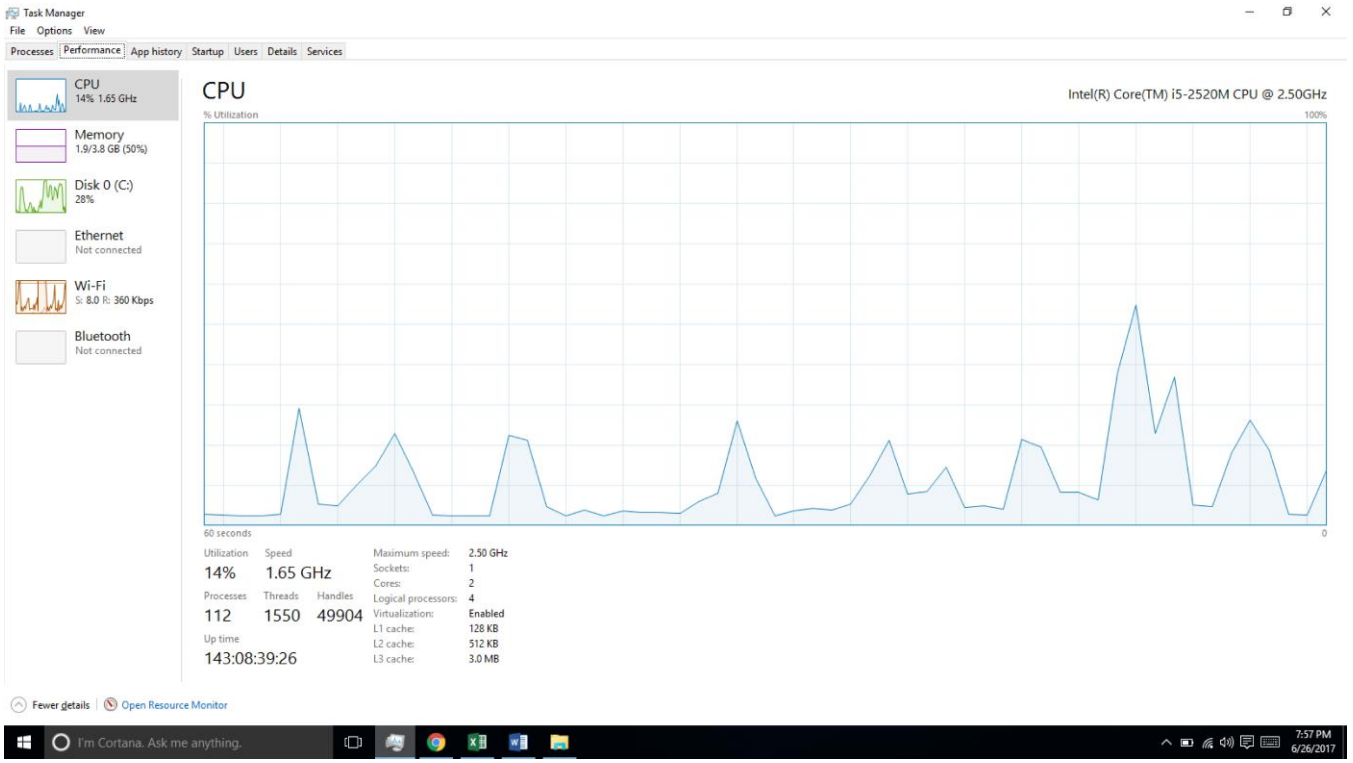
Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

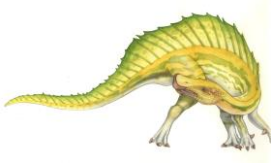




Performance Tuning

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior
- For example, “top” program or Windows Task Manager







Tracing

- Collects data for a specific event, such as steps involved in a system call invocation
- Tools include
  - strace – trace system calls invoked by a process
  - gdb – source-level debugger
  - perf – collection of Linux performance tools
  - tcpdump – collects network packets



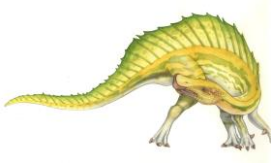


BCC

- Debugging interactions between user-level and kernel code nearly impossible without toolset that understands both and an instrument their actions
- BCC (BPF Compiler Collection) is a rich toolkit providing tracing features for Linux
  - See also the original DTrace
- For example, disksnoop.py traces disk I/O activity

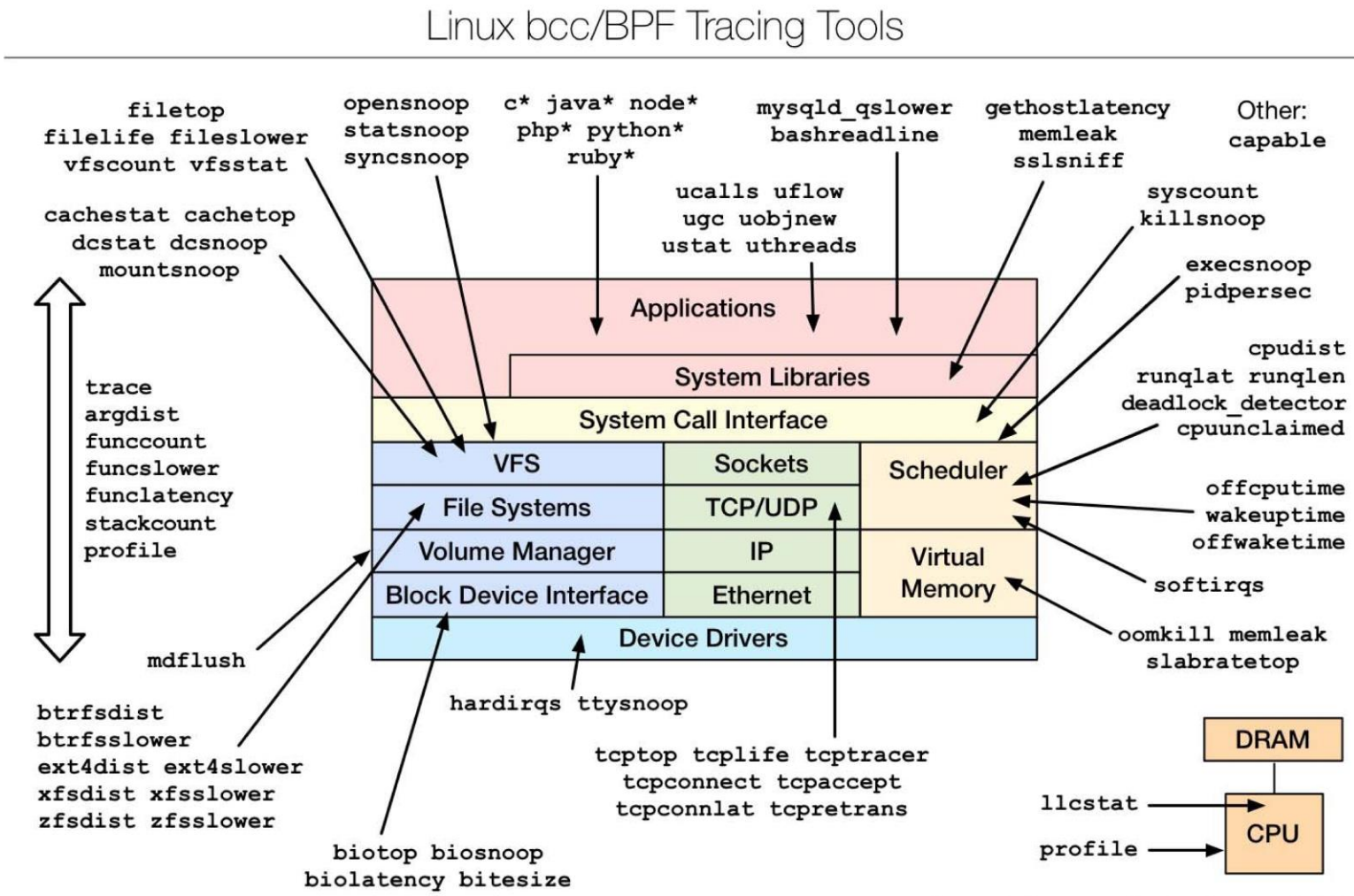
```
TIME(s)          T      BYTES      LAT(ms)
1946.29186700    R      8              0.27
1946.33965000    R      8              0.26
1948.34585000    W      8192           0.96
1950.43251000    R      4096           0.56
1951.74121000    R      4096           0.35
```

- Many other tools (next slide)



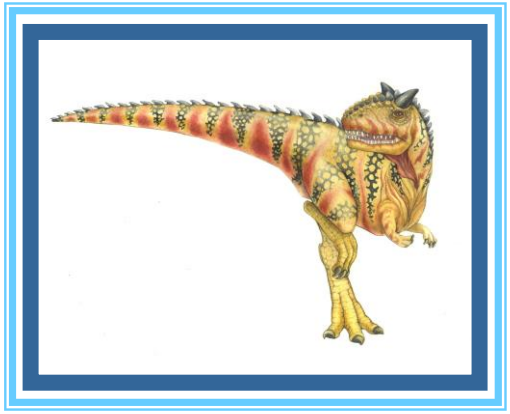


Linux bcc/BPF Tracing Tools



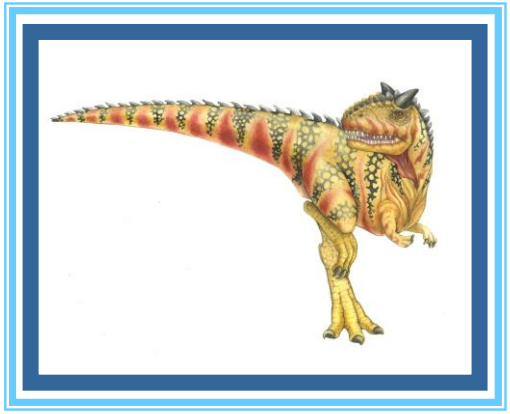


End of Chapter 2



Silberschatz, Galvin and Gagne ©2018

Chapter 3: Processes

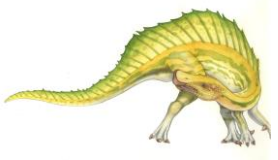


Silberschatz, Galvin and Gagne ©2018



Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Examples of IPC Systems
- Communication in Client-Server Systems





Objectives

- Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system.
- Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations.
- Describe and contrast interprocess communication using shared memory and message passing.
- Design programs that uses pipes and POSIX shared memory to perform interprocess communication.
- Describe client-server communication using sockets and remote procedure calls.
- Design kernel modules that interact with the Linux operating system.





Controlul proceselor

- proces = abstractia rularii unui program (instr + date)
  - in fapt, este programul + starea executiei sale la un moment dat (reflectata de registrele CPU si valorile variabilelor din program)
- multiprogramarea/multitasking-ul (concurenta virtuala/logica sau fizica, pt multiprocesoare) impun nevoia unui model de proces
  - fiecare proces e tratat ca si cand ar avea propriul CPU (abstractia de masina virtuala/extinsa)
- managerul de procese
  - componenta kernel care mentine informatii despre procesele din sistem
  - in principal, se bazeaza pe tabela de procese (contine PCB-uri) si cozi de procese (gestionate de CPU)
- PCB-ul contine: PID, prioritatea procesului, starea, valorile registrelor CPU, utilizarea memoriei, lista de fisiere deschise de proces, etc





Process Concept

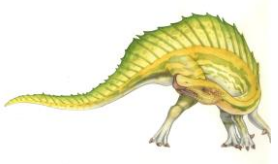
- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time





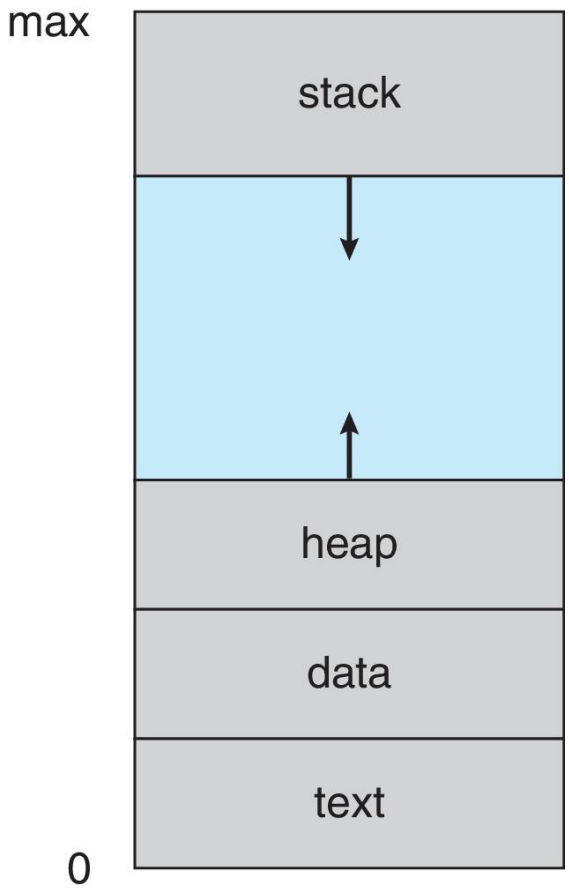
Process Concept (Cont.)

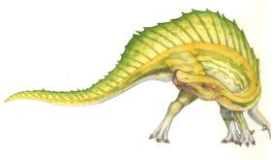
- Program is **passive** entity stored on disk (**executable file**); process is **active**
  - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
  - Consider multiple users executing the same program





Process in Memory

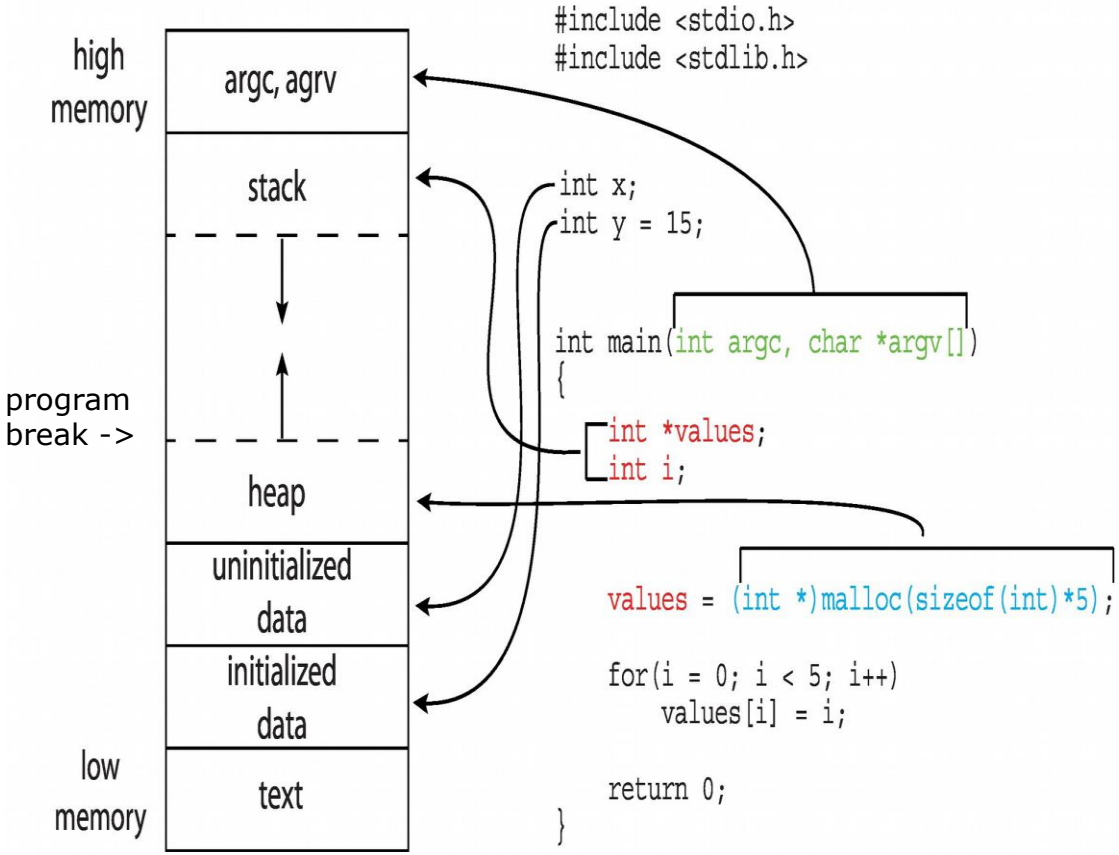






Imaginea unui program C in memorie

- codul si datele initializate sunt citite de *exec* din executabil (fisierul program de pe disc)
- datele neinitializate (bss) sunt initializate cu zero de *exec*
- heap-ul este gestionat ajutorul *program break*-ul (indica sfarsitul segmentului de date)
- valoarea break-ului este modificata prin apelurile de sistem *brk/sbrk*
  - cresterea valorii break-ului ⇔ alocare memorie dinamica
  - scaderea valorii break-ului ⇔ dealocare memorie dinamica
  - *sbrk(0)* intoarce valoarea curenta a program break-ului
  - in practica se evita folosirea *brk/sbrk* si se folosesc *malloc/free*
- stiva program (call stack) este gestionata cu ajutorul inregistrarilor de activare







Call stack

```
char buf[4096];

int main(int argc, char* argv[]) {
    int fdold, fdnew;

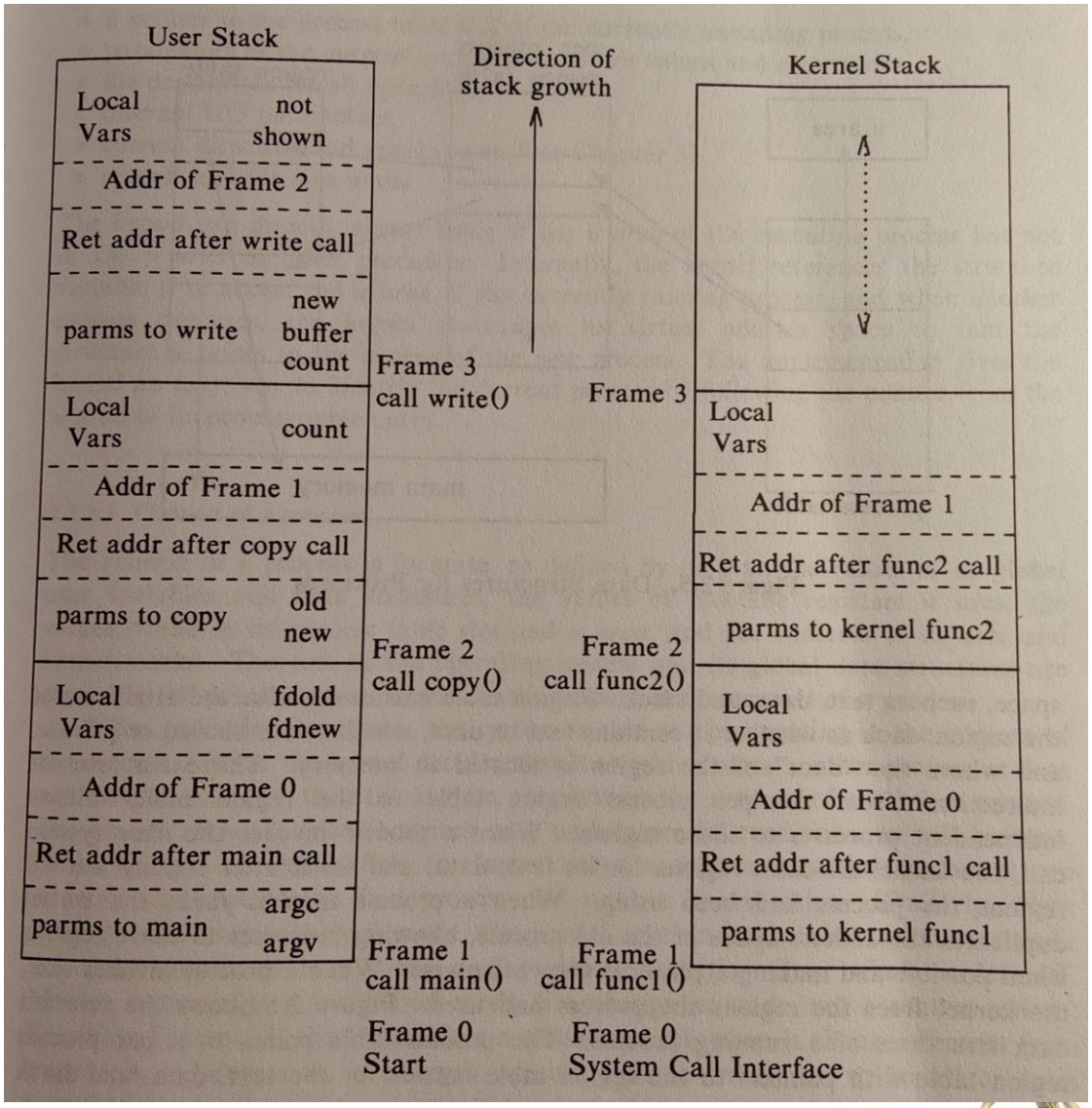
    if(argc != 3) return 0;

    if((fdold=open(argv[1],
            O_RDONLY)) < 0)
        exit(1);
   if((fdnew=creat(argv[2],0666))
            < 0)
        exit(1);

   copy(fdold, fdnew);
   exit(0);
}

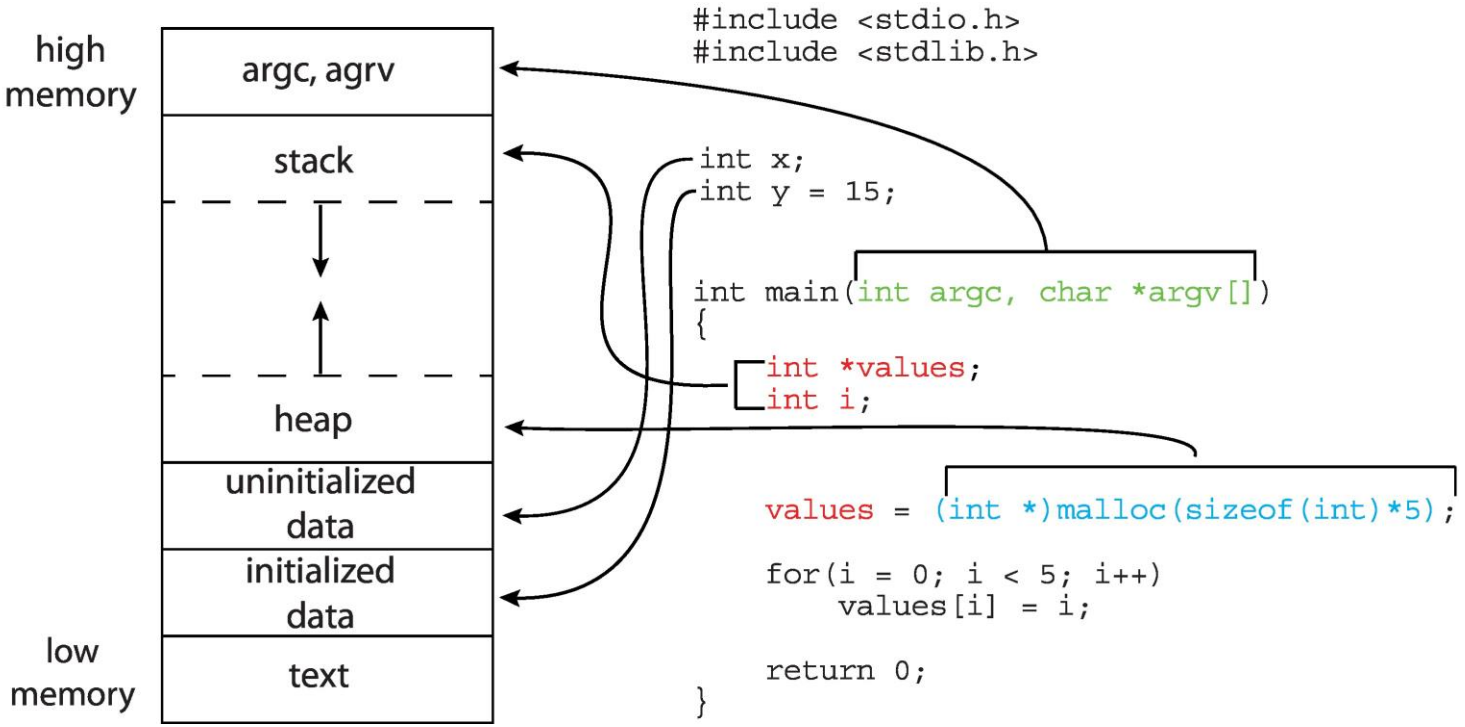
void copy(int old, int new) {
   int count;

   while((count =read(old, buf,
            4096)) > 0)
        write(new, buf, count);
}
```





Memory Layout of a C Program







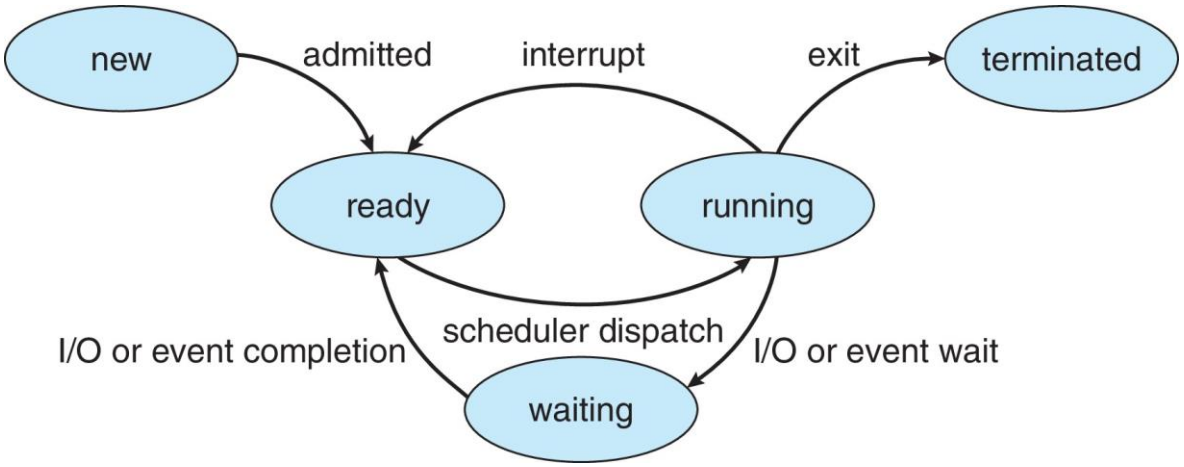
Process State

- As a process executes, it changes **state**
  - **New**: The process is being created
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
  - **Ready**: The process is waiting to be assigned to a processor
  - **Terminated**: The process has finished execution



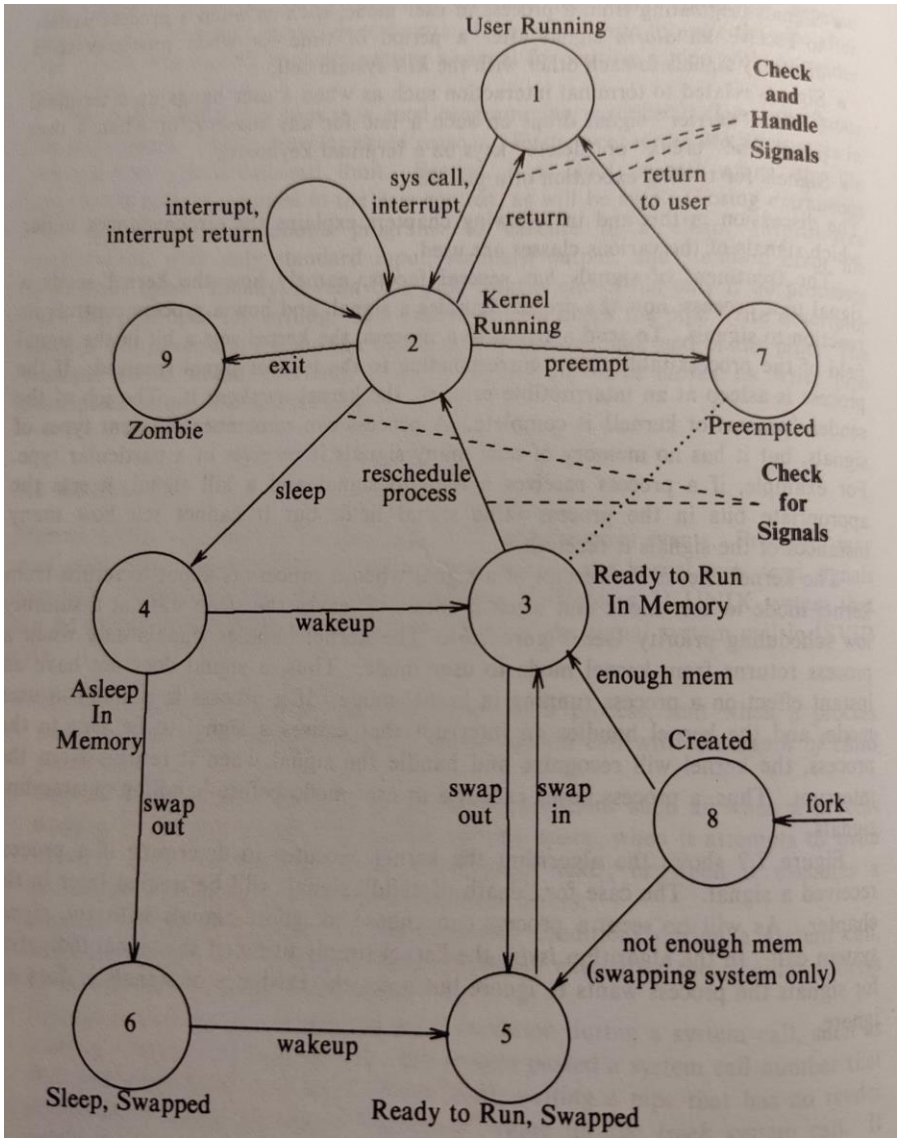


Diagram of Process State





Process State Diagram – Unix example

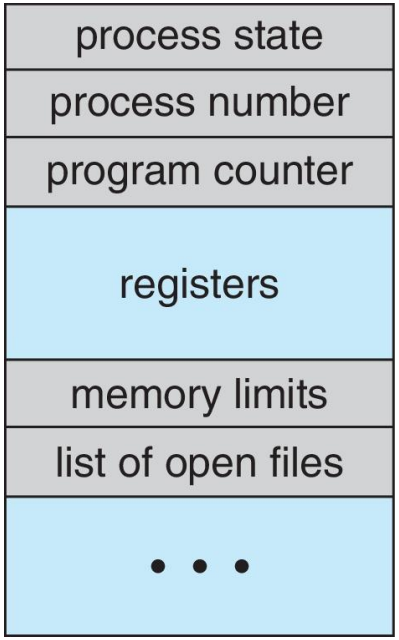


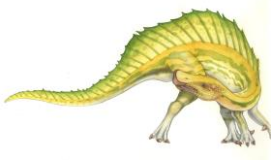


Process Control Block (PCB)

Information associated with each process(also called **task control block**)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files







Threads

- So far, process has a single thread of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
    - Multiple threads of control -> **threads**
- Must then have storage for thread details, multiple program counters in PCB
- Explore in detail in Chapter 4

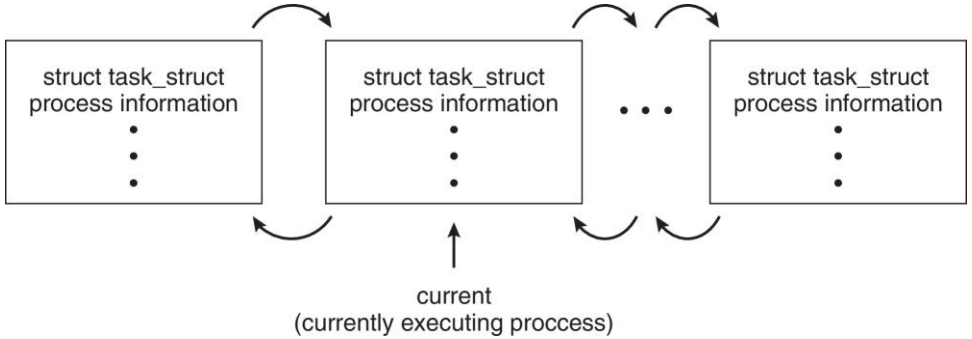




Process Representation in Linux

Represented by the C structure `task_struct`

```
pid t_pid;                    /* process identifier */
long state;                   /* state of the process */
unsigned int time_slice       /* scheduling information */
struct task_struct *parent;/* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files;/* list of open files */
struct mm_struct *mm;         /* address space of this
process */
```







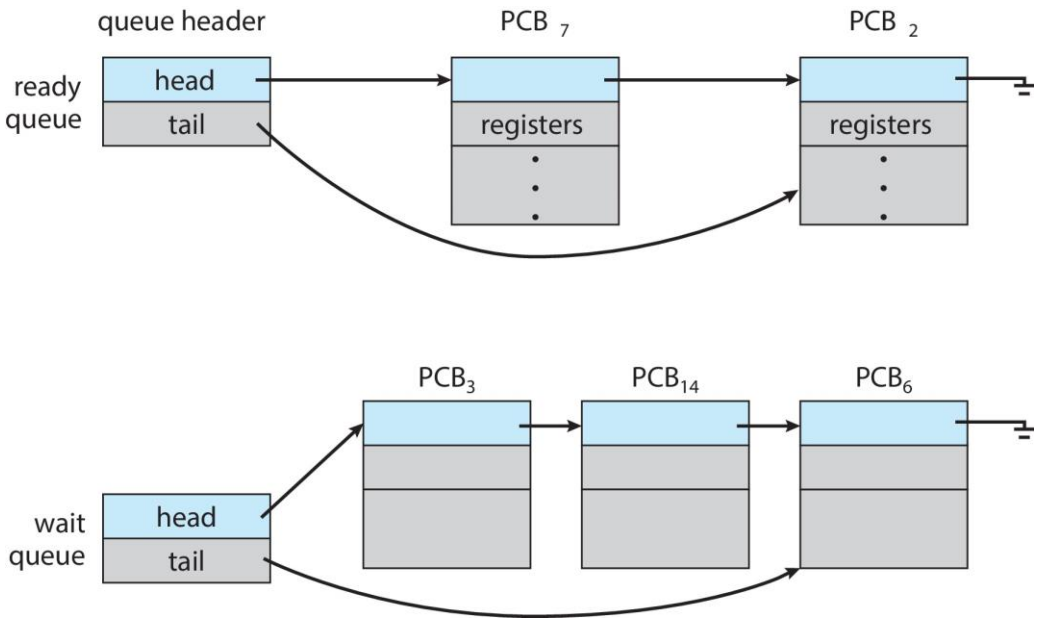
Process Scheduling

- **Process scheduler** selects among available processes for next execution on CPU core
- Goal -- Maximize CPU use, quickly switch processes onto CPU core
- Maintains **scheduling queues** of processes
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Wait queues** – set of processes waiting for an event (i.e., I/O)
  - Processes migrate among the various queues





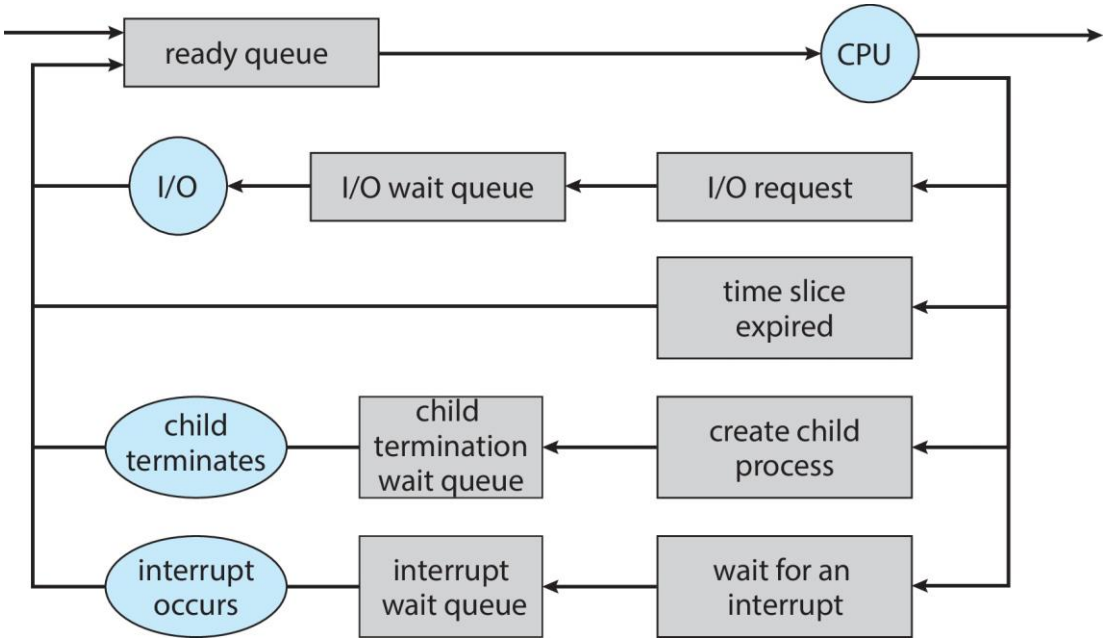
Ready and Wait Queues







Representation of Process Scheduling

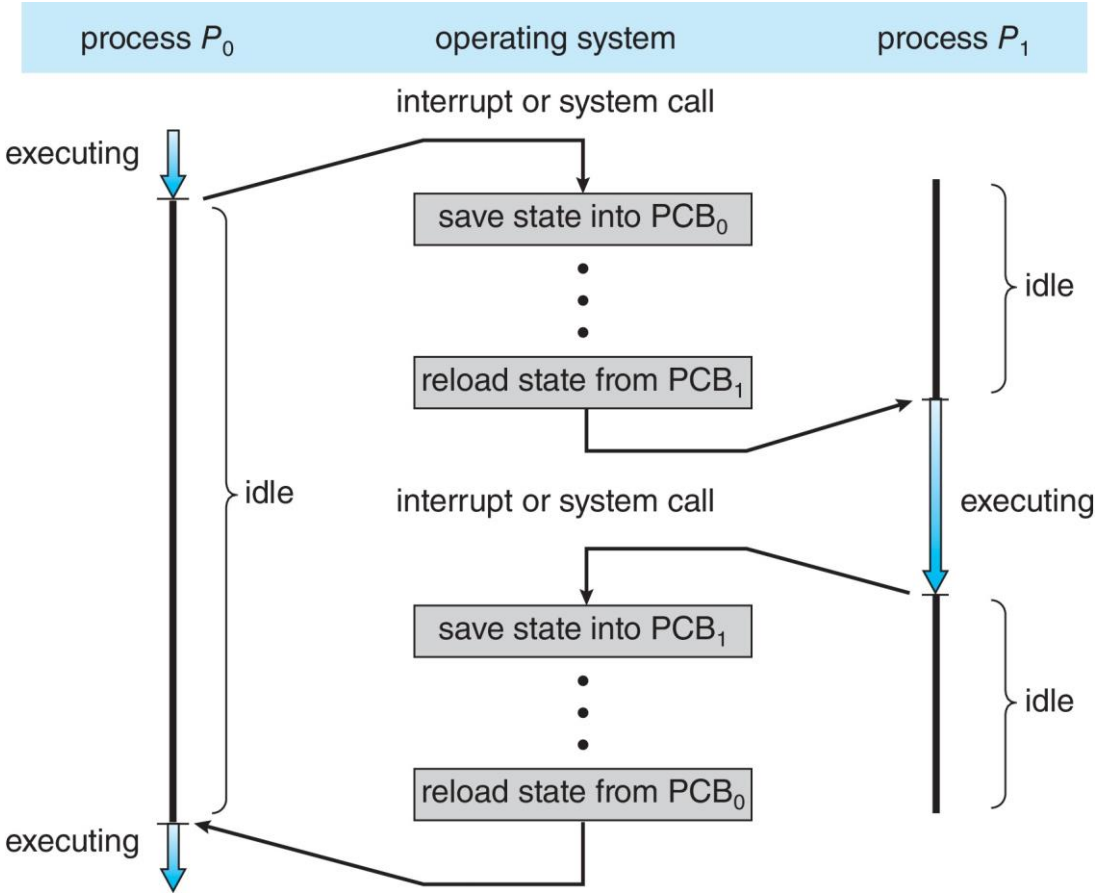






CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.







Context Switch

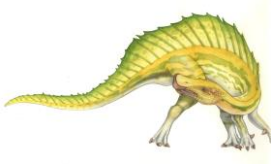
- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is pure overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once





Multitasking in Mobile Systems

- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- Due to screen real estate, user interface limits iOS provides for a
  - Single **foreground** process- controlled via user interface
  - Multiple **background** processes– in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Android runs foreground and background, with fewer limits
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use





Gestiunea proceselor

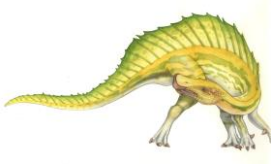
- necesita mecanisme de creare si terminare a proceselor (ca si de asteptare a terminarii unui alt proces)
- crearea proceselor
  - model arborescent: un proces parinte creeaza procese copil
  - la momentul crearii, procesul copil este o copie exacta a parintelui (cod + date)
  - implementarea duplicarii tine de optiuni si ratiuni de performanta: procesele pot partaja toate resursele, o parte dintre ele sau nu partajeaza resurse deloc
  - din momentul aparitiei procesului copil nu se mai poate conta in nici un fel pe ordinea executiei parintelui si a copilului: ambele procese se executa concurent sub controlul planificatorului de procese din kernel
  - exista posibilitatea coordonarii explicite intre parinte si copil (procesul parinte “asteapta” terminarea copilului, fie sincron, fie asincron)





Operations on Processes

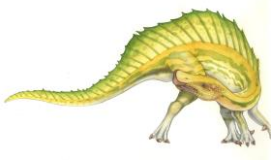
- System must provide mechanisms for:
  - Process creation
  - Process termination





Process Creation

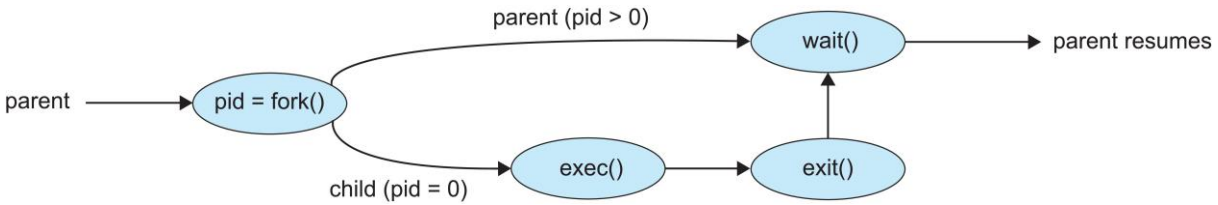
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

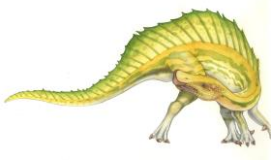




Process Creation (Cont.)

- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program
  - Parent process calls **`wait()`** waiting for the child to terminate

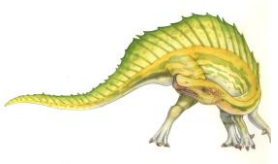






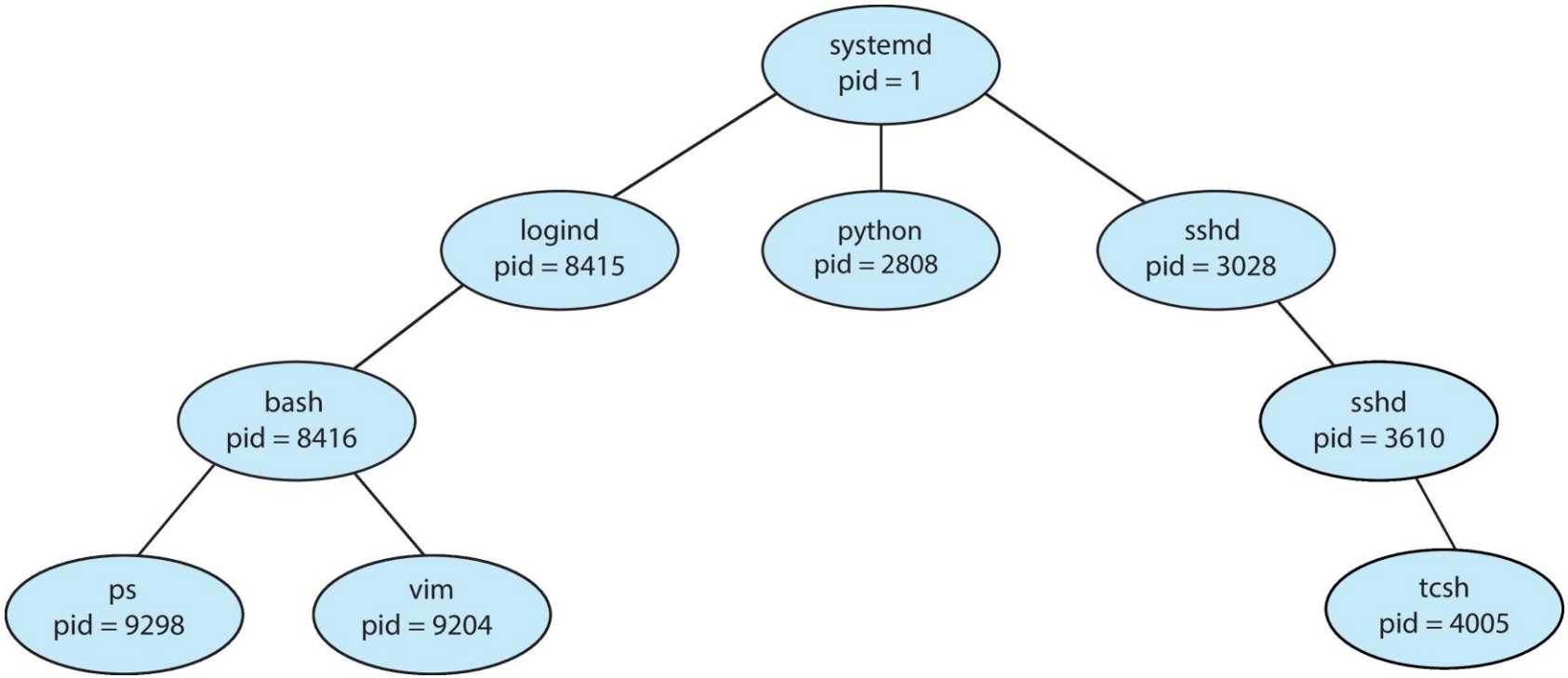
Executia fork in kernel

- kernelul verifica resursele disponibile
- aloca un PID unic si un PCB in tabela de procese
- verifica informatii de *quota* (de pilda daca utilizatorul nu are prea multe procese deschise)
- marcheaza starea procesului copil ca fiind *creat*
- copiaza PCB-ul parintelui in PCB-ul copilului (in particular, program counter-ul va fi util copilului pt reluarea executiei)
- incrementeaza referintele interne ale kernelului la directorul curent, directorul radacina si fisierele deschise
- copiaza (duplica) contextul de executie al parintelui (cod/text, date, stiva user si kernel)
- modifica in contextul procesului copil informatia necesara pt. ca acesta sa se poata recunoaste pe sine la reintoarcerea din apelul sistem
- apoi, parintele marcheaza starea copilului ca fiind *gata de rulare* si iese din kernel cu cod de retur egal cu noul PID alocat copilului
- copilul se intoarce si el din *fork* cu cod de retur zero





A Tree of Processes in Linux







C Program Forking Separate Process

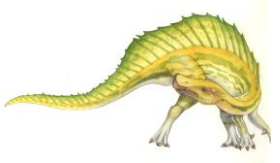
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```





Creating a Separate Process via Windows API

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
     "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
     NULL, /* don't inherit process handle */
     NULL, /* don't inherit thread handle */
     FALSE, /* disable handle inheritance */
     0, /* no creation flags */
     NULL, /* use parent's environment block */
     NULL, /* use parent's existing directory */
     &si,
     &pi))
    {
      fprintf(stderr, "Create Process Failed");
      return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

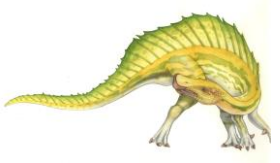
    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```





Terminarea proceselor

- normal, prin terminarea programului (“iesirea” din functia *main* in C) sau anormal, prin primirea unor semnale (exceptii software)
- dpdv tehnic, prin apel de sistem *exit*
  - intoarce starea procesului care termina in procesul parinte
  - dealoca resursele procesului
  - tratarea specifica a evenimentului se poate face prin handlere asociate terminarii procesului cu ajutorul *atexit* (de ex: tratarea specifica a dealocarii memoriei, a inchiderii fisierelor deschise, etc)
- procesul parinte asteapta terminarea copilului fie sincron, fie asincron cu ajutorul *wait*
  - asincron: cand copilul se termina inainte ca parintele sa cheme *wait* => procesul copil intra in starea de “zombie” (nu mai are PCB, kernelul mentine doar informatie minimala,i.e. codul de iesire, PID, informatie “contabila” despre procesul copil pentru a le putea furniza parintelui can apeleaza *wait*)
- parintele nu apeleaza *wait* si termina inaintea copilului, procesul copil devine orfan
  - in Unix, devine copilul procesului *init*, primul proces pornit de sistem (PID = 1)
  - *init* va executa un *wait* asincron cand primeste SIGCHLD de la kernel la terminarea copilului





Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates





Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

- If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
- If parent terminated without invoking **`wait()`**, process is an **orphan**





Android Process Importance Hierarchy

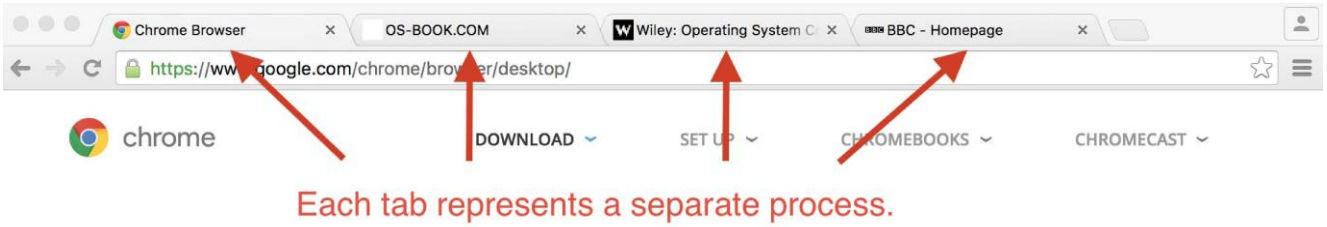
- Mobile operating systems often have to terminate processes to reclaim system resources such as memory. From **most** to **least** important:
  - Foreground process
  - Visible process
  - Service process
  - Background process
  - Empty process
- Android will begin terminating processes that are least important.

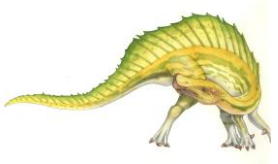




Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in







Interprocess Communication

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

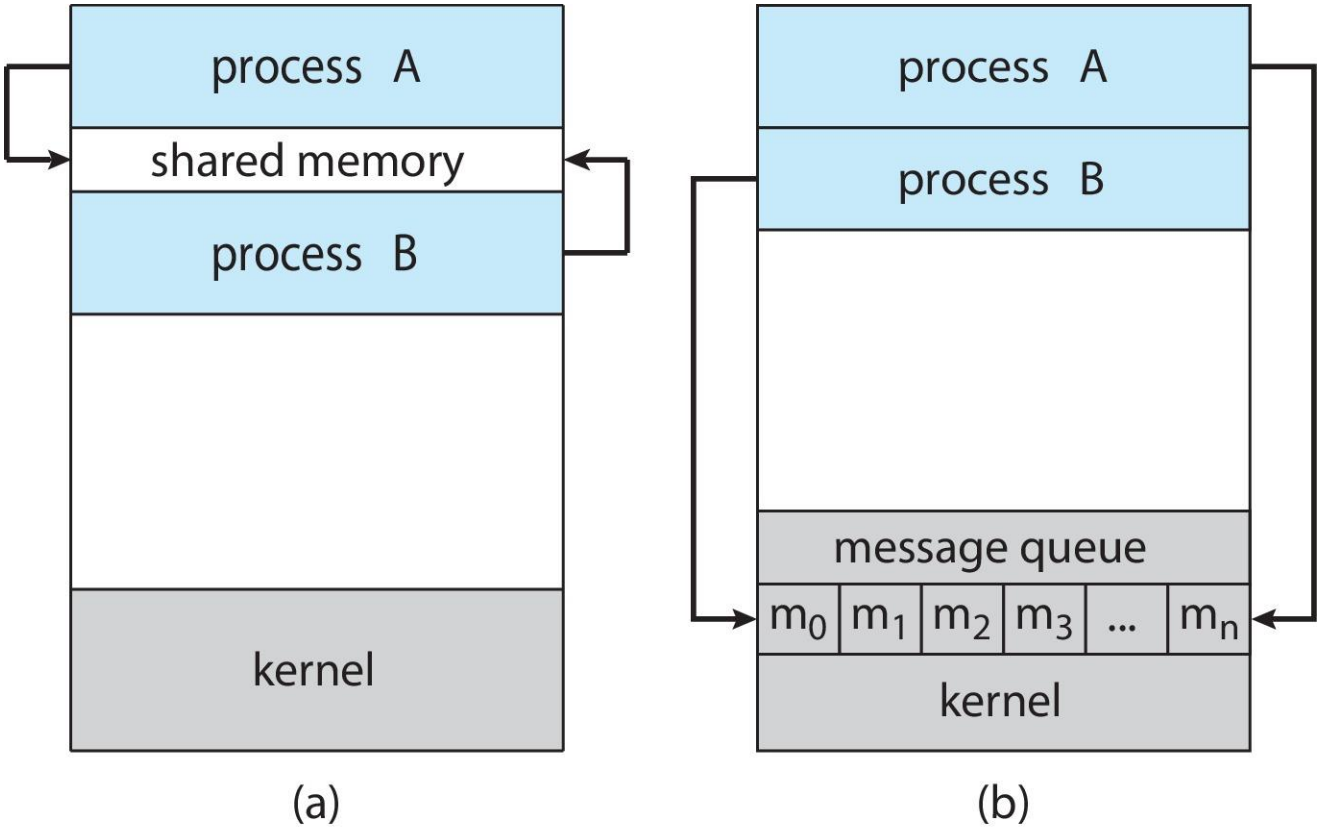


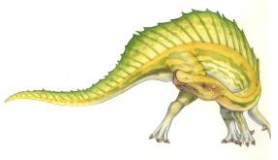


Communications Models

(a) Shared memory.

(b) Message passing.







Producer-Consumer Problem

- Paradigm for cooperating processes:
  - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
  - **unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume





IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in great details in Chapters 6 & 7.





Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use **`BUFFER_SIZE-1`** elements





Producer Process – Shared Memory

```
item next_produced;

while (true) {
   /* produce an item in next produced */
   while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
   buffer[in] = next_produced;
   in = (in + 1) % BUFFER_SIZE;
}
```





Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next consumed */
}
```





What about Filling all the Buffers?

- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers.
- We can do so by having an integer **`counter`** that keeps track of the number of full buffers.
- Initially, **`counter`** is set to 0.
- The integer **`counter`** is incremented by the producer after it produces a new buffer.
- The integer **`counter`** is and is decremented by the consumer after it consumes a buffer.





Producer

```
while (true) {
        /* produce an item in next produced */

        while (counter == BUFFER_SIZE)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```





Consumer

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next consumed */
}
```





Race Condition

- **`counter++`** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **`counter--`** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute `register1 = counter` {register1 = 5}
S1: producer execute `register1 = register1 + 1` {register1 = 6}
S2: consumer execute `register2 = counter` {register2 = 5}
S3: consumer execute `register2 = register2 - 1` {register2 = 4}
S4: producer execute `counter = register1` {counter = 6 }
S5: consumer execute `counter = register2` {counter = 4}





Race Condition (Cont.)

- Question – why was there no race condition in the first solution (where at most N – 1) buffers can be filled?
- More in Chapter 6.





IPC – Message Passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **`send`**(*message*)
  - **`receive`**(*message*)
- The *message* size is either fixed or variable





Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?





Implementation of Communication Link

- Physical:
  - Shared memory
  - Hardware bus
  - Network
- Logical:
  - Direct or indirect
  - Synchronous or asynchronous
  - Automatic or explicit buffering





Direct Communication

- Processes must name each other explicitly:
  - **`send`** (*P, message*) – send a message to process P
  - **`receive`**(*Q, message*) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional





Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional





Indirect Communication (Cont.)

- Operations
  - Create a new mailbox (port)
  - Send and receive messages through mailbox
  - Delete a mailbox
- Primitives are defined as:
  - **`send`**(*A, message*) – send a message to mailbox A
  - **`receive`**(*A, message*) – receive a message from mailbox A





Indirect Communication (Cont.)

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1, sends; P_2 and P_3 receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

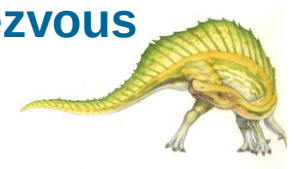




Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message
- Different combinations possible
  - If both send and receive are blocking, we have a **rendezvous**





Producer-Consumer: Message Passing

- Producer

```
message next_produced;
while (true) {
  /* produce an item in next_produced */

  send(next_produced);
}
```

- Consumer

```
message next_consumed;
while (true) {
  receive(next_consumed)

  /* consume the item in next_consumed */
}
```





Buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
  1. Zero capacity – no messages are queued on a link.
     Sender must wait for receiver (rendezvous)
  2. Bounded capacity – finite length of *n* messages
     Sender must wait if link full
  3. Unbounded capacity – infinite length
     Sender never waits





Examples of IPC Systems - POSIX

- POSIX Shared Memory
  - Process first creates shared memory segment
    `shm_fd = shm_open(name, O CREAT | O RDWR, 0666);`
  - Also used to open an existing segment
  - Set the size of the object

    `ftruncate(shm_fd, 4096);`
  - Use **`mmap()`** to memory-map a file pointer to the shared memory object
  - Reading and writing to shared memory is done by using the pointer returned by **`mmap()`**.





IPC POSIX Producer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

   /* create the shared memory object */
   shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);

   /* configure the size of the shared memory object */
   ftruncate(shm_fd, SIZE);

   /* memory map the shared memory object */
   ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);

   /* write to the shared memory object */
   sprintf(ptr,"%s",message_0);
   ptr += strlen(message_0);
   sprintf(ptr,"%s",message_1);
   ptr += strlen(message_1);

   return 0;
}
```





IPC POSIX Consumer

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>

int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;

    /* open the shared memory object */
    shm_fd = shm_open(name, O_RDONLY, 0666);

    /* memory map the shared memory object */
    ptr = mmap(0, SIZE, PROT_READ, MAP_SHARED, shm_fd, 0);

    /* read from the shared memory object */
    printf("%s",(char *)ptr);

    /* remove the shared memory object */
    shm_unlink(name);

    return 0;
}
```





Examples of IPC Systems - Mach

- Mach communication is message based
  - Even system calls are messages
  - Each task gets two ports at creation - Kernel and Notify
  - Messages are sent and received using the **`mach_msg()`** function
  - Ports needed for communication, created via

    `mach_port_allocate()`
  - Send and receive are flexible; for example four options if mailbox full:
    - Wait indefinitely
    - Wait at most n milliseconds
    - Return immediately
    - Temporarily cache a message





Mach Messages

```
#include<mach/mach.h>

struct message {
        mach_msg_header_t header;
        int data;
};

mach port t client;
mach port t server;
```





Mach Message Passing - Client

```
/* Client Code */

struct message message;

// construct the header
message.header.msgh_size = sizeof(message);
message.header.msgh_remote_port = server;
message.header.msgh_local_port = client;

// send the message
mach_msg(&message.header, // message header
   MACH_SEND_MSG, // sending a message
   sizeof(message), // size of message sent
   0, // maximum size of received message - unnecessary
   MACH_PORT_NULL, // name of receive port - unnecessary
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```





Mach Message Passing - Server

```
/* Server Code */

struct message message;

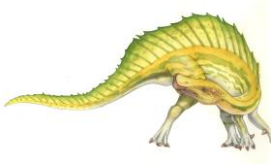
// receive the message
mach_msg(&message.header, // message header
   MACH_RCV_MSG, // sending a message
   0, // size of message sent
   sizeof(message), // maximum size of received message
   server, // name of receive port
   MACH_MSG_TIMEOUT_NONE, // no time outs
   MACH_PORT_NULL // no notify port
);
```





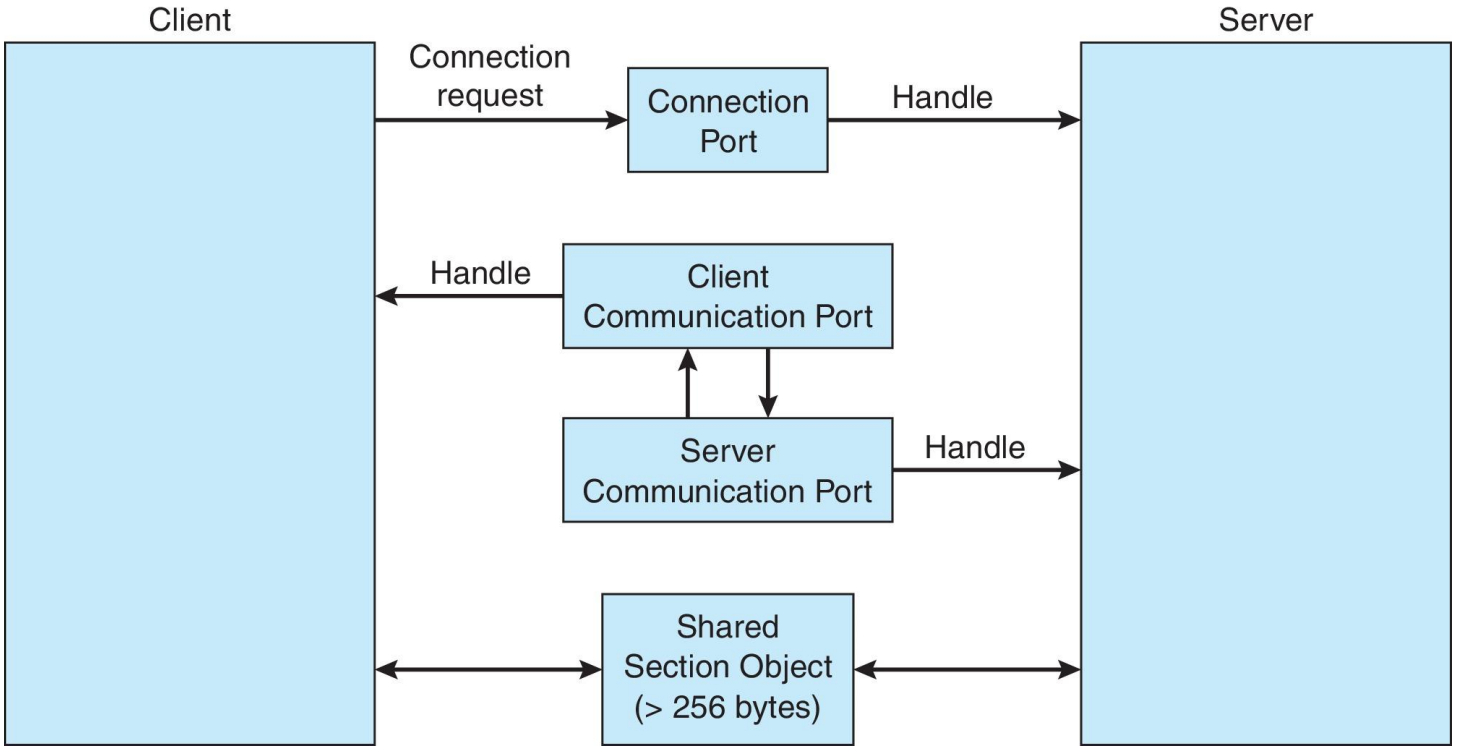
Examples of IPC Systems – Windows

- Message-passing centric via **advanced local procedure call (LPC)** facility
  - Only works between processes on the same system
  - Uses ports (like mailboxes) to establish and maintain communication channels
  - Communication works as follows:
    - The client opens a handle to the subsystem's **connection port** object.
    - The client sends a connection request.
    - The server creates two private **communication ports** and returns the handle to one of them to the client.
    - The client and server use the corresponding port handle to send messages or callbacks and to listen for replies.





Local Procedure Calls in Windows







Pipes

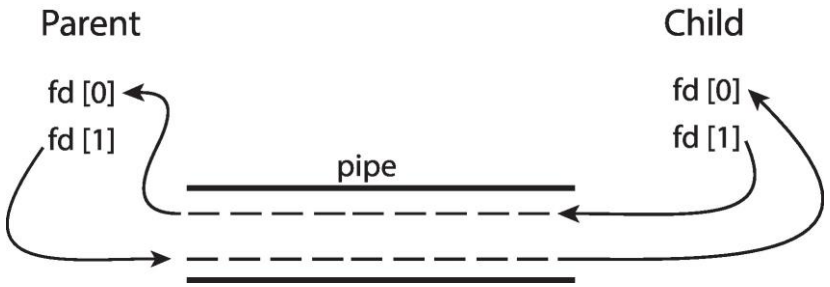
- Acts as a conduit allowing two processes to communicate
- Issues:
  - Is communication unidirectional or bidirectional?
  - In the case of two-way communication, is it half or full-duplex?
  - Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
  - Can the pipes be used over a network?
- **Ordinary pipes** – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- **Named pipes** – can be accessed without a parent-child relationship.



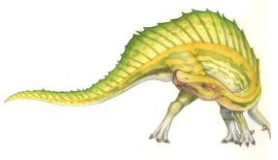


Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



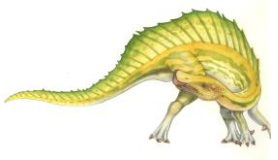
- Windows calls these **anonymous pipes**





Named Pipes

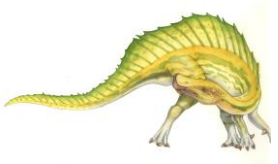
- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems





Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls





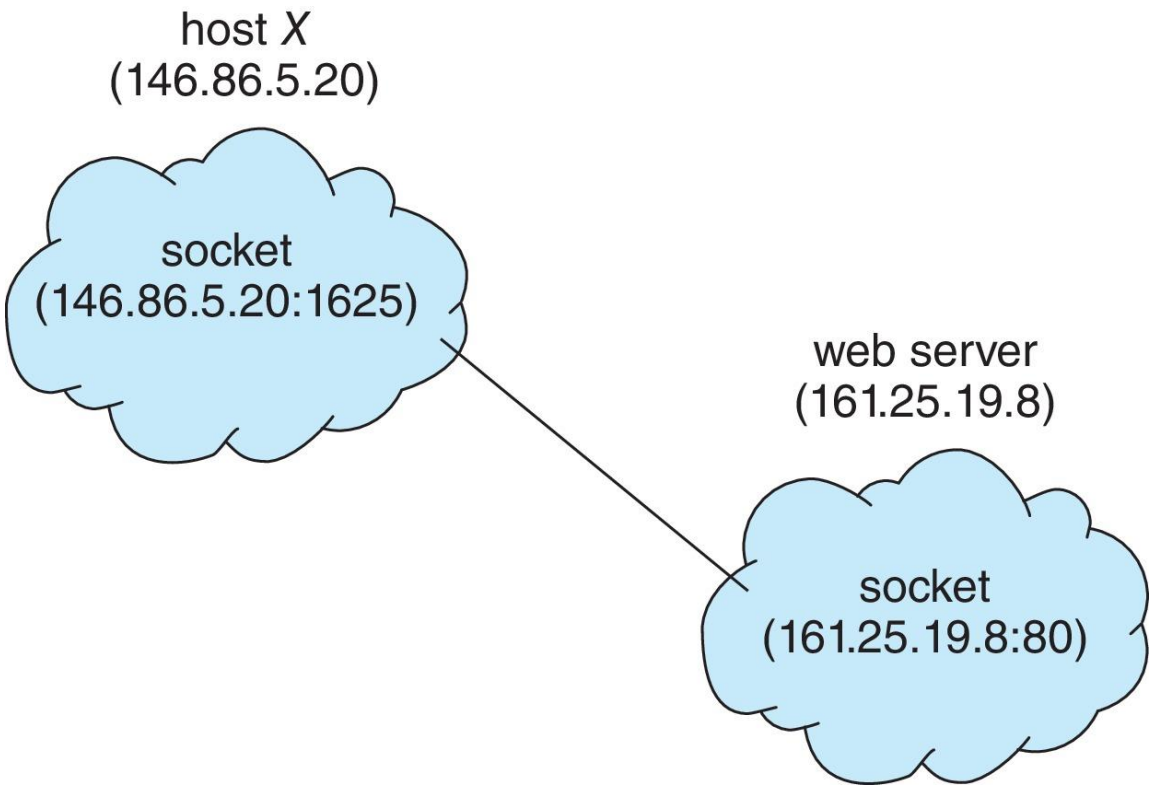
Sockets

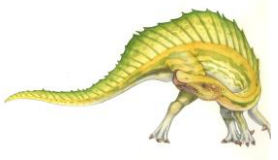
- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are ***well known***, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running





Socket Communication







Sockets in Java

- Three types of sockets
  - **Connection-oriented** (**TCP**)
  - **Connectionless** (**UDP**)
  - **`MulticastSocket`** class– data can be sent to multiple recipients
- Consider this “Date” server in Java:

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         /* now listen for connections */
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            /* write the Date to the socket */
            pout.println(new java.util.Date().toString());

            /* close the socket and resume */
            /* listening for connections */
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```





Sockets in Java

The equivalent Date client

```
import java.net.*;
import java.io.*;

public class DateClient
{
   public static void main(String[] args) {
      try {
         /* make connection to server socket */
         Socket sock = new Socket("127.0.0.1",6013);

         InputStream in = sock.getInputStream();
         BufferedReader bin = new
            BufferedReader(new InputStreamReader(in));

         /* read the date from the socket */
         String line;
         while ( (line = bin.readLine()) != null)
            System.out.println(line);

         /* close the socket connection*/
         sock.close();
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```





Dezavantaje paradigma message passing

- modelul de programare uzual pt sisteme centralizate nu functioneaza pt sisteme distribuite (nu exista reprezentarea resurselor distribuite
  - ideal ar fi ca modelul uzual de programare sa functioneze si pt sisteme distribuite
  - pt sisteme distribuite trebuie modificate programele obisnuite prin luarea in consideratie a comunicatiei (problema de I/O)
- apar dificultati de programare
  - un mesaj e doar un sir de biti, programele opereaza cu tipuri de date complexe
  - e nevoie de modalitati de pastrare a tipurilor de date la transmisia datelor peste retea
  - in plus, arhitecturile de calcul conectate in retea pot fi eterogene, cu reprezentare diferita a tipurilor de date

    ex: intregi reprezentati in format little (Intel) respective big (Sparc) endian

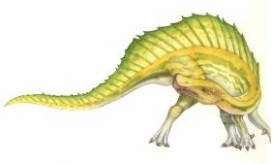




Remote Procedure Calls (RPC)

- Birell & Nelson, 1984
- incearca sa transceada limitelor masinii locale apeland proceduri aflate pe masini la distanta
- avantaj: model de programare centralizat, nu exista diferente intre apelurile locale si cele distante
- natura distribuita a sistemului impiedica totusi o identificare a celor doua tipuri de apeluri
  - procedura apelanta si cea apelata nu se afla in acelasi spatiu de adresa

  => pasarea parametrilor si intoarcerea rezultatului pot fi complicate
  - defectarea (“crash”) oricareia dintre cele doua masini fizice implicate creeaza probleme suplimentare, deopotriva tehnice si de semantica a apelului
- totusi modelul e popular pe arhitecturi NORMA
  - Java RMI, Web services





Functionarea RPC

- programul client se linkediteaza cu asa-numite rutine “stub” care vor fi invocate in locul procedurii propriu-zise aflate pe masina de la distanta
- invocarea stub-ului client determina urmatorul sir de actiuni:
  - (1) stub-ul converteste argumentele procedurii intr-un format independent de masina si le impacheteaza intr-un mesaj (“marshalling”)
  - (2) mesajul este livrat protocolului de transport din kernel (eg, TCP/UDP)
  - (3) protocolul de transport determina transmisia mesajului la distanta catre o rutina stub din server care despacheteaza argumentele procedurii (si identificatorul ei) si le converteste la reprezentarea masinii locale (“unmarshalling”)
  - (4) apoi, stub-ul server apeleaza procedura propriu-zisa



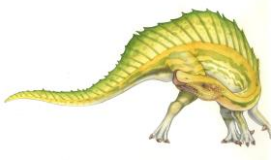


Functionarea RPC (cont.)

- programul client se linkediteaza cu asa-numite rutine “stub” care vor fi invocate in locul procedurii propriu-zise aflate pe masina de la distanta
- invocarea stub-ului client determina urmatorul sir de actiuni:

….

  - (5) rezultatele procedurii sunt supuse procedurii de marshalling intr-un mesaj de raspuns
  - (6) mesajul de raspuns e trimis inapoi clientului
  - (7) stub-ul client despacheteaza mesajul de raspuns si converteste datele la formatul local
  - (8) stub-ul client intoarce valoarea rezultatului RPC procedurii apelante





Transmiterea parametrilor in RPC

- call by value
  - se paseaza valoarea argumentului pe stiva
  - argumentul se poate modifica local in procedura, dar modificarile se pierd la return
  - in principiu, potrivit pt RPC
- call by reference
  - doar un pointer (adresa) catre date e pasat pe stiva
  - modificarile datelor facute prin indirectare supravietuiesc apelului (se vad la nivelul apelantului)
  - nu are sens pt RPC unde clientul si serverul nu partajeaza spatiile de adresa
- copy/restore
  - varianta asimilabila call by value
  - call by value dar la retur se copiaza inapoi valoarea argumentului suprascriindu-se valoarea apelantului
  - folosit in RPC





Reprezentarea datelor

- pana si tipurile de baza pot avea reprezentari diferite pe masini diferite (v. intregi)
- pt a functiona in medii eterogene parametrii se convertesc la o reprezentare standard folosita de stub-uri
  - ex: XDR (eXternal Data Representation), ASN.1 (Abstract Syntax Notation)
- in modelul OSI sunt transformari de nivel de prezentare a datelor (nivelul 6, imediat inainte de nivelul de aplicatie)
  - tot aici se pot opera transformari ale datelor de tip criptare (inclusiv in RPC)
- situatii clasice
  - intregi reprezentati little/big endian
  - string-uri codificate ASCII/EBCDIC/Unicode
- Obs: la intregi transformarea e simpla (se inverseaza bytes), dar la string-uri problema e mai complicata => nevoia de format standard
- optimizare: intre masini cu arhitecturi compatibile, convertirea e inutila, se recurge la notificarea tipului de format in mesaj





Protocolul de transport

- initial era prevazut un protocol dedicat
- UDP
  - fara overhead de setare a conexiunii
  - clientul trebuie sa trateze retransmisia pachetelor, erorile de transmisie, secventierea pachetelor
  - totusi, e protocolul preferat pe LAN unde pierderea de pachete si erorile sunt rare
  - oricum, mai putin performant decat un protocol dedicat (care n-ar face asamblarea/dezasamblarea fragmentelor, diverse checksum-uri, etc)
- TCP – potrivit pt WAN, unde tratarea automata a retransmisiei, erorilor si a secventierii mesajelor asigurata de TCP e un avantaj
- dimensiunea pachetului de retea folosita
  - overhead-ul RPC e fix, indiferent de marimea datelor trimise
  - pt performanta amortizata buna, se prefera transmisia mai multor date intr-un singur mesaj (aici conteaza si mediul de transmisie, v. Ethernet standard vs jumbo frames)





Semantica RPC

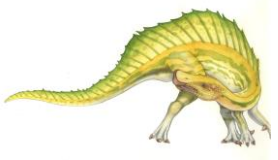
- diferita de semantica apelurilor locale
  - (1) se pot pierde mesaje
  - (2) se pot duplica mesaje
  - (3) serverul si/sau clientul se pot defecta/opri
- (1) & (2) se rezolva prin retransmisie si folosire nr de secventa
- operatii idempotente: efectul executiei multiple este acelasi cu cel al unei singure executii
- ex. operatii idempotente: setarea unui bit, citirea primului bloc de date dintr-un fisier
- ex. operatii care nu sunt idempotente: incrementarea unui intreg, adaugarea datelor la sfarsitul unui fisier (operatia de append)





Semantica RPC (cont.)

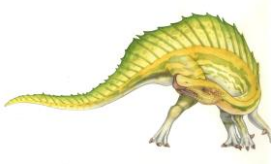
- *exactly once*
  - procedura de la distanta se executa o singura data
  - greu de asigurat
  - mesaje duplicate, rezolvate prin retransmisie si nr de secventa
  - server crash: rezolvare dificila, serverul poate crapa inainte de servirea cererii, dupa servirea ei dar inainte de transmisia unui raspuns sau dupa transmisia unui raspuns care se pierde
- *at most once*
  - procedura remote fie nu se executa deloc fie cel mult o data
  - in absenta unui raspuns de la server nu e clar daca operatia s-a executat sau nu
- *at least once*
  - procedura remote se executa cel putin o data (posibil de mai multe ori)
  - semantica potrivita pentru operatii idempotente
  - clientul retransmite cererea pana cand primeste ACK





Defectarea clientului, solutii

- *exterminare*
  - fiecare apel RPC se inregistreaza pe disc intr-un log
  - dupa reboot, daca vin raspunsuri de la server se ignora (“calculul e exterminat”)
- *reincarnare*
  - foloseste conceptul de epoca
  - la reboot clientul anunta prin broadcast o noua epoca tuturor serverelor, care in consecinta termina calculele aferente tuturor epocilor trecute
  - daca apar totusi raspunsuri din epoci trecute, clientul le ignora
- *reincarnare delicata*
  - varianta in care serverele notificate la inceputul unei noi epoci termina calculul doar daca nu pot identifica proprietarul ei





Defectarea clientului, solutii (cont.)

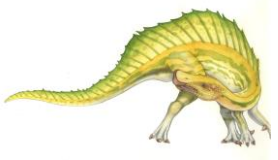
- *expirare*
  - fiecare RPC trebuie sa termine intr-un interval fix de timp T
  - dezavantaj: daca nu se poate termina in timp util, trebuie ceruta o noua perioada
  - Avantaj: daca dupa crash clientul asteapta o perioada T, atunci stie sigur ca nici un calcul nu mai e active
- Obs: nicio solutie nu e satisfacatoare (ex: dc. procedura remote care se termina fortat detine locks, resursele respective protejate de locks raman blocate definitive)





Binding

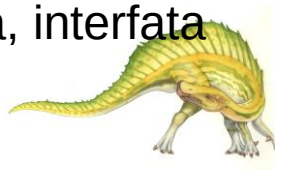
- procesul prin care clientul determina adresa serverului (host si port)
- (1) director de servicii global
  - serverele isi inregistreaza adresele si tipul de serviciu pe care il ofera in director
  - clientii cauta in director adresa unui server care ofera serviciul dorit
- (2) clientul stie adresa serverului
  - serverul ruleaza propriul director de servicii in care programele server isi inregistreaza serviciile
  - ex: SUN portmapper
- clientii folosesc porturi publice (“well-kown ports”) pentru a adresa cereri directorului de servicii





Suport RPC la nivelul limbajului

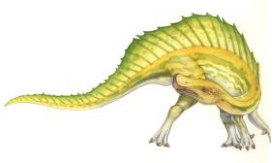
- RPC e o constructie de nivelul limbajelor de programare
- compilatorul trebuie informat ca procedura se va executa la distanta si ca apelurile se fac prin stub
- avand in vedere diferenta de semantica fata de apelurile obisnuite, se prefera ca apelurile RPC sa fie marcate explicit la nivel de limbaj prin keywords de tipul “remoteproc”
- solutia alternativa: Interface Definition Language
  - in loc sa se extinda compilatorul cu suport RPC, se defineste punctul de interactiune intre client si server ca fiind o *interfata*
  - interfata e definite intr-un limbaj special de definire a interfetei IDL
  - specificarea IDL a unei interfete se compileaza cu un compilator special care genereaza stub-uri client si server (ex: Sun rpcgen)
  - diferite compilatoare IDL pot fi folosite pt a genera strub-uri pt limbaje si conventii de apelare diferite
  - alternativ, exista limbaje care suporta nativ conceptul de interfata, interfata RPC nefiind decat un caz particular (ex: Java RMI)





Exemplu de implementare RPC

- Sun RPC are propriul IDL si un compilator specific (*rpcgen*) numit si “generator de stub-uri”
- fiecare masina server ruleaza un director de servicii numit *portmapper*
  - mapeaza identificatorul programului server pe un port de comunicatie
  - programul server e identificat prin nr de program si versiune
- exemplu de specificare a interfetei pt un serviciu simplu de fisiere
  - Interfata suporta operatia SFS_READ (operatia cu nr 1) care primeste un argument de tip *readargs* si intoarce un rezultat de tip *readres*
  - programul server are numarul 123321 si versiunea 9876
- interfata e definita in fisierul sfs.x iar stub-urile se genereaza cu comanda “rpcgen sfs.x”:
  - sfs_client.c – stub client
  - sfs_server.c – stub server
  - sfs_xdr.c – rutinele de marshalling/unmarshalling
  - sfs.h – interfata in limbaj C





Interfata IDL server de fisiere

```
 1 const SFS_MAXDATA = 4096;
 2
 3 struct readres {
 4   int count;
 5   opaque data<SFS_MAXDATA>;
 6 };
 7
 8 struct readargs {
 9   string file<SFS_MAXDATA>;
10   unsigned offset;
11   unsigned count;
12 };
13
14 program SFS_PROGRAM {
15   version SFS_VERSION {
16     readres SFS_READ(readargs) = 1;
17   } = 9876;
18 } = 123321;
```

Figure 2: Simple File Server interface definition file `sfs.x`.

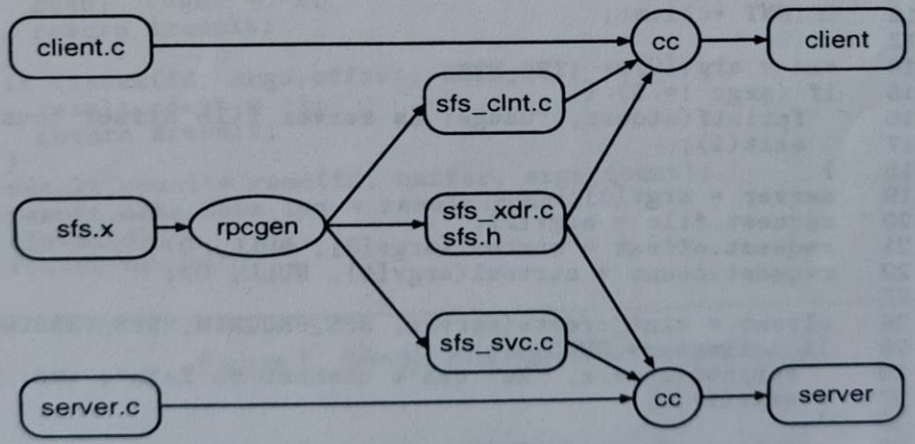


Figure 3: `rpcgen` operation.





Codul RPC client

- sfs_client.c
  - construieste o cerere
  - creaza un handle folosit pt. a executa RPC-uri pe server
  - *clnt_create* contacteaza portmapper-ul de pe server, cere portul corespuzatori perechii (SFS_PROGRAM,SFS_VERSION) si creeaza un socket UDP pt comunicare cu serverul
  - apelul RPC = apelul *sfs_read_9786*
    - foloseste o structura pt ca initial rpcgen-ul nu accepta decat proceduri cu un singur argument
    - versiunile moderne accepta mai multi parametri





Cod RPC client

```
 1 #include <rpc.h>
 2 #include "sfs.h"
 3
 4 char *cmd;
 5
 6 int
 7 main(int argc, char *argv[])
 8 {
 9   struct readargs request;
10   struct readres *result;
11   char *server;
12   CLIENT *client;
13
14   cmd = argv[0];
15   if (argc != 5) {
16     fprintf(stderr, "usage: %s server file offset count\n", cmd);
17     exit(1);
18   }
19   server = argv[1];
20   request.file = argv[2];
21   request.offset = strtoul(argv[3], NULL, 0);
22   request.count = strtoul(argv[4], NULL, 0);
23
24   client = clnt_create(server, SFS_PROGRAM, SFS_VERSION, "udp");
25   if (client == NULL) {
26     fprintf(stderr, "%s: can't connect to %s\n", cmd, server);
27     exit(1);
28   }
29   result = sfs_read_9876(request, client);
30   if (result == NULL) {
31     fprintf(stderr, "%s: read failed\n", cmd);
32     exit(1);
33   }
34   write(1, result->data.data_val, result->data.data_len);
35   exit(0);
36 }
```

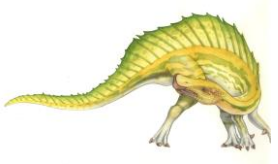
Figure 4: Simple File Server client code.





Codul RPC server

- sfs_server.c
  - apeleaza procedura *sfs_read_9786_svc*
  - aceasta deschide fisierul solicitat
  - muta file pointerul la offsetul solicitat
  - citeste nr de octeti solicitat
  - returneaza rezultatul





Cod RPC server

```
 1 #include <stdio.h>
 2 #include <fcntl.h>
 3 #include <rpc.h>
 4 #include "sfs.h"
 5
 6 readres *
 7 sfs_read_9876_svc(readargs args, struct svc_req *request)
 8 {
 9   static readres result;
10   static char buffer[SFS_MAXDATA];
11   int count;
12   int fd;
13
14   result.count = 0;
15   result.data.data_len = 0;
16   result.data.data_val = buffer;
17
18   if ((fd = open(args.file, O_RDONLY)) == -1) {
19     result.count = -1;
20     return &result;
21   }
22   if (lseek(fd, args.offset, SEEK_SET) == -1) {
23     result.count = -1;
24     return &result;
25   }
26   result.count = read(fd, buffer, args.count);
27   result.data.data_len = result.count;
28   close(fd);
29   return &result;
30 }
```

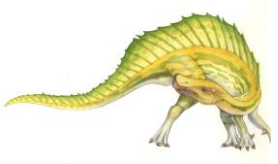
Figure 5: Simple File Server server code.





Remote Procedure Calls

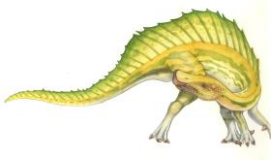
- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems
  - Again uses ports for service differentiation
- **Stubs** – client-side proxy for the actual procedure on the server
- The client-side stub locates the server and **marshalls** the parameters
- The server-side stub receives this message, unpacks the marshalled parameters, and performs the procedure on the server
- On Windows, stub code compile from specification written in **Microsoft Interface Definition Language** (**MIDL**)





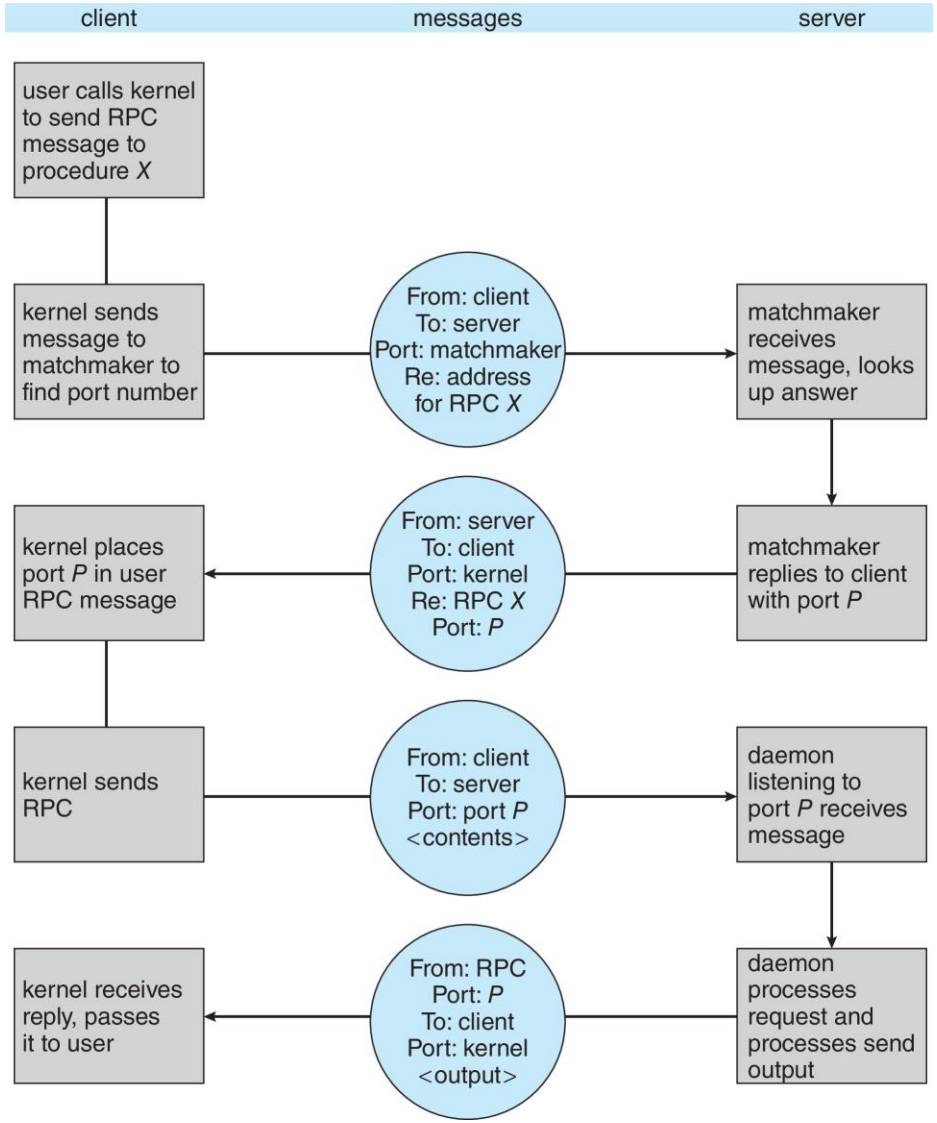
Remote Procedure Calls (Cont.)

- Data representation handled via **External Data Representation** (**XDL**) format to account for different architectures
  - **Big-endian** and **little-endian**
- Remote communication has more failure scenarios than local
  - Messages can be delivered ***exactly once*** rather than ***at most once***
- OS typically provides a rendezvous (or **matchmaker**) service to connect client and server



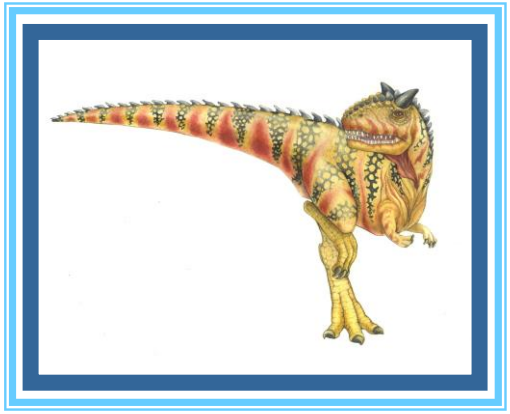


Execution of RPC



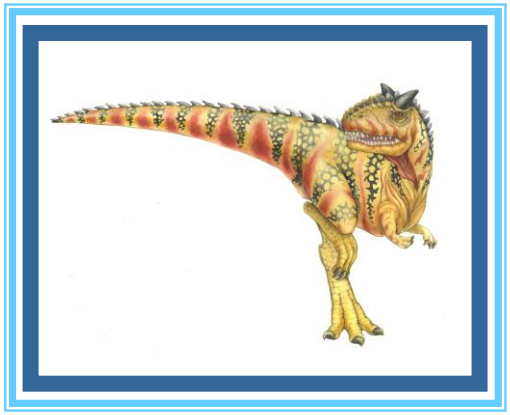


End of Chapter 3



Silberschatz, Galvin and Gagne ©2018

Chapter 4: Threads & Concurrency



Silberschatz, Galvin and Gagne ©2018



Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples





Objectives

- Identify the basic components of a thread, and contrast threads and processes
- Describe the benefits and challenges of designing multithreaded applications
- Illustrate different approaches to implicit threading including thread pools, fork-join, and Grand Central Dispatch
- Describe how the Windows and Linux operating systems represent threads
- Designing multithreaded applications using the Pthreads, Java, and Windows threading APIs





Motivation

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded





Aplicatii concurente

- proces = un singur punct de executie in aplicatie (o singura instanta in rulare a aplicatiei)
- unele aplicatii pot profita de existenta mai multor puncte de executie simultana in cadrul aplicatiei (mai ales pe multiprocesoare)
- ex. 1: procesor cu 4 core-uri + aplicatie de filtrare de imagini care imparte imaginea in 4 cadrane, fiecare core filtrand un cadran
  - avantaj: descompunerea activitarilor mari in activitati mai simple care ruleaza simultan => reducerea timpului de rulare
- ex. 2: un singur procesor + program de gestiune a ferestrelor in GUI
  - verifica si proceseaza input-ul de la tastatura, mouse si retea pt. a produce output pe ecran
  - activitati concurente: verificare/procesare input tastatura, mouse si , retea, afisare bitmap-uri pe ecran
  - toate aceste activitati sunt codate in module separate, cu date private si comunica intre ele prin date partajate





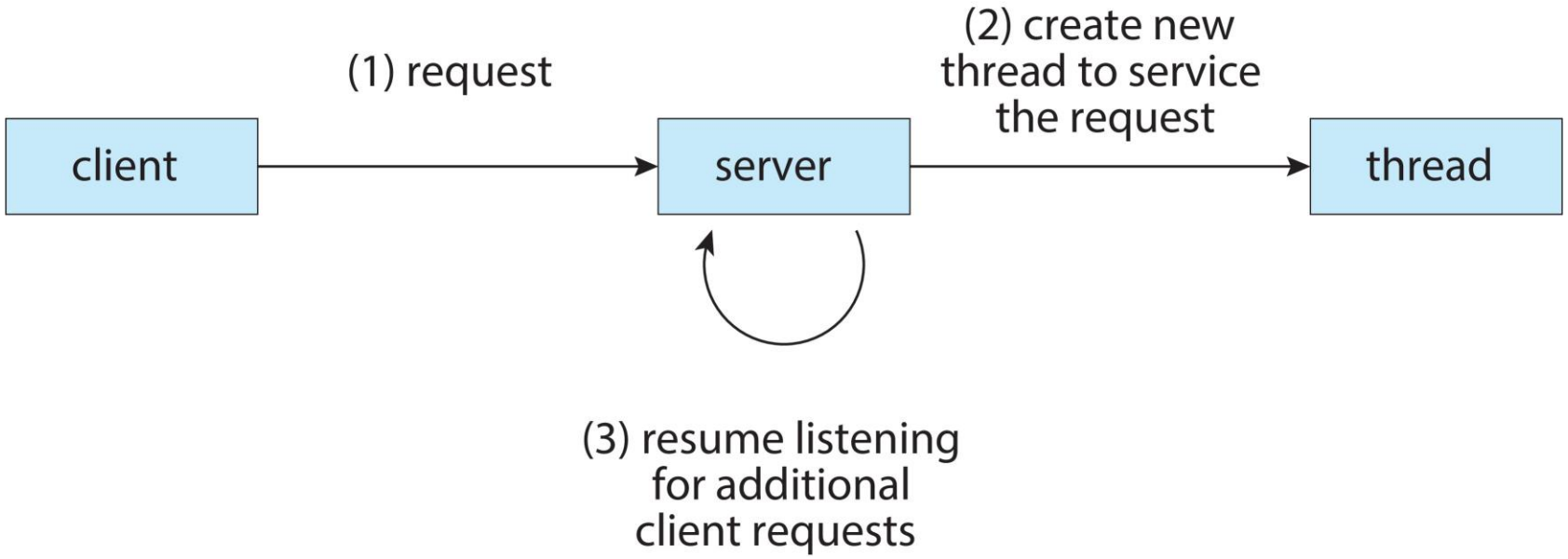
Aplicatii concurente (cont.)

- ex. 3: server de retea
  - activitatea principala: asteapta cereri de la client, le proceseaza si trimite inapoi raspunsurile
  - cu un singur punct de executie procesarea unei cereri particulare ia mult timp (de pilda, in asteptarea datelor de pe disc) => se intarzie foarte mult tratarea altor cereri (situatie similara cu cea care a condus la nevoia de multitasking)
  - solutie: fiecare cerere client e procesata in alt punct de executie al aplicatiei server





Multithreaded Server Architecture







Procese si date partajate

- crearea de puncte multiple de executie in program se poate face cu procese care partajeaza date (sa zicem prin memorie partajata, cu acces protejat cu semafoare/locks)
  - procesele au date private (datorita protectiei MMU a spatiului de adresa)
  - comunica intre ele prin IPC (memorie partajata)
  - daca exista capacitare de multiprocesare, procesele pot rula simultan pe mai multe procesoare
- puncte nevralgice
  - crearea proceselor
  - context-switch-ul
  - IPC-ul

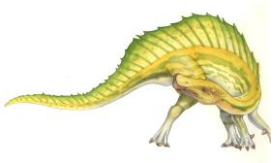




Crearea proceselor

- contextul unui proces e stufos (stocat in PCB in kernel)
- include starea completa a CPU, registre de gestiunea memoriei, tabele de pagini de memorie, descriptori de fisiere, actiuni asociate semnalelor, etc

  => cost semnificativ de creare a unui proces
- daca acest cost e prea mare, aplicatiile pot sa nu foloseasca eficient procesoarele:
  - ex aplicatie de filtrare imagini
  - daca timpul de creare a unui proces e comparabil cu timpul de filtrare al unui cadran, e posibil ca filtrarea intregii imagini cu 4 procese sa dureze mai mult decat filtrarea ei cu un singur proces (“slowdown”)





Context-switch

- salvarea contextul unui proces si incarcarea contextului unui nou proces poate implica un cost semnificativ avand in vedere bogatia de informatii din PCB
- ex: secventa de evenimente pt. producator-consumator cu zero buffering (“rendez-vous”)
  - producatorul produce un element si se blocheaza
  - sistemul face context-switch si aduce consumatorul pe procesor
  - consumatorul consuma elementul si se blocheaza
  - sistemul face context-switch si aduce producatorul inapoi pe procesor pt a produce un nou element
- daca timpul de context-switch e mare, viteza de transfer a datelor intre producator si consumator e serios afectata (fiecare transfer implica doua context-switch-uri); in orice caz, e imposibil sa se atinga viteza teoretica maxima de transfer





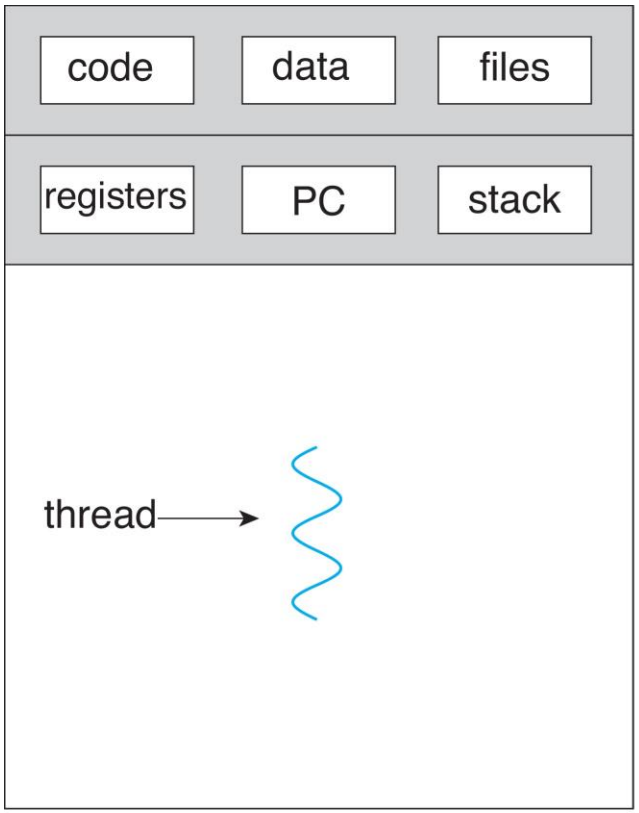
Fire de executie (threads of execution)

- modalitate de a reduce costurile crearii punctelor de executie in aplicatie si a schimbarii contextelor de executie intre ele
- idee: puncte de executie multiple din aplicatie partajeaza o parte din contextul programului (registrele de gestiune a memoriei, tabelele de pagini, descriptorii de fisiere deschise, samd)
- DAR, fiecare punct de executie are:
  - o copie individuala a unui subset al contextului de executie a aplicatiei (de ex. contextul CPU)
  - structuri de date necesare punctului de executie (de ex. stiva)
- apare o noua abstractie de nivel inalt, *firul de executie (thread)* = un punct de executie cu context redus in cadrul programului
  - referit uneori din cauza acestei viziuni ca fiind un “proces usor” (lightweight process, LWP)

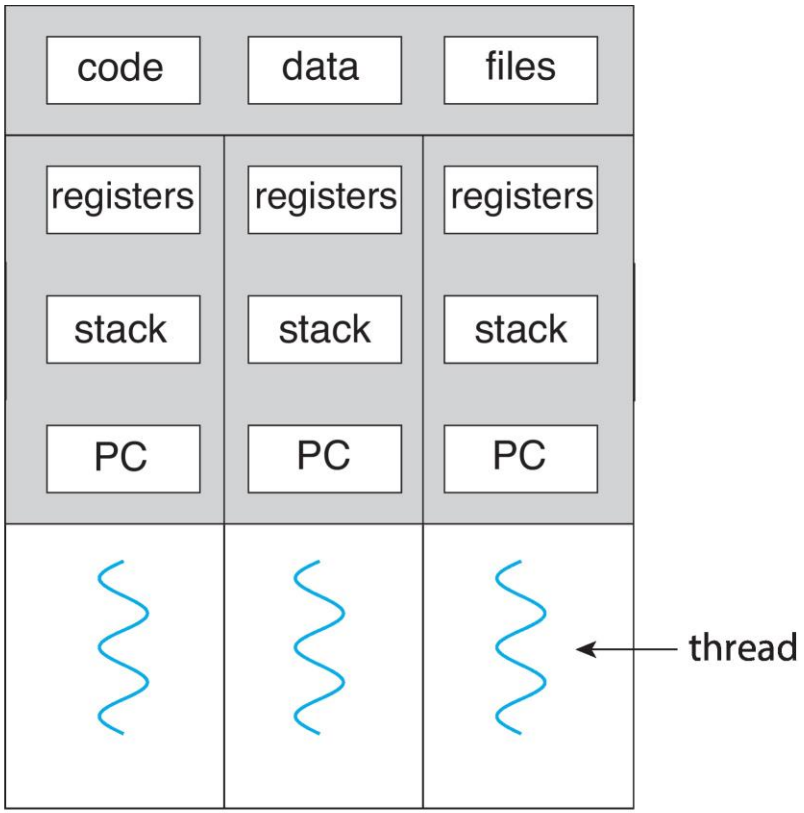




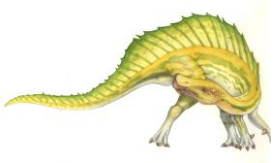
Single and Multithreaded Processes



single-threaded process



multithreaded process





Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multicore architectures





Caracteristici threaduri

- ruleaza secvential, au program counter si stiva propria
- multiplexeaza accesul la CPU ca si procesele (pe multiprocesoare ruleaza in paralel)
- pot crea alte threaduri
- pot executa apeluri de sistem
  - daca un thread se blocheaza intr-un apel sistem, alt thread primeste procesorul
- analogie posibila
  - threadul este fata de proces ceea ce procesul e in raport cu procesorul
  - procesul actioneaza ca un procesor virtual pe care ruleaza thread-ul





Diferente fata de procese

- threadurile aceluiasi proces partajeaza spatiul de adresa al procesului (de ex, partajeaza variabilele globale) => un thread poate distruge usor alt thread (nu exista protectia MMU ca in cazul proceselor diferite)
- lipsa protectiei intre threaduri
  - e inevitabila prin design
  - nu e necesara (threadurile sunt parte a aceluiasi program al unui anumit utilizator)
  - nici nu e de dorit (impunerea unor domenii de protectie conduce la probleme similare proceselor)
- alte resurse partajate: acelasi set de fisiere deschise, timere, semnale, etc





Alte caracteristici ale threadurilor

- stari (la fel ca la procese): in rulare, gata de rulare, blocat, terminat
- modele de utilizare
  - cooperativ, lucru in echipa (ex filtrare de imagini)
  - master-slave/worker (ex server)
  - pipeline (ex producator-consumator)
- avantaj principal: datele partajate sunt datele globale din proces (nu e nevoie de setarea unor mecanisme IPC de tipul memoriei partajate)
  - buffer global pt. producator-consumator
  - argument puternic pt sistemele multiprocesor unde threadurile pot rula pe CPU-uri diferie => partajarea implicita a datelor prin spatiul comun de adresa (nu e nevoie de mecanisme speciale de partajare ca in cazul proceselor, eg memorie partajata IPC)





Design-ul pachetelor de threaduri

- pachet de threaduri = colectie de primitive (apeluri de biblioteca) pt lucrul cu thread-uri
- (1) gestiunea threadurilor
  - *creare thread*: primeste ca argumente functia care reprezinta punctul de executie initial, o stiva privata si o prioritate de planificare pe procesor; intoarce un TID (Thread ID)
  - *terminare thread*: explicit prin apel *exit* sau semnal de tip *kill* de la alt thread/proces
  - primitive pt. mecanisme de sincronizare, necesare datorita existentei datelor partajate (uzual mutex-uri, dar in mod notabil si variabile de conditie folosite in conexiune cu un mutex)
  - ex: *acquire/release* resource folosind mutex+condition variable
- (2) planificare (aceeasi algoritmi ca si la procese, vom discuta la planificarea proceselor/threadurilor)





Design-ul pachetelor de threaduri (cont.)

- (3) probleme de reentranta
  - scenariu: doua threaduri T1 si T2 executa concurent apeluri sistem, T1 reuseste, T2 esueaza
  - daca T1 nu evalueaza *errno* inainte ca T2 sa execute apelul sistem care esueaza, T1 va crede eronat ca apelul sau sistem a esuat
  - problema principiala: *errno* e variabila globala
  - solutii posibile: (a) protejarea *errno* cu mutex-uri
- (b) crearea unei copii private a lui *errno* prin intermediul unor variabile globale thread-ului apelant, dar private (invizibile) celorlalte thread-uri => TLS (Thread-Local Storage)





Implementarea threadurilor kernel

- pachetele de threaduri se pot implementa in kernel sau in spatiul utilizator
- threadurile kernel separa campurile din PCB care ajuta la crearea unui punct de executie si le stocheaza intr-un TCB
- astfel, un proces cu un singur punct de executie e reprezentat in kernel de un PCB si un TCB
- operatia de creare a unui thread (*thread_create*) este un apel sistem
  - aloca un TCB
  - aloca stive kernel si user
  - le leaga la PCB-ul procesului in care s-a facut apelul sistem



Exemplu cu doua threaduri kernel

- un thread suspendat in kernel
- altul ruland in spatiul utilizator
- pt ca sunt implementate in kernel si seamana cu procesele, threadurile kernel se mai cheama si *procese usoare* (*lightweight processes, LWP*)

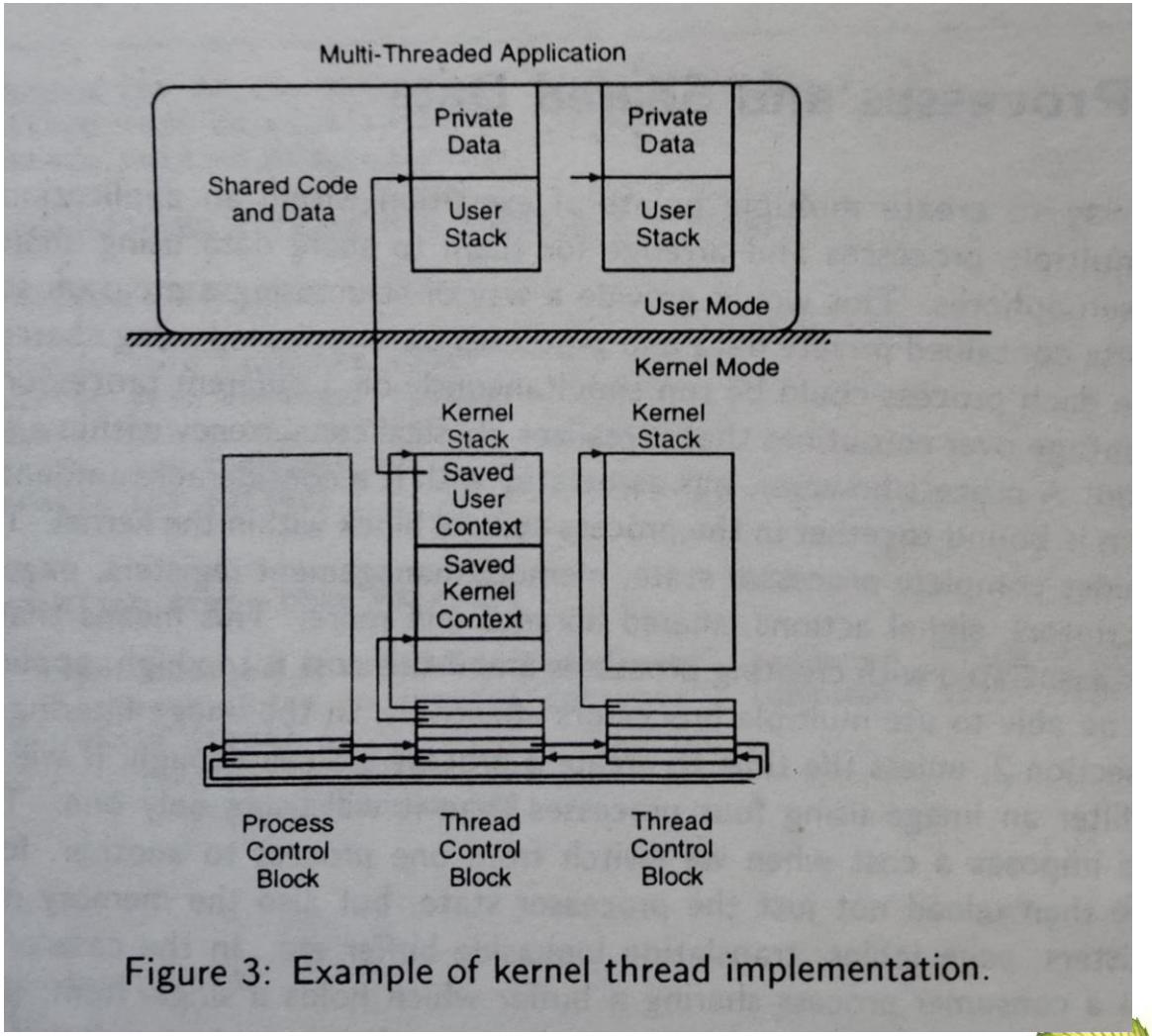


Figure 3: Example of kernel thread implementation.



Costuri threaduri kernel

- costul crearii unui thread kernel << costul crearii unui proces
  - se aloca si initializeaza doar TCB-ul si stivele
  - restul contextului exista deja creat in PCB
- context switch-ul intre doua threaduri **ale aceluiasi proces** dureaza << schimbarea contextului a doua procese, pt ca nu trebuie schimbat restul contextului din PCB
- context switch-ul intre doua threaduri din procese **diferite** are acelasi cost ca si context switch-ul de procese (nu se schimba doar TCB-urile, ci si PCB-urile)



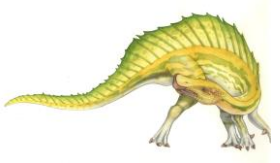


Planificarea kernel threadurilor kernel

- threadurile ruleaza asincron unele fata de celelalte si pot pierde procesorul la fel ca si procesele

=> accesul la datele partajate trebuie sincronizat cand se doreste IPC

- planificatorul kernel alege urmatorul thread care trebuie sa ruleze => daca aplicatia are propria politica de planificare (de ex bazata pe prioritati) trebuie sa o comunice intr-un fel sau altul kernelului
- in cazul multiprocesoarelor, planificatorul poate asigna mai multe CPU-uri unui singur proces pt. ca threadurile sa ruleze in paralel (*gang scheduling*)
- daca un thread se blocheaza in kernel (de ex prin apel de sistem I/O) si cuanta de timp alocata procesului nu a expirat, planificatorul cauta in lista de TCB-uri un thread gata de rulare **din acelasi proces** si ii acorda procesorul





Protectia threadurilor kernel

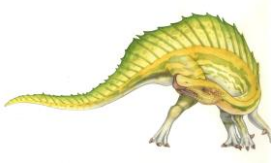
- observatia generala despre threaduri e valabila si pt threaduri kernel
- un thread poate corupe stiva altui thread, de pilda => se distrug datele private ale altui thread (variabilele automatice/locale)
- solutie: implementarea stivelor in spatii de adresa diferite, dar asta mareste costul context switch-ului
- mai exact, ar fi nevoie de salvarea si reincarcarea contextelor de executie referitoare la gestiunea memoriei, pe langa contextul uzual din TCB





Dezavantajele threadurilor kernel

- desi mai putin costisitoare ca procesele, anumite aspecte le fac nepotrivite pt utilizatori
- (1) *thread_create* este apel sistem => costisitor pt procese care creeaza multe threaduri
- (2) context switching-ul de threaduri necesita intrarea si iesirea in/din kernel mode => overhead aditional context switching-ului obisnuit
- (3) implementarea in kernel e **inflexibila**
  - impune un model de threaduri care nu e potrivit pt orice aplicatie
  - codul planificatorului (scheduler) nu e accesibil (fiind in kernel) => greu de adaptat pt cerintele specifice ale unei anumite aplicatii (politica de planificare nu se poate schimba usor)





User-level threads

- daca planificatorul si TCB-urile sunt implementate in spatiul utilizator costurile scad pentru ca nu mai e nevoie de transgresarea granitei kernel/user
- comparatie calitativa, intr-un ex in care un apel de procedura cost 7 usec, iar un apel sistem (trap) 19 usec

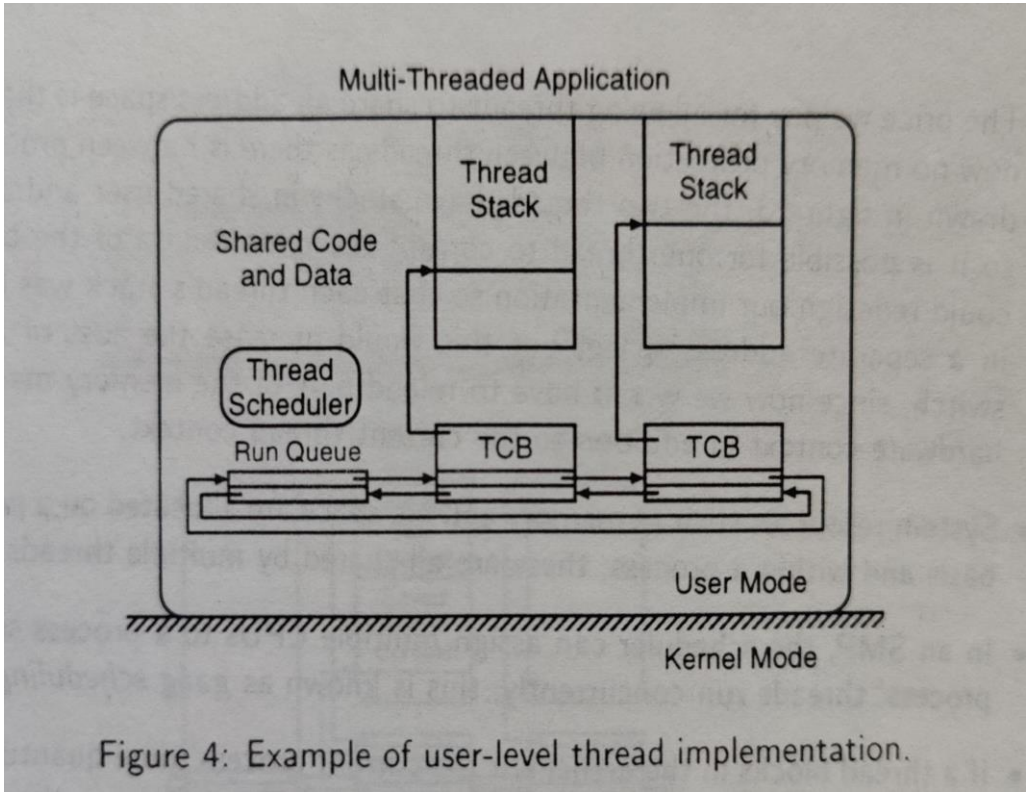


Figure 4: Example of user-level thread implementation.

Operatie	Thread user	Thread kernel	Proces Unix
fork	34 usec	948 usec	11300 usec
IPC synch	37 usec	441 usec	1840 usec





Caracteristici threaduri utilizator

- nu apeleaza serviciile kernel pt creare si context switch => aceste operatii sunt f. rapide (nu se trece granita user-kernel si nu e nevoie de verificarea parametrilor apelului sistem de catre kernel)
- aplicatiile pot furniza propriul planificator (*thread scheduler*) customizat cf unor cerinte specifice
- nu necesita nici un fel de suport explicit din partea kernelului; procesul e privit ca un procesor virtual pt. threaduri
- fiind mult mai rapide decat threadurile kernel, de regula threadurile user se implementeaza deasupra threadurilor kernel

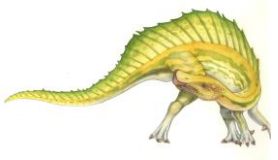
  => planificatorul de threaduri user trateaza threadurile kernel ca pe procesoare virtuale si multiplexeaza mai multe threaduri user pe unul sau mai multe threaduri kernel





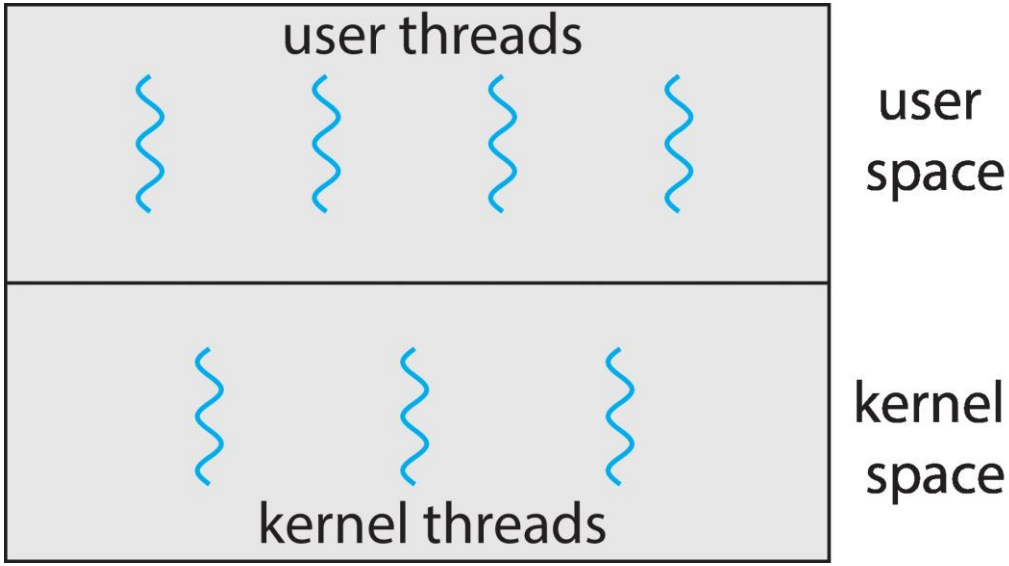
User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general-purpose operating systems, including:
  - Windows
  - Linux
  - Mac OS X
  - iOS
  - Android





User and Kernel Threads

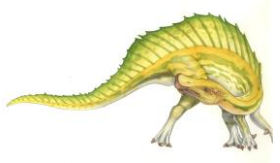






Multithreading Models

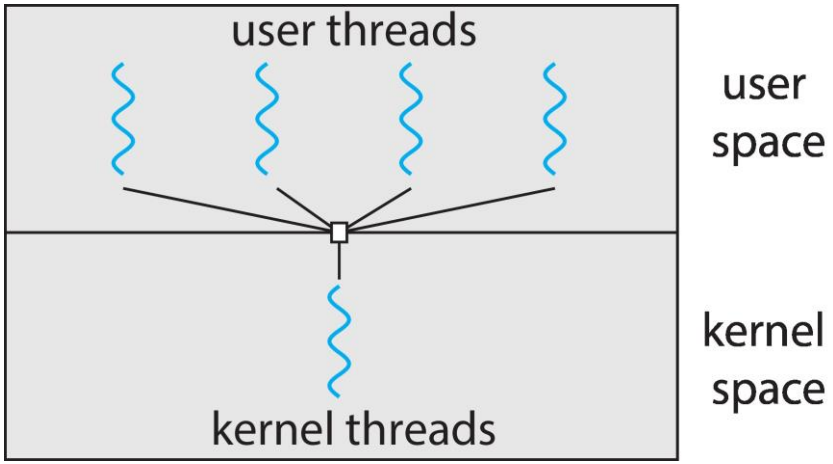
- Many-to-One
- One-to-One
- Many-to-Many





Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**

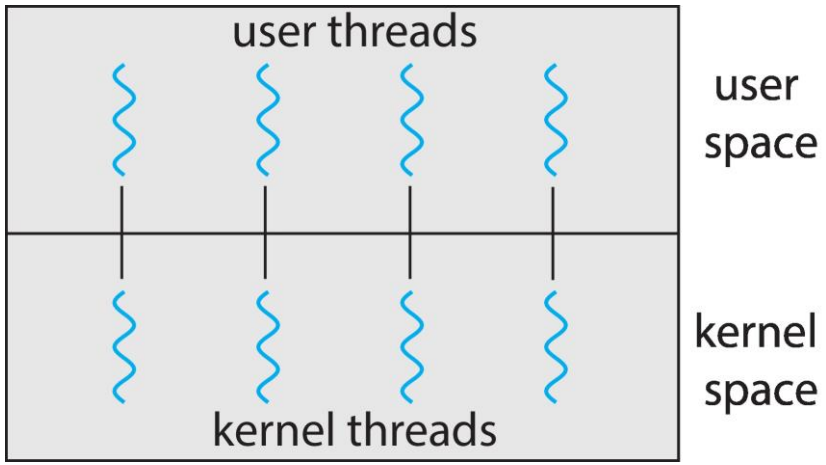






One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux

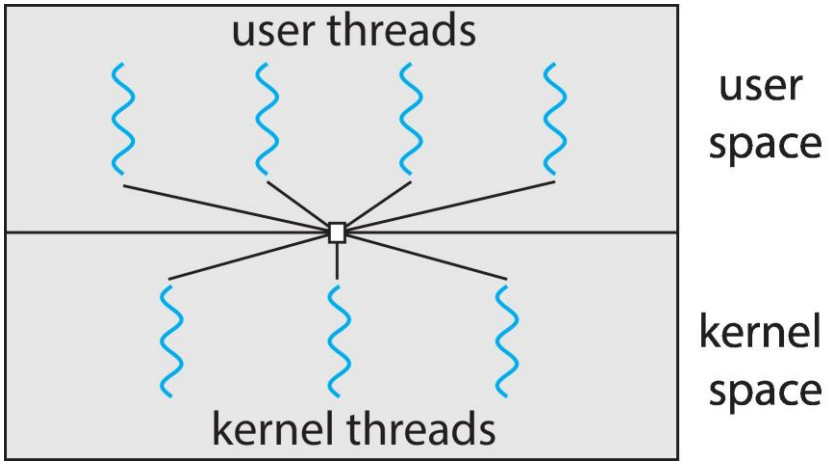


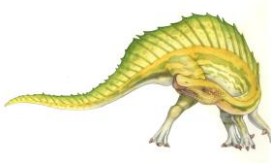




Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Windows with the *ThreadFiber* package
- Otherwise not very common

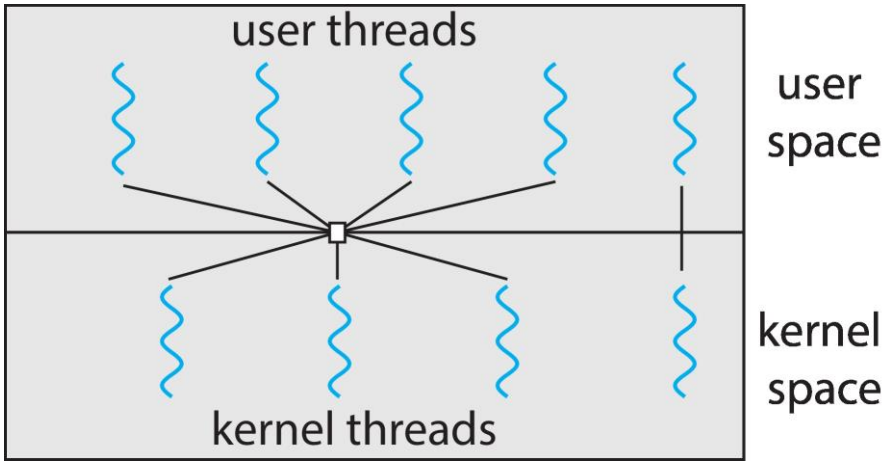






Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread







Probleme comune threadurilor k. si u.

- reentranta: multe apeluri de biblioteca sunt ne-reentrante (apelurile reentrante sunt desemnate in paginile de manual ca fiind “thread safe”)
  - ex: trimiterea unui mesaj in retea in 2 pasi: (a) asamblare mesaj in buffer + (b) transmisie (*syscall*)
  - pierderea procesorului intre (a) si (b) poate conduce la suprascrierea bufferului
  - alte ex: errno, malloc, apeluri stdio
- tratarea semnalelor
  - ex: un thread trateaza un semnal in vreme ce alt thread vrea ca semnalul respectiv sa termine aplicatia (procesul, mai exact)
  - se poate intampla daca se folosesc apeluri de biblioteca si runtime user impreuna
  - problema deriva din faptul ca semnalele sunt definite per proces si nu per thread





Dezavantaje threaduri utilizator

- determinate de faptul ca existenta lor e necunoscuta kernelului
- (1) thread user executa apel sistem blocant in kernel, planificatorul kernel (agnostic cu privire la threadurile user) considera intreg procesul blocat si aloca CPU altui proces (sau kernel thread), *chiar daca procesul curent mai are si alte threaduri user gata de rulare si nu si-a consumat toata cuanta de timp CPU !*
- (2) thread user comite *page fault* (acces la pagina de memorie inexistenta/nealocata) => acelasi efect ca mai sus, planificatorul alege alt procesor/kernel thread in vreme ce pagina de memorie e adusa de pe disc => aplicatia ruleaza cu mai putine CPU decat e necesar
- (3) nefiind constient de existenta threadurilor user, kernelul poate lua procesorul unui thread user care detine un spinlock pe care nu l-a eliberat => scadere dramatica de performanta pt aplicatiile care folosesc threaduri user in paralel
  - efectul e si mai dramatic daca schedulerul alege sa ruleze alte threaduri care vor sa obtina acelasi spinlock
- problema de fond: lipsa de coordonare intre schedulerul si implementarea pachetului de threaduri user





Scheduler activations

- metoda de coordonare kernel – pachet de threaduri utilizator
- model: aplicatia ruleaza pe un multiprocesor virtual
  - exista apeluri sistem pt a suplimenta/diminua nr de procesoare virtuale alocate de kernel (“adauga CPU”, “acest CPU este idle”)
  - kernelul decide daca onoreaza cererea sau nu
- scheduler activation e aproximarea unui kernel thread
  - are stive kernel si user
  - ofera context de executie pt un thread user
  - in plus, ofera conceptul de *upcall* pt evenimente de mai multe tipuri
  - fiecare upcall creeaza o noua activare (optimizare: folosirea vechilor activari)





Tipuri de evenimente upcall

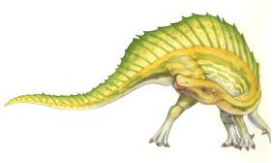
- *adauga procesor* – consecinta este executia unui thread user
- *procesor preemptat* – adauga threadul user care se executa in activarea care a pierdut CPU in coada de threaduri gata de rulare
- *activare blocata* – activarea s-a blocat (eg, apel sistem blocant) si nu mai utilizeaza procesorul
- *activare deblocata* – pune in lista gata de rulare threadul care se executa in contextul activarii blocate





Functionarea scheduler activations

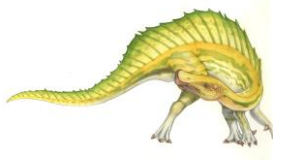
- la pornirea procesului
  - kernelul aloca o activare + notifica aplicatia (*upcall* “adauga CPU”) dupa ce i-a asigurat un CPU
  - sistemul de gestiune al threadurilor user primeste notificarea si foloseste noua activare drept context de executie pt initializarea sa si a threadului *main* (ulterior se pot crea noi threaduri si cere noi procesoare)
- la cerere de suplimentare a concurentei (la adaugarea CPU sau pt. ca un thread se blocheaza)
  - kernelul salveaza starea threadului user in activarea curenta
  - aloca o noua activare si cheama aplicatia in contextul noii activari





Functionarea scheduler activations (cont)

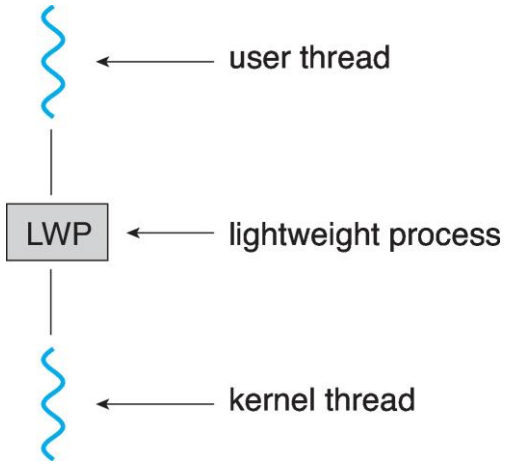
- la blocarea unui thread user in kernel
  - nu se continua in kernel la deblocare
  - se creeaza o noua activare
  - se notifica aplicatia
  - schedulerul user copiaza starea threadului blocat din vechea activare si notifica kernelul ca vechea activare poate fi refolosita
- cand un thread user care detine un spinlock pierde procesorul
  - kernelul genereaza un *upcall*
  - sistemul de gestiune a threadurilor verifica daca threadul e in sectiune critica
  - in caz afirmativ, threadul este continuat temporar printr-un context switch in user space
  - la iesirea din sectiunea critica, se da controlul inapoi *upcall-ului* tot prin context switch in user space





Scheduler Activations

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Typically use an intermediate data structure between user and kernel threads – **lightweight process** (**LWP**)
  - Appears to be a virtual processor on which process can schedule user thread to run
  - Each LWP attached to kernel thread
  - How many LWPs to create?
- Scheduler activations provide **upcalls** - a communication mechanism from the kernel to the **upcall handler** in the thread library
- This communication allows an application to maintain the correct number kernel threads







Thread Libraries

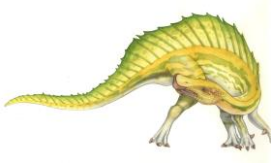
- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
  - Library entirely in user space
  - Kernel-level library supported by the OS





Exemple de pachete de threaduri user

- Solaris si POSIX
- threaduri Solaris
  - exista si la nivel de kernel (s.n. LWPs, sunt multiplexate pe CPU existente) si la nivel de utilizator
  - pachetul de threaduri (*libthread*) planifica threadurile user pe o colectie de LWPs
  - uzual, threadurile user sunt create nelegate (*unbound*), adica se pot muta intre LWP-uri si asa se si folosesc in general
  - daca un thread user e asignat unui LWP se spune ca e legat (*bound*)
  - controlul kernelului asupra threadurilor user se exercita prin bound threads cu ajutorul setarii prioritatii LWP-ului asociat (de ex, thread user pt stream audio cu prioritate mare in aplicatie multimedia)





Exemple de pachete de threaduri user

- threaduri Solaris
- ….
  - biblioteca de threaduri asigura ca exista suficiente LWP-uri active pt ca procesul sa poata continua
  - daca un proces determina toate LWP-urile sale sa se blocheze, biblioteca va crea LWP-uri aditionale pt. ca threadurile neblocate sa poata rula
  - daca LWP-urile sunt idle prea mult timp, biblioteca le da inapoi kernelului pt a fi folosite de alte aplicatii

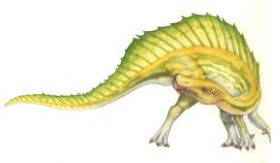




Exemple de pachete de threaduri user

- exista apel de setare explicita a nr de LWP-uri la un nou nivel

  thr_setconcurrency(int new_level)
- mecanisme IPC: mutex si variabile conditie
- implementeaza TLS (Thread-Local Storage), capacitate privata de stocare a variabilelor in afara stivei
  - variabile locale nepartajate, “statice” (globale pt threadul respectiv)
  - declarate cu #pragma, de ex #pragma unshared errno;
- threaduri POSIX (pthreads)
  - standard IEEE, Portable Operating System Interface (e o specificatie, nu o implementare)
  - asemanatoare threadurilor Solaris





Solaris threads

```
int thr_create(void *stack_base, size_t stack_size,
               void *(*function)(void *), void *arg, long flags)
void thr_exit(void *status)
int thr_join(thread_t wait_for, thread_t *joiner_id, void *status)
int mutex_init(mutex_t *mutex, int type, void *arg)
int mutex_lock(mutex_t *mutex)
int mtex_unlock(mutex_t *mutex);
int cond_init(cond_t *cond, int type, int arg)
int cond_wait(cond_t *cond, mutex_t *mutex)
int cond_signal(cond_t *cond)
int cond_broadcast(cond_t *cond)
```

Figure 5: Some commonly used UI thread operations.





Tratarea semnalelor

- fiecare thread are propria masca de semnale
- toate threadurile unui proces partajeaza handlerele de semnal
- cand SIG_IGN/SIG_DFL sunt setate, se aplica tuturor threadurilor procesului
- semnalele de tip *trap* (sincrone cu executia threadului, ex SIGFPE, SIGSEGV) executate exclusiv de threadul care le-a generat => mai multe threaduri pot genera si trata acelasi tip de semnal simultan
- semnalele de tip interupere (generate asincron cu executia threadului, ex SIGIO, SIGINT) pot fi tratate de orice thread care nu mascheaza/blocheaza semnalul respectiv (in cazul mai multor astfel de threaduri, se alege unul dintre ele pt tratarea semnalului)
- daca toate threadurile au mascat un anume semnal, se asteapta dupa primul thread care deblocheaza tratarea semnalului respectiv





Tratarea semnalelor (cont.)

- *thread_kill*: trimite un semnal catre un thread din acelasi proces
  - semnalul e de tip trap, tratat doar de threadul caruia ii este adresat
  - nu se pot trimite semnale unui *anume* thread dintr-un *alt* proces
- threadurile Solaris implementeaza un nou semnal, SIGWAITING (SIG_IGN implicit)
  - trimis procesului cand toate LWP-urile sunt blocate indefinit in asteptarea unui eveniment extern
  - poate fi folosit pt crearea de noi LWP-uri pt a evita blocarea intregului proces
  - idee similara cu scheduler activations, dar coarse-grain (nu merge pt short-term blocking, doar pt evenimente de tip page faults, filesystem I/O)





Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)

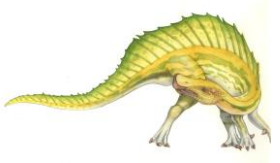




POSIX threads

```
int pthread_create(pthread_t *thread_pointer,
                   pthread_attr_t *attributes,
                    void *(*function)(void *), void *arg)
void pthread_exit(void *status)
int pthread_join(pthread_t thread, void **status)
void pthread_mutex_init(pthread_mutex_t *mutex,
                        pthread_mutexattr_t *attr)
int pthread_mutex_lock(pthread_mutex_t *mutex)
int pthread_mutex_unlock(pthread_mutex_t *mutex)
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *attr)
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)
int pthread_cond_signal(pthread_cond_t *cond)
int pthread_cond_broadcast(pthread_cond_t *cond)
```

Figure 6: Some commonly used Pthreads operations.





Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

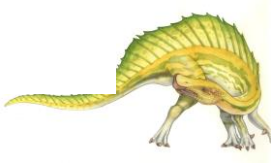
#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```



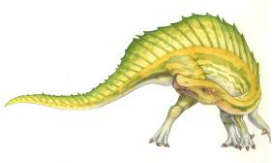


Pthreads Example (Cont.)

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```





Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```





Implicit Threading

- Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads
- Creation and management of threads done by compilers and run-time libraries rather than programmers
- Five methods explored
  - Thread Pools
  - Fork-Join
  - OpenMP
  - Grand Central Dispatch
  - Intel Threading Building Blocks

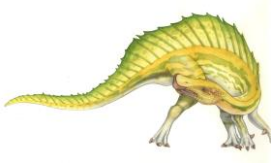




Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
  - Usually slightly faster to service a request with an existing thread than create a new thread
  - Allows the number of threads in the application(s) to be bound to the size of the pool
  - Separating task to be performed from mechanics of creating task allows different strategies for running task
    - i.e,Tasks could be scheduled to run periodically
- Windows API supports thread pools:

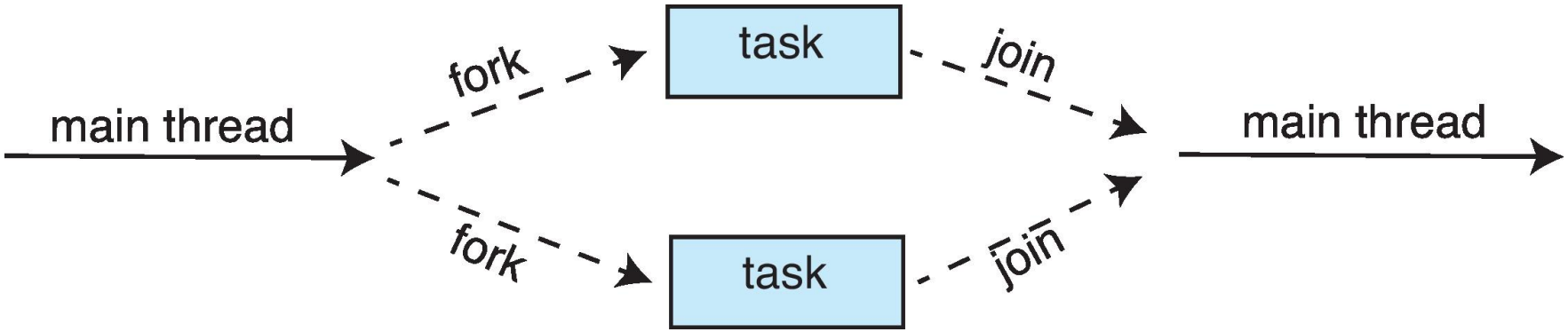
```
DWORD WINAPI PoolFunction(AVOID Param) {
    /*
    * this function runs as a separate thread.
    */
}
```





Fork-Join Parallelism

- Multiple threads (tasks) are **forked**, and then **joined**.







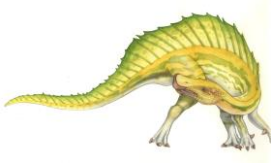
Fork-Join Parallelism

- General algorithm for fork-join strategy:

```
Task(problem)
   if problem is small enough
     solve the problem directly
   else
     subtask1 = fork(new Task(subset of problem)
     subtask2 = fork(new Task(subset of problem)

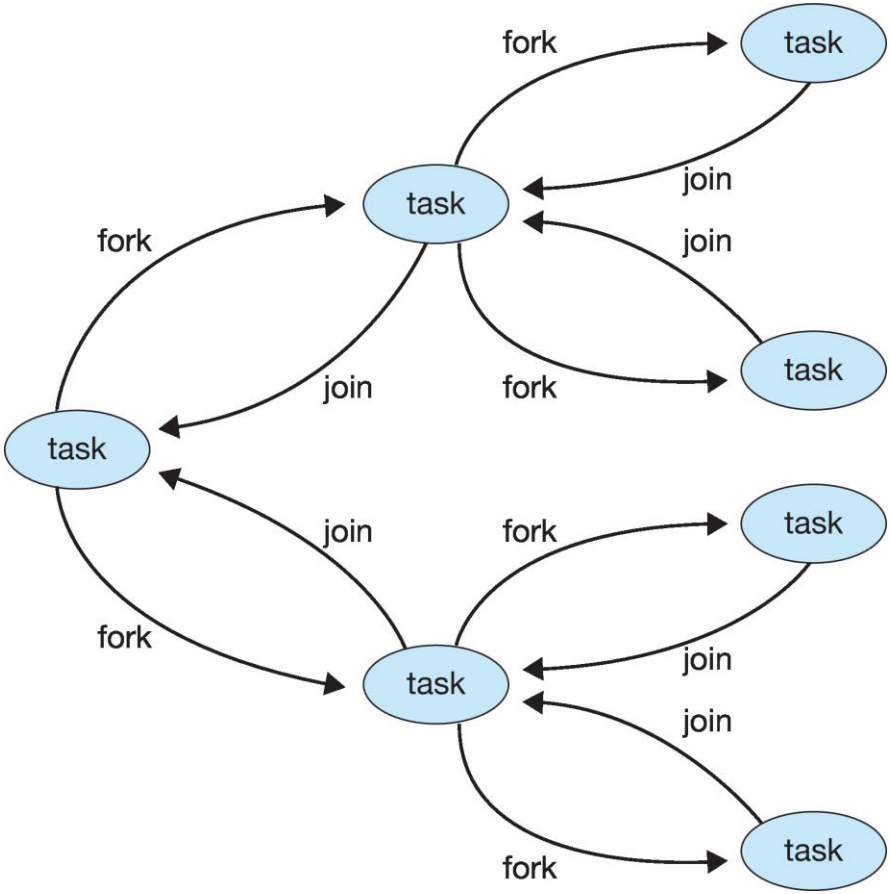
     result1 = join(subtask1)
     result2 = join(subtask2)

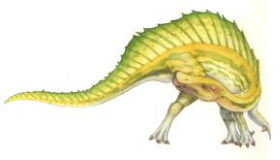
     return combined results
```





Fork-Join Parallelism







Threading Issues

- Semantics of **fork()** and **exec()** system calls
- Signal handling
  - Synchronous and asynchronous
- Thread cancellation of target thread
  - Asynchronous or deferred
- Thread-local storage
- Scheduler Activations





Semantics of fork() and exec()

- Does **`fork()`** duplicate only the calling thread or all threads?
  - Some UNIXes have two versions of fork
- **`exec()`** usually works as normal – replace the running process including all threads





Signal Handling

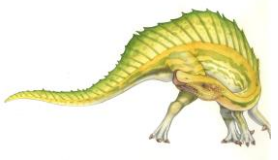
- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
  1. Signal is generated by particular event
  2. Signal is delivered to a process
  3. Signal is handled by one of two signal handlers:
     1. default
     2. user-defined
- Every signal has **default handler** that kernel runs when handling signal
  - **User-defined signal handler** can override default
  - For single-threaded, signal delivered to process





Signal Handling (Cont.)

- Where should a signal be delivered for multi-threaded?
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process





Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
  - **Asynchronous cancellation** terminates the target thread immediately
  - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

  . . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid,NULL);
```





Thread Cancellation (Cont.)

- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
  - Cancellation only occurs when thread reaches **cancellation point**
    - i.e., **`pthread_testcancel()`**
    - Then **cleanup handler** is invoked
- On Linux systems, thread cancellation is handled through signals





Thread-Local Storage

- **Thread-local storage** (**TLS**) allows each thread to have its own copy of data
- Useful when you do not have control over the thread creation process (i.e., when using a thread pool)
- Different from local variables
  - Local variables visible only during single function invocation
  - TLS visible across function invocations
- Similar to `static` data
  - TLS is unique to each thread





Multicore Programming

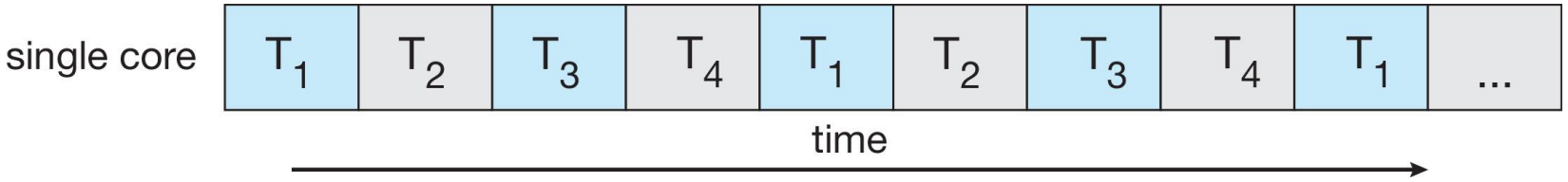
- **Multicore** or **multiprocessor** systems puts pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency



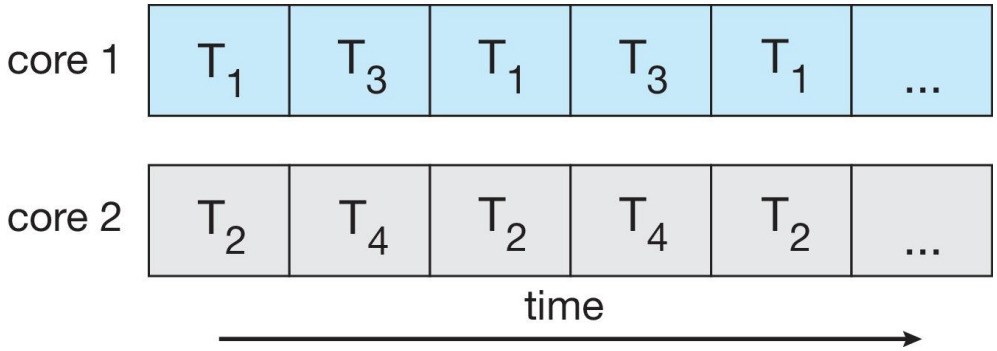


Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



- **Parallelism on a multi-core system:**







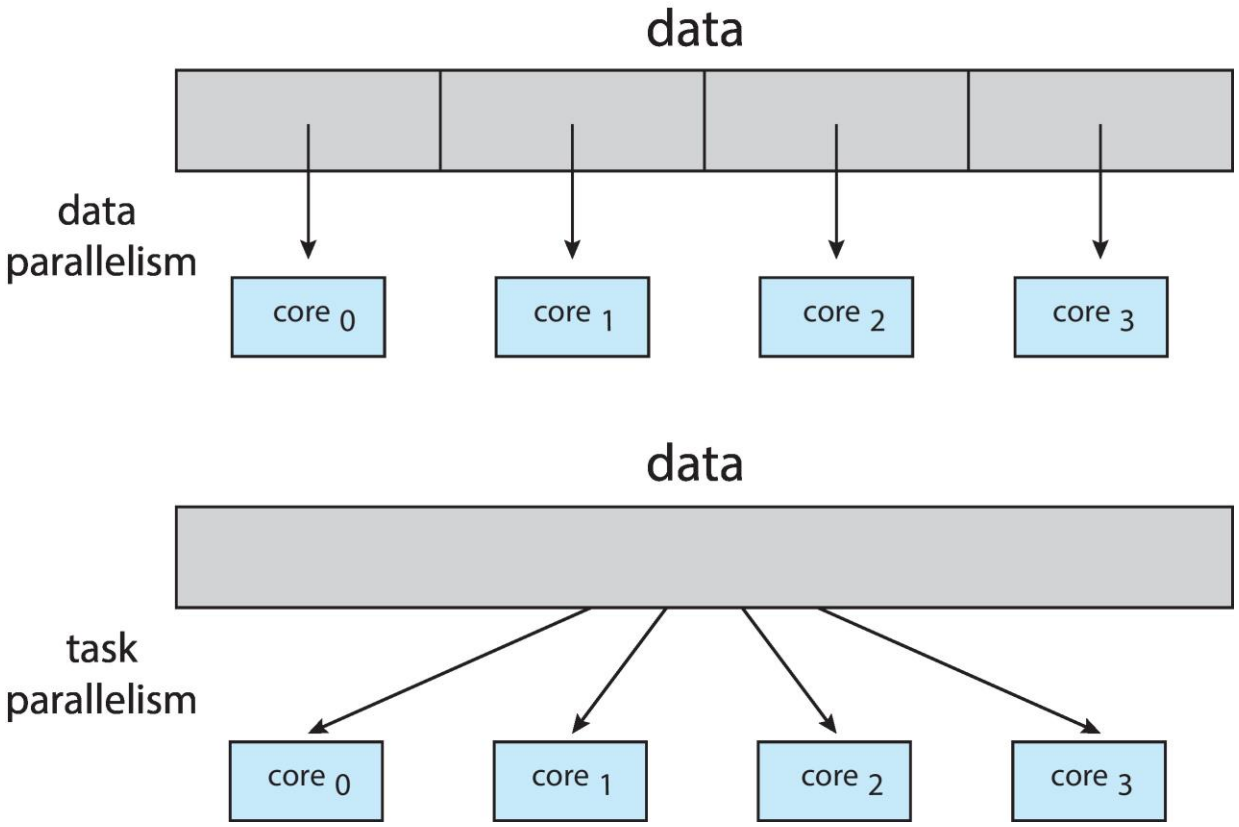
Multicore Programming

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation





Data and Task Parallelism







Amdahl's Law

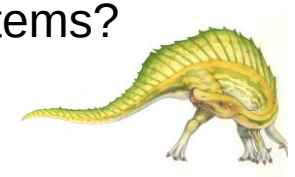
- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- *S* is serial portion
- *N* processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As *N* approaches infinity, speedup approaches 1 / *S*

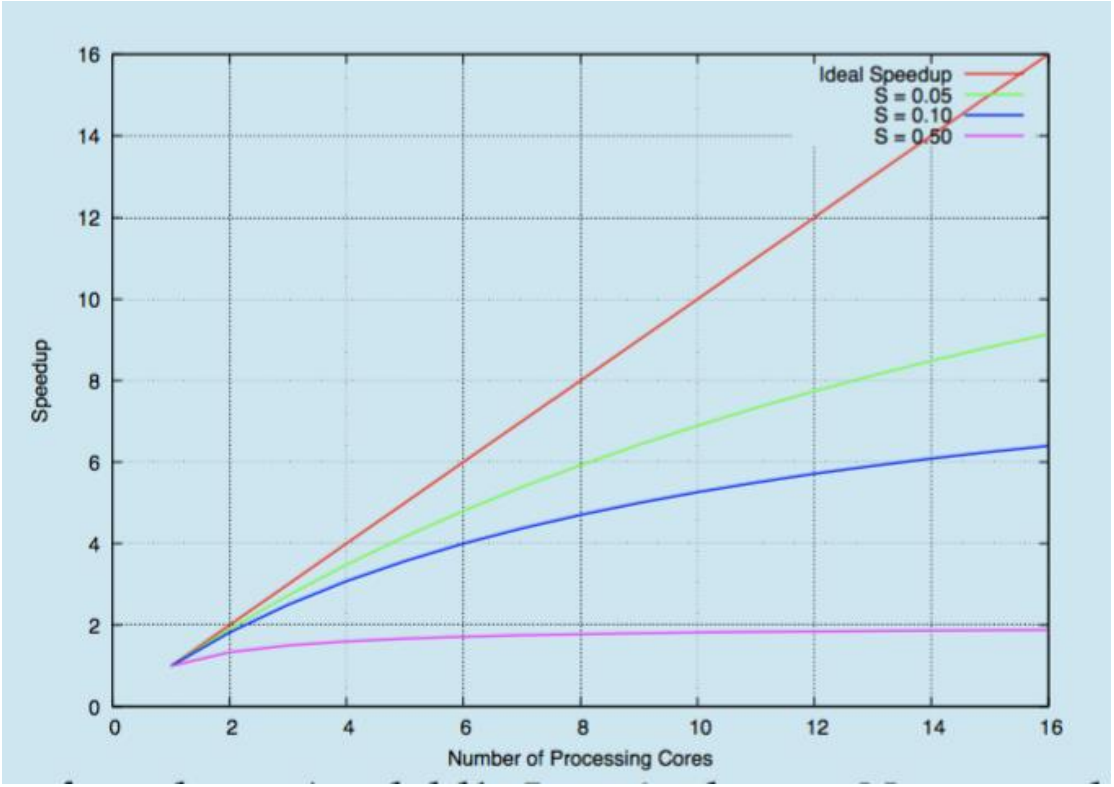
Serial portion of an application has disproportionate effect on performance gained by adding additional cores

- But does the law take into account contemporary multicore systems?





Amdahl's Law


Ideal Speedup
S = 0.05
S = 0.10
S = 0.50
Speedup
Number of Processing Cores
0
2
4
6
8
10
12
14
16

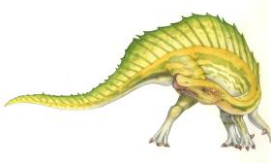




Windows Multithreaded C Program

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
   DWORD Upper = *(DWORD*)Param;
   for (DWORD i = 1; i <= Upper; i++)
      Sum += i;
   return 0;
}
```





Windows Multithreaded C Program (Cont.)

```
int main(int argc, char *argv[])
{
   DWORD ThreadId;
   HANDLE ThreadHandle;
   int Param;

   Param = atoi(argv[1]);
   /* create the thread */
   ThreadHandle = CreateThread(
      NULL, /* default security attributes */
      0, /* default stack size */
      Summation, /* thread function */
      &Param, /* parameter to thread function */
      0, /* default creation flags */
      &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
   WaitForSingleObject(ThreadHandle,INFINITE);

   /* close the thread handle */
   CloseHandle(ThreadHandle);

   printf("sum = %d\n",Sum);
}
```





Java Threads

- Java threads are managed by the JVM
- Typically implemented using the threads model provided by underlying OS
- Java threads may be created by:
  - Extending Thread class
  - Implementing the Runnable interface

```
public interface Runnable
{
    public abstract void run();
}
```

  - Standard practice is to implement Runnable interface





Java Threads

Implementing Runnable interface:

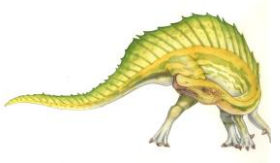
```
class Task implements Runnable
{
   public void run() {
      System.out.println("I am a thread.");
   }
}
```

Creating a thread:

```
Thread worker = new Thread(new Task());
worker.start();
```

Waiting on a thread:

```
try {
   worker.join();
}
catch (InterruptedException ie) { }
```





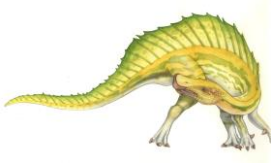
Java Executor Framework

- Rather than explicitly creating threads, Java also allows thread creation around the Executor interface:

```
public interface Executor
{
   void execute(Runnable command);
}
```

- The Executor is used as follows:

```
Executor service = new Executor;
service.execute(new Task());
```





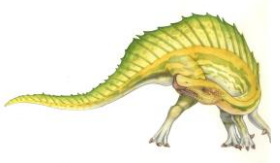
Java Executor Framework

```
import java.util.concurrent.*;

class Summation implements Callable<Integer>
{
   private int upper;
   public Summation(int upper) {
      this.upper = upper;
   }

   /* The thread will execute in this method */
   public Integer call() {
      int sum = 0;
      for (int i = 1; i <= upper; i++)
         sum += i;

      return new Integer(sum);
   }
}
```





Java Executor Framework (Cont.)

```
public class Driver
{
 public static void main(String[] args) {
    int upper = Integer.parseInt(args[0]);

    ExecutorService pool = Executors.newSingleThreadExecutor();
    Future<Integer> result = pool.submit(new Summation(upper));

    try {
        System.out.println("sum = " + result.get());
    } catch (InterruptedException | ExecutionException ie) { }
  }
}
```





Java Thread Pools

- Three factory methods for creating thread pools in Executors class:
  - `static ExecutorService newSingleThreadExecutor()`
  - `static ExecutorService newFixedThreadPool(int size)`
  - `static ExecutorService newCachedThreadPool()`





Java Thread Pools (Cont.)

```
import java.util.concurrent.*;

public class ThreadPoolExample
{
public static void main(String[] args) {
   int numTasks = Integer.parseInt(args[0].trim());

   /* Create the thread pool */
   ExecutorService pool = Executors.newCachedThreadPool();

   /* Run each task using a thread in the pool */
   for (int i = 0; i < numTasks; i++)
      pool.execute(new Task());

   /* Shut down the pool once all threads have completed */
   pool.shutdown();
}
```

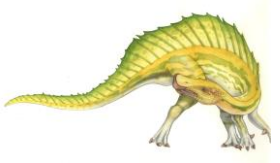




Fork-Join Parallelism in Java

```
ForkJoinPool pool = new ForkJoinPool();
// array contains the integers to be summed
int[] array = new int[SIZE];

SumTask task = new SumTask(0, SIZE - 1, array);
int sum = pool.invoke(task);
```





Fork-Join Parallelism in Java

```
import java.util.concurrent.*;

public class SumTask extends RecursiveTask<Integer>
{
   static final int THRESHOLD = 1000;

   private int begin;
   private int end;
   private int[] array;

   public SumTask(int begin, int end, int[] array) {
      this.begin = begin;
      this.end = end;
      this.array = array;
   }

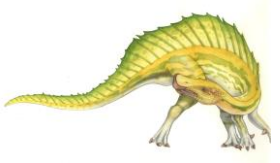
   protected Integer compute() {
      if (end - begin < THRESHOLD) {
         int sum = 0;
         for (int i = begin; i <= end; i++)
            sum += array[i];

         return sum;
      }
      else {
         int mid = (begin + end) / 2;

         SumTask leftTask = new SumTask(begin, mid, array);
         SumTask rightTask = new SumTask(mid + 1, end, array);

         leftTask.fork();
         rightTask.fork();

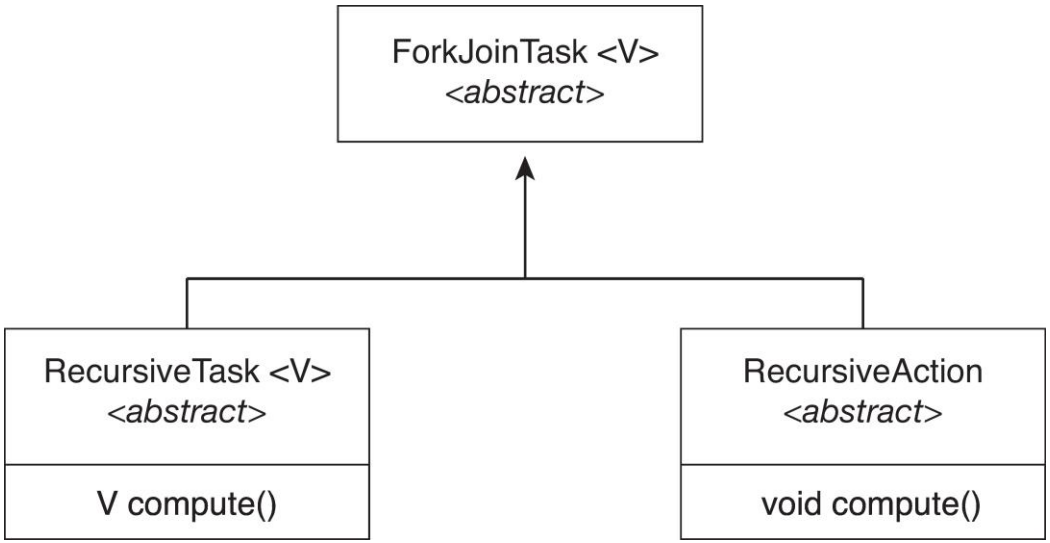
         return rightTask.join() + leftTask.join();
      }
   }
}
```





Fork-Join Parallelism in Java

- The **`ForkJoinTask`** is an abstract base class
- **`RecursiveTask`** and **`RecursiveAction`** classes extend **`ForkJoinTask`**
- **`RecursiveTask`** returns a result (via the return value from the **`compute()`** method)
- **`RecursiveAction`** does not return a result







OpenMP

- Set of compiler directives and an API for C, C++, FORTRAN
- Provides support for parallel programming in shared-memory environments
- Identifies **parallel regions** – blocks of code that can run in parallel

`#pragma omp parallel`

Create as many threads as there are cores

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
   /* sequential code */

   #pragma omp parallel
   {
      printf("I am a parallel region.");
   }

   /* sequential code */

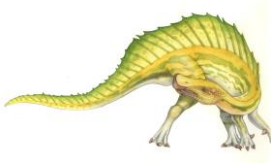
   return 0;
}
```





- Run the for loop in parallel

```
#pragma omp parallel for
for (i = 0; i < N; i++) {
   c[i] = a[i] + b[i];
}
```





Grand Central Dispatch

- Apple technology for macOS and iOS operating systems
- Extensions to C, C++ and Objective-C languages, API, and run-time library
- Allows identification of parallel sections
- Manages most of the details of threading
- Block is in “^{ }” :

```
^{ printf("I am a block"); }
```

- Blocks placed in dispatch queue
  - Assigned to available thread in thread pool when removed from queue





Grand Central Dispatch

- Two types of dispatch queues:
  - **serial** – blocks removed in FIFO order, queue is per process, called **main queue**
    - Programmers can create additional serial queues within program
  - **concurrent** – removed in FIFO order but several may be removed at a time
    - Four system wide queues divided by quality of service:
      - `QOS_CLASS_USER_INTERACTIVE`
      - `QOS_CLASS_USER_INITIATED`
      - `QOS_CLASS_USER_UTILITY`
      - `QOS_CLASS_USER_BACKGROUND`





Grand Central Dispatch

- For the Swift language a task is defined as a closure – similar to a block, minus the caret
- Closures are submitted to the queue using the `dispatch_async()` function:

```
let queue = dispatch_get_global_queue
   (QOS_CLASS_USER_INITIATED, 0)

dispatch_async(queue,{ print("I am a closure.") })
```





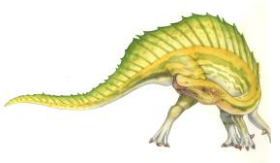
Intel Threading Building Blocks (TBB)

- Template library for designing parallel C++ programs
- A serial version of a simple for loop

```
for (int i = 0; i < n; i++) {
   apply(v[i]);
}
```

- The same for loop written using TBB with **`parallel_for`** statement:

```
parallel_for (size_t(0), n, [=](size_t i) {apply(v[i]);});
```





Thread Cancellation in Java

- Deferred cancellation uses the **`interrupt()`** method, which sets the interrupted status of a thread.

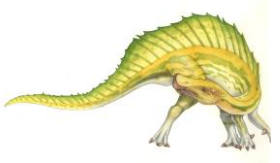
```
Thread worker;

. . .

/* set the interruption status of the thread */
worker.interrupt()
```

- A thread can then check to see if it has been interrupted:

```
while (!Thread.currentThread().isInterrupted()) {
    . . .
}
```





Operating System Examples

- Windows Threads
- Linux Threads





Windows Threads

- Windows API – primary API for Windows applications
- Implements the one-to-one mapping, kernel-level
- Each thread contains
  - A thread id
  - Register set representing state of processor
  - Separate user and kernel stacks for when thread runs in user mode or kernel mode
  - Private data storage area used by run-time libraries and dynamic link libraries (DLLs)
- The register set, stacks, and private storage area are known as the **context** of the thread





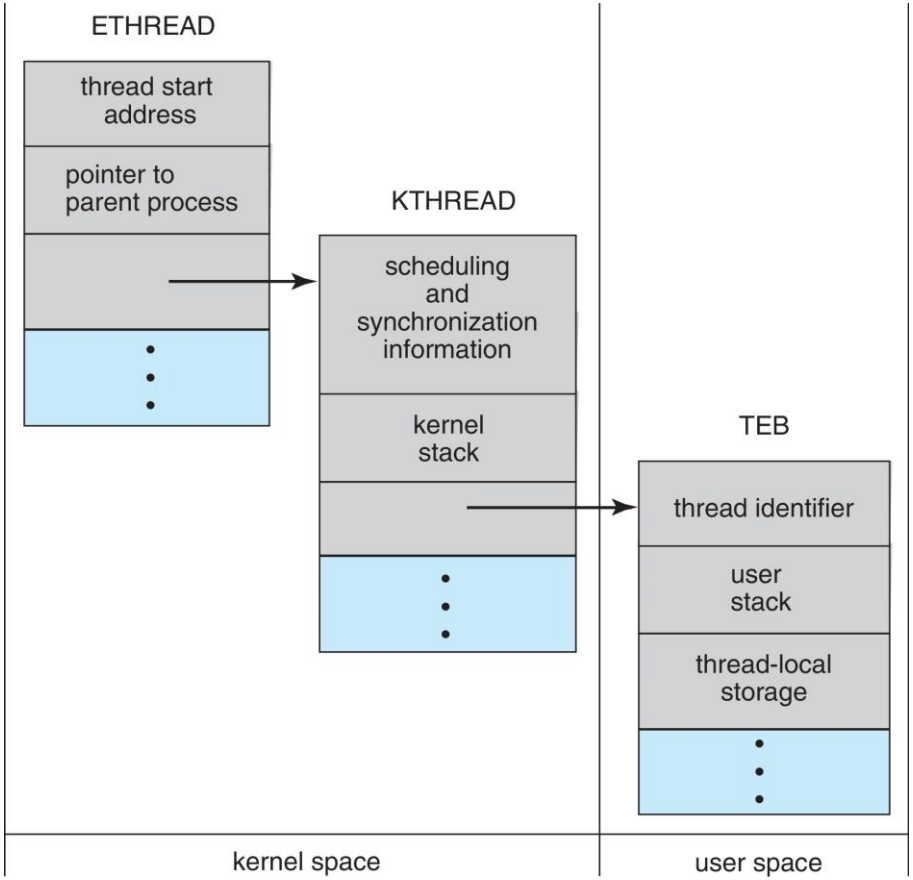
Windows Threads (Cont.)

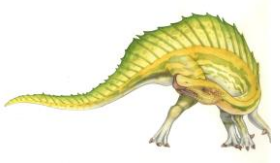
- The primary data structures of a thread include:
  - ETHREAD (executive thread block) – includes pointer to process to which thread belongs and to KTHREAD, in kernel space
  - KTHREAD (kernel thread block) – scheduling and synchronization info, kernel-mode stack, pointer to TEB, in kernel space
  - TEB (thread environment block) – thread id, user-mode stack, thread-local storage, in user space





Windows Threads Data Structures





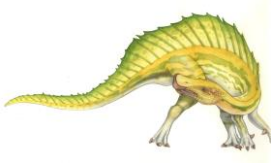


Linux Threads

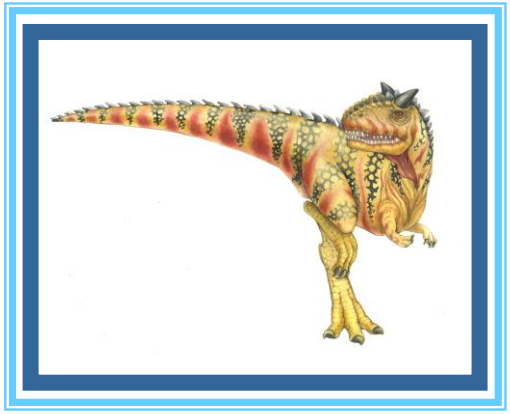
- Linux refers to them as ***tasks*** rather than ***threads***
- Thread creation is done through **`clone()`** system call
- **`clone()`** allows a child task to share the address space of the parent task (process)
  - Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

- **`struct task_struct`** points to process data structures (shared or unique)

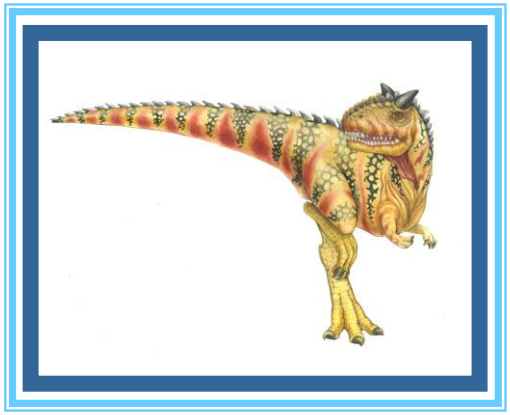


End of Chapter 4



Silberschatz, Galvin and Gagne ©2018

Chapter 6: Synchronization Tools



Silberschatz, Galvin and Gagne ©2018



Outline

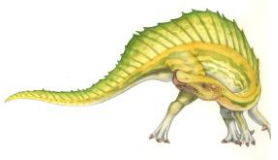
- Background
- The Critical-Section Problem
- Peterson's Solution
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Evaluation





Objectives

- Describe the critical-section problem and illustrate a race condition
- Illustrate hardware solutions to the critical-section problem using memory barriers, compare-and-swap operations, and atomic variables
- Demonstrate how mutex locks, semaphores, monitors, and condition variables can be used to solve the critical section problem
- Evaluate tools that solve the critical-section problem in low-, Moderate-, and high-contention scenarios





Background

- Processes can execute concurrently
  - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- We illustrated in chapter 4 the problem when we considered the Bounded Buffer problem with use of a counter that is updated concurrently by the producer and consumer,. Which lead to race condition.





Determinism in sistemele de operare

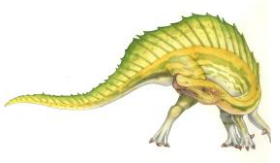
- SO sunt deopotriva deterministe si nedeterministe
- cand ruleaza un proces, garanteaza ca rularea va produce mereu aceleasi rezultate daca se furnizeaza aceleasi date de intrare
- natura impredictibila a evenimentelor externe (intreruperi sau cereri de a rula programe carora trebuie sa le raspunda) implica nedeterminism in privinta ordinii de executie a proceselor
- vom numi procese *concurente* procese care exista simultan in sistem, indiferent daca ruleaza time-shared pe un singur CPU (concurenta logica) sau in paralel pe mai multe CPU (concurenta fizica)
- data fiind nedeterminarea inerenta sistemului de operare, nu exista garantii despre starea unui proces la un moment dat
- din aceasta cauza, vom numi procesele *asincrone*





Definitii IPC

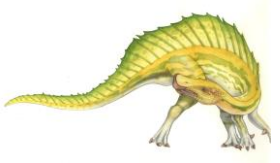
- procese *independente* – procese care nu partajeaza resurse
- procese *dependente* – partajeaza resurse pt a-si indeplini obiectivele
  - ex resursa partajata: imprimanta
  - prin intermediul primitivelor IPC, procesele dependente isi coordoneaza accesul la resursele patajate cf unui anumit protocol
- *sectiune/regiune critica* – portiune de cod care acceseaza o resursa partajata
- *race condition* – situatie in care executia intretesuta (interleaved) a mai multor procese care acceseaza o resursa partajata induce rezultate nedeterministe
- *excludere mutuala* – situatie in care cel mult un proces are acces la un moment dat la o resursa partajata





Definitii IPC (cont'd)

- *deadlock* – situatie in care nici un proces nu poate continua pt. ca resursele de care are nevoie sunt detinute de un alt proces
  - pp. implicit accesul mutual exclusiv la resursele partajate si o forma sau alta de asteptare circulara (ciclu intr-un graf de alocare a resurselor)
- *sincronizare* – cerinta ca un proces sa fi atins o anumita etapa in calculul sau inainte ca alt proces sa poata continua
- *starvation* – resursele necesare unui proces nu ii sunt niciodata puse la dispozitie
  - ex: un proces de prioritate mica P_l este impiedicat sa acceseze o resursa care e constant obtinuta de un proces de prioritate mare P_h
  - spunem ca P_l "moare de foame"





Corectitudinea partajarii resurselor

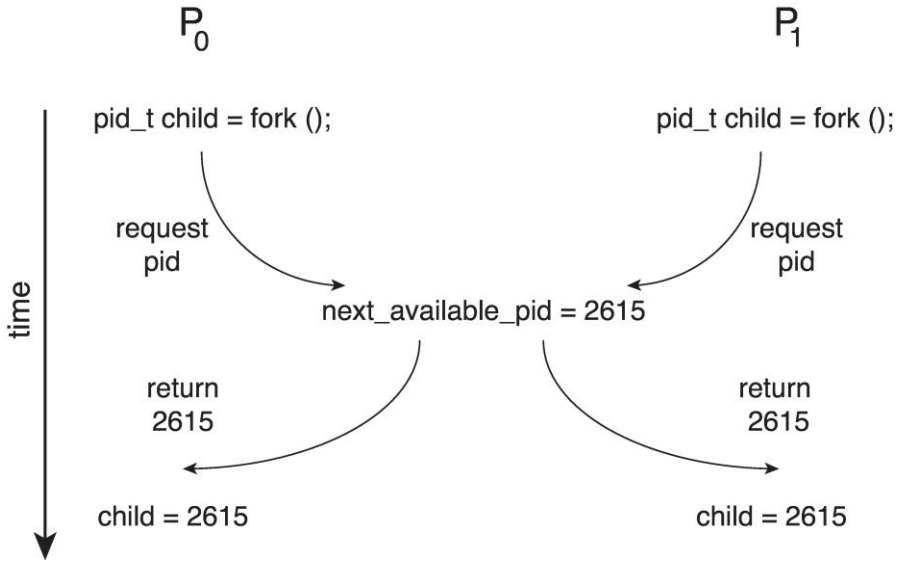
- resursele partajate sunt folosite corect daca procesele concurente evita race condition-uri, starvation si deadlock
- mecanismele utilizate in acest scop sunt: sectiunile critice, excluderea mutuala si sincronizarea
- cerinte necesare unei solutii corecte de partajare a resurselor de catre procese concurente
  - *nu e permisa nici o presupunere vis-à-vis de viteza relativa de executie a proceselor concurente*
  - ***excludere mutuala*** – cel mult un proces poate fi in sectiune critica la orice moment dat
  - ***asteptare limitata*** – daca un proces solicita accesul in sectiune critica trebuie sa i se garanteze ca-l va obtine candva
  - ***progres*** – un proces care ruleaza in afara sectiunii critice nu trebuie sa blocheze alt proces care vrea sa intre in sectiunea critica



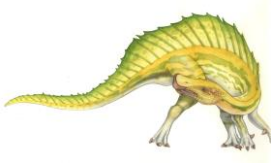


Race Condition

- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)



- Unless there is a mechanism to prevent P_0 and P_1 from accessing the variable `next_available_pid` the same pid could be assigned to two different processes!





Critical Section Problem

- Consider system of ***n*** processes {p_0, p_1, … p_{n-1}}
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc.
  - When one process in critical section, no other may be in its critical section
- ***Critical section problem*** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

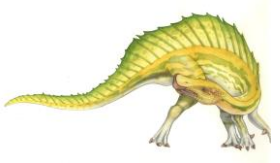




Critical Section

- General structure of process P_i

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

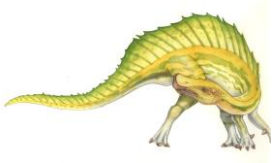




Critical-Section Problem (Cont.)

Requirements for solution to critical-section problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning **relative speed** of the ***n*** processes





Interrupt-based Solution

- Entry section: disable interrupts
- Exit section: enable interrupts
- Will this solve the problem?
  - What if the critical section is code that runs for an hour?
  - Can some processes starve – never enter their critical section.
  - What if there are two CPUs?





Dezactivarea intreruperilor

- bazata pe instructiuni privilegiate (eg Intel cli & sti) inaccesibile din user-mode
- are ca efect cresterea latentei intreruperilor => afecteaza intregul sistem, inclusiv operatii de I/O fara legatura cu sectiunea critica
- daca se dezactiveaza intreruperea de ceas, e posibil ca unele procese sa nu ajunga sa fie planificate pt rulare pe procesor
- intr-un sistem multiprocesor metoda e ineficienta (nu se poate garanta ca alte procese concurente care vor sa intre in sectiune critica nu vor rula pe alte procesoare)
- tehnica utila in kernele uniprocesor pt sectiuni critice scurte





Software Solution 1

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share one variable:
  - **`int turn;`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- initially, the value of **`turn`** is set to *i*





Alternanta stricta

```
int turn = 0;
P0 ()
{
        while(1)
        {
                while(turn != 0)
                        ;
                sectiune_critica();
                turn = 1;
                sectiune_necritica();
        }
}
```

```
P1 ()
{
        while(1)
        {
                while(turn != 1)
                        ;
                sectiune_critica();
                turn = 0;
                sectiune_necritica();
        }
}
```





Observatii

- procesele asteapta sa le vina randul sa intre in sectiunea critica ocupand procesorul (*busy waiting*)
  - pe un sistem uniprocesor time-shared, daca procesul aflat in sectiune critica pierde CPU, celalalt proces va fi planificat pt rulare si isi va consuma intreaga cuanta de timp alocata in busy waiting
  - pe sisteme multiprocesor, cele doua procese ruleaza in paralel pe CPU diferite si, cand unul, cand celalalt, consuma inutil cicluri procesor in busy waiting pt a intra in sectiune critica

  => din ratiuni de eficienta, sectiunile critice implementate in busy waiting trebuie sa fie cat mai scurte





Corectitudinea alternantei stricte

- pp. viteze de executie egale pt cele doua procese
- excludere mutuala
  - la inceput, P0 intra in SC, iar P1 asteapta ocupat in ciclul while
  - dupa ce P0 iese din SC, indica ca e randul lui P1 sa intre in SC, si continua cu sectiunea necritica
  - P1 intra in SC si la iesire indica ca e randul lui P0 sa intre in SC

  => doar un proces se afla in SC la un moment dat
- alternanta stricta poate sa nu fie neaparat dezirabila (sau realista)
- oricum, cerinta de progres nu e respectata
  - daca unul dintre procese petrece mult timp in afara SC, celalalt proces e in mod nejustificat impiedicat sa intre SC
- asteptarea limitata e respectata datorita alternantei stricte





Correctness of the Software Solution

- Mutual exclusion is preserved

  P_i enters critical section only if:

  `turn = i`

  and **`turn`** cannot be both 0 and 1 at the same time
- What about the Progress requirement?
- What about the Bounded-waiting requirement?





Peterson's Solution

- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
  - **`int turn;`**
  - **`boolean flag[2]`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- The **`flag`** array is used to indicate if a process is ready to enter the critical section.
  - **`flag[i] = true`** implies that process P_i is ready!

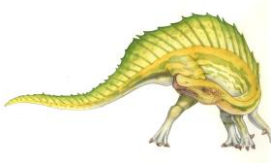




Algoritmul lui Peterson

- notatie: cele doua procese sunt P_i si P_j, unde j = 1 – i

```
1       const int N = 2;
2       bool flag[N];
3       int turn;
4       void func(int i)
5       {
6               j = 1 – i;
7               flag[i] = true;
8               turn = j;
9               while(flag[j] && (turn == j))
10                      ;
11              sectiune_critica()
12              flag[i] = false;
13              sectiune_necritica();
14      }
```





Correctness of Peterson's Solution

- Provable that the three CS requirement are met:
  1. Mutual exclusion is preserved

     $\mathbf{P_i}$ enters CS only if:

     either **`flag[j] = false`** or **`turn = i`**
  2. Progress requirement is satisfied
  3. Bounded-waiting requirement is met





Corectitudinea solutiei lui Peterson

- excludere mutuala
  - P_i intra in SC doar daca flag[j] == false sau turn == i
  - pp atat P_i cat si P_j sunt simultan in SC => ambele au executat liniile 7 & 8, deci flag[i] == flag[j] == true
  - cf. celor doua ipoteze de mai sus, P_i si P_j nu puteau executa linia 9 in acelasi timp, pt ca turn nu poate fi si 0 si 1 simultan

  => unul dintre procese, sa zicem P_j trebuie sa fi executat cu succes linia 9 (instructiunea while) in vreme ce celalalt a executat turn = j;
  - dar, in acest moment, flag[j]=true && turn == j si P_i trebuie sa astepte pana cand P_j iese din SC => excluderea mutuala e indeplinita
- progres
- asteptare limitata





Corectitudinea solutiei lui Peterson

- progres
  - P_i nu poate intra in SC doar daca flag[j]=true si turn == j
  - daca P_j nu este interesat sa intre in SC flag[j] == false si P_i poate intra in SC
  - altfel, la iesirea din SC, P_j seteaza flag[j] = false si poate petrece oricat timp in sectiune necritica fara sa impiedice P_i sa intre in SC
- asteptare limitata
  - daca P_j vrea sa intre in SC, seteaza flag[j]=true dar si turn = i
  - pt ca P_i nu schimba valoarea lui turn in linia 9 (instructiunea while), va intra in SC dupa cel mult o intrare a lui P_j in SC





Peterson's Solution and Modern Architecture

- Although useful for demonstrating an algorithm, Peterson's Solution is not guaranteed to work on modern architectures.
  - To improve performance, processors and/or compilers may reorder operations that have no dependencies
- Understanding why it will not work is useful for better understanding race conditions.
- For single-threaded this is ok as the result will always be the same.
- For multithreaded the reordering may produce inconsistent or unexpected results!





Modern Architecture Example

- Two threads share the data:

```
boolean flag = false;
int x = 0;
```

- Thread 1 performs

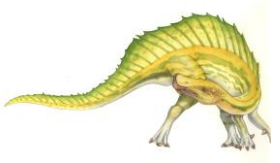
```
while (!flag)
  ;
print x
```

- Thread 2 performs

```
x = 100;
flag = true
```

- What is the expected output?

  100





Modern Architecture Example (Cont.)

- However, since the variables `flag` and `x` are independent of each other, the instructions:

```
flag = true;
x = 100;
```

for Thread 2 may be reordered

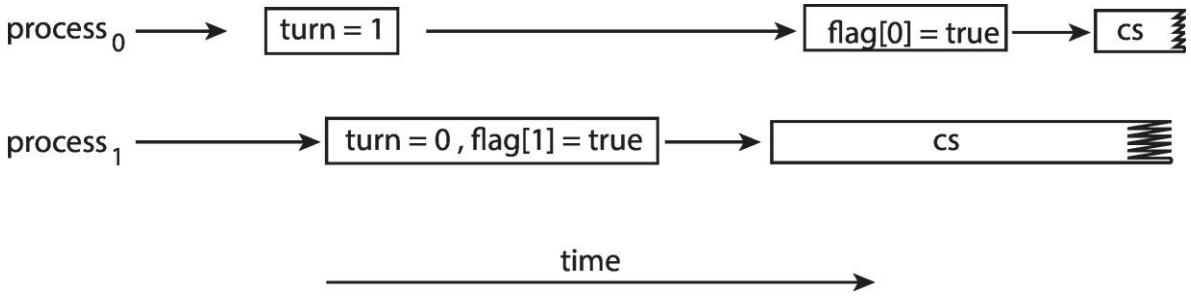
- If this occurs, the output may be 0!





Peterson's Solution Revisited

- The effects of instruction reordering in Peterson's Solution



- This allows both processes to be in their critical section at the same time!
- To ensure that Peterson's solution will work correctly on modern computer architecture we must use **Memory Barrier**.





Memory Barrier

- **Memory model** are the memory guarantees a computer architecture makes to application programs.
- Memory models may be either:
  - **Strongly ordered** – where a memory modification of one processor is immediately visible to all other processors.
  - **Weakly ordered** – where a memory modification of one processor may not be immediately visible to all other processors.
- A **memory barrier** is an instruction that forces any change in memory to be propagated (made visible) to all other processors.





Memory Barrier Instructions

- When a memory barrier instruction is performed, the system ensures that all loads and stores are completed before any subsequent load or store operations are performed.
- Therefore, even if instructions were reordered, the memory barrier ensures that the store operations are completed in memory and visible to other processors before future load or store operations are performed.





Memory Barrier Example

- Returning to the example of slides 6.17 - 6.18
- We could add a memory barrier to the following instructions to ensure Thread 1 outputs 100:
- Thread 1 now performs

```
while (!flag)
  memory_barrier();
print x
```

- Thread 2 now performs

```
x = 100;
memory_barrier();
flag = true
```

- For Thread 1 we are guaranteed that that the value of `flag` is loaded before the value of `x`.
- For Thread 2 we ensure that the assignment to `x` occurs before the assignment `flag`.





Synchronization Hardware

- Many systems provide hardware support for implementing the critical section code.
- Uniprocessors – could disable interrupts
  - Currently running code would execute without preemption
  - Generally too inefficient on multiprocessor systems
    - Operating systems using this not broadly scalable
- We will look at three forms of hardware support:

1. Hardware instructions
2. Atomic variables





Hardware Instructions

- Special hardware instructions that allow us to either *test-and-modify* the content of a word, or to *swap* the contents of two words atomically (uninterruptedly.)
  - **Test-and-Set** instruction
  - **Compare-and-Swap** instruction





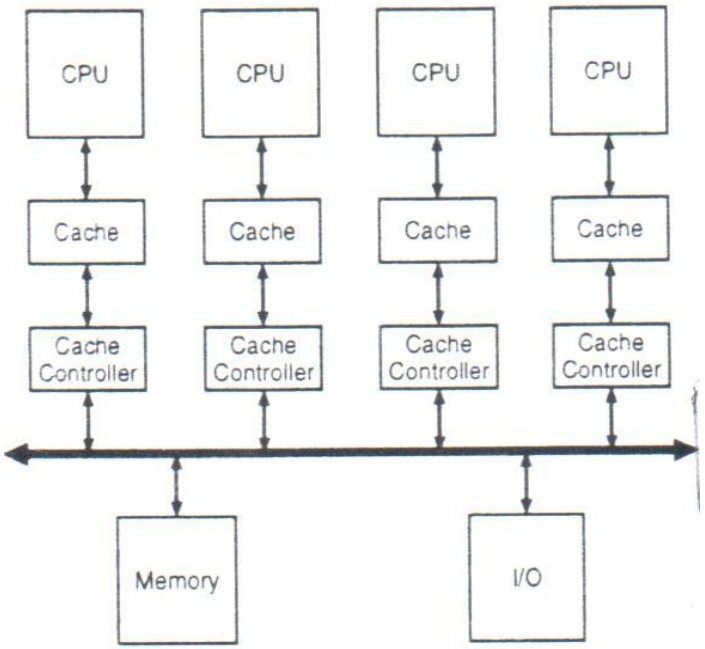
Sisteme multiprocesor (MP)

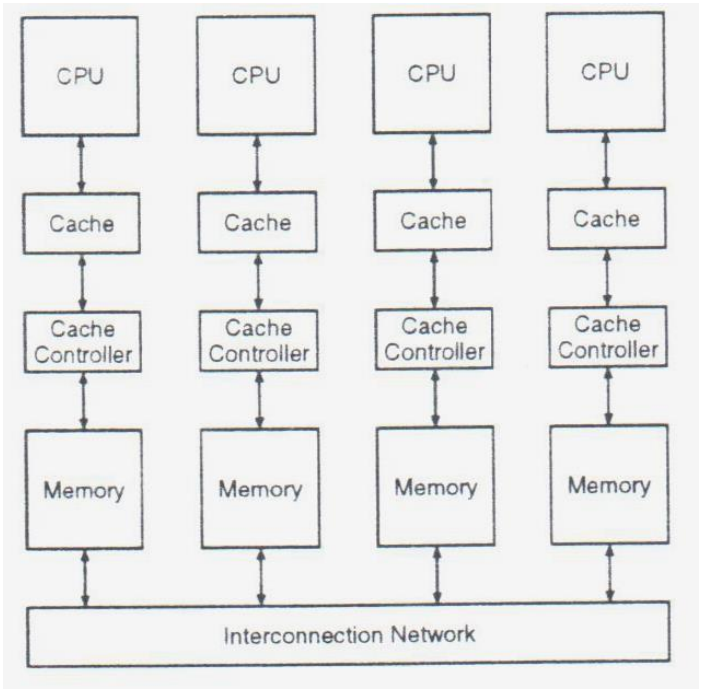
- taxonomie arhitecturi multiprocesor
  - Flynn: SISD, SIMD, MISD, MIMD
  - MIMD sunt referite uzual ca sisteme multiprocesor
- felul in care se interconecteaza procesoarele individuale in sistemele multiprocesor determina in general si tipul de acces la memorie
  - UMA, NUMA, NORMA
- in sistemele UMA/NUMA procesoarele partajeaza date protejat prin variabile partajate de memorie accesate sincronizat (cu ajutorul lock-urilor, de pilda)
- in sistemele NORMA accesul coordonat la date se face prin message passing
- vom restrange discutia la multiprocesoare interconectate printr-o singura magistrala





MP cu memorie partajata









Multiprocesoare cu o singura magistrala

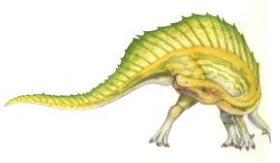
- magistrala unica reprezinta principala limitare pt. cresterea nr. de CPU
- folosirea cache-urilor reduce traficul pe magistrala si permite cresterea nr de procesoare in arhitecturile UMA
- existenta cache-urilor in general (si pt cc-NUMA) => copii multiple ale aceleiasi locatii de memorie => potential de inconsistenta daca un CPU modifica datele (locale)
- accesul uncached la datele partajate nu e indicat
  - scade viteza accesului la date
  - creste traficul pe magistrala ceea ce incetineste si accesul celorlalte procesoare inclusiv la datele nepartajate
- ideal, cand un procesor citeste date partajate pot exista copii multiple si in alte cache-uri, dar la scriere e nevoie de acces exclusiv la date urmat de invalidarea celorlalte copii existente





Multiprocesoare cu o singura magistrala

- solutia: protocoale de mentinere a coerentei cache-urilor locale
- ex de protocol uzual: *snoop coherency*
  - controlerele cache-urilor monitorizeaza magistrala pt traficul care afecteaza datele din cache-urile lor
  - *read miss*: controlerul localizeaza o copie actualizata a datelor (posibil aflata in cache-ul altui CPU)
  - *write*: exista doua protocoale posibile, *write-invalidate* si *write-update*
  - *write-invalidate* invalideaza toate copiile celorlalte procesoare inainte de a scrie datele in cache-ul local (eg. Intel MESI)
  - *write-update* scrie datele modificate pe magistrala printr-o operatie de broadcast
    - controlerele de cache isi pot actualiza copiile locale ale datelor la valoarea modificata





Suport HW pentru sincronizare

- multe procecesoare ofera suport HW pt implementarea sectiunilor critice
- uniprocesoare – pot dezactiva intreruperile
  - codul current se executa fara a fi interrupt (preempted)
  - ineficient in sisteme multiprocesor
- instructiuni HW atomice
  - instructiuni procesor speciale executate *in mod atomic (neintrerupt)*
  - fie testeaza si modifica (*test-and-modify)* continutul unui cuvant de memorie, **Test-and-Set (TAS)**
  - fie schimba (*swap)* continutul a doua cuvinte de memorie (sau mai multe), **Compare-and-Swap** (**CAS**, cu varianta **CAS2**)



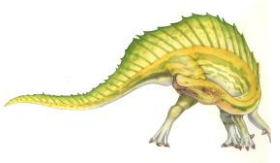


Test-and-set (TAS)

- definitie

```
int tas(int *lock)
  {
        int rv = *lock;
        *lock = 1;
        return rv:
  }
```

- proprietati
  - executie atomica
  - intoarce valoarea originala a parametrului pasat functiei
  - seteaza noua valoarea a parametrului pe 1





Spinlock-uri cu TAS

- locks (lacat, zavor), concept (abstractie) de nivel inalt pt. protectia sectiunilor critice
- ofera doua operatii: *acquire* (apelata la intrarea in sectiune critica) si respectiv *release* (apelata la iesirea din sectiune critica)
- cand un proces/thread detine lock-ul, celelalte sunt in busy waiting (“spinning in the while loop”), a.k.a *spinlocks*
- pt a se reduce busy waiting, sectiunile critice implementate cu spinlocks trebuie sa fie foarte scurte

```
int lock = 0;
void acquire(int *lock)
{
        while(tas(lock))
                ;
}
void release(int *lock)
{
        *lock = 0;
}
```





Instructiuni atomice multiprocesor

- se folosesc instructiuni atomice de tip TAS/CAS
- TAS in sisteme multiprocesor
  - cand un CPU executa TAS, arbitrul de magistrala da drept de folosinta exclusiva a magistralei procesorului respectiv pe durata executiei instructiunii
  - toate celelalte surse generatoare de accese de memorie sunt blocate pe durata TAS
  - se executa ciclul read-modify-write
  - la final, se deblocheaza magistrala pt uzul altor procesoare





Consecinte ciclu read-modify-write

- intr-un astfel de ciclu se citeste si se scrie o valoare in mod atomic, iar datele nu sunt cached

=> operatie mai costisitoare decat operatiile de citire/scriere obisnuite

=> magistrala e ocupata mai mult timp

=> operatiile TAS afecteaza semnificativ rata de transfer pe magistrala

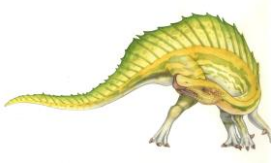
- alte consecinte privesc implementarea spinlock-urilor in sisteme multiprocesor





Spinlock-uri TAS MP

```
void acquire(int *lock_ptr)
{
        disable_interrupts();
        while(tas(lock_ptr))
                ;
}
void release(int *lock_ptr)
{
        *lock_ptr = 0;
        enable_interrupts();
}
```





Observatii

- dezactivarea intreruperilor in *acquire* impiedica pierderea procesorului pe durata detinerii lock-ului
- pierderea procesorului ar insemna ca procese/thread-uri de pe alte CPU nu pot intra in sectiune critica din cauza unui proces/thread care nu ruleaza !
- pe durata detinerii unui lock, celelalte procesoare executa TAS in bucla si consuma inutil largimea de banda a magistralei afectand procesoare care nu sunt implicate in accesul la sectiunea critica !
- la *release*, procesorul care detine lock-ul concureaza cu celelalte procesoare pt accesul la magistrala inainte de a putea ceda lock-ul
- solutie: *test-and-test-and-set* (TATAS)





TATAS

- traficul pe magistrala se poate reduce daca procesoarele cicleaza in *acquire* pe o copie locala a spinlock-ului
- pt ca *release* e in fond o operatie de scriere a spinlock-ului, protocolul de coerenta snoop detecteaza scrierea si invalideaza copiile locale ale celorlalte procesoare (in cazul protocolului *write-invalidate*)
- cand celelalte procesoare acceseaza din nou spinlock-ul => cache miss
- la cache miss copia locala se va actualiza si va reflecta starea de lock free
- procesorul incearca din nou sa execute operatia TAS sperand ca acum lock-ul e probabil free

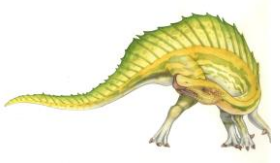




Spinlock-uri TATAS

```
void acquire(int *lock_ptr)
{
        disable_interrupts();
        while(*lock_ptr || tas(lock_ptr))
                ;
}
void release(int *lock_ptr)
{
        *lock_ptr = 0;
        enable_interrupts();
}
```

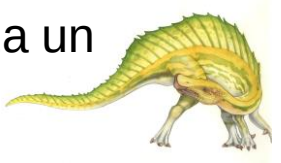
Obs: codul se bazeaza pe evaluarea scurtcircuitata a conditiilor in C





TATAS vs TAS

- pt sectiuni critice scurte si protocol *write-invalidate*, costurile sunt asemanatoare
  - dupa ce un CPU elibereaza lock-ul si altul il obtine, dureaza pana cand celelalte procesoare ajung sa cicleze pe copia locala a lock-ului
  - in tot acest timp, magistrala e saturata cu trafic de invalidare, read miss si TAS
- secventa tipica de evenimente
  - eliberare lock => invalidare copii din cache-urile altor CPU
  - invalidarile genereaza read miss pt toate CPU care cicleaza, toate citesc noua valoare a lock-ului de pe bus si ordinea e seriala !
  - primul CPU care obtine noua valoare a lock-ului, executa TAS si obtine lock-ul
  - restul CPUs primesc noua valoare, executa TAS si esueaza
  - fiecare TAS invalideaza copiile locale din cache-uri si forteaza un read miss





TATAS cu backoff

- competitia pt. magistrala partajata seamana cu protocolul de transmisie a datelor pe Ethernet (CSMA/CD)
  - analogia nu e stricta: pe Ethernet toti transmitatorii simultani esueaza, la TAS pe MP arbitrul de magistrala garanteaza un castigator
- analogia sugereaza insa scheme de backoff pt. reducerea competitiei la magistrala a procesoarelor care cicleaza
- metoda introduce o intarziere (delay) inainte de a incerca din nou operatia TAS
- intarzierea poate fi statica sau dinamica
- intarzierea statica:
  - fiecare CPU are un slot
  - merge bine pentru multe CPU-uri
  - intarzie nejustificat un singur CPU chiar daca lock-ul e liber





TATAS cu backoff (cont.)

- intarzierea dinamica:
  - la inceput, toate CPU-urile aleg o intarziere mica => coliziuni
  - dupa detectarea coliziunilor, fiecare CPU mareste intarzierea
  - overhead-ul intarzierilor mici de la inceput face metoda mai costisitoare decat alocarea statica a intarzierilor
- Obs: folosirea protocolului *write-update* poate imbunatati performanta TATAS prin reducerea traficului pe magistrala
  - cand un lock e eliberat, noua sa valoare poate fi transmisa prin broadcast
  - fiecare CPU care monitorizeaza magistrala isi poate astfel actualiza copia locala fara a mai genera read miss





Spinlock-uri TATAS cu backoff

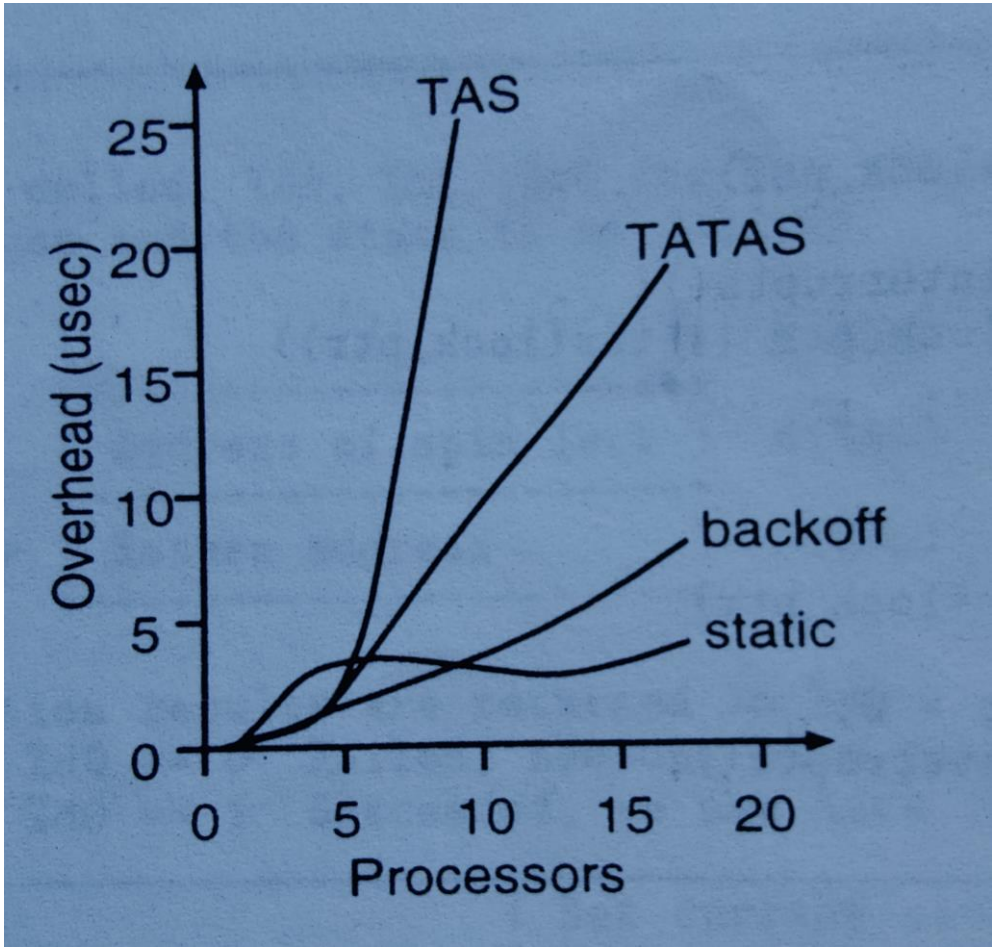
```
void acquire(int *lock_ptr)
{
        disable_interrupts();
        while(*lock_ptr || tas(lock_ptr))
        {
                while(*lock_ptr)
                        ;
                delay();
        }
}
```

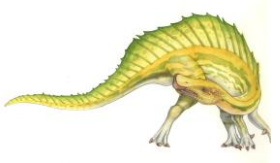
```
void release(int *lock_ptr)
{
        *lock_ptr = 0;
        enable_interrupts();
}
```





Performanta TAS

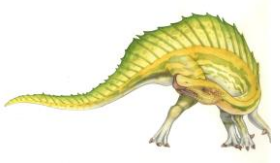






Dezavantaje instructiuni atomice

- instructiunile atomice simplifica IPC pe multiprocesoare, dar au si dezavantaje
  - pot avea performanta redusa daca sunt folosite neglijent
  - daca un proces/thread pierde CPU sau e fortat sa astepte o perioada lunga (eg, page fault) in timp ce detine lock-ul, celelalte procese nu pot avansa pana cand detinatorul lock-ului revine pe CPU si elibereaza lock-ul
  - daca un proces/thread se termina anormal in timp ce detine un lock, nici un alt proces/thread nu poate intra in sectiune critica
  - inversarea prioritatilor – cand un proces/thread cu prioritate mica detine lock-ul si impiedica un proces cu prioritate marea sa intre in sectiune critica
- solutie: sincronizare *wait-free* (*lock-free*), Herlihy 1991





Sincronizare wait-free (lock-free)

- idee centrala: se incearca executia operatiei, dar se lasa datele intr-o stare consistenta daca operatia esueaza
  - analogie cu tranzactiile din baze de date
  - model de control optimist al concurentei
- se pot folosi instructiuni HW de tipul CAS/CAS2 sau LL/SC
- model Herlihy:
  - sistem cu *n* procese secventiale care comunica prin memorie partajata
  - *obiect concurent* = ({tip+valori}, set operatii, specificatie secventiala)
  - *obiect concurent neblocant* – procesul care executa una dintre operatiile sale trebuie o termine intr-un nr finit de pasi
  - *obiect concurent wait-free* – fiecare proces din sistem trebuie sa termine operatia intr-un nr finit de pasi





Sincronizare wait-free (cont.)

- conditia de operare *nonblocanta* - unele procese/threaduri vor face intotdeauna progres indiferent de intarzierile sau opririle fortate ale altor procese/threaduri
- conditia de operare *wait-free* e mai puternica => toate procesele/threadurile **care nu sunt oprite** progreseaza indiferent de terminarile anormale sau intarzierile altor procese/threaduri
- ambele conditii **exclud** folosirea sectiunilor critice ca metoda de implementare
  - un proces/thread care se termina anormal (cu eroare) in mijlocul unei sectiuni critice va bloca pt totdeauna celelalte procese care asteapta sa intre in sectiunea critica
- **rezultat teoretic**: *este imposibil sa se construiasca implementari neblocante sau wait-free ale tipurilor de date simple folosind operatii de citire/scriere, TAS, fetch-and-add, swap de memorie cu registre* (eg, instructiunea Intel *xchg*)



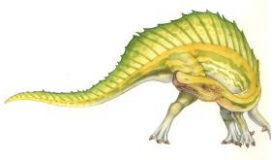


Semantica compare-and-swap (CAS)

- definitie

```
int compare_and_swap(int *value, int expected, int new_value)
   {
    int temp = *value;
     if (*value == expected)
         *value = new_value;
      return temp;
  }
```

- proprietati
  - executie atomica
  - intoarce valoarea originala a parametrului **`value`**
  - seteaza variabila **`value`** la valoarea parametrului **`new_value`** doar daca e indeplinita conditia **`*value == expected`**, adica, schimbul de valori se face doar daca e indeplinita conditia





Incrementare atomica wait-free cu CAS

```
1       void increment(atomic_int *v)
2       {
3               int temp;
4               do {
5                       temp = *v;
6               }while (temp != (compare_and_swap(v,temp,temp+1));
7       }
```

Scenariu de executie:

- pp initial v = 10
- pp. proces de prioritate mica P_l executa codul primul
- P_l pierde CPU intre liniile 5 si 6, moment in care un proces cu prioritate mare P_h incepe executia
- starea P_l este v=10, temp=10, iar P_h incheie cu succes incrementarea => v = 11
- cand P_l se reia, **cas(11,10,11)** esueaza si se re-executa linia 5 => temp = v = 11 si **cas(11, 11, 12)** reuseste = > v = 12

Obs: spre deosebire de TAS care modifica operandul intotdeauna, CAS/CAS2 modifica operandul *doar daca reuseste comparatia* !





Suport HW RISC pentru sectiuni critice

- TAS, CAS specifice procesoarelor CISC, dar secvente atomice de tip *read-modify-write* nu se pot implementa pe procesoare RISC (arhitecturi load/store)
- operatii speciale RISC: Load Linked (LL) si Store Conditional (SC)
- LL incarca o variabila din memorie intr-un registru CPU si apoi verifica activ daca variabila din memorie este modificata de alte procesoare
- SC verifica daca au existat modificari ale variabilei de memorie de la ultimul LL
  - daca nu, valorea registrului se stocheaza in memorie cu success (variabila din memorie este modificata) iar continutul registrului este setat pe 1
  - daca da, stocarea esueaza (variabila de memorie ramane nemodificata) iar valoarea registrului se seteaza pe 0

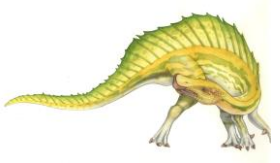




Incrementare atomica wait-free cu LL/SC

- in fapt, o forma elementara de memorie tranzactionala (zona de memorie restransa la un byte/word)
- functioneaza cf. principiilor de control optimist al concurentei din baze de date
- simuleaza executia unei tranzactii din baze de date cf principiilor ACID (Atomicity, Consistency, Isolation, Durability)

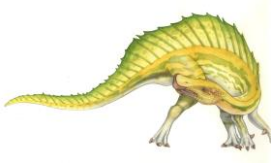
```
1   increment:
2           ll      $2, count               ; incarca valoarea counter-ului
3           addu    $3, $2, 1               ; incrementeaza valoare counter
4           sc      $3, count               ; incearca sa stocheze noua valoare
5           beq     $3, zero, increment             ; zero inseamna esec
6           j       $31                     ; return din rutina
```





Proprietati ACID incrementare LL/SC

- *atomicitate* - modificarile (incrementarea counter-ului) sunt executate ca si cand operatia ar fi atomica, i.e. fie toate modificarile sunt executate, fie niciuna
- *consistenta* - datele (*count*) sunt intr-o stare consistenta cand tranzactia incepe si cand se termina, i.e. *count* e incrementat corect, chiar daca se executa mai multe tranzactii simultan, eventual intretesut
- *izolare* - starea intermediara a tranzactiei (apelul procedurii si valorile registrelor) e invizibila altor tranzactii (altor apeluri ale aceleiasi proceduri)
  - tranzactiile concurente (apelurile concurente ale procedurii *increment*) par a fi serializate
- *durabilitate* – dupa incheierea cu succes a tranzactiei (apelul procedurii *increment*), datele sunt salvate in memoria principala RAM si modificarea e finala
  - analogia cu bazele de date se opreste aici, stocarea in RAM nu e persistenta





Comparatie CAS vs. LL/SC

- daca au aparut modificari ale counter-ului, SC va esua garantat, chiar daca valoarea initial citita de LL a fost restaurata
  - ex: problema ABA, P1 si P2 partajeaza o variabila de memorie
  - P1 citeste A si pierde procesorul, P2 scrie B si apoi A, P1 revine pe procesor si citeste A
- daca se incearca aceeasi secventa de operatii cu CAS, adica
  - se citeste valoarea counter-ului
  - apoi se executa CAS
  - daca vechea valoare a fost restaurata, nu se vor detecta modificarile intermediare aparute intre operatia de read si cea de CAS

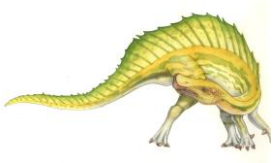
  => semantica LL/SC e mai puternica decat CAS
- atat CAS cat si LL/SC se pot folosi pentru implementarea sincronizarii *wait-free*





Memorie tranzactionala

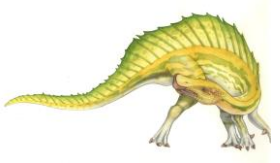
- software transactional memory, Herlihy & Moss 1993
- am vazut deja ex. LL/SC & CAS
- exploateaza ideea de tranzactii din baze de date (v. proprietati ACID) si controlul optimist al concurentei
- un thread termina modificarile operate pe o zona de memorie partajata fara a se coordona cu alte threaduri
- se inregistreaza fiecare operatie de citire/scriere intr-un log
- la finalul tranzactiei se incearca operatia de *commit*
  - daca reuseste (i.e., nici o alta tranzactie nu a apucat sa faca un *commit* reusit intre timp), tranzactia e validata si modificarile devin permanente
  - daca esueaza, tranzactia e abandonata (*aborted*) si se re-executa de la inceput pana reuseste (*transaction roll-back*)
- beneficiu major: grad crescut de concurenta, thread-urile nu au nevoie sa se sincronizeze prin lock-uri la accesul memoriei partajate





Suport HW pt. memorie tranzactionala

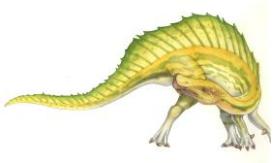
- ex. Intel TSX (Transactional Synchronization Extension), extensie ISA x86, microarhitectura Haswell (2013)
- accesibila prin intermediul a doua interfete
  - HLE (Hardware Lock Elision) – backward compatibility
  - RTM (Restricted Transactional Memory)
- HLE adauga doua prefixuri de instr. XACQUIRE & XRELEASE
  - se pot folosi doar pt anumite instructiuni care trebuie prefixate explicit cu LOCK
  - permite executia optimista a sectiunii critice sarind scrierea lock-ului, a.i. acesta apare liber pt alte threaduri
  - o tranzactie esuata determina reluarea operatiei de la instructiunea prefixata cu XACQUIRE, dar instructiunea se va reexecuta ca si cand nu a fost prefixata cu XACQUIRE





Suport HW pt. memorie tranzactionala

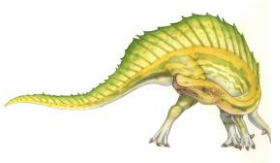
- RTM adauga la ISA trei noi instructiuni
  - XBEGIN – marcheaza inceputul zonei de memorie tranzactionala
  - XEND - marcheaza sfarsitul zonei de memorie tranzactionala
  - XABORT – abandoneaza explicit o tranzactie
  - esecul tranzactiei redirecteaza executia catre codul specificat de instr. XBEGIN, iar codul de eroare e stocat in registrul EAX
- instructiunea XTEST permite testarea starii procesorului (este sau nu in mijlocul executiei unei regiuni de memorie tranzactionala)





RTM lock elision in glibc

```
void elided_lock_wrapper(lock) {
        if(_xbegin() == _XBEGIN_STARTED) { // start tranzactie
                if(lock  e liber)
                        return;    // executa sectiunea critica in tranzactie
                _xabort(0xff);     // abandoneaza tranzactia
        }
        obtine lock
}
void elided_unlock_wrapper(lock) {
        if (lock e liber)
                _xend();           // comite tranzactia
        else
                elibereaza lock;
}
```



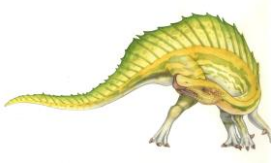


The test_and_set Instruction

- Definition

```
boolean test_and_set (boolean *target)
   {
          boolean rv = *target;
          *target = true;
          return rv:
   }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter
  - Set the new value of passed parameter to **`true`**





Solution Using test_and_set()

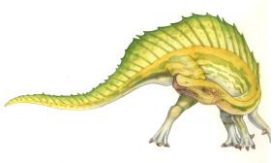
- Shared boolean variable **`lock`**, initialized to **`false`**
- Solution:

```
do {
        while (test_and_set(&lock))
        ; /* do nothing */

            /* critical section */

    lock = false;
            /* remainder section */
 } while (true);
```

- Does it solve the critical-section problem?



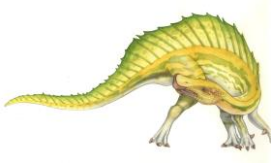


The compare_and_swap Instruction

- Definition

```
int compare_and_swap(int *value, int expected, int new_value)
 {
   int temp = *value;
   if (*value == expected)
       *value = new_value;
    return temp;
 }
```

- Properties
  - Executed atomically
  - Returns the original value of passed parameter **`value`**
  - Set the variable **`value`** the value of the passed parameter **`new_value`** but only if **`*value == expected`** is true. That is, the swap takes place only under this condition.





Solution using compare_and_swap

- Shared integer **`lock`** initialized to 0;
- Solution:

```
while (true){
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing */

    /* critical section */

    lock = 0;

    /* remainder section */
}
```

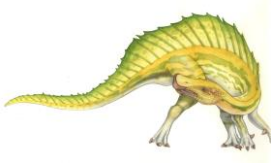
- Does it solve the critical-section problem?





Bounded-waiting with compare-and-swap

```
while (true) {
   waiting[i] = true;
   key = 1;
   while (waiting[i] && key == 1)
      key = compare_and_swap(&lock,0,1);
   waiting[i] = false;
   /* critical section */
   j = (i + 1) % n;
   while ((j != i) && !waiting[j])
      j = (j + 1) % n;
   if (j == i)
      lock = 0;
   else
      waiting[j] = false;
   /* remainder section */
}
```





Atomic Variables

- Typically, instructions such as compare-and-swap are used as building blocks for other synchronization tools.
- One tool is an **atomic variable** that provides *atomic* (uninterruptible) updates on basic data types such as integers and booleans.
- For example:
  - Let **`sequence`** be an atomic variable
  - Let **`increment()`** be operation on the atomic variable **`sequence`**
  - The Command:

    `increment(&sequence);`

    ensures **`sequence`** is incremented without interruption:





Atomic Variables

- The **`increment()`** function can be implemented as follows:

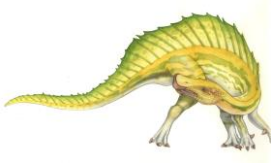
```
void increment(atomic_int *v)
{
     int temp;
     do {
            temp = *v;
     }
     while (temp != (compare_and_swap(v,temp,temp+1));
}
```





Mutex Locks

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
  - Boolean variable indicating if lock is available or not
- Protect a critical section by
  - First **`acquire()`** a lock
  - Then **`release()`** the lock
- Calls to **`acquire()`** and **`release()`** must be **atomic**
  - Usually implemented via hardware atomic instructions such as compare-and-swap.
- But this solution requires **busy waiting**
  - This lock therefore called a **spinlock**





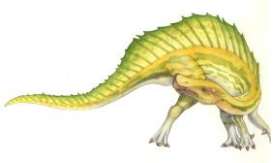
Solution to CS Problem Using Mutex Locks

```
while (true) {
        acquire lock

            critical section

        release lock

remainder section
}
```





Sleep/wakeup

- spinlock-urile sufera de busy-waiting si ca atare de problemele discutate anterior
- busy-waiting-ul se poate evita cu ajutorul unei primitive **sleep**
- **sleep** blocheaza procesele care nu au dreptul sa intre in sectiune critica (pierd procesorul)
- cand procesul aflat in sectiune critica paraseste sectiunea critica, apeleaza primitiva **wakeup** care trezeste toate procesele blocate in sleep
- problema producator-consumator implementata cu **sleep/wakeup**





Producator-consumator

```
count = 0
N = dim buffer
producator()
{
    while(1)
    {
            produce_item();
            if(count == N) sleep();
            enter_item();
            count++;
            if(count == 1)
                wakeup(consumator)
    }
}
```

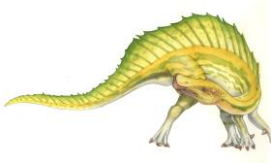
```
consumator()
{
    while(1)
    {
            if(count == 0) sleep();
            remove_item();
            count--;
            if(count == N - 1)
                wakeup(producator)
            consuma_item();
    }
}
```





Race condition solutie sleep/wakeup

- pp buferul e gol, consumatorul a citit *count* == 0 si inainte sa execute testul schedulerul da controlul producatorului
- producatorul produce un obiect, observa ca buferul era gol, crede ca procesul consumator executa **sleep** si apeleaza **wakeup** pt a-l notifica
- cand revine pe CPU, consumatorul executa testul si apoi **sleep** => notificarea **wakeup** s-a pierdut !
- pt ca procesul consumator nu a apucat sa consume obiectul produs, producatorul continua sa produca obiecte pana umple buferul complet si e nevoit sa apeleze **sleep**
- in acest moment, ambele procese sunt blocate => **deadlock !**
- problema esentiala: accesul nesincronizat la variabila *count*
- solutie posibila: **wakeup** seteaza un waiting bit pe care consumatorul il citeste si nu intra in **sleep**
  - limitare neplacuta: in general e nevoie de un nr arbitar de waiting bits => e necesara o primitiva speciala (semafoare)





Semafoare

- structura de date speciala (Dijkstra 1965)
  - are contor care numara **wakeup**-urile
  - are coada de procese blocate in **sleep**
  - **down**
    - operatie care decrementeaza **atomic** contorul daca e > 0 si permite procesului sa continue
    - daca contorul e zero, blocheaza procesul si il pune in coada
  - **up** – verifica daca exista procese blocate in coada
    - daca da, alege unul dintre ele si il deblocheaza
    - daca nu, incrementeaza **atomic** contorul
- un semafor cu contor initial 1 este de fapt un mutex (semafor binar)
- **up/down** generalizeaza **sleep/wakeup** DAR sunt operatii **atomice** !
- obs: **up** nu blocheaza niciodata procese !





Semaphore Usage Example

- Solution to the CS Problem
  - Create a semaphore "**mutex**" initialized to 1

```
down(mutex);
    CS
up(mutex);
```

- Consider P_1 and P_2 that with two statements S_1 and S_2 and the requirement that S_1 to happen before S_2
  - Create a semaphore "**synch**" initialized to 0

```
P1:
    S1;
    up(synch);
P2:
    down(synch);
    S2;
```





Producator-consumator cu semafoare

```
item_t buffer[N]
semaphore mutex = {1}, space = {N},
items = {0};
void producer(void)
{
        item_t item;
        while(true) {
            produce(&item);
            down(space);
            down(mutex);
            put(buffer, &item);
            up(mutex);
            up(items);
        }
}
```

```
void consumer(void)
{
        item_t item;
        while(true) {
            down(items);
            down(mutex);
            get(buffer, &item);
            up(mutex);
            up(space);
            consume(&item);
        }
}
```





Semaphore Implementation

- Must guarantee that no two processes can execute the **`down()`** and **`up()`** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **`down`** and **`up`** code are placed in the critical section
- Could now have **busy waiting** in critical section implementation
  - But implementation code is short
  - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution





Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - Value (of type integer)
  - Pointer to next record in the list
- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue





Implementation with no Busy waiting (Cont.)

- Waiting queue

```
typedef struct {
   int value;
   struct process *list;
} semaphore;
```





Implementation with no Busy waiting (Cont.)

```
down(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

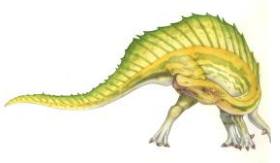
up(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```





Problems with Semaphores

- Incorrect use of semaphore operations:
  - **`up(mutex)` …. `down(mutex)`**
  - **`down(mutex)` … `down(mutex)`**
  - Omitting of **`down (mutex)`** and/or **`up (mutex)`**
- These – and others – are examples of what can occur when semaphores and other synchronization tools are used incorrectly.





Ex. greseli producator-consumator

- producatorul inverseaza secventa de accesare a semafoarelor

  down(mutex);

  down(space);

- daca buferul e plin, producatorul se blocheaza in operatia down(space), iar mutex = 0
- consumatorul porneste si executa

  down(items);

  down(mutex)

=> Deadlock ! (mutex = 0)

- concluzie: e nevoie de mecanisme de sincronizare de nivel inalt asistate de compilator !





Monitors

- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- *Abstract data type*, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time
- Pseudocode syntax of a monitor:

```
monitor monitor-name
{
  // shared variable declarations
  procedure P1 (…) { …. }

  procedure P2 (…) { …. }

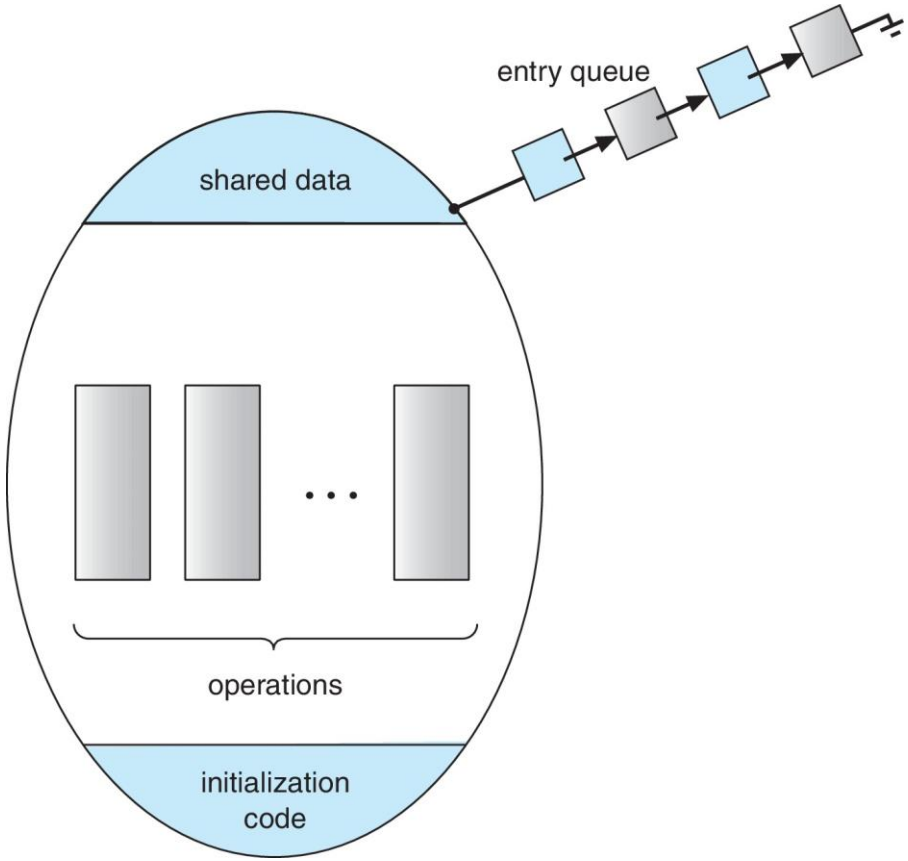
  procedure Pn (…) {……}

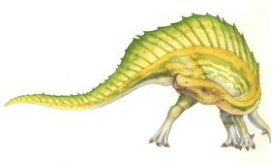
  initialization code (…) { … }
}
```





Schematic view of a Monitor







Asistenta compilatorului

- procedurile monitorului sunt instrumentate de compilator sa execute o secventa speciala de apelare
- cand un proces apeleaza o procedura din monitor
  - primele instructiuni ale procedurii verifica daca exista alt proces activ in interiorul monitorului
  - daca da, procesul e suspendat pana cand celalalt proces paraseste monitorul
  - daca nu, procesul apelant poate intra in monitor
  - implementarea se face in mod uzual folosind un semafor
- avantaj: sectiunile critice se implementeaza ca proceduri de monitor, iar compilatorul genereaza automat cod de excludere mutuala





Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex
mutex = 1
```

- Each procedure ***P*** is replaced by

```
down(mutex);
    …
    body of P;
    …
up(mutex);
```

- Mutual exclusion within a monitor is ensured





Variabile conditie

- problema: ce se intampla daca un proces aflat in interiorul monitorului **trebuie** sa se blocheze? (eg., situatia producatorului cand buferul e plin)
- solutie: **variabile conditie**
- suporta doua operatii
  - **wait** – blocheaza procesul apelant si elibereaza monitorul pt ca alte procese blocate la intrarea in monitor sa poata accesa monitorul
  - **signal** – trezeste procesul care a apelat anterior *wait* pe aceasta variabila conditie
- problema **signal** – daca trezeste un proces care apelase anterior **wait**, vor exista simultan doua procese in interiorul monitorului
- solutii
  - Hoare – procesul care executa **signal** e suspendat, si celalalt proces este lasat sa ruleze in monitor
  - Hansen – **signal** nu se poate executa decat ca ultima operatie a unei proceduri de monitor (i.e., procesul care cheama **signal** paraseste imediat monitorul dupa executia operatiei)
  - uzual, se foloseste solutia Hansen pt. simplitatea implementarii





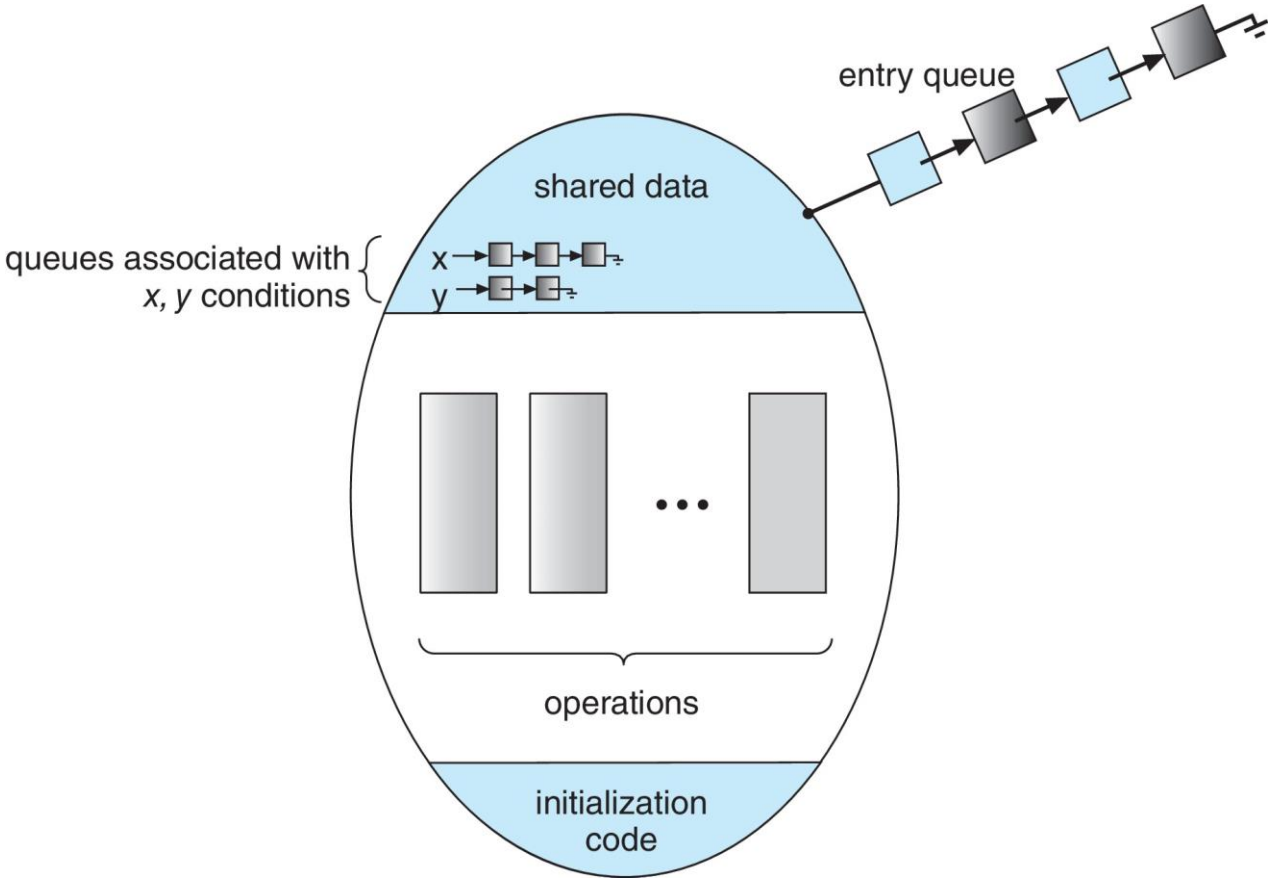
Condition Variables

- `condition x, y;`
- Two operations are allowed on a condition variable:
  - **`x.wait()`** – a process that invokes the operation is suspended until **`x.signal()`**
  - **`x.signal()`** – resumes one of processes (if any) that invoked **`x.wait()`**
    - If no **`x.wait()`** on the variable, then it has no effect on the variable





Monitor with Condition Variables







Observatii

- variabilele conditie nu contorizeaza “semnalele”, i.e. o operatie **signal** pe o variabila conditie pe care nu asteapta nimeni se pierde
  - **wait** trebuie executat inainte de **signal**
- **wait/signal** aproximeaza comportamentul **sleep/wakeup**, DAR race condition-ul **sleep/wakeup** e inlaturat de excluderea mutuala automata a accesului in monitor
- monitoarele sunt un concept de nivel de limbaj de programare, spre deosebire de semafoare care pot fi implementate si ca apeluri de biblioteca (cu suport din partea sistemului de operare, evident)
- atat semafoarele cat si monitoarele functioneaza doar pe sisteme cu memorie partajata (inclusiv multiprocesoare), NU pe sisteme distribuite





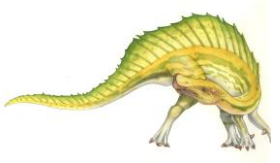
Usage of Condition Variable Example

- Consider P_1 and P_2 that that need to execute two statements S_1 and S_2 and the requirement that S_1 to happen before S_2
  - Create a monitor with two procedures F_1 and F_2 that are invoked by P_1 and P_2 respectively
  - One condition variable “x” initialized to 0
  - One Boolean variable “done”
  - **F1:**

```
S1;
done = true;
x.signal();
```

  - **F2:**

```
if done = false
   x.wait()
S2;
```





Echivalenta primitivelor de sincronizare

- event counters, sequencers, path expressions, serializers, toate sunt semantic echivalente
- exemplu practice: echivalenta monitoarelor cu semafoarele
- pp. sistemul de operare ofera semafoare
- dezvoltatorul de compilatoare scrie urmatoarele rutine

```
semaphore_t mutex = {1};
void enter_monitor(void)     { down(mutex); }
void leave_normally(void)    { up(mutex); }

void leave_with_signal(semaphore_t c)  // c = 0 initial
{   up(c); // iese cu mutex = 0, i.e. nici un alt proces nu poate intra
           // (signal e ultima instructiune din procedura)
}
void wait(semaphore_t c)   { up(mutex); down(c); }
```

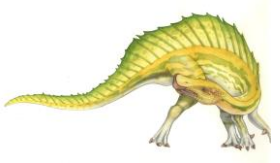




Implementare semafoare cu monitoare

- exemplu in Java

```
public class semaphore
{       unsigned int counter;
        public synchronized void down()
        {
                if(counter > 0)
                        { counter--, return; }
                wait();
        }
        public synchronized void up()
        {       counter++, notify();
        }
}
```





Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex;  // (initially  = 1)
semaphore next;   // (initially  = 0)
int next_count = 0; // number of processes waiting
                       inside the monitor
```

- Each function ***P*** will be replaced by

```
down(mutex);
    …
   body of P;
    …
if (next_count > 0)
  up(next)
else
  up(mutex);
```

- Mutual exclusion within a monitor is ensured





Implementation – Condition Variables

- For each condition variable ***x***, we have:

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- The operation **`x.wait()`** can be implemented as:

```
x_count++;
if (next_count > 0)
    up(next);
else
   up(mutex);
down(x_sem);
x_count--;
```





Implementation (Cont.)

- The operation **`x.signal()`** can be implemented as:

```
if (x_count > 0) {
   next_count++;
   up(x_sem);
   down(next);
   next_count--;
}
```



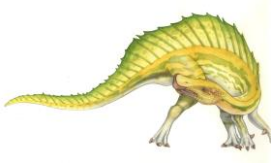


Resuming Processes within a Monitor

- If several processes queued on condition variable **`x`**, and **`x.signal()`** is executed, which process should be resumed?
- FCFS frequently not adequate
- Use the **conditional-wait** construct of the form

  `x.wait(c)`

  where:
  - **`c`** is an integer (called the priority number)
  - The process with lowest number (highest priority) is scheduled next





Single Resource allocation

- Allocate a single resource among competing processes using priority numbers that specifies the maximum time a process plans to use the resource

```
R.acquire(t);
    ...
  access the resource;
    ...

R.release;
```

- Where R is an instance of type **ResourceAllocator**





Single Resource allocation

- Allocate a single resource among competing processes using priority numbers that specifies the maximum time a process plans to use the resource
- The process with the shortest time is allocated the resource first
- Let R is an instance of type **`ResourceAllocator`** (next slide)
- Access to **`ResourceAllocator`** is done via:

```
R.acquire(t);
    ...
  access the resurce;
    ...
R.release;
```

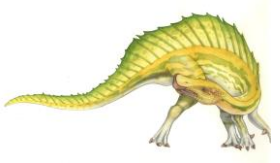
- Where **`t`** is the maximum time a process plans to use the resource





A Monitor to Allocate Single Resource

```
monitor ResourceAllocator
{
   boolean busy;
   condition x;
   void acquire(int time) {
            if (busy)
               x.wait(time);
            busy = true;
   }
   void release() {
            busy = false;
            x.signal();
   }
   initialization code() {
    busy = false;
   }
}
```



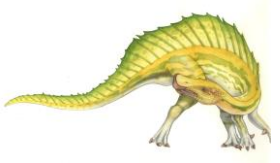


Single Resource Monitor (Cont.)

- Usage:

```
acquire
  ...
release
```

- Incorrect use of monitor operations
  - `release()` … `acquire()`
  - `acquire()` … `acquire())`
  - Omitting of `acquire()` and/or `release()`





Lock-uri adaptive

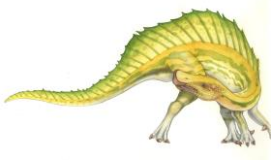
- primitive de sincronizare Solaris: lock-uri adaptive (adaptive locks), variabile conditie, semafoare, reader-writer locks, turnstiles
- lock-uri adaptive
  - comportament polimorf spinlock-semafor
  - pt sectiuni critice scurte (cateva sute de instructiuni)
    - daca un thread vrea acces la date protejate de un lock detinut de un alt thread care ruleaza pe alt CPU, lock-ul se comporta ca un spinlock
  - daca lock-ul e detinut de un thread care nu ruleaza momentan pe nici un procesor, threadul care incearca sa obtina lock-ul va fi pus in stare dormanta (sleep), i.e., lock-ul se comporta ca un semafor
  - pe un sistem uniprocesor, cand un thread incearca sa obtina un lock detinut de alt thread, comportamentul lock-ului e intotdeauna de tip semafor





Liveness

- Processes may have to wait indefinitely while trying to acquire a synchronization tool such as a mutex lock or semaphore.
- Waiting indefinitely violates the progress and bounded-waiting criteria discussed at the beginning of this chapter.
- **Liveness** refers to a set of properties that a system must satisfy to ensure processes make progress.
- Indefinite waiting is an example of a liveness failure.



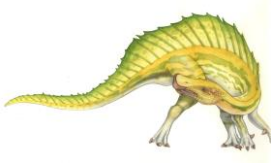


Liveness

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let ***S*** and ***Q*** be two semaphores initialized to 1

P_0	P_1
`down(S);`	`down(Q);`
`down(Q);`	`down(S);`
`...`	`...`
`up(S);`	`up(Q);`
`up(Q);`	`up(S);`

- Consider if P_0 executes down(S) and P_1 down(Q). When P_0 executes down(Q), it must wait until P_1 executes up(Q)
- However, P_1 is waiting until P_0 execute up(S).
- Since these up() operations will never be executed, P_0 and P_1 are **deadlocked**.





Conditii necesare pt. deadlock

- Coffman et al., 1971
- (1) **excludere mutuala (mutual exclusion)** – doar un proces/thread utilizeaza o resursa la un moment dat
- (2) **hold & wait** – procesul/threadul detine o resursa si asteapta sa obtina alte resurse detinute de alte procese/threaduri
- (3) **fara preemptiune (no preemption)** – resursele detinute de un proces/thread nu pot fi confiscate fortat de la un proces/thread care le detine, ci doar cedate voluntar la sfarsitul taskului
- (4) **asteptare circulara (circular wait)** – un set de procese/threaduri aflate intr-un lant de asteptare (P_i asteapta o resursa detinuta de P_{i+1})
- producerea unui deadlock presupune ca toate cele 4 conditii sa fie respectate !
- obs: (4) => (2), adica nu toate conditiile sunt independente unele de celelalte





Liveness

- Other forms of deadlock:
- **Starvation** – indefinite blocking
  - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
  - Solved via **priority-inheritance protocol**





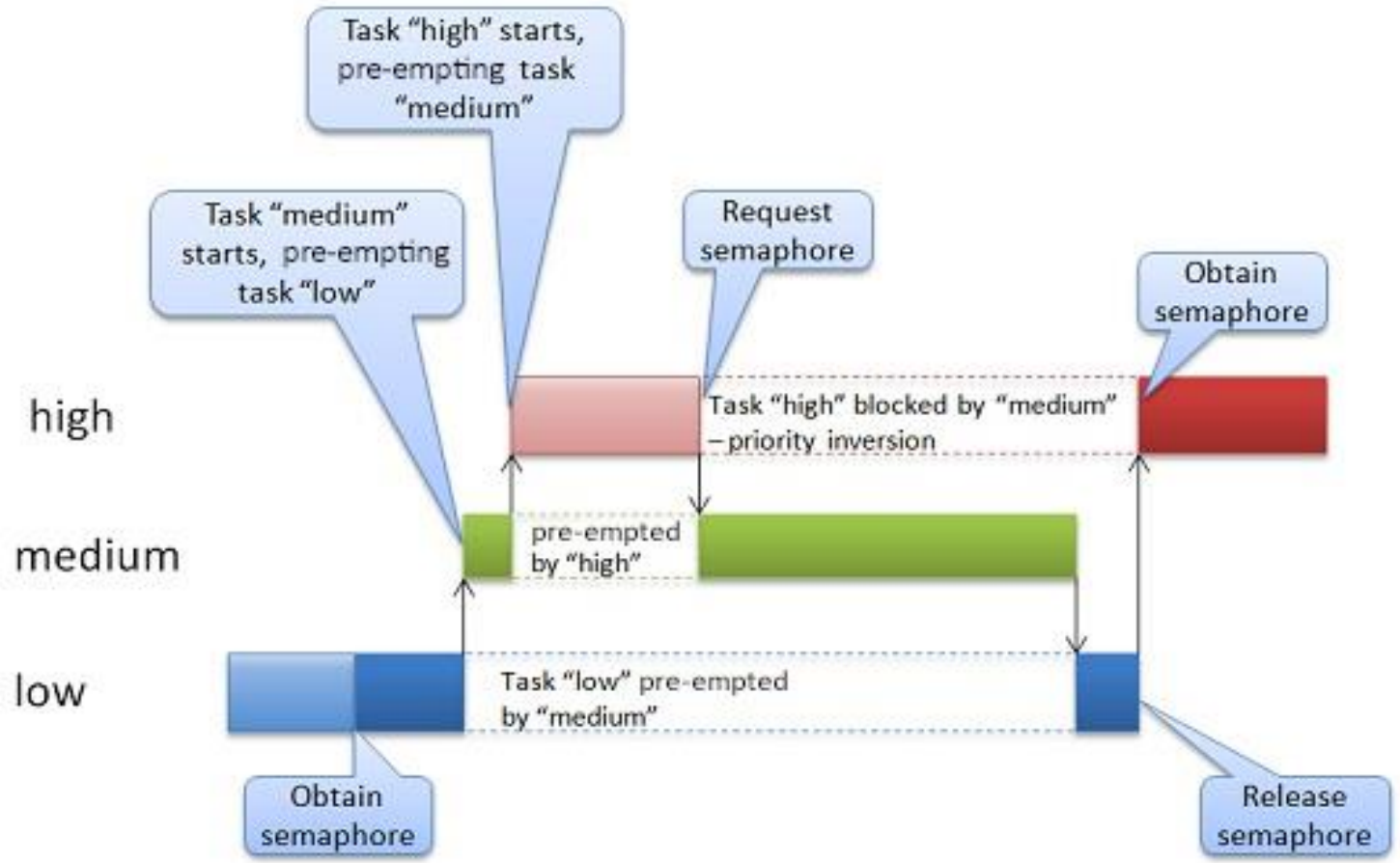
Incidentul Mars Pathfinder

- aplicatiile roverului rulau pe VxWorks (sistem de operare de timp real)
- taskurile erau planificate in functie de prioritati
  - task de mica prioritate care aduna date meteorologice, accesa magistrala folosind un mutex
  - task de gestiune a magistralei de prioritate mare, accesa magistrala folosind acelasi mutex
  - task de comunicare de lunga durata, cu prioritate medie, impiedica taskul meteorologic sa ruleze
- problema: un timer de tip watchdog reseta periodic sistemul observand ca taskul de gestiune a magistralei nu a mai rulat de mult si concluzionand ca s-a defectat





Inversiunea de prioritati





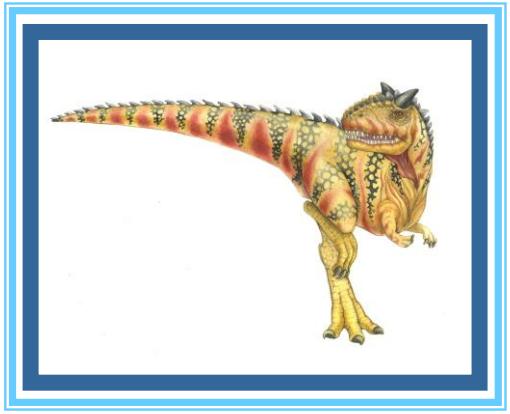


Mostenirea prioritatii

- solutia problemei: *priority inheritance*
- taskul care detine mutex-ul (semaforul) mosteneste prioritatea taskului cu prioritate mare pe durata sectiunii critice
- astfel, taskul de prioritate mica (eg., taskul meteorologic) primeste procesorul in locul taskului de prioritate medie (eg., taskul de comunicare)
- in aceste conditii, taskul de prioritate mica care si-a marit temporar prioritatea
  - termina sectiunea critica fara sa piarda procesorul
  - elibereaza mutex-ul (resursa partajata, eg., magistrala)
  - revine la prioritatea mica

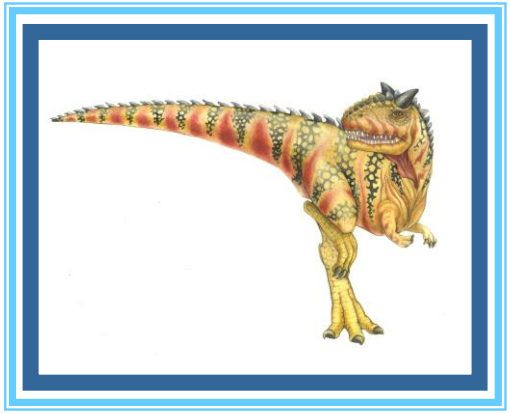


End of Chapter 6



Silberschatz, Galvin and Gagne ©2018

Chapter 7: Synchronization Examples



Silberschatz, Galvin and Gagne ©2018



Outline

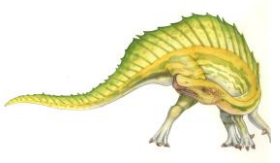
- Explain the bounded-buffer synchronization problem
- Explain the readers-writers synchronization problem
- Explain and dining-philosophers synchronization problems
- Describe the tools used by Linux and Windows to solve synchronization problems.
- Illustrate how POSIX and Java can be used to solve process synchronization problems





Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
  - Bounded-Buffer Problem
  - Readers and Writers Problem
  - Dining-Philosophers Problem





Readers-Writers Problem

- A data set is shared among a number of concurrent processes
  - **Readers** – only read the data set; they do ***not*** perform any updates
  - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
  - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities





Readers-Writers Problem (Cont.)

- Shared Data
  - Data set
  - Semaphore **`rw_mutex`** initialized to 1
  - Semaphore **`mutex`** initialized to 1
  - Integer **`read_count`** initialized to 0

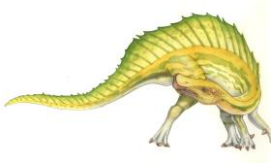




Readers-Writers Problem (Cont.)

- The structure of a writer process

```
while (true) {
   down(rw_mutex);
         ...
      /* writing is performed */
         ...
    up(rw_mutex);
}
```





Readers-Writers Problem (Cont.)

- The structure of a reader process

```
while (true){
        down(mutex);
        read_count++;
        if (read_count == 1) /* first reader */
             down(rw_mutex);
        up(mutex);
        ...
        /* reading is performed */
        ...
        down(mutex);
        read_count--;
        if (read_count == 0) /* last reader */
             up(rw_mutex);
        up(mutex);
}
```





Readers-Writers Problem Variations

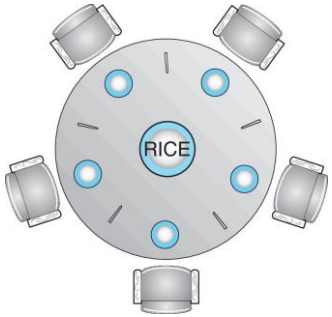
- The solution in previous slide can result in a situation where a writer process never writes. It is referred to as the “First reader-writer” problem.
- The “Second reader-writer” problem is a variation the first reader-writer problem that state:
  - Once a writer is ready to write, no “newly arrived reader” is allowed to read.
- Both the first and second may result in starvation, leading to even more variations
- Problem is solved on some systems by kernel providing reader-writer locks





Dining-Philosophers Problem

- N philosophers' sit at a round table with a bowel of rice in the middle.



- They spend their lives alternating thinking and eating.
- They do not interact with their neighbors.
- Occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers, the shared data
  - Bowl of rice (data set)
  - Semaphore chopstick [5] initialized to 1





Dining-Philosophers Problem Algorithm

- Semaphore Solution
- The structure of Philosopher i :

```
while (true){
    down (chopstick[i] );
    down (chopStick[ (i + 1) % 5] );

     /* eat for awhile */

    up (chopstick[i] );
    up (chopstick[ (i + 1) % 5] );

     /* think for awhile */

}
```

- What is the problem with this algorithm?





Solutie corecta cu semafoare

```
#define THINKING        0
#define HUNGRY          1
#define EATING          2
int state[N];
semaphore_t   mutex = {1};
semaphore_t   ph[N];   // toate initializate cu 0
void philosopher(int i) {
        while(true) {
                think();
                take_forks(i);
                eat();
                put_forks(i);
        }
}
```





Solutie corecta cu semafoare

```
void take_forks(int i) {
        down(mutex);
        state[i] = HUNGRY;
        test(i);
        up(mutex);
        down(ph[i]);
}
void put_forks(int i) {
        down(mutex);
        state[i] = THINKING;
        test((i – 1) % N));
        test((i + 1) % N));
        up(mutex);
}
```

```
void test(int i) {
        if(state[i] == HUNGRY &&
           state[(i-1)%N] != EATING) &&
           state[(i+1)%N] != EATING)
        {
                state[i] = EATING;
                up(ph[i]);
        }
}
```





Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
   enum {THINKING; HUNGRY, EATING} state [5];
   condition self [5];

   void pickup (int i) {
          state[i] = HUNGRY;
          test(i);
          if (state[i] != EATING) self[i].wait;
   }

   void putdown (int i) {
          state[i] = THINKING;
                  // test left and right neighbors
          test((i + 4) % 5);
          test((i + 1) % 5);
   }
```

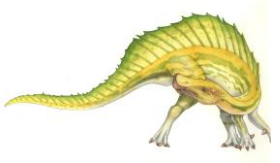




Solution to Dining Philosophers (Cont.)

```
    void test (int i) {
            if ((state[(i + 4) % 5] != EATING) &&
            (state[i] == HUNGRY) &&
            (state[(i + 1) % 5] != EATING) ) {
                 state[i] = EATING ;
                 self[i].signal () ;
            }
    }

        initialization_code() {
           for (int i = 0; i < 5; i++)
           state[i] = THINKING;
         }
}
```





Solution to Dining Philosophers (Cont.)

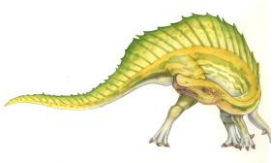
- Each philosopher "i" invokes the operations **`pickup()`** and **`putdown()`** in the following sequence:

```
DiningPhilosophers.pickup(i);

/** EAT **/

DiningPhilosophers.putdown(i);
```

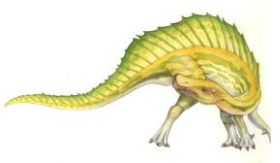
- No deadlock, but starvation is possible





POSIX Semaphores

- POSIX provides two versions – **named** and **unnamed**.
- Named semaphores can be used by unrelated processes, unnamed cannot.





POSIX Named Semaphores

- Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- Another process can access the semaphore by referring to its name **SEM**.
- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```





POSIX Unnamed Semaphores

- Creating an initializing the semaphore:

```
#include <semaphore.h>
sem_t sem;

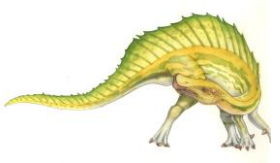
/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- Acquiring and releasing the semaphore:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```





POSIX Condition Variables

- Since POSIX is typically used in C/C++ and these languages do not provide a monitor, POSIX condition variables are associated with a POSIX mutex lock to provide mutual exclusion: Creating and initializing the condition variable:

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;

pthread_mutex_init(&mutex,NULL);
pthread_cond_init(&cond_var,NULL);
```





POSIX Condition Variables

- Thread waiting for the condition **`a == b`** to become true:

```
pthread_mutex_lock(&mutex);
while (a != b)
     pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

- Thread signaling another thread waiting on the condition variable:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```





Producator-consumator POSIX

```c
#include <stdio.h>
#include <pthread.h>

#define ITEMS           8
#define BUFFER_SIZE     4
int buffer[BUFFER_SIZE];

void *producer(void*);
void *consumer(void*);

int spaces, items, tail;
pthread_cond_t space, item;
pthread_mutex_t buffer_mutex;

int main()
{
        pthread_t producer_thread, consumer_thread;
        void *thread_return;
        int result;

        spaces = BUFFER_SIZE;
        items = 0;
        tail = 0;

        pthread_mutex_init(&buffer_mutex, NULL);
        pthread_cond_init(&space, NULL);
        pthread_cond_init(&item, NULL);

        if(pthread_create(&producer_thread, NULL, producer, NULL) ||
           pthread_create(&consumer_thread, NULL, consumer, NULL))
                exit(1);

        if(pthread_join(producer_thread, &thread_return))
                exit(1);
        else
                printf("producer returns with %d\n", (int)thread_return);

        if(pthread_join(consumer_thread, &thread_return))
                exit(1);
        else
                printf("consumer returns with %d\n", (int)thread_return);
        exit(0);
}

```





Producator-consumator POSIX

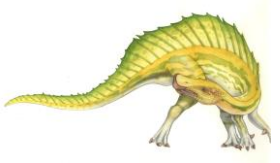
```
1  void *producer(void *arg)
2  {
3          int i;
4
5          for(i = 0; i < ITEMS; i++)
6          {
7                  pthread_mutex_lock(&buffer_mutex);
8                  while(spaces == 0)
9                          pthread_cond_wait(&space, &buffer_mutex);
10                 buffer[(tail + items) % BUFFER_SIZE] = i;
11                 items += 1;
12                 spaces -= 1;
13                 printf("producer puts %d\n", i);
14                 pthread_mutex_unlock(&buffer_mutex);
15                 pthread_cond_signal(&item);
16         }
17         pthread_exit(0);
18         return (void*)0;
19 }
20
21 void *consumer(void *arg)
22 {
23         int i, b;
24
25         for(i = 0; i < ITEMS; i++)
26         {
27                 pthread_mutex_lock(&buffer_mutex);
28                 while(items == 0)
29                         pthread_cond_wait(&item, &buffer_mutex);
30                 b = buffer[tail % BUFFER_SIZE];
31                 tail += 1;
32                 items -= 1;
33                 spaces += 1;
34                 printf("consumer gets %d\n", b);
35                 pthread_mutex_unlock(&buffer_mutex);
36                 pthread_cond_signal(&space);
37         }
38         pthread_exit(0);
39         return (void*)0;
40 }
```





Kernel Synchronization - Windows

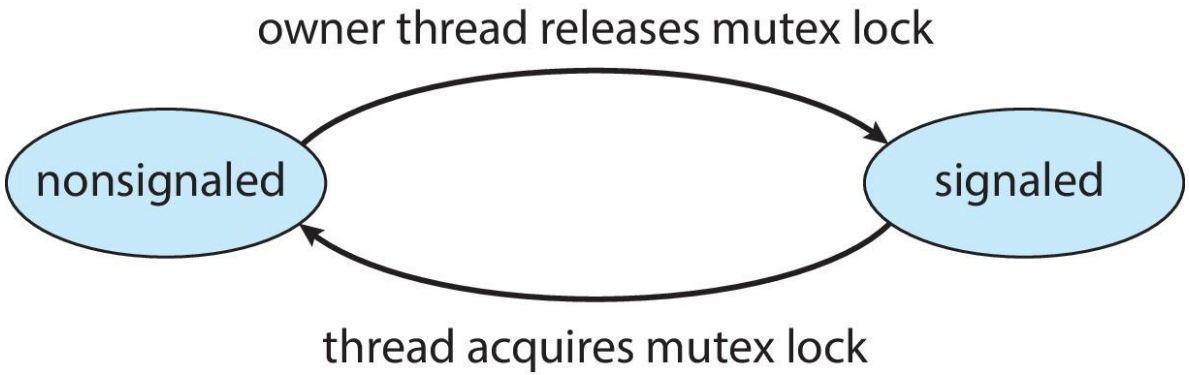
- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
  - Spinlocking-thread will never be preempted
- Also provides **dispatcher objects** user-land which may act mutexes, semaphores, events, and timers
  - **Events**
    - An event acts much like a condition variable
  - Timers notify one or more thread when time expired
  - Dispatcher objects either **signaled-state** (object available) or **non-signaled state** (thread will block)





Kernel Synchronization - Windows

- Mutex dispatcher object







Linux Synchronization

- Linux:
  - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
  - Version 2.6 and later, fully preemptive
- Linux provides:
  - Semaphores
  - Atomic integers
  - Spinlocks
  - Reader-writer versions of both
- On single-CPU system, spinlocks replaced by enabling and disabling kernel preemption





Linux Synchronization

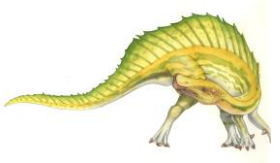
- Atomic variables

  `atomic_t` is the type for atomic integer

- Consider the variables

```
atomic_t counter;
int value;
```

Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`





Sincronizare in Solaris

- lock-uri adaptive (adaptive locks), variabile de conditie, semafoare, reader-writer locks, turnstiles
- lock-uri adaptive
  - folosite pt sectiuni critice scurte (cateva sute de instructiuni)
  - comportament polimorf spinlock-semafor
  - daca un thread vrea acces la date protejate de un lock detinut de un alt thread care ruleaza pe alt CPU, lock-ul se comporta ca un spinlock
  - daca lock-ul e detinut de un thread care nu ruleaza momentan pe nici un procesor, threadul care incearca sa obtina lock-ul va fi pus in stare dormanta (sleep), i.e., lock-ul se comporta ca un semafor
  - pe un sistem uniprocesor, cand un thread incearca sa obtina un lock detinut de alt thread, comportamentul lock-ului e intotdeauna de tip semafor





POSIX Synchronization

- POSIX API provides
  - mutex locks
  - semaphores
  - condition variable
- Widely used on UNIX, Linux, and macOS





POSIX Mutex Locks

- Creating and initializing the lock

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

- Acquiring and releasing the lock

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```





Transactional Memory

- Consider a function update() that must be called atomically. One option is to use mutex locks:

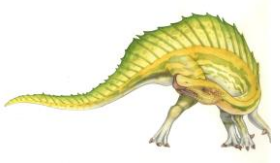
```
void update ()
{
   acquire();

   /* modify shared data */

   release();
}
```

- A **memory transaction** is a sequence of read-write operations to memory that are performed atomically. A transaction can be completed by adding **`atomic{S}`** which ensure statements in **`S`** are executed atomically:

```
void update ()
{
   atomic {
      /* modify shared data */
   }
}
```





Memorie tranzactionala

- software transactional memory, Herlihy & Moss 1993
- am vazut deja ex. LL/SC
- exploateaza ideea de tranzactii din baze de date (v. proprietati ACID) si controlul optimist al concurentei
- un thread termina modificarile operate pe o zona de memorie partajata fara a se coordona cu alte threaduri
- se inregistreaza fiecare operatie de citire/scriere intr-un log
- la finalul tranzactiei se incearca operatia de *commit*
  - daca reuseste (i.e., nici o alta tranzactie nu a apucat sa faca un commit reusit intre timp), tranzactia e validata si modificarile devin permanente
  - daca esueaza, tranzactia e abandonata (aborted) si se re-executa de la inceput pana reuseste (*transaction roll-back*)
- beneficiu major: grad crescut de concurenta, thread-urile nu au nevoie sa se sincronizeze prin lock-uri la accesul memoriei partajate





Suport HW pt. memorie tranzactionala

- ex. Intel TSX (Transactional Synchronization Extension), extensie ISA x86, microarhitectura Haswell (2013)
- accesibila prin intermediul a doua interfete
  - HLE (Hardware Lock Elision) – backward compatibility
  - RTM (Restricted Transactional Memory)
- HLE adauga doua prefixuri de instr. XACQUIRE & XRELEASE
  - se pot folosi doar pt anumite instructiuni care trebuie prefixate explicit cu LOCK
  - permite executia optimista a sectiunii critice sarind scrierea lock-ului, a.i. acesta apare liber pt alte threaduri
  - o tranzactie esuata determina reluarea operatiei de la instructiunea prefixata cu XACQUIRE





Suport HW pt. memorie tranzactionala

- RTM adauga la ISA trei noi instructiuni
  - XBEGIN – marcheaza inceputul zonei de memorie tranzactionala
  - XEND - marcheaza sfarsitul zonei de memorie tranzactionala
  - XABORT – abandoneaza explicit o tranzactie
  - esecul tranzactiei redirecteaza executia catre codul specificat de instr. XBEGIN, iar codul de eroare e stocat in registrul EAX
- instructiunea XTEST permite testarea starii procesorului (este sau nu in mijlocul executiei unei regiuni de memorie tranzactionala)





RTM elision pt. pthread_mutex_lock

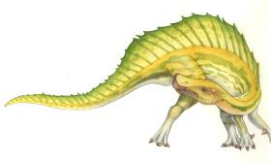
```
void elided_lock_wrapper(lock) {
        if(_xbegin() == _XBEGIN_STARTED) { // start tranzactie
                if(lock  e liber)
                        return;    // executa SC in tranzactie
                _xabort(0xff);       // abandoneaza tranzactia
        }
        obtine lock
}
void elided_unlock_wrapper(lock) {
        if (lock e liber)
                _xend();             // comite tranzactia
        else
                elibereaza lock;
}
```





Java Synchronization

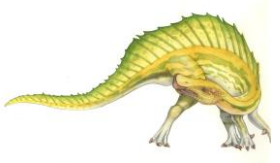
- Java provides rich set of synchronization features:
  - Java monitors
  - Reentrant locks
  - Semaphores
  - Condition variables





Java Monitors

- Every Java object has associated with it a single lock.
- If a method is declared as **`synchronized`**, a calling thread must own the lock for the object.
- If the lock is owned by another thread, the calling thread must wait for the lock until it is released.
- Locks are released when the owning thread exits the **`synchronized`** method.





Bounded Buffer – Java Synchronization

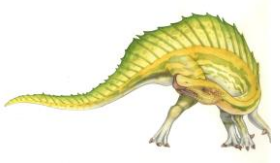
```
public class BoundedBuffer<E>
{
   private static final int BUFFER_SIZE = 5;

   private int count, in, out;
   private E[] buffer;

   public BoundedBuffer() {
      count = 0;
      in = 0;
      out = 0;
      buffer = (E[]) new Object[BUFFER_SIZE];
   }

   /* Producers call this method */
   public synchronized void insert(E item) {
      /* See Figure 7.11 */
   }

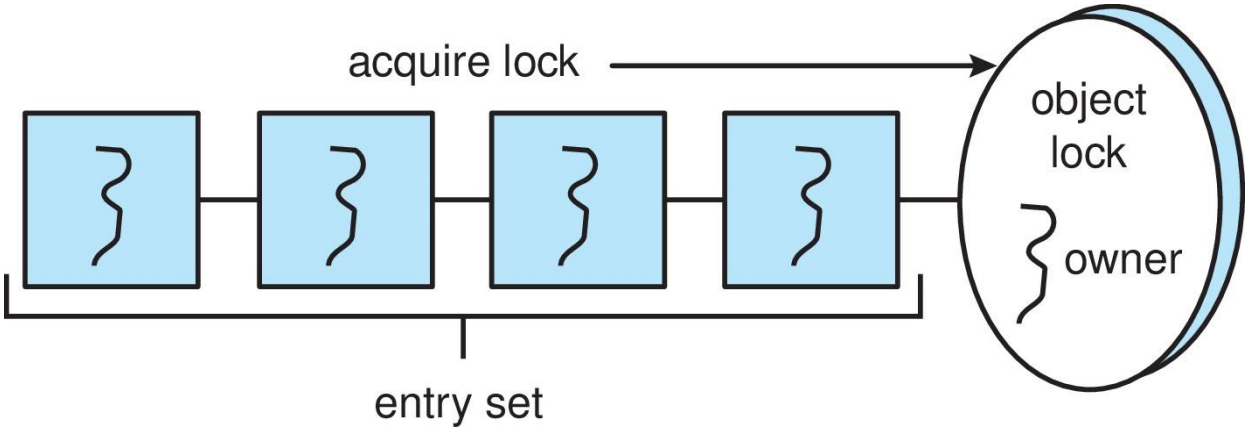
   /* Consumers call this method */
   public synchronized E remove() {
      /* See Figure 7.11 */
   }
}
```





Java Synchronization

- A thread that tries to acquire an unavailable lock is placed in the object's **entry set**:

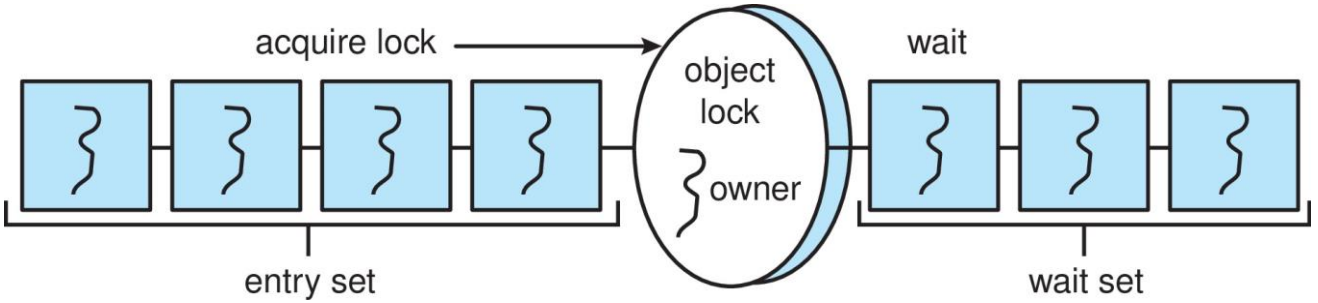






Java Synchronization

- Similarly, each object also has a **wait set**.
- When a thread calls **`wait()`**:
  1. It releases the lock for the object
  2. The state of the thread is set to blocked
  3. The thread is placed in the wait set for the object

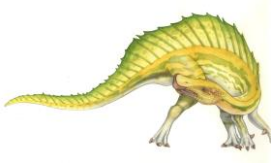






Java Synchronization

- A thread typically calls wait() when it is waiting for a condition to become true.
- How does a thread get notified?
- When a thread calls **`notify()`**:
  1. An arbitrary thread T is selected from the wait set
  2. T is moved from the wait set to the entry set
  3. Set the state of T from blocked to runnable.
- T can now compete for the lock to check if the condition it was waiting for is now true.



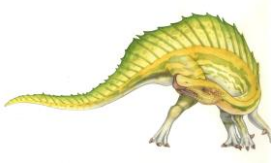


Bounded Buffer – Java Synchronization

```
/* Producers call this method */
public synchronized void insert(E item) {
   while (count == BUFFER_SIZE) {
      try {
         wait();
      }
      catch (InterruptedException ie) { }
   }

   buffer[in] = item;
   in = (in + 1) % BUFFER_SIZE;
   count++;

   notify();
}
```





Bounded Buffer – Java Synchronization

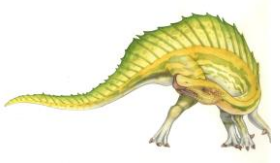
```
/* Consumers call this method */
public synchronized E remove() {
   E item;

   while (count == 0) {
      try {
         wait();
      }
      catch (InterruptedException ie) { }
   }

   item = buffer[out];
   out = (out + 1) % BUFFER_SIZE;
   count--;

   notify();

   return item;
}
```



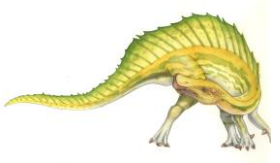


Java Reentrant Locks

- Similar to mutex locks
- The **`finally`** clause ensures the lock will be released in case an exception occurs in the **`try`** block.

```
Lock key = new ReentrantLock();

key.lock();
try {
   /* critical section */
}
finally {
   key.unlock();
}
```





Java Semaphores

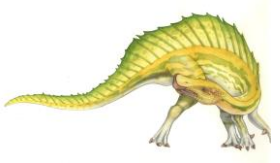
- Constructor:

```
Semaphore(int value);
```

- Usage:

```
Semaphore sem = new Semaphore(1);

try {
   sem.acquire();
   /* critical section */
}
catch (InterruptedException ie) { }
finally {
   sem.release();
}
```



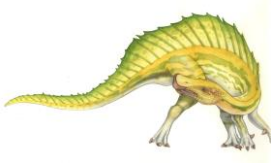


Java Condition Variables

- Condition variables are associated with an **`ReentrantLock`**.
- Creating a condition variable using **`newCondition()`** method of **`ReentrantLock`**:

```
Lock key = new ReentrantLock();
Condition condVar = key.newCondition();
```

- A thread waits by calling the **`await()`** method, and signals by calling the **`signal()`** method.



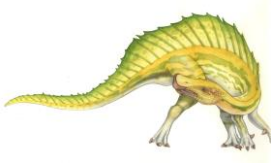


Java Condition Variables

- Example:
- Five threads numbered 0 .. 4
- Shared variable **`turn`** indicating which thread's turn it is.
- Thread calls **`doWork()`** when it wishes to do some work. (But it may only do work if it is their turn.
- If not their turn, wait
- If their turn, do some work for awhile ……
- When completed, notify the thread whose turn is next.
- Necessary data structures:

```
Lock lock = new ReentrantLock();
Condition[] condVars = new Condition[5];

for (int i = 0; i < 5; i++)
   condVars[i] = lock.newCondition();
```





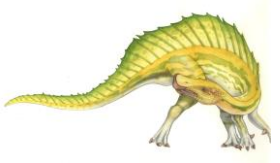
Java Condition Variables

```
/* threadNumber is the thread that wishes to do some work */
public void doWork(int threadNumber)
{
   lock.lock();

   try {
      /**
       * If it's not my turn, then wait
       * until I'm signaled.
       */
      if (threadNumber != turn)
         condVars[threadNumber].await();

      /**
       * Do some work for awhile ...
       */

      /**
       * Now signal to the next thread.
       */
      turn = (turn + 1) % 5;
      condVars[turn].signal();
   }
   catch (InterruptedException ie) { }
   finally {
      lock.unlock();
   }
}
```





Alternative Approaches

- Transactional Memory
- OpenMP
- Functional Programming Languages



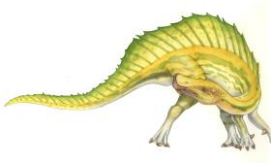


OpenMP

- OpenMP is a set of compiler directives and API that support parallel progamming.

```
void update(int value)
 {
      #pragma omp critical
      {
           count += value
      }
  }
```

- The code contained within the **`#pragma omp critical`** directive is treated as a critical section and performed atomically.



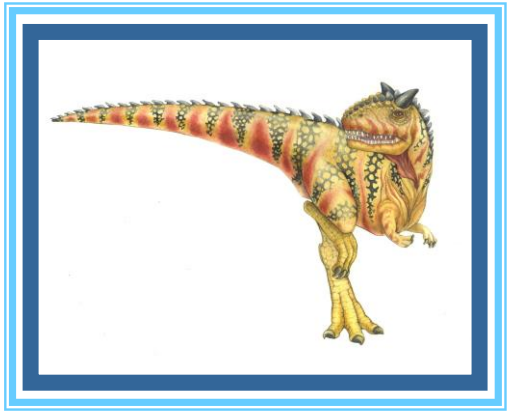


Functional Programming Languages

- Functional programming languages offer a different paradigm than procedural languages in that they do not maintain state.
- Variables are treated as immutable and cannot change state once they have been assigned a value.
- There is increasing interest in functional languages such as Erlang and Scala for their approach in handling data races.

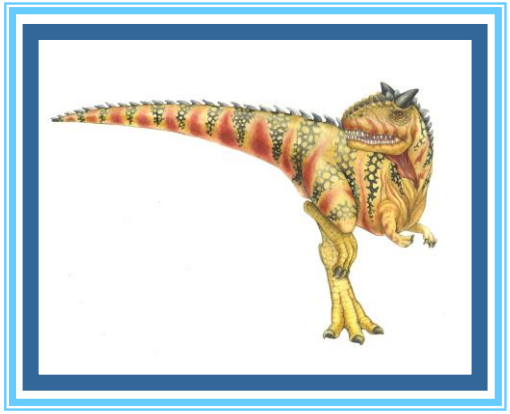


End of Chapter 7



Silberschatz, Galvin and Gagne ©2018

Chapter 5: CPU Scheduling



Silberschatz, Galvin and Gagne ©2018



Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Operating Systems Examples
- Algorithm Evaluation





Objectives

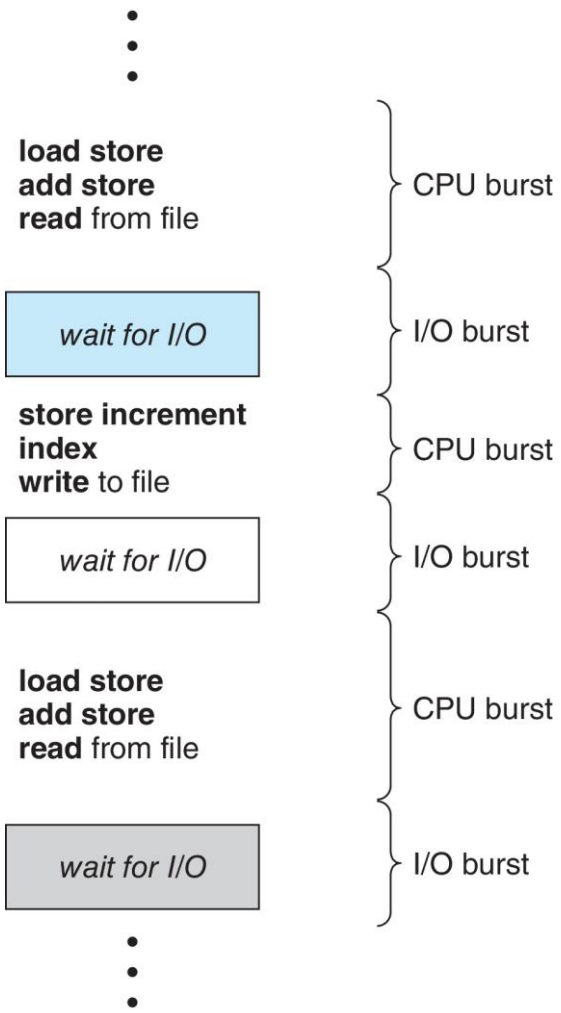
- Describe various CPU scheduling algorithms
- Assess CPU scheduling algorithms based on scheduling criteria
- Explain the issues related to multiprocessor and multicore scheduling
- Describe various real-time scheduling algorithms
- Describe the scheduling algorithms used in the Windows, Linux, and Solaris operating systems
- Apply modeling and simulations to evaluate CPU scheduling algorithms





Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



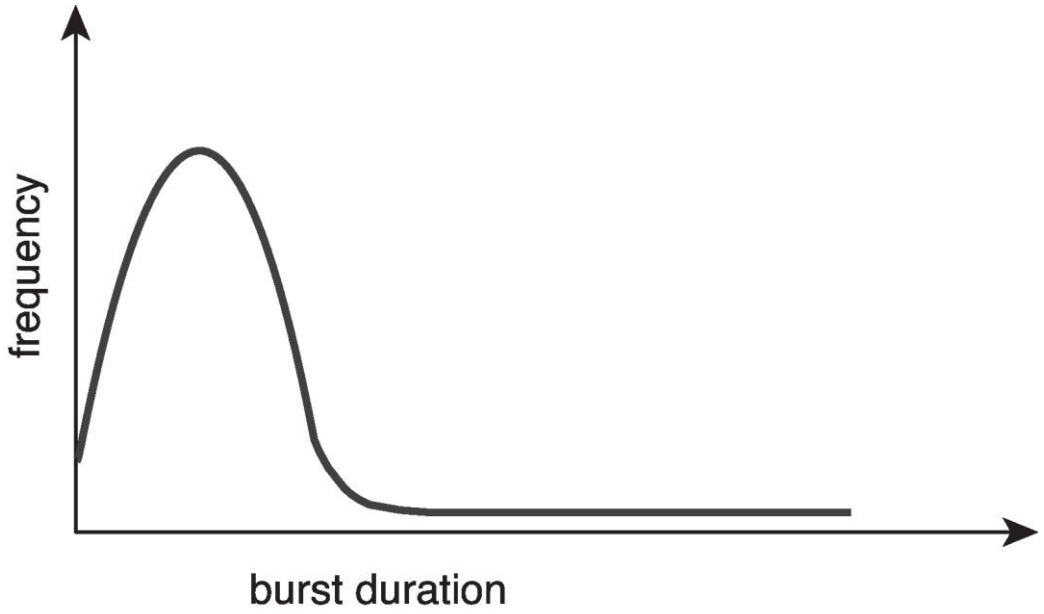


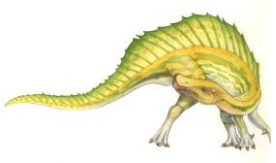


Histogram of CPU-burst Times

Large number of short bursts

Small number of longer bursts

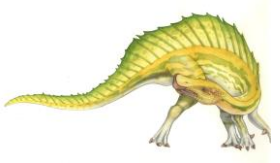






CPU Scheduler

- The **CPU scheduler** selects from among the processes in ready queue, and allocates a CPU core to one of them
  - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates
- For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution.
- For situations 2 and 3, however, there is a choice.





Preemptive and Nonpreemptive Scheduling

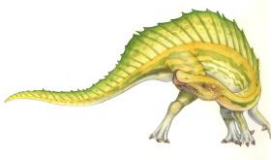
- When scheduling takes place only under circumstances 1 and 4, the scheduling scheme is **nonpreemptive**.
- Otherwise, it is **preemptive**.
- Under Nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state.
- Virtually all modern operating systems including Windows, MacOS, Linux, and UNIX use preemptive scheduling algorithms.





Preemptive Scheduling and Race Conditions

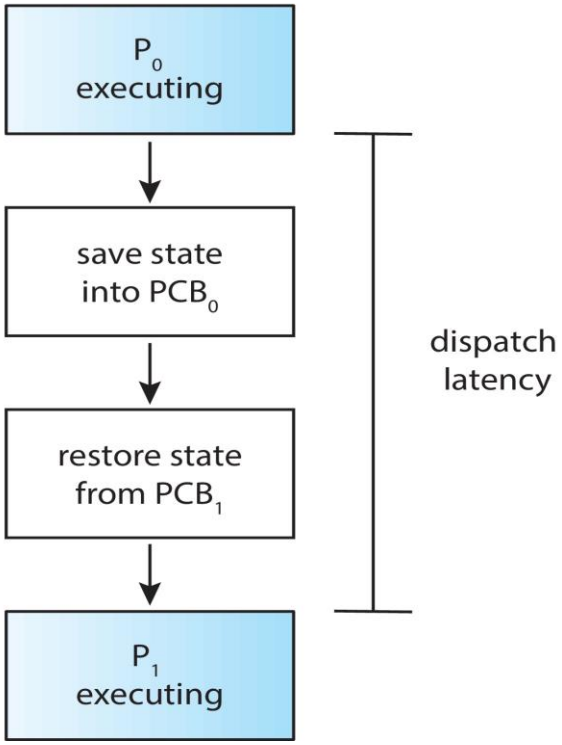
- Preemptive scheduling can result in race conditions when data are shared among several processes.
- Consider the case of two processes that share data. While one process is updating the data, it is preempted so that the second process can run. The second process then tries to read the data, which are in an inconsistent state.
- This issue will be explored in detail in Chapter 6.

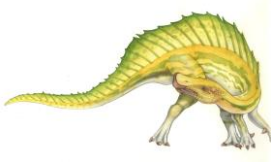




Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the CPU scheduler; this involves:
  - Switching context
  - Switching to user mode
  - Jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running







Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced.





Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time





Scheduling Algorithm Optimization Criteria

- in general, se optimizeaza media
- alegeri alternative:
  - min/max
  - varianta (dispersie)
- exemple:
  - minimizarea timpului de raspuns maxim
  - minimizarea variantei timpului de raspuns in sisteme interactive (desktop)
    - favorizeaza predictibilitatea raspunsului vs. raspuns rapid pe medie dar cu variabilitate mare





First- Come, First-Served (FCFS) Scheduling

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
  The Gantt Chart for the schedule is:



- Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17





FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2 , P_3 , P_1$$

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6; P_2 = 0; P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- Much better than previous case
- **Convoy effect** - short process behind long process
  - Consider one CPU-bound and many I/O-bound processes

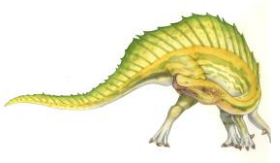




Shortest-Job-First (SJF) Scheduling

- initial folosit pt sisteme batch, cand timpii de rulare sunt cunoscuti a priori
- se alege cel mai scurt job primul
- SJF e optimal
  - Pp t_1, t_2, … t_n timpii de rulare
  - Job 1 termina la t_1
  - Job 2 termina la $t_1 + t_2$
  - …
  - Job n termina la $t_1 + t_2 + \ldots + t_n$

  $\Rightarrow$ timp mediu de asteptare = $\frac{1}{n}$(n t_1+ (n-1) t_2 + … t_n)
- contributia lui t_1 e cea mai mare => valoarea lui t_1 trebuie sa fie cea mai mica pt a obtine valoarea minima a timpului mediu de asteptare
- similar, t_2 e urmatorul cel mai mic “contribuitor”, samd

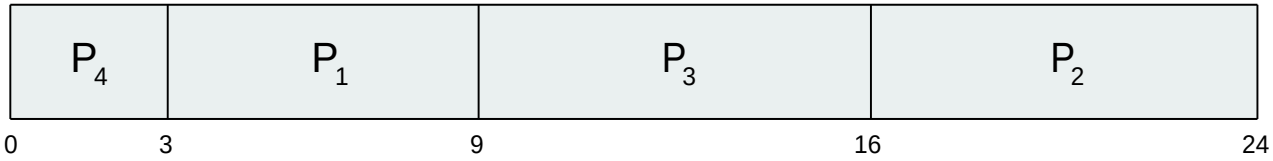




Example of SJF

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart



- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7
- Average waiting time FCFS = 10.25





Shortest-Job-First (SJF) Scheduling

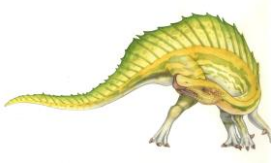
- Obs: *SJF e optimal doar daca joburile sunt disponibile simultan !*
- contraexemplu:
  - 5 joburi A, B, C, D, E cu duratele 2,4,1,1,1 si timpi de sosire 0,0,3,3,3
  - SJF impune ordinea A, B, C, D, E => TA mediu = 4.6
  - in schimb, o planificare B, C, D, E, A => TA mediu = 4.4





Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst
  - Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
- Preemptive version called **shortest-remaining-time-first**
- How do we determine the length of the next CPU burst?
  - Could ask the user
  - Estimate





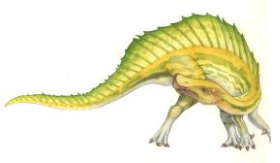
Determining Length of Next CPU Burst

- Can only estimate the length – should be similar to the previous one
  - Then pick process with shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using exponential averaging

  1. t_n = actual length of nth CPU burst
  2. τ_{n+1} = predicted value for the next CPU burst
  3. α, $0 \leq \alpha \leq 1$
  4. Define:

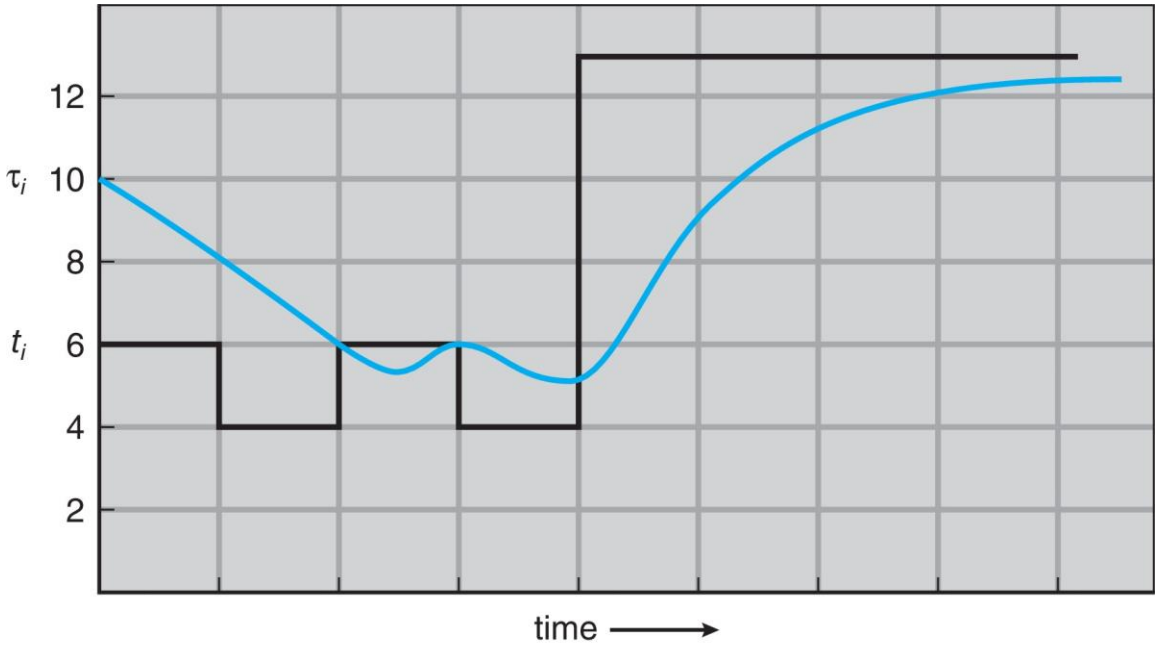
$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\tau_n.$$

- Commonly, α set to ½

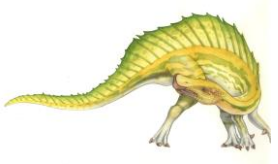




Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...





Examples of Exponential Averaging

- $\alpha = 0$
  - $\tau_{n+1} = \tau_n$
  - Recent history does not count
- $\alpha = 1$
  - $\tau_{n+1} = \alpha\, t_n$
  - Only the actual last CPU burst counts
- If we expand the formula, we get:

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\alpha\, t_{n-1} + \ldots$$
$$+ (1 - \alpha)^j \alpha\, t_{n-j} + \ldots$$
$$+ (1 - \alpha)^{n+1} \tau_0$$

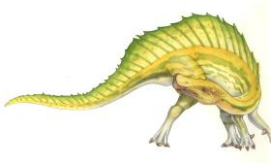
- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor





Shortest Remaining Time First Scheduling

- Preemptive version of SJN
- Whenever a new process arrives in the ready queue, the decision on which process to schedule next is redone using the SJN algorithm.
- Is SRT more “optimal” than SJN in terms of the minimum average waiting time for a given set of processes?



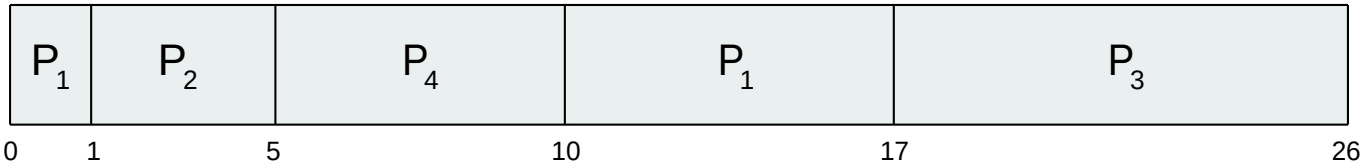


Example of Shortest-remaining-time-first

- Now we add the concepts of varying arrival times and preemption to the analysis

Process	*Arrival* Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- *Preemptive* SJF Gantt Chart



- Average waiting time = [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5
- SJF (i.d., nepreemptiv) are average waiting time = 7.75 ms





Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum** *q*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are *n* processes in the ready queue and the time quantum is *q*, then each process gets 1/*n* of the CPU time in chunks of at most *q* time units at once. No process waits more than (*n*-1)*q* time units.
- Timer interrupts every quantum to schedule next process
- Performance
  - *q* large $\Rightarrow$ FIFO (FCFS)
  - *q* small $\Rightarrow$ RR
- Note that q must be large with respect to context switch, otherwise overhead is too high





Observatii RR

- run/ready queue e implementata circular, mutarea procesului la sf. listei revine la a muta un pointer circular
- alegerea duratei cuantei de timp
  - dc. context-switch-ul dureaza mult (ex. 5 ms) fata de durata cuantei (ex. 20 ms) => 20 ms de lucru util urmat de 5 ms context switch => 20% overhead administrativ !

  $\Rightarrow$ e bine sa alegem cuanta >> durata context switch-ului

  Q: cat de mare?

  Pp. q = 500ms si 10 useri interactivi care lanseaza simultan 10 programe => primul user vede imediat o parte din rezultate, al doilea dupa ½ s, samd Al 10-lea asteapta 5 s !!!
- concluzie: - cuanta scurta implica multe context switch-uri si reduce eficienta utilizarii CPU

  - cuanta prea mare reduce timpul de raspuns pt programele interactive

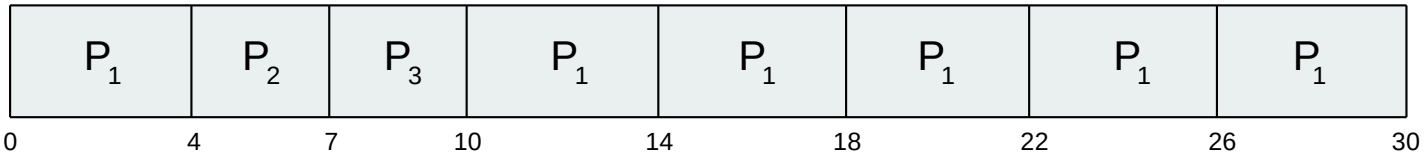




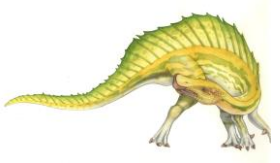
Example of RR with Time Quantum = 4

Process	Burst Time
P_1	24
P_2	3
P_3	3

- The Gantt chart is:

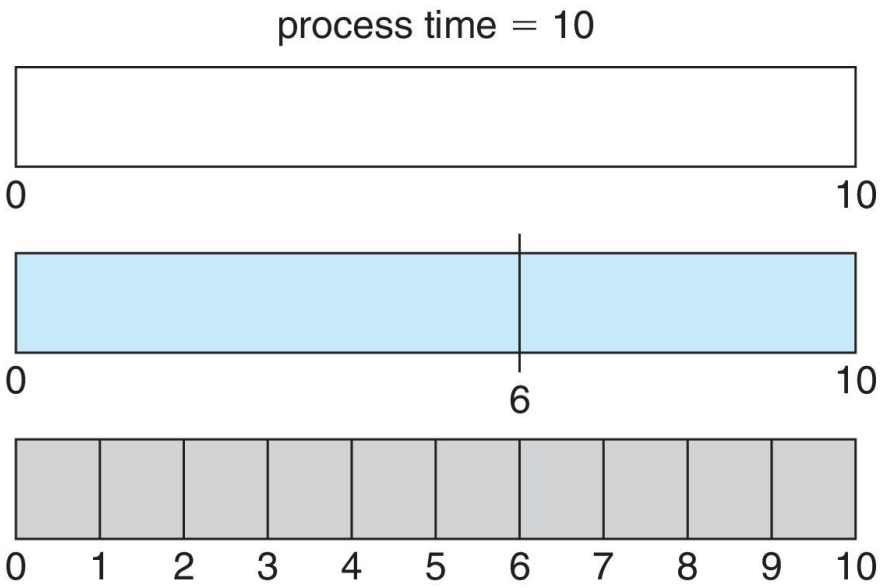


- Average waiting time = 17/3 = 5.66 ms
- Typically, higher average turnaround than SJF, but better ***response***
- q should be large compared to context switch time
  - q usually 10 milliseconds to 100 milliseconds,
  - Context switch < 10 microseconds





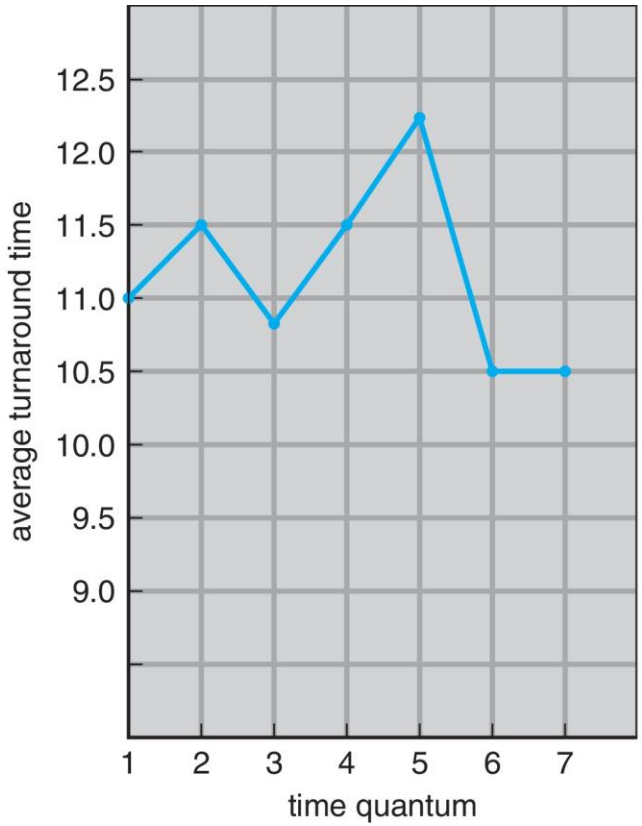
Time Quantum and Context Switch Time



quantum	context switches
12	0
6	1
1	9



Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts should be shorter than q



Planificarea cu prioritati

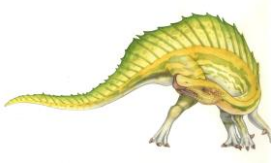
- RR pp. ca toate procesele au aceeasi importanta, nerealist
- cuantificarea “importantei” se face cu ajutorul prioritatilor
- procesul cel mai important are prioritate maxima
- planificatorul alege intotdeauna procesul cu prioritate maxima
- pentru a impiedica procesele cu prioritate maxima sa ruleze indefinit, planificatorul poate reduce prioritatea procesului care ruleaza cu fiecare tact de ceas
- cand prioritatea scade sub cea a urmatorului proces prioritar => context switch
- prioritatile se pot asigna fie static fie dinamic
- asignare dinamica:
  - procesele I/O-bound ruleaza putin, e bine sa primeasca imediat CPU pt a lansa cererea de I/O urmatoare





Planificarea cu prioritati

- asignare dinamica:
  - procesele I/O-bound ruleaza putin, e bine sa primeasca imediat CPU pt a lansa cererea de I/O urmatoare
  - algoritm:
    - procesul I/O-bound primeste prioritatea *1 / f*
    - f = fractiunea din cuanta CPU utilizata la ultima rulare de catre proces => cu cat ruleaza mai putin, cu are prioritate mai mare
- clase de prioritate + RR:
  - *n* clase de prioritate
  - se ruleaza RR procesele din clasa de prioritate maxima
  - daca nu exista procese in clasa de prioritate maxima, se trece la clasa urmatoare de prioritate
  - si aici se ajusteaza prioritatile pt a evita starvation in clasele de prioritate mica





Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority)
  - Preemptive
  - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem ≡ **Starvation** – low priority processes may never execute
- Solution ≡ **Aging** – as time progresses increase the priority of the process

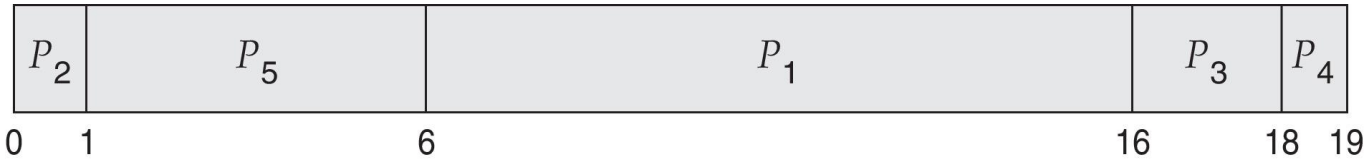




Example of Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart



- Average waiting time = 8.2





Priority Scheduling w/ Round-Robin

- Run the process with the highest priority. Processes with the same priority run round-robin
- Example:

Process	Burst Time	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

- Gantt Chart with time quantum = 2







Multilevel Queue

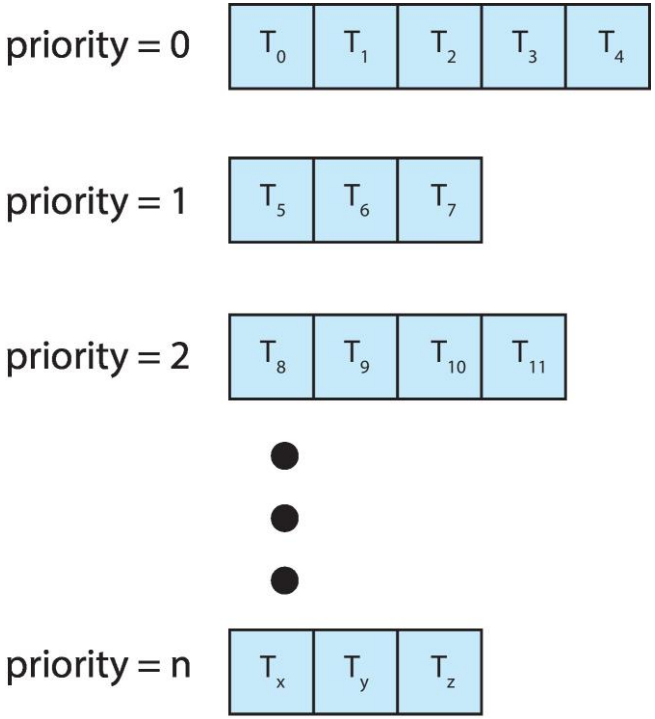
- The ready queue consists of multiple queues
- Multilevel queue scheduler defined by the following parameters:
  - Number of queues
  - Scheduling algorithms for each queue
  - Method used to determine which queue a process will enter when that process needs service
  - Scheduling among the queues





Multilevel Queue

- With priority scheduling, have separate queues for each priority.
- Schedule the process in the highest-priority queue!

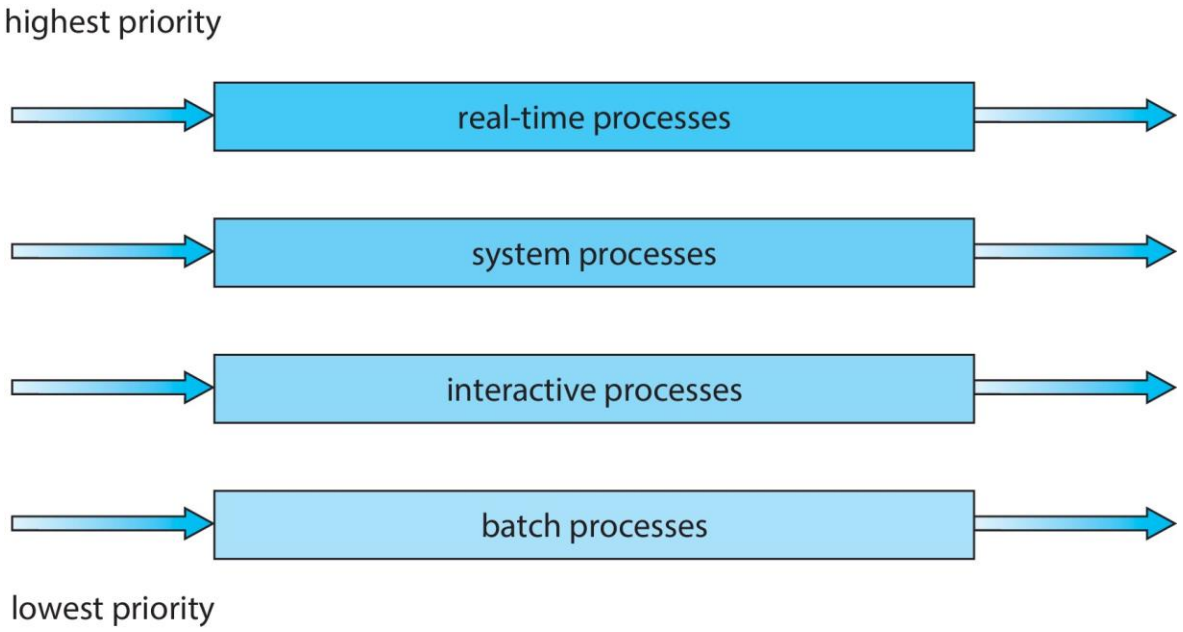






Multilevel Queue

- Prioritization based upon process type







Multilevel Feedback Queue

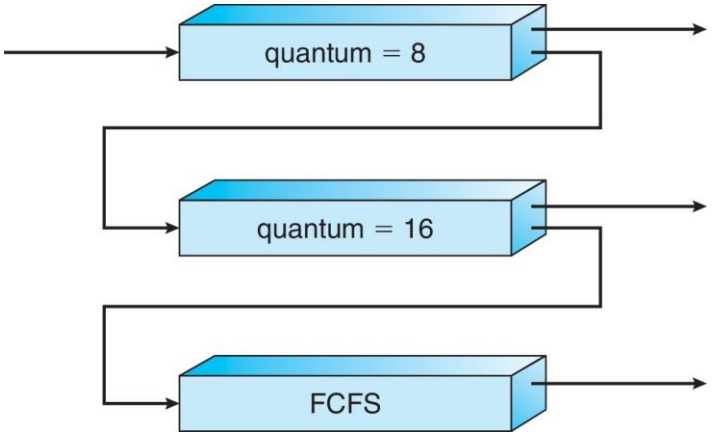
- A process can move between the various queues.
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - Number of queues
  - Scheduling algorithms for each queue
  - Method used to determine when to upgrade a process
  - Method used to determine when to demote a process
  - Method used to determine which queue a process will enter when that process needs service
- Aging can be implemented using multilevel feedback queue





Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new process enters queue Q_0 which is served in RR
    - When it gains CPU, the process receives 8 milliseconds
    - If it does not finish in 8 milliseconds, the process is moved to queue Q_1
  - At Q_1 job is again served in RR and receives 16 additional milliseconds
    - If it still does not complete, it is preempted and moved to queue Q_2

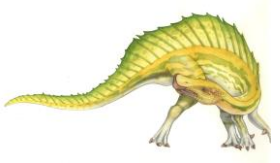






Thread Scheduling

- Distinction between user-level and kernel-level threads
- When kernel threads supported, kernel threads scheduled, not processes
- Many-to-one and many-to-many models, thread library schedules user-level threads to run on LWP
  - Known as **process-contention scope** (**PCS**) since scheduling competition is within the process
  - Typically done via priority set by programmer
- Kernel thread scheduled onto available CPU is **system-contention scope** (**SCS**) – competition among all threads in system





Pthread Scheduling

- API allows specifying either PCS or SCS during thread creation
  - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
  - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling
- Can be limited by OS – Linux and macOS only allow PTHREAD_SCOPE_SYSTEM





Pthread Scheduling API

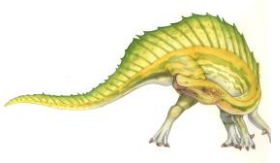
```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
   int i, scope;
   pthread_t tid[NUM THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* first inquire on the current scope */
   if (pthread_attr_getscope(&attr, &scope) != 0)
      fprintf(stderr, "Unable to get scheduling scope\n");
   else {
      if (scope == PTHREAD_SCOPE_PROCESS)
         printf("PTHREAD_SCOPE_PROCESS");
      else if (scope == PTHREAD_SCOPE_SYSTEM)
         printf("PTHREAD_SCOPE_SYSTEM");
      else
         fprintf(stderr, "Illegal scope value.\n");
   }
```





Pthread Scheduling API

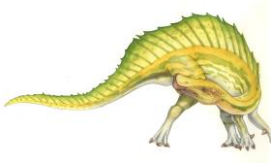
```
   /* set the scheduling algorithm to PCS or SCS */
   pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
   /* create the threads */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_create(&tid[i],&attr,runner,NULL);
   /* now join on each thread */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_join(tid[i], NULL);
}
/* Each thread will begin control in this function */
void *runner(void *param)
{
   /* do some work ... */
   pthread_exit(0);
}
```





Multiple-Processor Scheduling

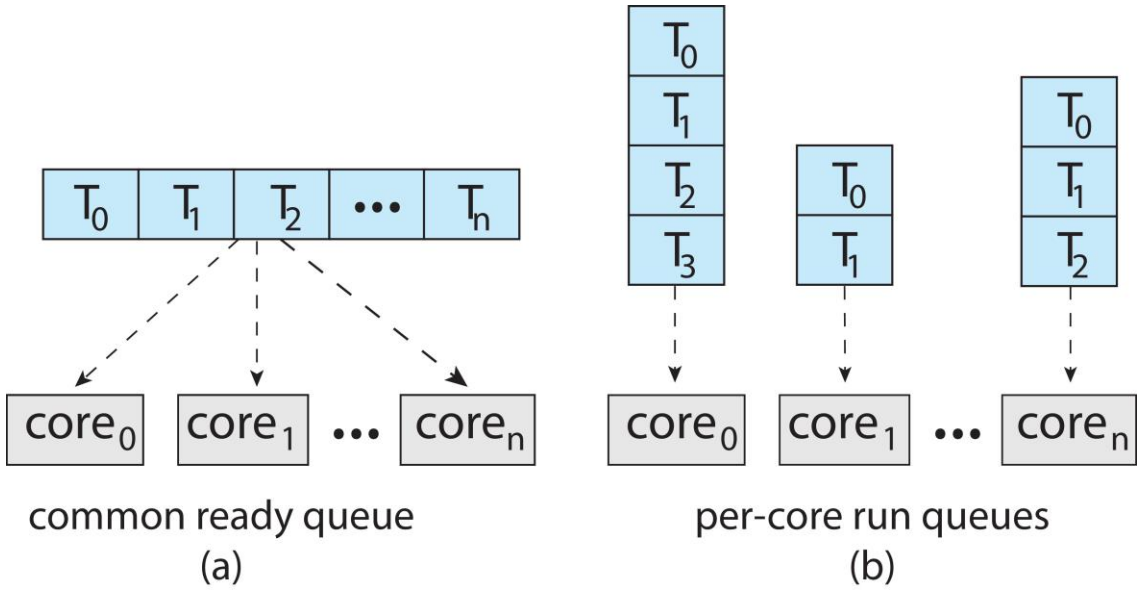
- CPU scheduling more complex when multiple CPUs are available
- Multiprocess may be any one of the following architectures:
  - Multicore CPUs
  - Multithreaded cores
  - NUMA systems
  - Heterogeneous multiprocessing





Multiple-Processor Scheduling

- Symmetric multiprocessing (SMP) is where each processor is self scheduling.
- All threads may be in a common ready queue (a)
- Each processor may have its own private queue of threads (b)

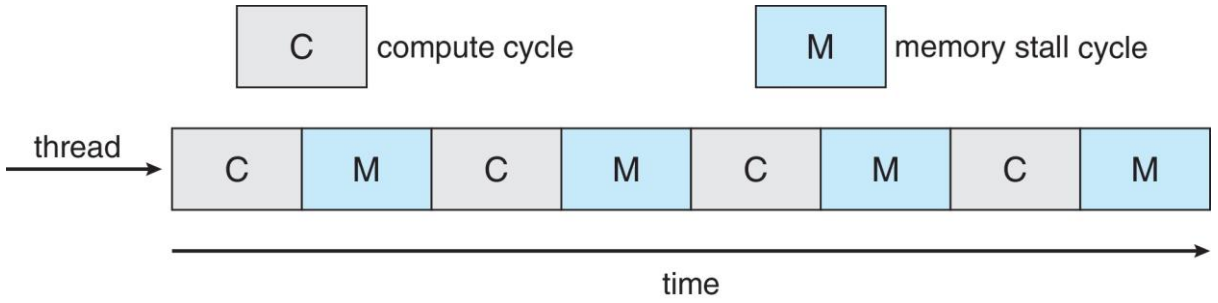






Multicore Processors

- Recent trend to place multiple processor cores on same physical chip
- Faster and consumes less power
- Multiple threads per core also growing
  - Takes advantage of memory stall to make progress on another thread while memory retrieve happens
- Figure

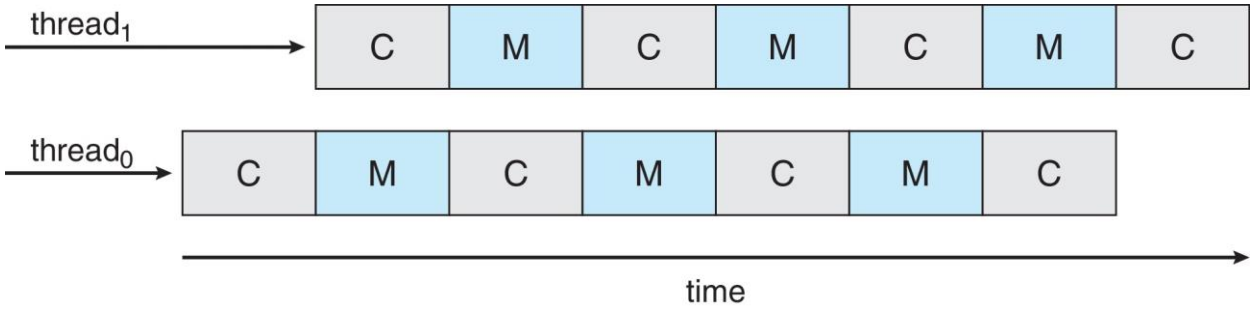






Multithreaded Multicore System

- Each core has > 1 hardware threads.
- If one thread has a memory stall, switch to another thread!
- Figure



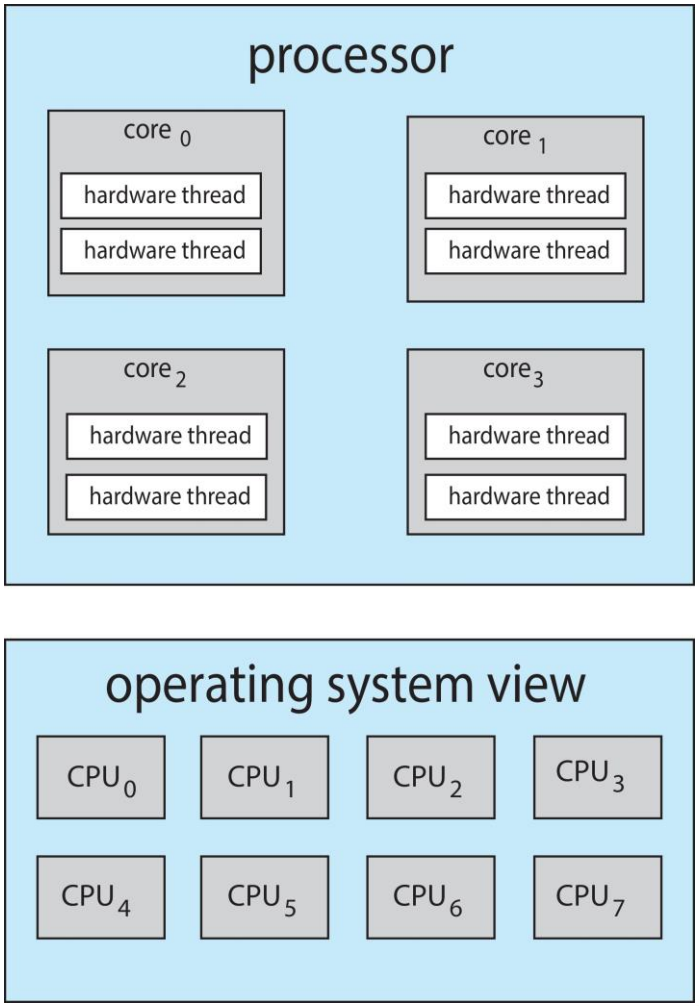


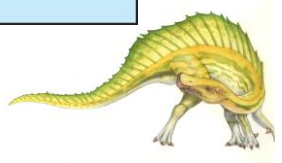


Multithreaded Multicore System

- **Chip-multithreading** (CMT) assigns each core multiple hardware threads. (Intel refers to this as **hyperthreading**.)

- On a quad-core system with 2 hardware threads per core, the operating system sees 8 logical processors.

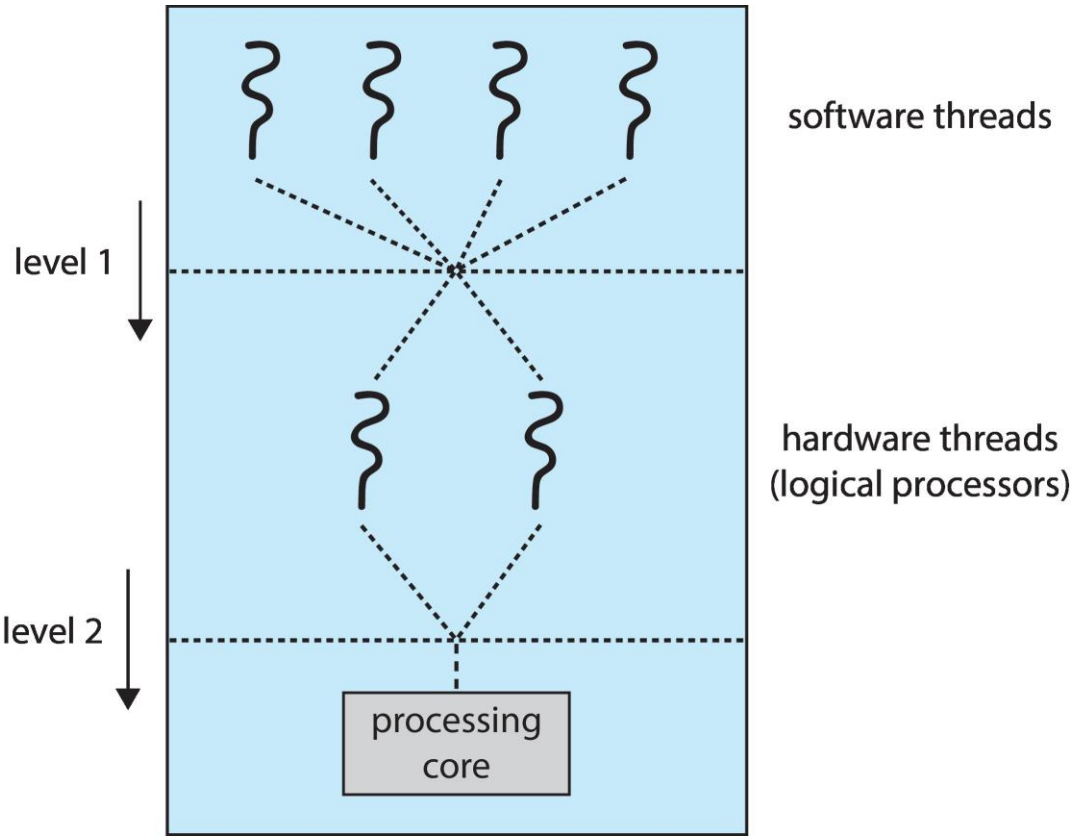


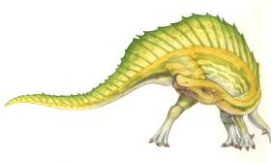




Multithreaded Multicore System

- Two levels of scheduling:
  1. The operating system deciding which software thread to run on a logical CPU
  2. How each core decides which hardware thread to run on the physical core.







Multiple-Processor Scheduling – Load Balancing

- If SMP, need to keep all CPUs loaded for efficiency
- **Load balancing** attempts to keep workload evenly distributed
- **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
- **Pull migration** – idle processors pulls waiting task from busy processor
- legatura cu acceleratia (si legile lui Amdahl/Gustafson)
  - *eficienta*: fractiunea de timp efectiv utilizata de proces

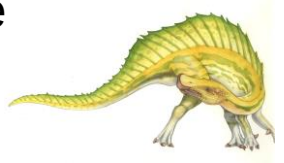
p = nr procesoare

n = dimensiunea problemei

$$S_p(n) = \frac{T_1(n)}{T_p(n)}$$

$$E_p(n) = \frac{S_p(n)}{p} = \frac{T_1(n)}{pT_p(n)}$$

Valoare ideala: $E_p(n) = 1$ (i.e., accelerare a calculului de *p* ori, nici un procesor nu e idle vreodata, *practic imposibil* !)





Multiple-Processor Scheduling – Processor Affinity

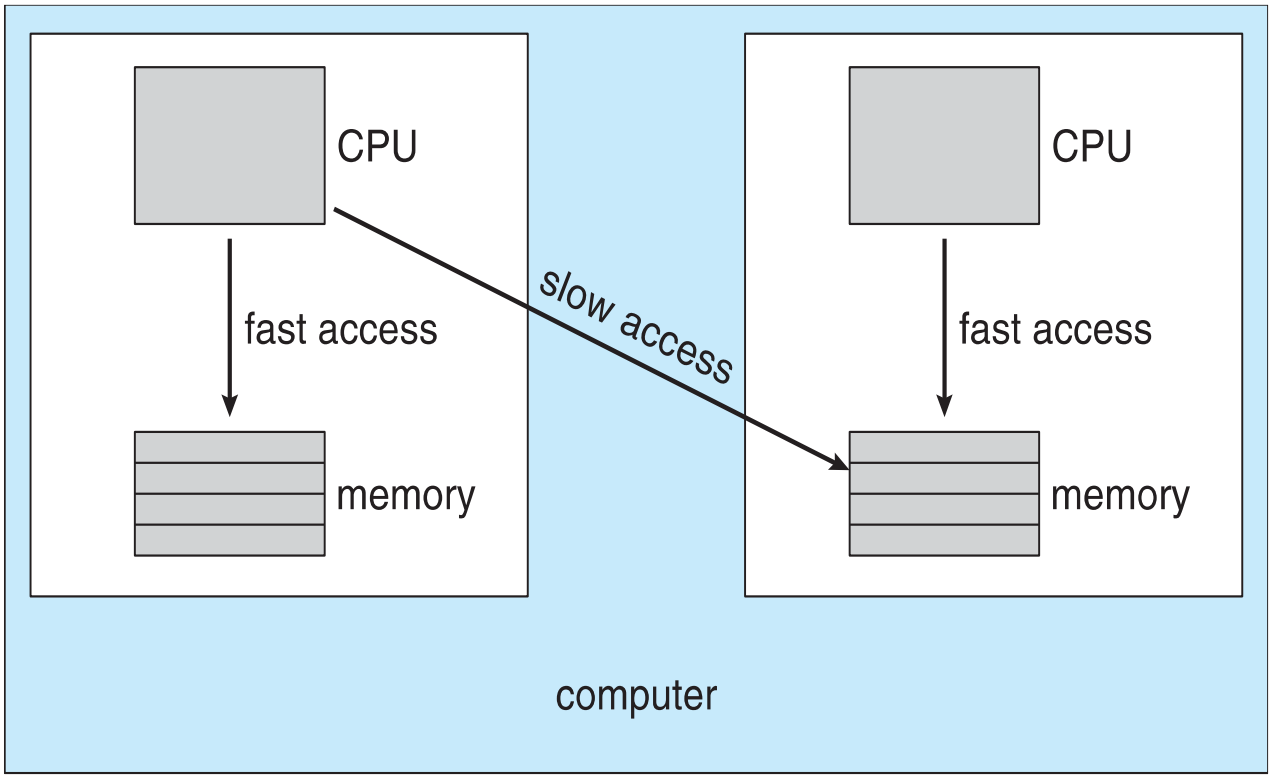
- When a thread has been running on one processor, the cache contents of that processor stores the memory accesses by that thread.
- We refer to this as a thread having affinity for a processor (i.e., “processor affinity”)
- Load balancing may affect processor affinity as a thread may be moved from one processor to another to balance loads, yet that thread loses the contents of what it had in the cache of the processor it was moved off of.
- **Soft affinity** – the operating system attempts to keep a thread running on the same processor, but no guarantees.
- **Hard affinity** – allows a process to specify a set of processors it may run on.
- Obs: afinitatea de procesor si echilibrarea incarcarii procesoarelor sunt tinte contradictorii !





NUMA and CPU Scheduling

If the operating system is **NUMA-aware**, it will assign memory closes to the CPU the thread is running on.



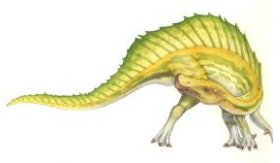




NORMA Scheduling

- in sistemele distribuite, fiecare procesor executa local algoritmul de planificare
- Q: cum se comporta un grup de procese care interactioneaza strans si ruleaza pe masini diferite?
- ex: procesele A, B ruleaza pe P1; C si D pe P2
- masinile P1 si P2 ruleaza RR cu q=10ms
- A si C ruleaza in cuantele pare, B si D in cele impare
- A porneste si cheama D in cuanta para, D nu ruleaza (C ruleaza in cuantele pare)
- dupa 10ms, D primeste mesajul lui A si raspunde; cuanta e impara, A nu ruleaza (B ruleaza)
- dupa inca 10 ms, A primeste mesajul lui D

$\Rightarrow$ schimbul de mesaje are un overhead de 20 ms datorat exclusiv planificarii (se adauga minimal costurile comunicatiei)





Co-scheduling

- idee: procesele care comunica frecvent sa ruleze simultan pe masini diferite
- implementare co-scheduling cu matrici
  - fiecare coloana contine procesele rulate pe un anume procesor (masina)
  - fiecare linie contine procesele care ruleaza simultan pe procesoare
  - i.e., coloanele reprezinta procesoare(masini), liniile cuante de timp
  - fiecare procesor planifica RR local
  - procesoarele isi sincronizeaza cuantele RR folosind bcast (un ceas sincron ca la SIMD, posibil implementat prin NTP)
  - co-scheduling: toate procesele unui grup au alocate aceleasi cuante de timp (i.e., apar pe aceleasi linii ale matricii in diferite coloane) => maximizarea paralelismului disponibil si a throughput-ului comunicatiei





Planificare de timp real

- *sistem de timp real* = sistem in care corectitudinea executiei unui program nu depinde doar de corectitudinea rezultatelor calculate ci si de **timpul** la care sunt livrate rezultatele
  - **hard** – toate rezultatele trebuie livrate la timp
  - **soft** – rezultatele calculate pot fi acceptate si cu intarziere
- definitii
  - task = o singura executie a unui bloc de cod
  - timp de sosire al taskului = timpul la care sistemul devine constient ca taskul trebuie executat
- taskuri *periodice*
  - sosesc la intervale regulate de timp
  - T(t) = timp fix inter-sosiri (perioada)
  - C(t) = timpul de calcul





Planificare de timp real

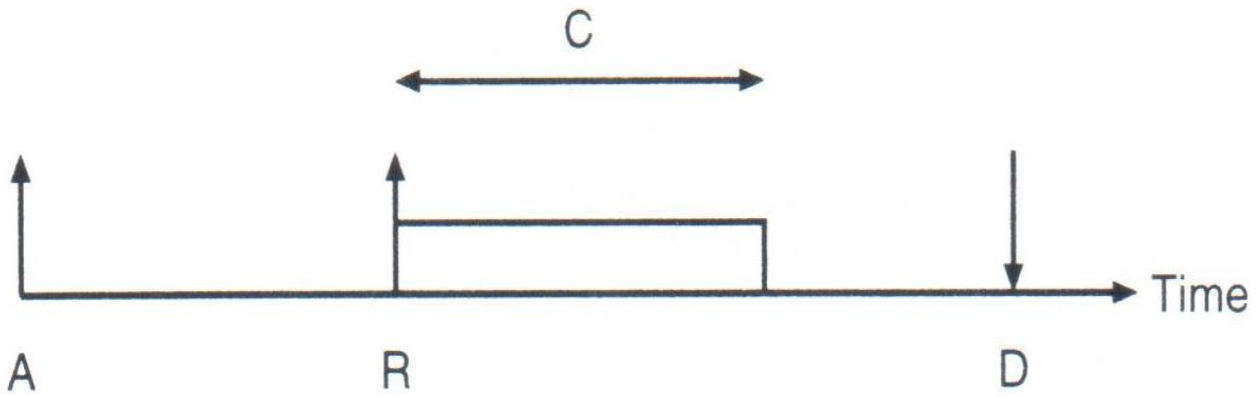
- taskuri *aperiodice*
  - caracterizate de rate de sosiri stochastice
  - pot veni “in rafala” => mai multe instante ale unui task aperiodic pot veni intr-un interval scurt de timp => supraincarcare sistem
  - in consecinta, se pp. ca exista un timp minim intre sosiri succesive ale aceluiasi task
  - asemenea taskuri s.n. *sporadice*
  - si taskurile sporadice pot veni intr-un interval scurt de timp; daca sistemul n-are suficiente resurse disponibile se ajunge la nerespectarea deadline-urilor
  - se spune ca sistemul se afla in stare de *supraincarcare trecatoare* (“transient overload”)





Planificare de timp real

- timpul de eliberare (“release time”) = timpul la care taskului i se permite sa porneasca executia
- deadline = timpul pana la care trebuie sa se termine executia taskului
  - trebuie notificat planificatorului kernel
  - kernelul utilizeaza o politica de *admission-control* pt a decide daca accepta task-ul sau nu
- diagrama de timp
  - A -> R -> C -> D (arrival/sosire, release, calcul, deadline)







Real-Time CPU Scheduling

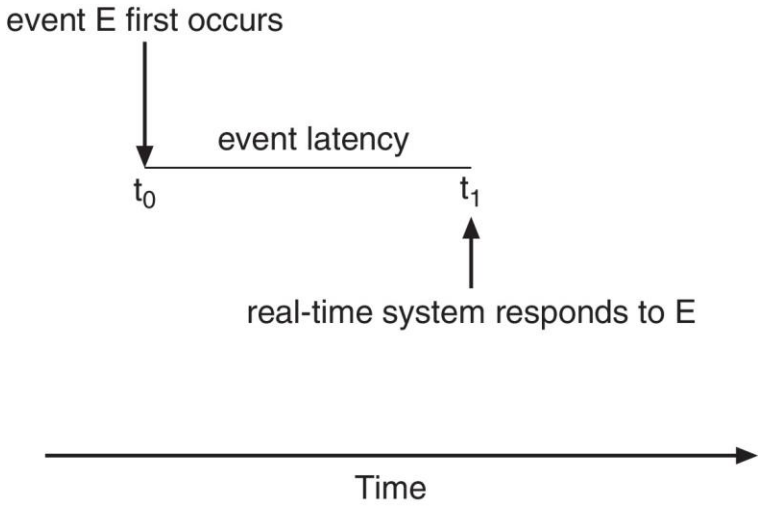
- Can present obvious challenges
- **Soft real-time systems** – Critical real-time tasks have the highest priority, but no guarantee as to when tasks will be scheduled
- **Hard real-time systems – task must be serviced by its deadline**





Real-Time CPU Scheduling

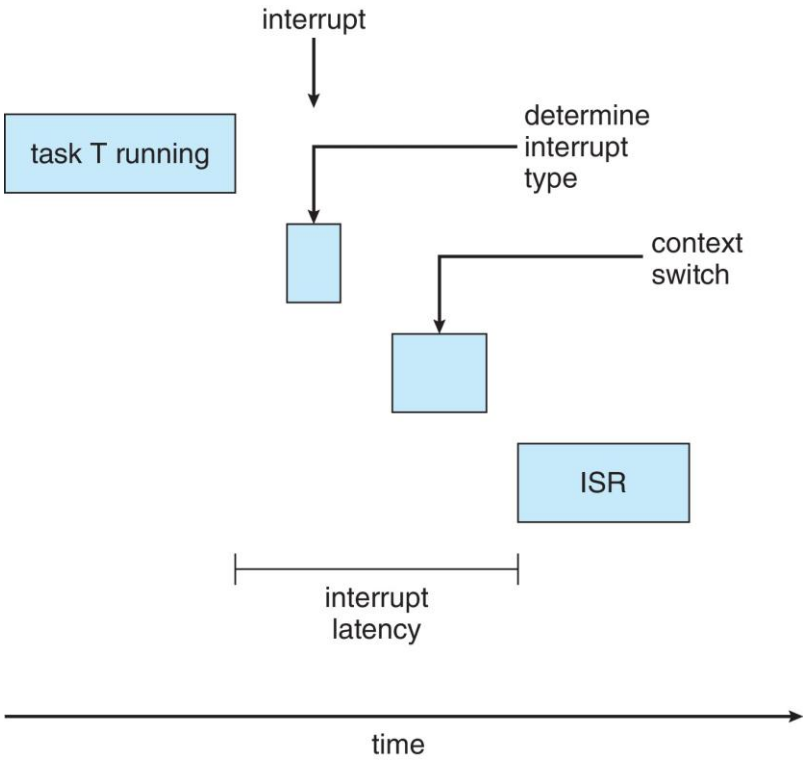
- Event latency – the amount of time that elapses from when an event occurs to when it is serviced.
- Two types of latencies affect performance
  1. **Interrupt latency** – time from arrival of interrupt to start of routine that services interrupt
  2. **Dispatch latency** – time for schedule to take current process off CPU and switch to another







Interrupt Latency

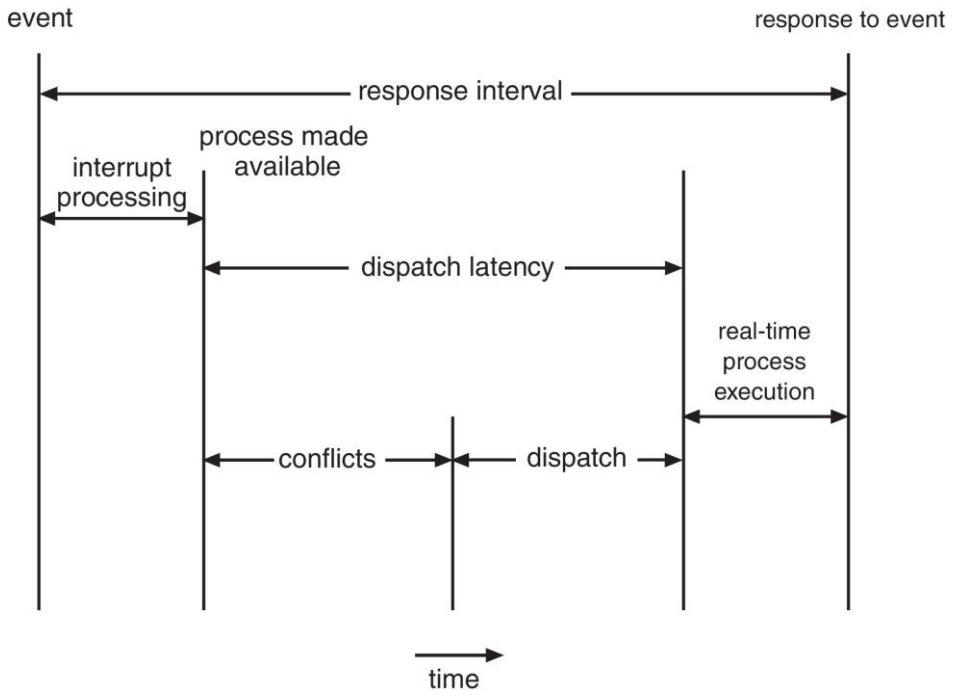






Dispatch Latency

- Conflict phase of dispatch latency:
  1. Preemption of any process running in kernel mode
  2. Release by low-priority process of resources needed by high-priority processes

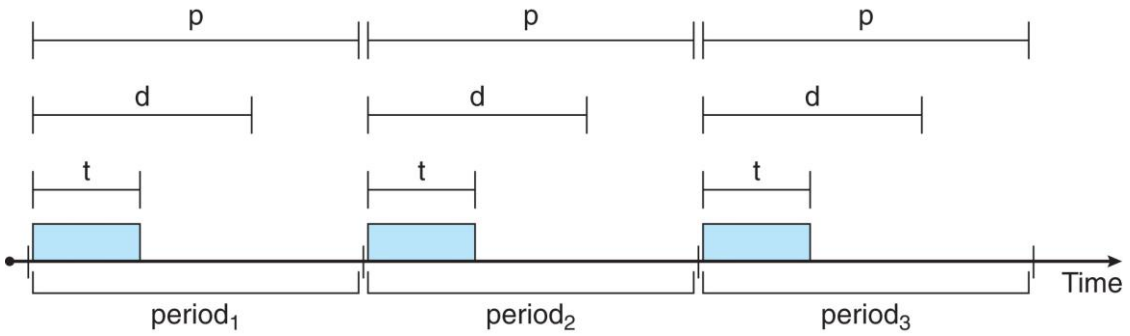






Priority-based Scheduling

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
  - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
- Processes have new characteristics: **periodic** ones require CPU at constant intervals
  - Has processing time *t*, deadline *d,* period *p*
  - $0 \leq t \leq d \leq p$
  - **Rate** of periodic task is $1/p$

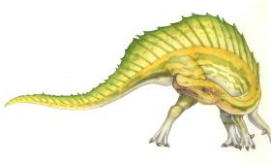






Rate Monotonic (RM)

- algoritm pt taskuri periodice (interesante pt modelarea usoara si folosirea pe scara larga in industrie/automatizari)
- RM este un algoritm offline de planificare a unor taskuri care respecta urmatoarele constrangeri
  - (1) cererile taskurilor cu deadline hard sunt periodice, cu interval constant intre cereri
  - (2) deadline-urile constau exclusiv in constrangeri de executabilitate, i.e., fiecare task trebuie sa termine inainte sa apara urmatoarea cerere pt task
  - (3) taskurile sunt independente, cererile pt un anumit task nu depind de initierea sau terminarea cererilor pt alte taskuri
  - (4) timpul de executie al fiecarui task este constant (nu variaza in timp); timpul de executie aici inseamna timpul CPU necesar sa execute taskul fara intrerupere





Rate Monotonic (RM)

- RM este un algoritm offline de planificare a unor taskuri care respecta urmatoarele constrangeri

  ….

  - (5) orice task neperiodic din sistem este special: rutina de initializare sau de recuperare din eroare
    - inlocuiesc taskurile periodice cand ruleaza dar nu deadline-uri hard, critice





Algoritmul RM

- taskurile au asignate prioritati statice a.i. taskurile cu rate de sosiri mari au prioritate mare (echivalent, perioada mica => prioritate mare)
- taskurile se executa preemptiv
- planificatorul alege intotdeauna taskul cu prioritate maxima
- utilizarea *U* a procesorului de catre *n* taskuri

$$U = \sum_{i=1}^{n} \frac{C_i}{Ti}$$

- **Lema**: daca U < ln 2, planificarea RM e fezabila (toate deadline-urile sunt respectate)
- in practica, U < ln 2 e o cerinta conservatoare, exista seturi de taskuri care se pot planifica RM si a caror utilizare depaseste ln 2





Exemplu RM

- sistem multimedia
  - task audio cu perioada T_a = 10 ms si C_a = 2 ms
  - task video cu perioada T_v = 30 ms si C_v = 10 ms

  => $U_a = 2/10 = 0.2$

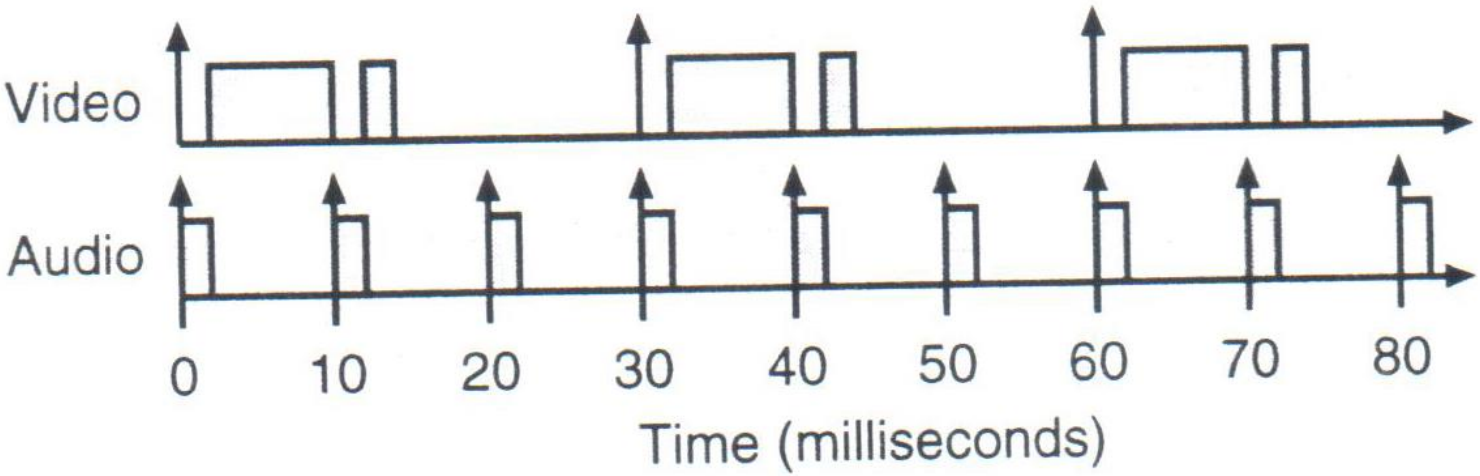
  $U_v = 10/30 = 0.33$

  => $U = 0.53 < \ln 2 \approx 0.69$ => taskurile sunt planificabile
- taskul audio are frecventa maxima (perioada cea mai scurta) => are si prioritate maxima
- pp. C_v = 20 ms => $U_v = 20/30 = 0.66$ => $U = 0.86 > \ln 2$; totusi, o diagrama de timp arata ca taskurile sunt planificabile RM daca taskul audio are prioritate maxima





Exemplu RM, diagrama de timp

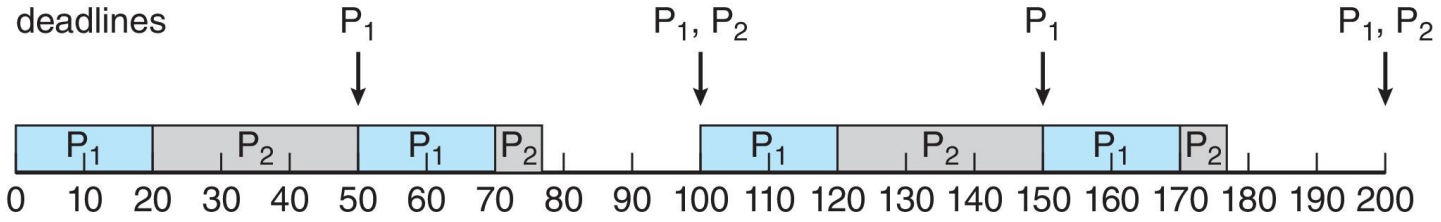






Rate Monotonic Scheduling

- A priority is assigned based on the inverse of its period
- Shorter periods = higher priority;
- Longer periods = lower priority
- P_1 is assigned a higher priority than P_2.

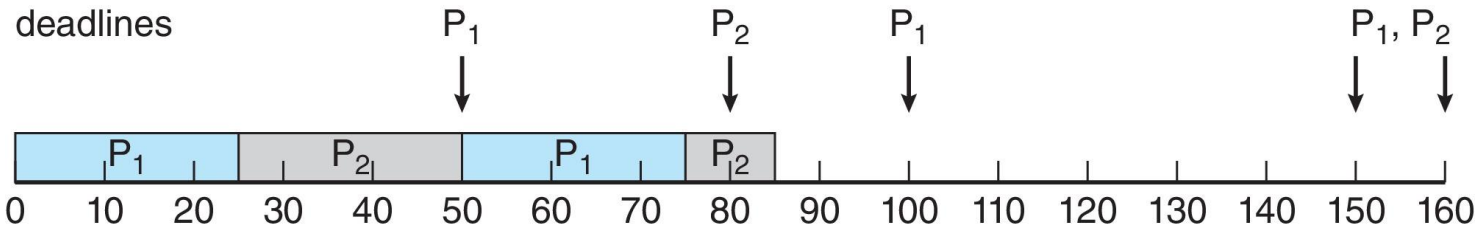






Missed Deadlines with Rate Monotonic Scheduling

- Process P_2 misses finishing its deadline at time 80
- Figure





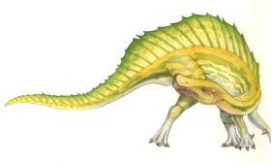


Earliest Deadline First (EDF)

- taskul cu deadline-ul cel mai apropiat are prioritate maxima
- un set de *n* taskuri e planificabil daca

$$U = \sum_{i=1}^{n} \frac{C_i}{Ti} <= 1$$

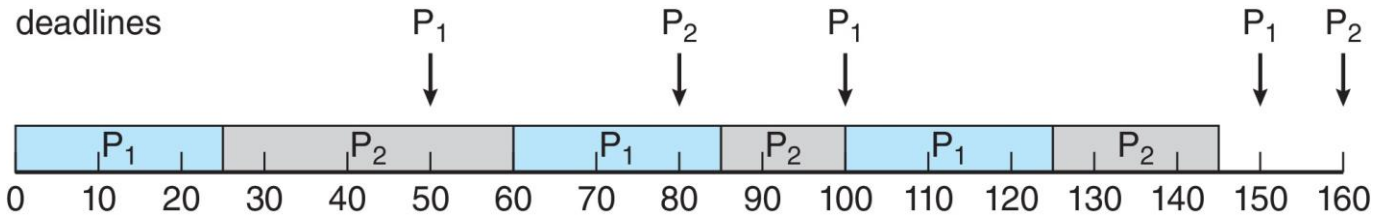
- ambele exemple de la RM sunt planificabile si EDF





Earliest Deadline First Scheduling (EDF)

- Priorities are assigned according to deadlines:
  - The earlier the deadline, the higher the priority
  - The later the deadline, the lower the priority
- Figure







Least Slack First (LSF)

- *slack* = durata maxima de timp cu care un task poate fi intarziat fara a-si pierde deadline-ul
- LSF alege pt rulare taskul cu slack-ul cel mai mic





Planificarea taskurilor aperiodice

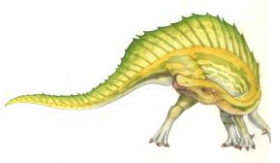
- EDF si LSF se pot folosi si pt. planificarea taskurilor aperiodice
- se poate demonstra ca EDF este optimal
  - daca orice alt algoritm poate produce o planificare fezabila pt un set de taskuri ale caror timpi de sosire, timpi de calcul si deadline-uri sunt cunoscute, atunci si EDF o poate face
- exista o demonstratie similara de optimalitate si pt LSF





Proportional Share Scheduling

- *T* shares are allocated among all processes in the system
- An application receives *N* shares where *N* < *T*
- This ensures each application will receive ***N*** / *T* of the total processor time
- si aici exista mecanism de admission-control pt a verifica daca un proces poate primi partea procentuala de timp procesor pe care o solicita





Alocare proportionala de resurse

- planificarea de timp real functioneaza bine in sisteme embedded (mix-ul de job-uri e cunoscut a priori)
- in medii dinamice sistemul poate deveni supraincarcat iar aplicatiile de timp real nu-si pot respecta deadline-urile
- solutiile de planificare bazate pe prioritati nu sunt intotdeauna utile
  - ex: task computational intensiv (video) de prioritate mare intarzie indefinit taskuri non real-time de prioritate mica (compilari)
- abordare noua in planificarea de procese
  - renuntam la conditia ca toate procesele sa termine cu siguranta inainte de deadline
  - planificatorul controleaza rata de executie a proceselor variind “proportia” de resurse (timp CPU) pe care procesele o primesc





Planificare de tip loterie

- Lottery Scheduling (LS), Waldspurger & Weihl 1994
- un algoritm randomizat de alocare proportionala a resurselor in general
  - timp CPU
  - largime de banda I/O
  - alocare de memorie
  - accesul la lock-uri
- controleaza ratele relative de executie ale proceselor variind procentul de alocare al resursei (eg, timpul CPU pe care-l primeste procesul)
- rata de “consum” a resursei de catre un proces activ este proportionala cu procentul de alocare a resursei respective detinut de proces





Tichete de loterie

- drepturile asupra resurselor sunt exprimate prin intermediul unor *tichete de loterie*
- alocarea resursei se face prin organizarea unei loterii:
  - resursa e acordata procesului care detine tichetul castigator
  - alocarea e proportionala cu nr de tichete detinut de proces
- tichetele de loterie
  - reprezinta dreptul de a folosi o fractiune din resursa (suma tichetelor detinute de un proces defineste procentul total de utilizare a resursei la care are dreptul procesul)
  - *abstracte*
  - *relative*
  - *uniforme*





Tichete de loterie (cont'd)

- tichetele de loterie
  - *abstracte*: cuantifica dreptul de folosire a resursei independent de detaliile HW
  - *relative*: fractiunea de resursa pe care o cuantifica variaza proportional cu nr total de tichete detinute
    - variatia e dinamica, in functie de competitia la resursa
    - un proces va obtine o fractiune mai mare a unei resurse cu competitie mica decat in cazul uneia cu competitie mare
    - in cel mai rau caz, un proces primeste o fractiune de resursa proportionala cu nr sau de tichete
  - *uniforme*: indiferent de tipul de resursa, procentele de alocare ale proceselor sunt reprezentate omogen prin tichete





Algoritm LS

- la inceputul fiecarei cuante, planificatorul organizeaza o loterie si resursa (CPU) e acordata detinatorului tichetului castigator
- tichetul castigator este generat aleator cf unei distributii uniforme
- apoi se cauta in lista de procese gata de rulare participantul care detine tichetul castigator
  - i.e., participantul pentru care suma cumulata a tichetelor de pana la el este mai mare decat tichetul castigator
- ex: nr total de tichete = 20, 5 procese cu [9, 3, 5, 1, 2] tichete

  random[0..19] = 15

  S1 = 9; S1 > 15? NU

  S2 = 12; S2 > 15? NU

  S3 = 17; S3 > 15? DA => procesul cu nr. 3 (detine 5 tichete) primeste procesorul





Analiza algoritmului LS

- running time
  - generarea de nr aleatoare e rapida (cca 10 instr RISC)
  - traversarea listei de procese O(n)
  - acumularea sumei partiale
- optimizari pt reducerea nr de procese ce trebuie examinate
  - pt distributie inegala de tichete, ordoneaza descrescator lista de procese cf nr de tichete detinute

    => lungimea medie de cautare scade pt ca procesele cu nr mare de tichete sunt frecvent selectate
  - pt *n* mare, se poate folosi un arbore cu sume partiale in nodurile interne si nr de tichete ale proceselor pe frunze

    => localizarea tichetului castigator se face pornind din radacina

    catre procesul (frunza) castigator => complexitate O(log n)





Caracteristici LS

- (1) *fair* dpdv probabilistic
  - dupa o perioada suficient de lunga de timp, valoarea estimata a fractiunii de alocare a resursei este proportionala cu nr de tichete detinut de proces
  - obs: nu e garantat ca un proces cu *t* tichete din totalul *T* va primi **exact** *t / T* din utilizarea resursei !
  - diferenta scade insa pe masura ce nr de alocari creste

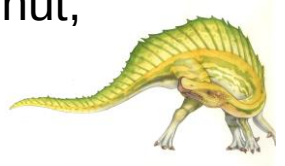
  Argument: fie X = nr de loterii castigate de un proces (variabila aleatoare cu distributie binomiala, proces Bernoulli)

  Fie $p = t / T$ probabilitatea ca un proces cu *t* tichete sa castige loteria

  => dupa *n* loterii identice $E[X] = np$ iar $Var(X) = np(1 – p)$

  $\Rightarrow$ coef. de variatie = $\frac{stdev(X)}{E[X]} = \sqrt{\frac{1-p}{np}}$

  i.e., throughput-ul procesului e proportional cu nr de tichete detinut, iar acuratetea estimarii se imbunatateste cu viteza lui $\sqrt{n}$





Caracteristici LS (cont'd)

- (1) *fair* dpdv probabilistic
  - fie Y = nr de loterii necesare ca un proces sa castige prima data loteria (variabila aleatoare cu distributie geometrica)

$$E[Y] = \frac{1}{p}$$

$$Var(Y) = \frac{1-p}{p^2}$$

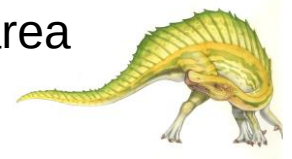
  $\Rightarrow$ timpul mediu de raspuns este invers proportional cu nr de tichete detinut de proces





Caracteristici LS (cont'd)

- (2) tichetele se pot transfera de la un proces la altul atunci cand acesta se blocheaza dintr-un motiv oarecare
  - ex: proces client asteapta raspuns de la server, clientul poate transfera temporar tichetele sale serverului
  - clientul poate transfera tichetele sale catre mai multe servere de la care asteapta un raspuns
  - solutie pt. priority inversion
- (3) *inflatia* de tichete: alternativa la transferul de tichete
  - procesul isi mareste dreptul la resurse prin creearea artificiala de tichete pt sine
  - metoda se poate folosi intre procese care au incredere reciproca intre ele
  - prin inflatie/deflatie de tichete procesele controleaza alocarea resursei fara sa comunice explicit intre ele
  - in general este de evitat, pt ca poate conduce la monopolizarea resursei





Caracteristici LS (cont'd)

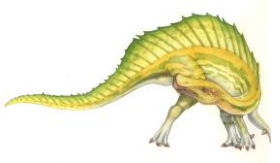
- (4) *tichete de compensare*
  - cand un proces consuma doar o fractiune *f* din cuanta alocata pentru utilizarea resursei, el primeste un *tichet de compensare*
  - tichetul de compensare ii creste procesului valoarea tichetelor cu *1 / f* pana cand procesul primeste urmatoarea cuanta
  - astfel, consumul resursei de *f * p* este ajustat cu *1 / f* ca sa corespunda fractiunii reprezentate de p
  - in absenta unei astfel de metode, procesul care nu consuma intreaga fractiune alocata utilizeaza resursa mai putin decat are dreptul





POSIX Real-Time Scheduling

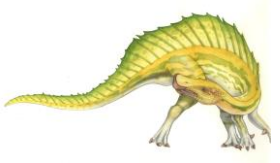
- The POSIX.1b standard
- API provides functions for managing real-time threads
- Defines two scheduling classes for real-time threads:
  1. SCHED_FIFO - threads are scheduled using a FCFS strategy with a FIFO queue. There is no time-slicing for threads of equal priority
  2. SCHED_RR - similar to SCHED_FIFO except time-slicing occurs for threads of equal priority
- Defines two functions for getting and setting scheduling policy:
  1. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
  2. `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`





POSIX Real-Time Scheduling API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
   int i, policy;
   pthread_t tid[NUM_THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* get the current scheduling policy */
   if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
      fprintf(stderr, "Unable to get policy.\n");
   else {
      if (policy == SCHED_OTHER) printf("SCHED_OTHER\n");
      else if (policy == SCHED_RR) printf("SCHED_RR\n");
      else if (policy == SCHED_FIFO) printf("SCHED_FIFO\n");
   }
```

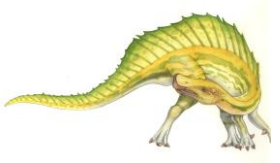




POSIX Real-Time Scheduling API (Cont.)

```
   /* set the scheduling policy - FIFO, RR, or OTHER */
   if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
      fprintf(stderr, "Unable to set policy.\n");
   /* create the threads */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_create(&tid[i],&attr,runner,NULL);
   /* now join on each thread */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
   /* do some work ... */
   pthread_exit(0);
}
```





Operating System Examples

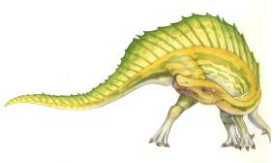
- Linux scheduling
- Windows scheduling
- Solaris scheduling





Linux Scheduling Through Version 2.5

- Prior to kernel version 2.5, ran variation of standard UNIX scheduling algorithm
- Version 2.5 moved to constant order *O*(1) scheduling time
  - Preemptive, priority based
  - Two priority ranges: time-sharing and real-time
  - **Real-time** range from 0 to 99 and **nice** value from 100 to 140
  - Map into global priority with numerically lower values indicating higher priority
  - Higher priority gets larger q
  - Task run-able as long as time left in time slice (**active**)
  - If no time left (**expired**), not run-able until all other tasks use their slices
  - All run-able tasks tracked in per-CPU **runqueue** data structure
    - Two priority arrays (active, expired)
    - Tasks indexed by priority
    - When no more active, arrays are exchanged
  - Worked well, but poor response times for interactive processes





Linux Scheduling in Version 2.6.23 +

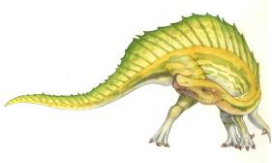
- **Completely Fair Scheduler (CFS)**
- **Scheduling classes**
  - Each has specific priority
  - Scheduler picks highest priority task in highest scheduling class
  - Rather than quantum based on fixed time allotments, based on proportion of CPU time
  - Two scheduling classes included, others can be added
    1. default
    2. real-time





Linux Scheduling in Version 2.6.23 + (Cont.)

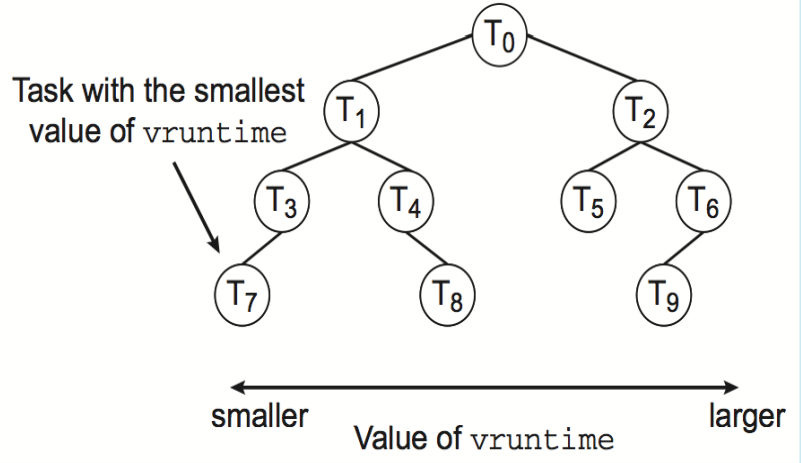
- Quantum calculated based on **nice value** from -20 to +19
  - Lower value is higher priority
  - Calculates **target latency** – interval of time during which task should run at least once
  - Target latency can increase if say number of active tasks increases
- CFS scheduler maintains per task **virtual run time** in variable **`vruntime`**
  - Associated with decay factor based on priority of task – lower priority is higher decay rate
  - Normal default priority yields virtual run time = actual run time
- To decide next task to run, scheduler picks task with lowest virtual run time





CFS Performance

The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



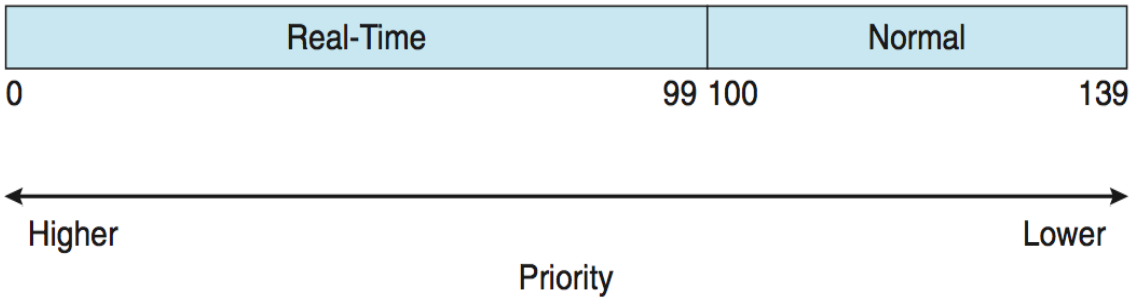
When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.





Linux Scheduling (Cont.)

- Real-time scheduling according to POSIX.1b
  - Real-time tasks have static priorities
- Real-time plus normal map into global priority scheme
- Nice value of -20 maps to global priority 100
- Nice value of +19 maps to priority 139

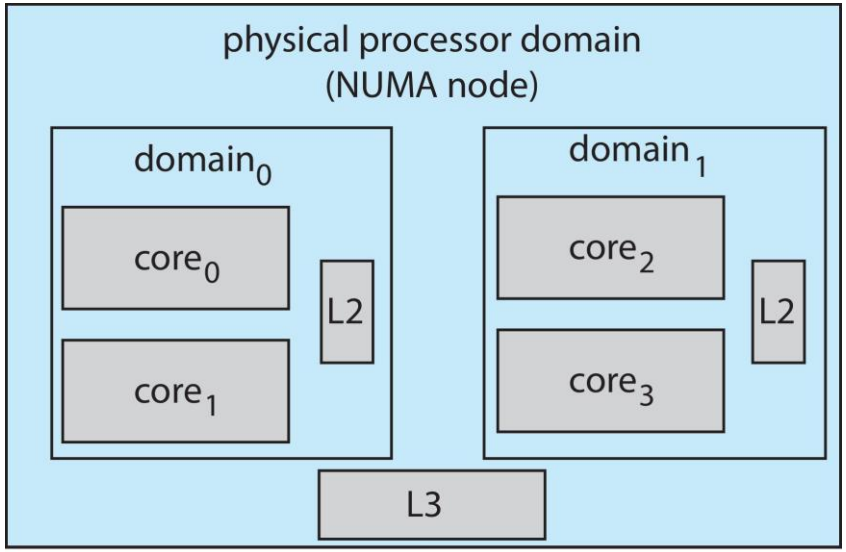






Linux Scheduling (Cont.)

- Linux supports load balancing, but is also NUMA-aware.
- **Scheduling domain** is a set of CPU cores that can be balanced against one another.
- Domains are organized by what they share (i.e., cache memory.) Goal is to keep threads from migrating between domains.







Windows Scheduling

- Windows uses priority-based preemptive scheduling
- Highest-priority thread runs next
- **Dispatcher** is scheduler
- Thread runs until (1) blocks, (2) uses time slice, (3) preempted by higher-priority thread
- Real-time threads can preempt non-real-time
- 32-level priority scheme
- **Variable class** is 1-15, **real-time class** is 16-31
- Priority 0 is memory-management thread
- Queue for each priority
- If no run-able thread, runs **idle thread**





Windows Priority Classes

- Win32 API identifies several priority classes to which a process can belong
  - REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS,NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS
  - All are variable except REALTIME
- A thread within a given priority class has a relative priority
  - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
- Priority class and relative priority combine to give numeric priority
- Base priority is NORMAL within the class
- If quantum expires, priority lowered, but never below base





Windows Priority Classes (Cont.)

- If wait occurs, priority boosted depending on what was waited for
- Foreground window given 3x priority boost
- Windows 7 added **user-mode scheduling** (**UMS**)
  - Applications create and manage threads independent of kernel
  - For large number of threads, much more efficient
  - UMS schedulers come from programming language libraries like C++ **Concurrent Runtime** (ConcRT) framework





Windows Priorities

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1





Solaris

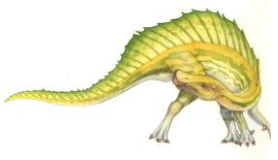
- Priority-based scheduling
- Six classes available
  - Time sharing (default) (TS)
  - Interactive (IA)
  - Real time (RT)
  - System (SYS)
  - Fair Share (FSS)
  - Fixed priority (FP)
- Given thread can be in one class at a time
- Each class has its own scheduling algorithm
- Time sharing is multi-level feedback queue
  - Loadable table configurable by sysadmin





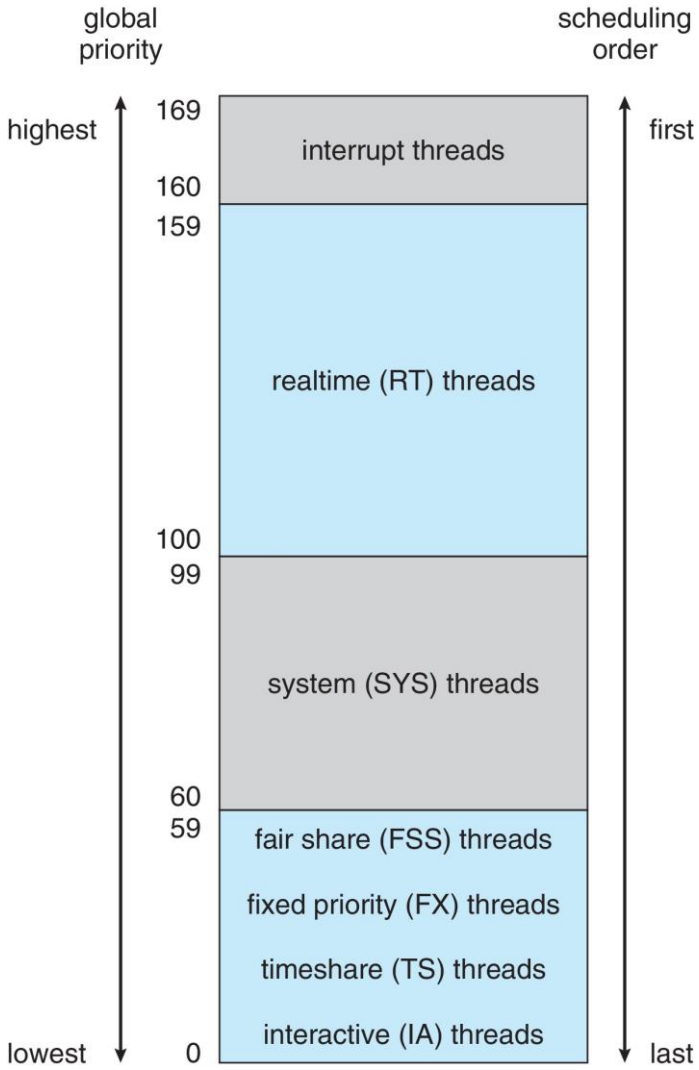
Solaris Dispatch Table

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59





Solaris Scheduling







Solaris Scheduling (Cont.)

- Scheduler converts class-specific priorities into a per-thread global priority
  - Thread with highest priority runs next
  - Runs until (1) blocks, (2) uses time slice, (3) preempted by higher-priority thread
  - Multiple threads at same priority selected via RR





Algorithm Evaluation

- How to select CPU-scheduling algorithm for an OS?
- Determine criteria, then evaluate algorithms
- **Deterministic modeling**
  - Type of **analytic evaluation**
  - Takes a particular predetermined workload and defines the performance of each algorithm for that workload
- Consider 5 processes arriving at time 0:

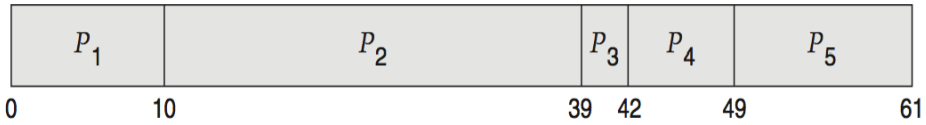
Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12



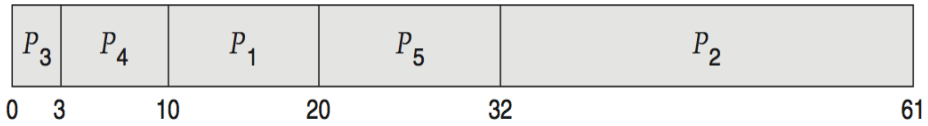


Deterministic Evaluation

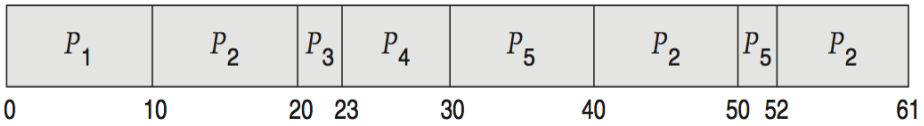
- For each algorithm, calculate minimum average waiting time
- Simple and fast, but requires exact numbers for input, applies only to those inputs
  - FCS is 28ms:



  - Non-preemptive SFJ is 13ms:



  - RR is 23ms:







Queueing Models

- Describes the arrival of processes, and CPU and I/O bursts probabilistically
  - Commonly exponential, and described by mean
  - Computes average throughput, utilization, waiting time, etc.
- Computer system described as network of servers, each with queue of waiting processes
  - Knowing arrival rates and service rates
  - Computes utilization, average queue length, average wait time, etc.





Little's Formula

- n = average queue length
- W = average waiting time in queue
- λ = average arrival rate into queue
- Little's law – in steady state, processes leaving queue must equal processes arriving, thus:

  $n = \lambda \times W$

  - Valid for any scheduling algorithm and arrival distribution
- For example, if on average 7 processes arrive per second, and normally 14 processes in queue, then average wait time per process = 2 seconds





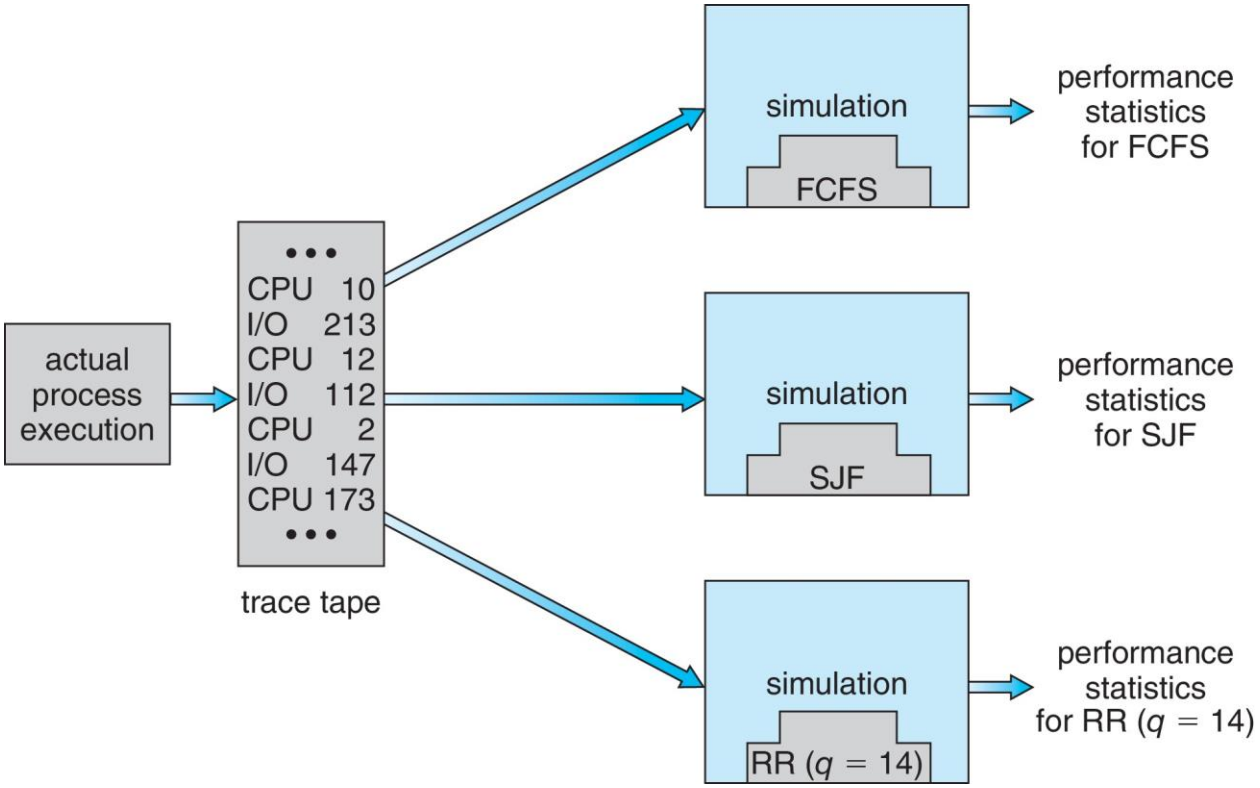
Simulations

- Queueing models limited
- **Simulations** more accurate
  - Programmed model of computer system
  - Clock is a variable
  - Gather statistics indicating algorithm performance
  - Data to drive simulation gathered via
    - Random number generator according to probabilities
    - Distributions defined mathematically or empirically
    - Trace tapes record sequences of real events in real systems





Evaluation of CPU Schedulers by Simulation





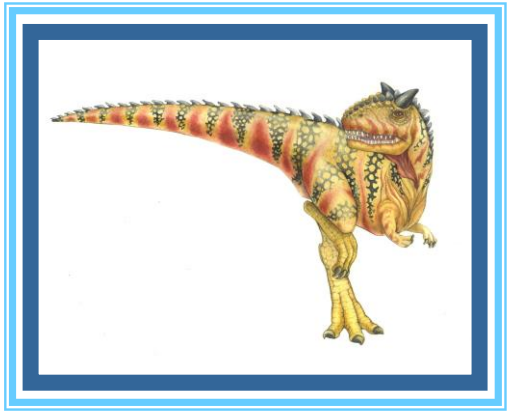


Implementation

- Even simulations have limited accuracy
- Just implement new scheduler and test in real systems
  - High cost, high risk
  - Environments vary
- Most flexible schedulers can be modified per-site or per-system
- Or APIs to modify priorities
- But again environments vary

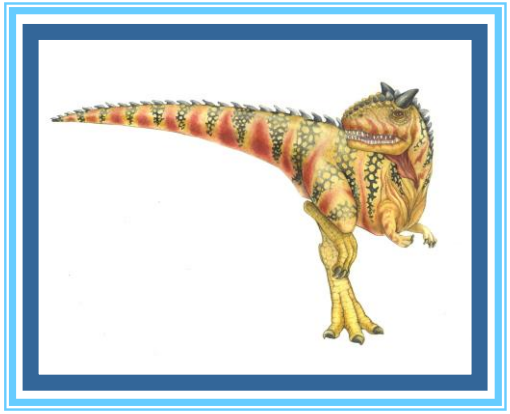


End of Chapter 5



Silberschatz, Galvin and Gagne ©2018

Chapter 9: Main Memory



Silberschatz, Galvin and Gagne ©2018



Chapter 9: Memory Management

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping
- Example: The Intel 32 and 64-bit Architectures
- Example: ARMv8 Architecture





Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques,
- To provide a detailed description of the Intel Pentium, which supports both pure segmentation and segmentation with paging





Gestiunea memoriei

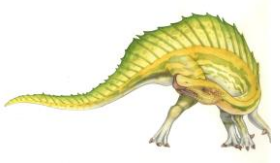
- controleaza partile din memoria RAM utilizate si respective libere (neutilizate)
- aloca/dealoca memorie pt. procese
- gestioneaza spatiul de swap
  - schimb de segmente de memorie intre disc si memoria principala atunci cand aceasta nu poate tine toate procesele rezidente
- in absenta multiprogramarii si a swapping-ului:
  - un singur proces e rezident in memoria principala la un moment dat
  - aproape toata memoria principala e disponibila procesului (mai putin memoria ocupata de sistemul de operare)





Background

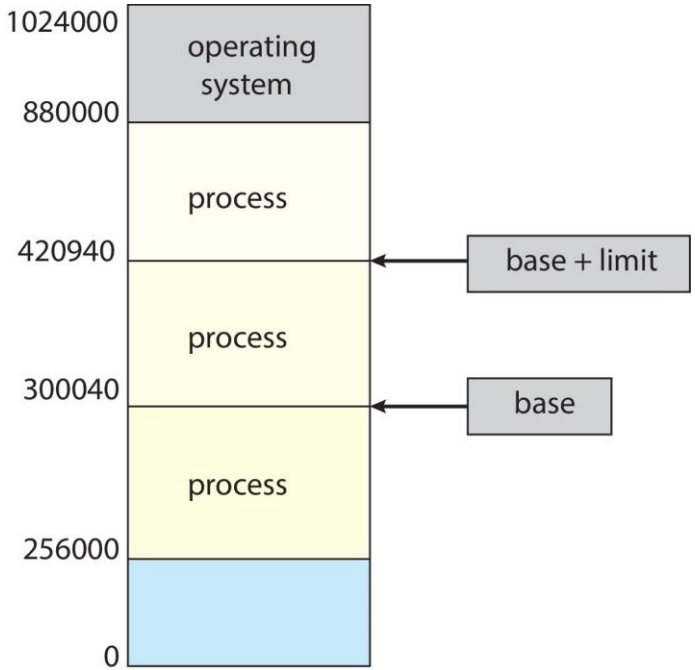
- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of:
  - addresses + read requests, or
  - address + data and write requests
- Register access is done in one CPU clock (or less)
- Main memory may take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation





Protection

- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process

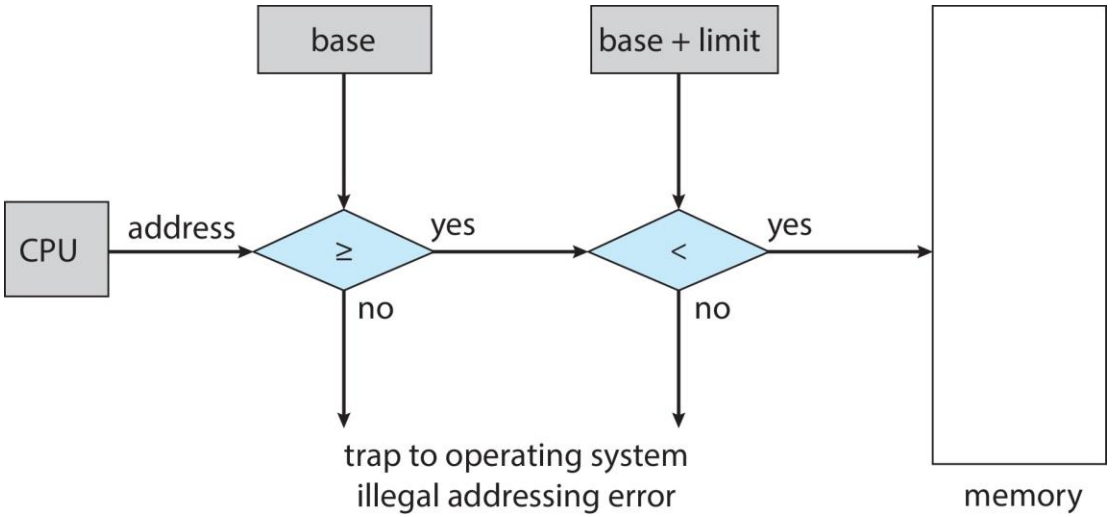






Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



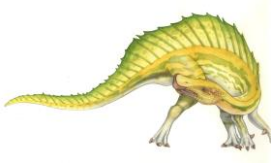
- the instructions to loading the base and limit registers are privileged
- obs: kernelul are acces nerestrictionat la intreaga memorie, deopotriva zona de memorie a sistemului de operare cat si a proceselor





Address Binding

- Programs on disk, ready to be brought into memory to execute form an **input queue**
  - Without support, must be loaded into address 0000
- Inconvenient to have first user process physical address always at 0000
  - How can it not be?
- Addresses represented in different ways at different stages of a program's life
  - Source code addresses usually symbolic
  - Compiled code addresses **bind** to relocatable addresses
    - i.e., "14 bytes from beginning of this module"
  - Linker or loader will bind relocatable addresses to absolute addresses
    - i.e., 74014
  - Each binding maps one address space to another





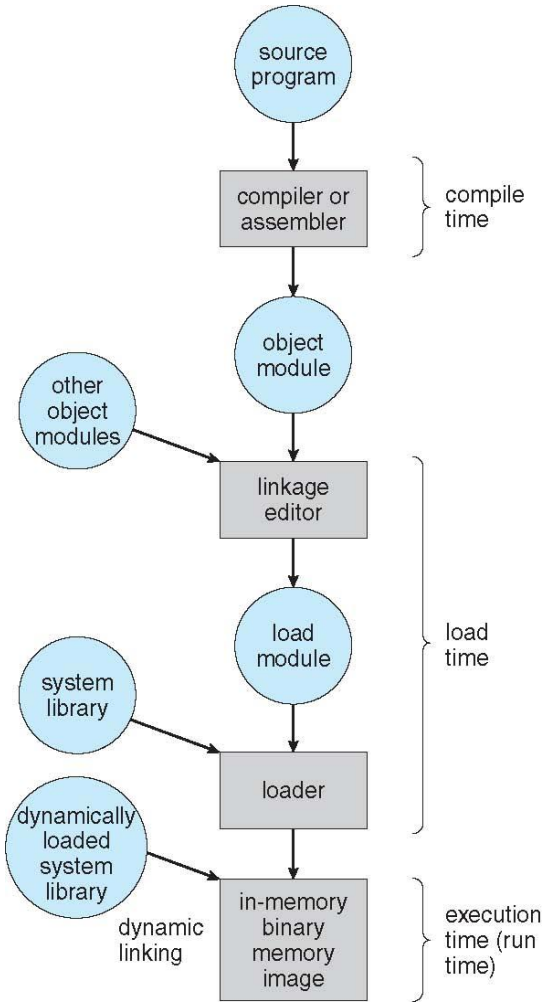
Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
  - **Compile time**: If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
  - **Load time**: Must generate **relocatable code** if memory location is not known at compile time
  - **Execution time**: Binding delayed until run time if the process can be moved during its execution from one memory segment to another
    - Need hardware support for address maps (e.g., base and limit registers)





Multistep Processing of a User Program

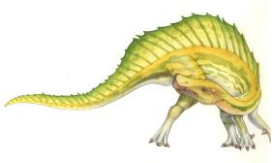






Logical vs. Physical Address Space

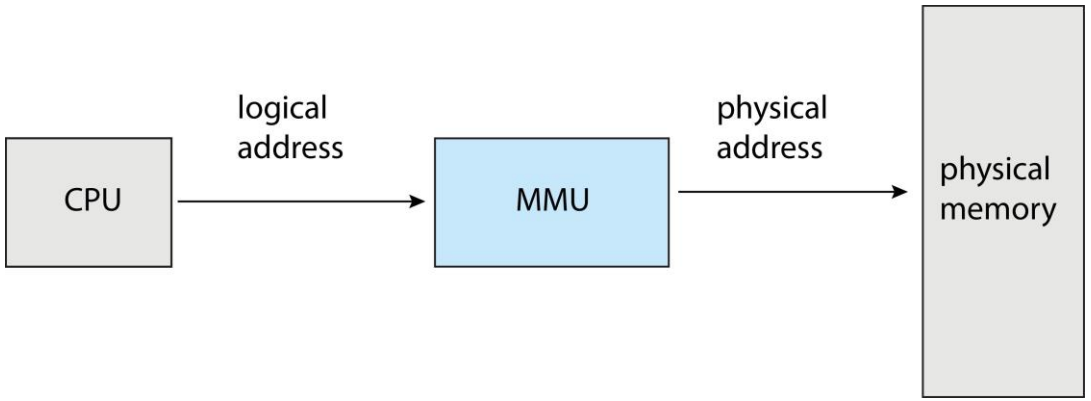
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program





Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address



- Many methods possible, covered in the rest of this chapter





Memory-Management Unit (Cont.)

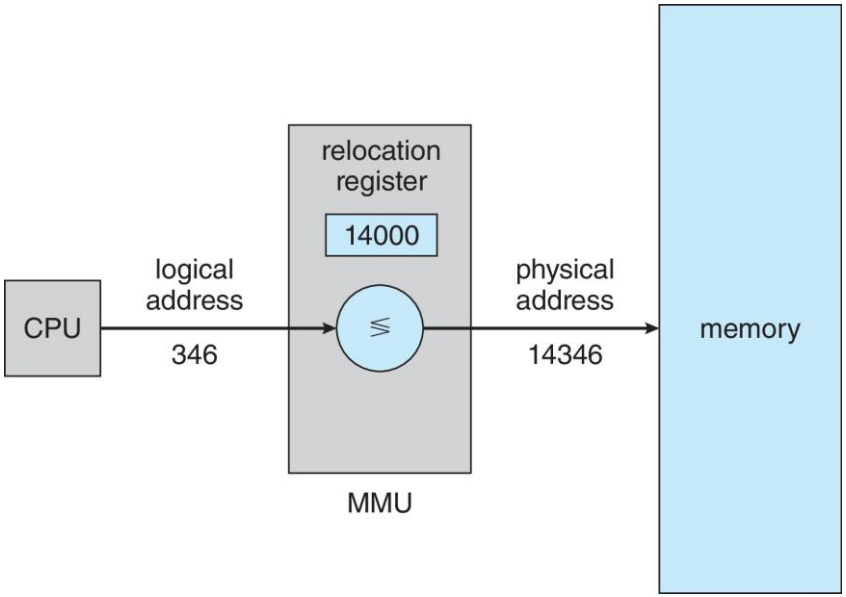
- Consider simple scheme which is a generalization of the base-register scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses

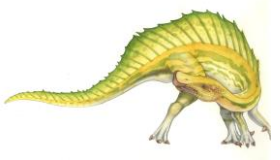




Memory-Management Unit (Cont.)

- Consider simple scheme. which is a generalization of the base-register scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory

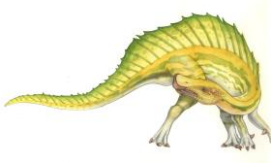






Dynamic Loading

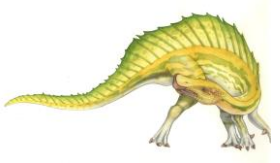
- The entire program does need to be in memory to execute
- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
  - Implemented through program design
  - OS can help by providing libraries to implement dynamic loading





Dynamic Linking

- **Static linking** – system libraries and program code combined by the loader into the binary program image
- Dynamic linking –linking postponed until execution time
- Small piece of code, **stub**, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
  - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**
- Consider applicability to patching system libraries
  - Versioning may be needed
- Obs: spre deosebire de dynamic loading, dynamic linking are nevoie de suport OS (pp. protectia memoriei intre procese)





Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought **back** into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
  - Creste gradul de multiprogramare
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk





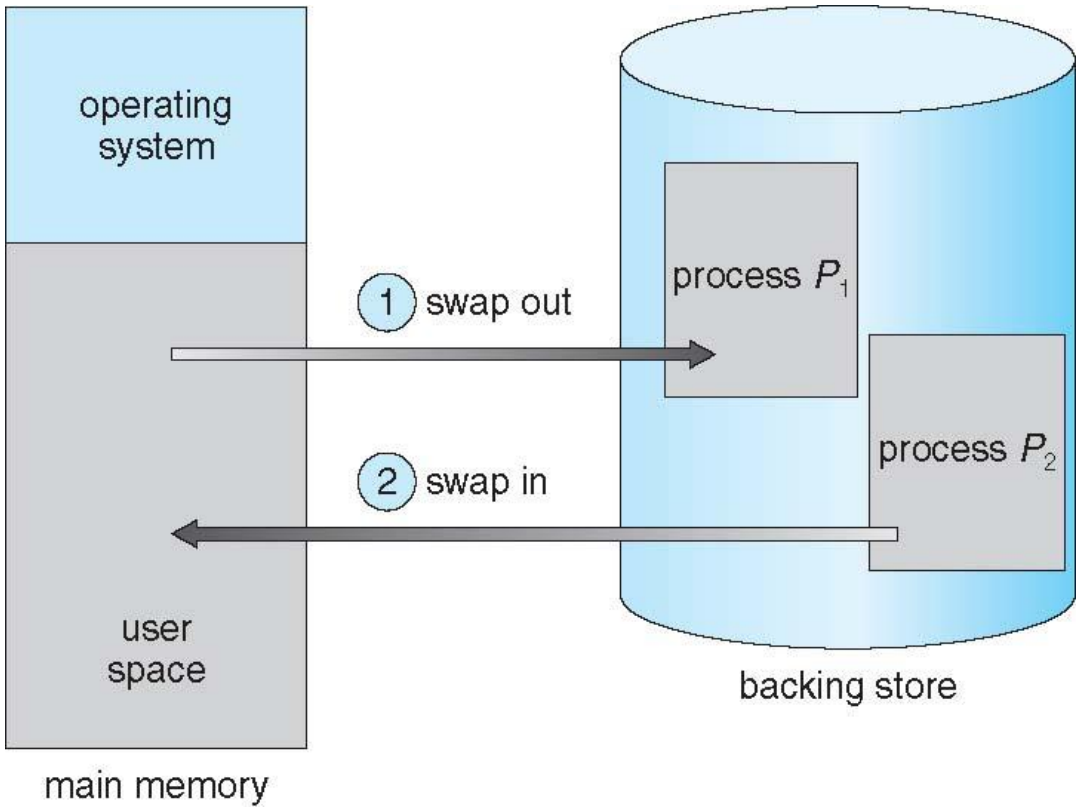
Swapping (Cont.)

- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
  - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
  - Swapping normally disabled
  - Started if more than threshold amount of memory allocated
  - Disabled again once memory demand reduced below threshold





Schematic View of Swapping

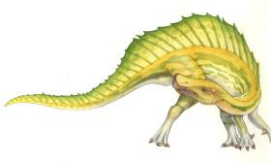






Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
  - Swap out time of 2000 ms
  - Plus swap in of same sized process
  - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
  - System calls to inform OS of memory use via `request_memory()` and `release_memory()`





Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
  - Pending I/O – can't swap out as I/O would occur to wrong process
  - Or always transfer I/O to kernel space, then to I/O device
    - Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
  - But modified version common
    - Swap only when free memory extremely low
    - Sau, swap doar parti ale procesului, in sisteme care implementeaza memorie virtuala





Swapping on Mobile Systems

- Not typically supported
  - Flash memory based
    - Small amount of space
    - Limited number of write cycles
    - Poor throughput between flash memory and CPU on mobile platform
- Instead use other methods to free memory if low
  - iOS ***asks*** apps to voluntarily relinquish allocated memory
    - Read-only data thrown out and reloaded from flash if needed
    - datele modificate (eg, stiva) nu se elimina din memorie
    - Failure to free can result in termination
  - Android terminates apps if low free memory, but first writes **application state** to flash for fast restart
  - Both OSes support paging as discussed below





Contiguous Allocation

- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is one early method
- Main memory usually into two **partitions**:
  - Resident operating system, usually held in low memory with interrupt vector
  - User processes then held in high memory
  - Each process contained in single contiguous section of memory





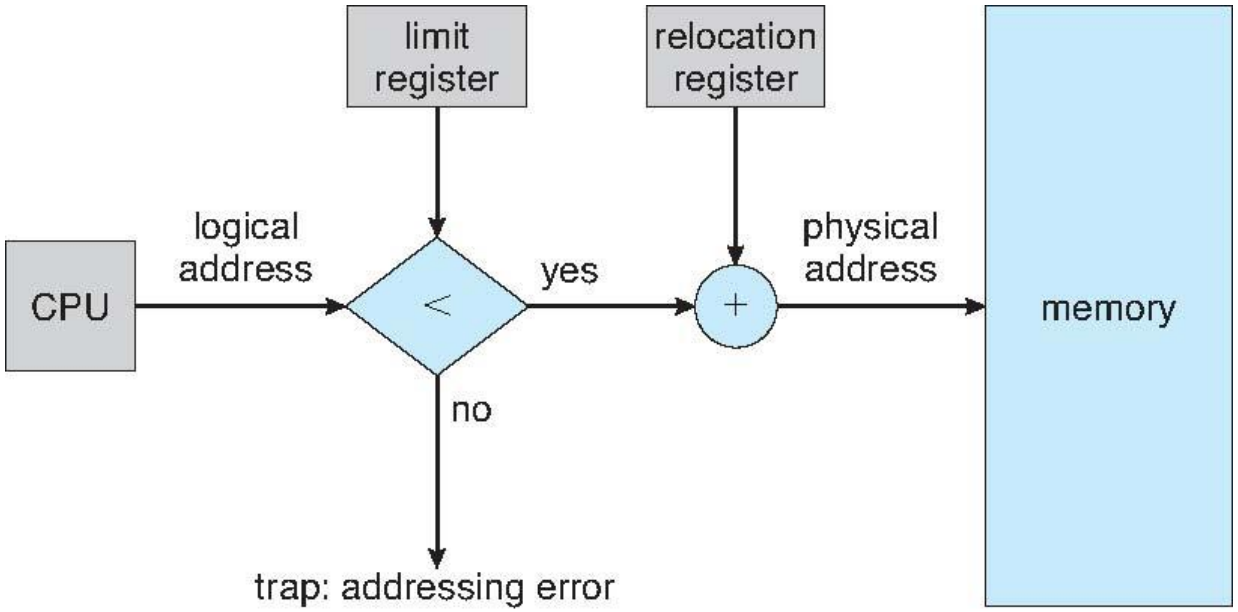
Contiguous Allocation (Cont.)

- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
  - Base register contains value of smallest physical address
  - Limit register contains range of logical addresses – each logical address must be less than the limit register
  - MMU maps logical address *dynamically*
  - Can then allow actions such as kernel code being **transient** and kernel changing size





Hardware Support for Relocation and Limit Registers

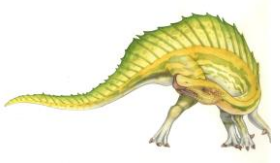






Alocarea memoriei cu partitii fixe

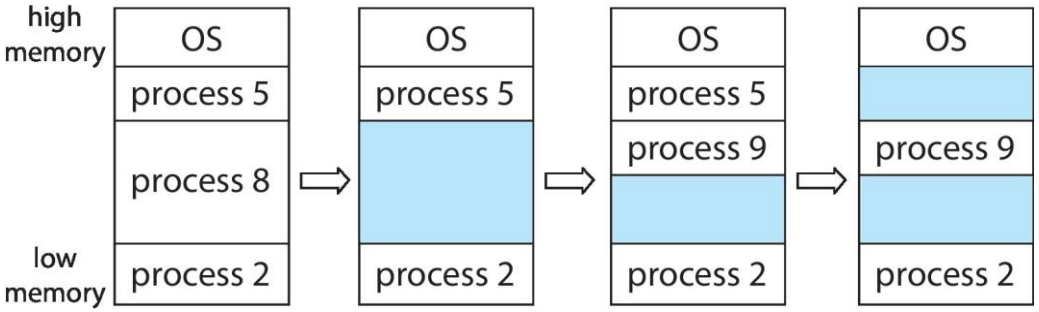
- memoria principala e impartita intr-un nr fix de partitii (partitiile pot fi inegale, in cazul cel mai general)
- fiecare partitie contine un singur proces => gradul de multiprogramare e limitat de nr de partitii
- procesele sosite in sistem sunt puse intr-o coada de asteptare pt partitii libere; partitia trebuie sa fie suficient de mare ca sa contina programul
- odata disponibila, programul se incarca in partitie si se executa
- la terminare, partitia se elibereaza si devine disponibila pt alt process
- problema: dimensiunea fixa a partitiilor implica existenta unui spatiu neutilizat (“pierdut”) de catre proces pe durata rularii (“fragmentare interna”)
  - sol. 1: cozi separate pt fiecare partitie (dezavantaj: cozi de partitii mari goale, cozi de partitii mici aglomerate)
  - sol 2: coada unica; se alege cel mai mic proces care “incape” in partitia eliberata (dezavantaj: discriminare procese mici)





Variable Partition

- Multiple-partition allocation
  - Degree of multiprogramming limited by number of partitions
  - **Variable-partition** sizes for efficiency (sized to a given process' needs)
  - **Hole** – block of available memory; holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Process exiting frees its partition, adjacent free partitions combined
  - Operating system maintains information about:
    a) allocated partitions b) free partitions (hole)







Dynamic Storage-Allocation Problem

How to satisfy a request of size ***n*** from a list of free holes?

- **First-fit**: Allocate the ***first*** hole that is big enough
- **Best-fit**: Allocate the ***smallest*** hole that is big enough; must search entire list, unless ordered by size
  - Produces the smallest leftover hole
- **Worst-fit**: Allocate the ***largest*** hole; must also search entire list
  - Produces the largest leftover hole
- **Next-fit**: similar cu first-fit, dar cautarea continua din punctul in care reamasese ultima cautare
  - performanta mai proasta decat first-fit
- **Quick-fit**: mentine liste separate pentru cele mai frecvent cerute dimensiuni de segmente
  - f. rapid, folosit des nu pt procese, ci pt alte scopuri (eg, pachete de retea Ethernet, cu dimensiune a priori cunoscuta)

First-fit and best-fit better than worst-fit in terms of speed and storage utilization





Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- First fit analysis reveals that given *N* blocks allocated, 0.5 *N* blocks lost to fragmentation
  - 1/3 may be unusable -> **50-percent rule**
  - explicatie: pp. sistem in echilibru, un proces oarecare in mijlocul memoriei
    - pe durata executiei procesului, ½ din operatiile cu segmentele de “deasupra” lui sunt alocari, ½ dealocari

    ⇒ ½ din timp procesul are ca vecin alt proces, ½ din timp are un segment neutilizat

    ⇒ pe medie, ½ din blocurile alocate sunt gauri





Analiza fragmentarii externe

- *f* = fractiunea de memorie aferenta gaurilor
- *s* = dimensiunea medie a *n* procese
- $k * s$ = dimensiunea medie a gaurilor, k > 0
- m = dimensiunea memoriei in biti

$\Rightarrow$ $n/2$ gauri de memorie ocupa $m - n * s$ biti

$\Rightarrow$ $(n/2) * k * s = m - n * s$

$\Rightarrow$ $m = n * s * (1 + k / 2)$

$\Rightarrow$ $f = ((n / 2) * k * s) / m = k / (k + 2)$

$\Rightarrow$ pt $k = ½$ (dimensiunea gaurii e ½ din dimensiunea medie a procesului)

f = 20% pierdere de memorie

pt. $k = ¼ \Rightarrow f = 11\%$





Fragmentation (Cont.)

- Reduce external fragmentation by **compaction**
  - Shuffle memory contents to place all free memory together in one large block
  - Compaction is possible *only* if relocation is dynamic, and is done at execution time
  - I/O problem
    - Latch job in memory while it is involved in I/O
    - Do I/O only into OS buffers
- In general fragmentarea externa se rezolva prin alocarea necontigua a spatiului logic (virtual) de adrese
  - solutii complementare: segmentare si/sau paginare
  - paginarea rezolva in buna masura si problema fragmentarii interne
- Now consider that backing store has same fragmentation problems





Paging

- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
  - Avoids external fragmentation
  - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
  - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size ***N*** pages, need to find ***N*** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- Backing store likewise split into pages
- Still have internal fragmentation





Paginare (cont'd)

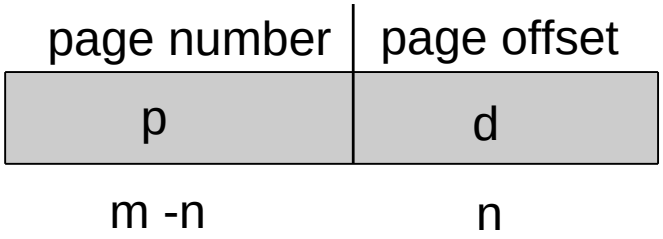
- la incarcarea programelor in memorie nu e nevoie de relocare
  - fiecare proces are tabela de pagini proprie spatiului sau de adrese virtual
  - spatiul de adrese virtual e contiguu, alocarea efectiva de memorie fizica e necontigua
- permite implementarea sistemelor de memorie virtuala
  - spatiul virtual de adresa e mai mare decat spatiul de adrese fizice (i.e., memoria calculatorului)
  - ex: spatiu de adresa pe 64 de biti (2EB, 2048 PB) chiar daca memoria e semnificativ mai mica
  - o parte din pagini sunt rezidente in memorie, restul se afla pe disc
- se potriveste multiprogramarii
  - accesul la paginile nerezidente in memorie (*page-fault*) determina oprirea procesului pana la incarcarea paginii de pe disc in memorie, prilej ideal pt a oferi procesorul altui proces





Address Translation Scheme

- Address generated by CPU is divided into:
  - **Page number** (***p***) – used as an index into a **page table** which contains base address of a frame ***f*** in physical memory
  - **Page offset** (***d***) – combined with base address to define the physical memory address that is sent to the memory unit
  - *perspectiva alternativa, analogie cu relocarea*: ***f*** este base register, iar limita este data de dimensiunea paginii/frame-ului

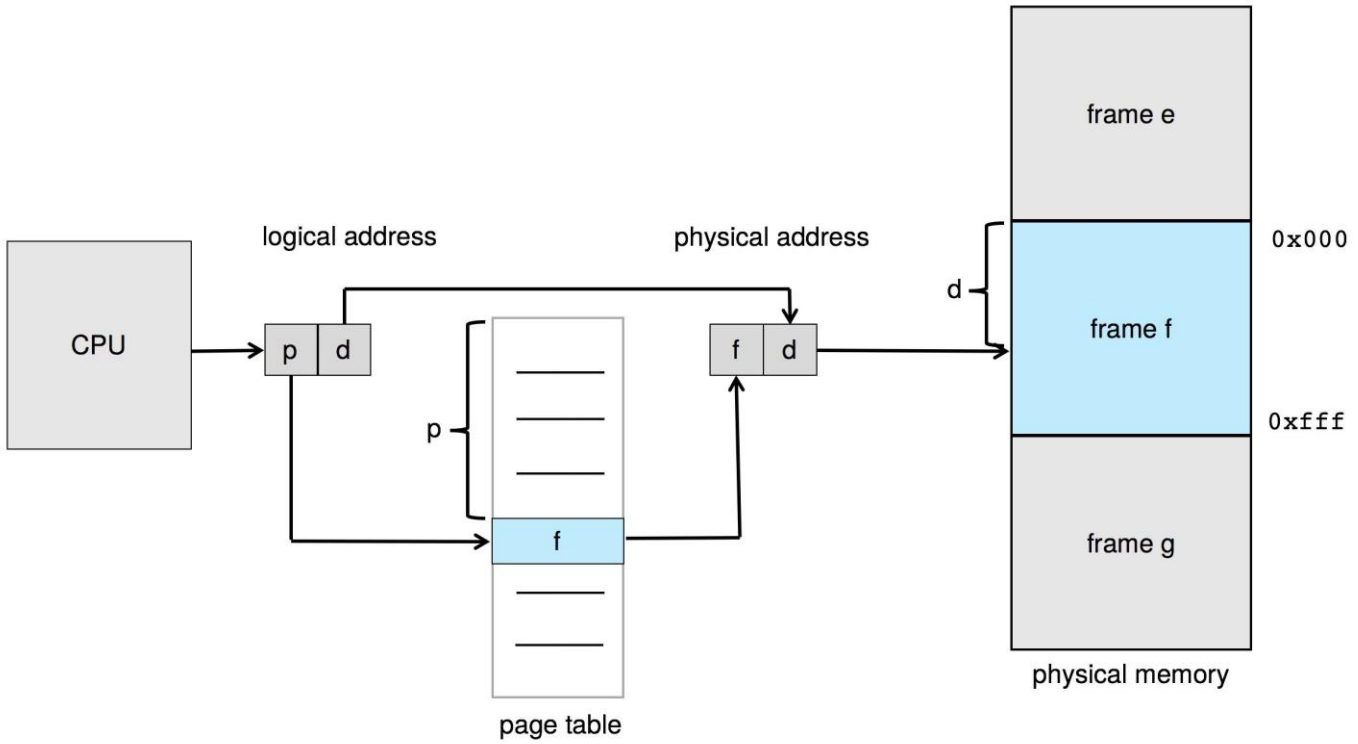


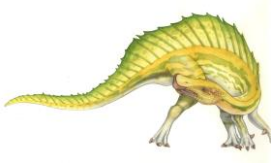
  - For given logical address space 2^m and page size 2^n



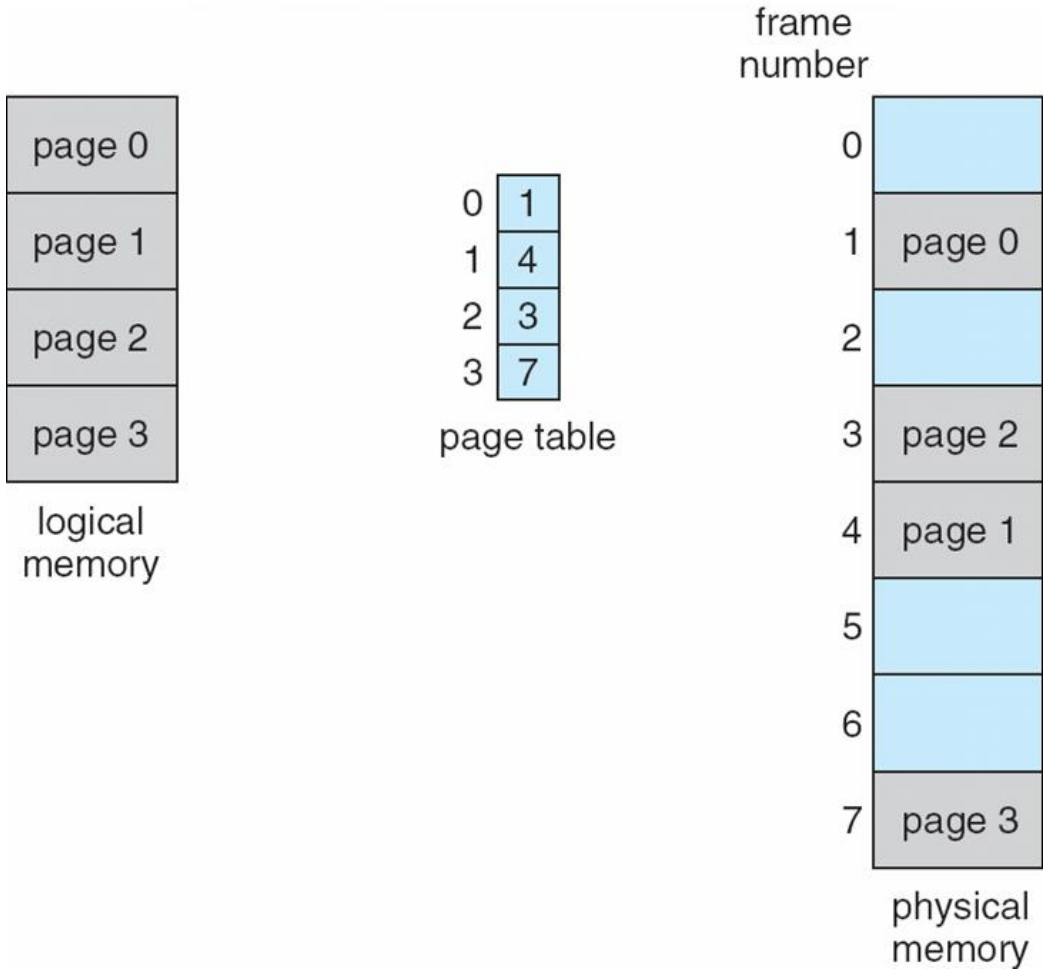


Paging Hardware



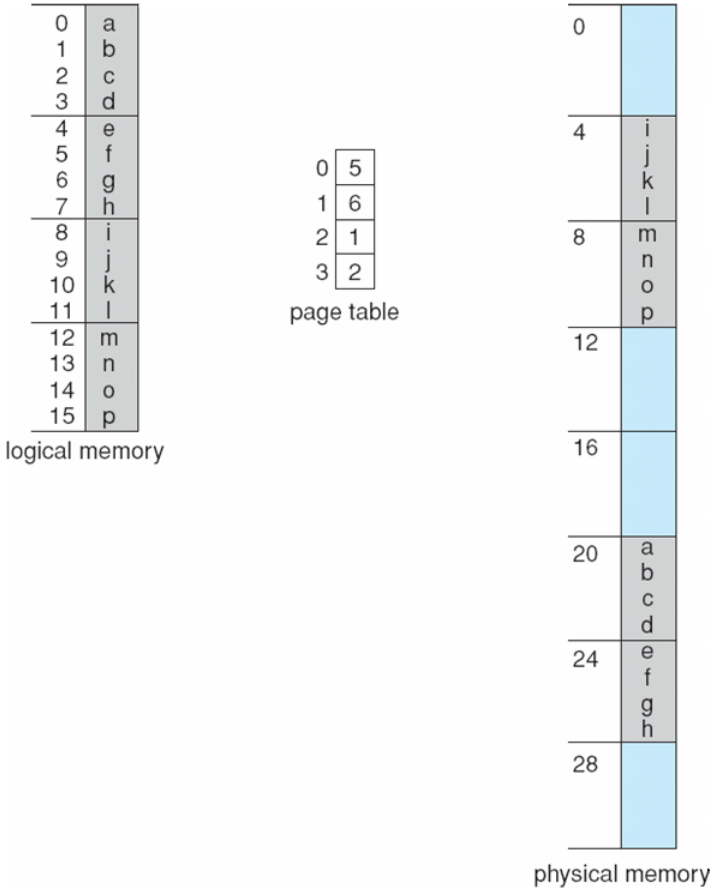


Paging Model of Logical and Physical Memory



Paging Example

- Logical address: n = 2 and m = 4. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)





Paging -- Calculating internal fragmentation

- paginarea exclude fragmentarea externa (orice frame liber poate fi alocat unui proces care are nevoie de el)
- in schimb, nu elimina complet fragmentarea interna, de ex:
  - Page size = 2,048 bytes
  - Process size = 72,766 bytes
  - 35 pages + 1,086 bytes
  - Internal fragmentation of 2,048 - 1,086 = 962 bytes
- Worst case fragmentation = 1 frame – 1 byte
- On average fragmentation = 1 / 2 frame size
- So small frame sizes desirable?
- But each page table entry takes memory to track
- Page sizes growing over time
  - Solaris supports two page sizes – 8 KB and 4 MB





Analiza optimalitatii dimensiunii paginii

- Q: care e dimensiunea optima a paginii?
- pagini mici => fragmentare interna mica, dar tabela de pagini mare (cu multe intrari)
- paginile mari => tabela de pagini mai mica, dar si I/O mai eficient cu discul (totusi timpul de acces la disc e dominat de componenta de rotatie si cautare, adica mutarea capului de citire)
- analiza sumara:
  - pp. *s* = dimensiunea medie a procesului in bytes

  iar *p* = dimensiunea paginii in bytes

  $\Rightarrow s / p$ pagini per proces si $s * e / p$ bytes per tabela de pagini

  unde *e* = nr de bytes din PTE (Page Table Entry)
  - in medie, spatiul de memorie pierdut din cauza fragmentarii interne a ultimei pagini este $p / 2$

  => overhead total de memorie: $s * e / p + p / 2$





Analiza optimalitatii dimensiunii paginii

- obs: cei doi termeni ai overheadului evolueaza contrar in functie de valoarea lui *p*
  - creste *p*, scade dimensiunea tabelei de pagini si creste fragmentarea interna
  - scade *p*, scade fragmentarea interna si creste dimensiunea tabelei de pagini
- pt a determina valoarea optima a lui *p*, derivam si rezolvam ecuatia

$$-\frac{s*e}{p^2}+\frac{1}{2}=0$$

$$\Rightarrow \quad p=\sqrt{2se}$$

- ex:

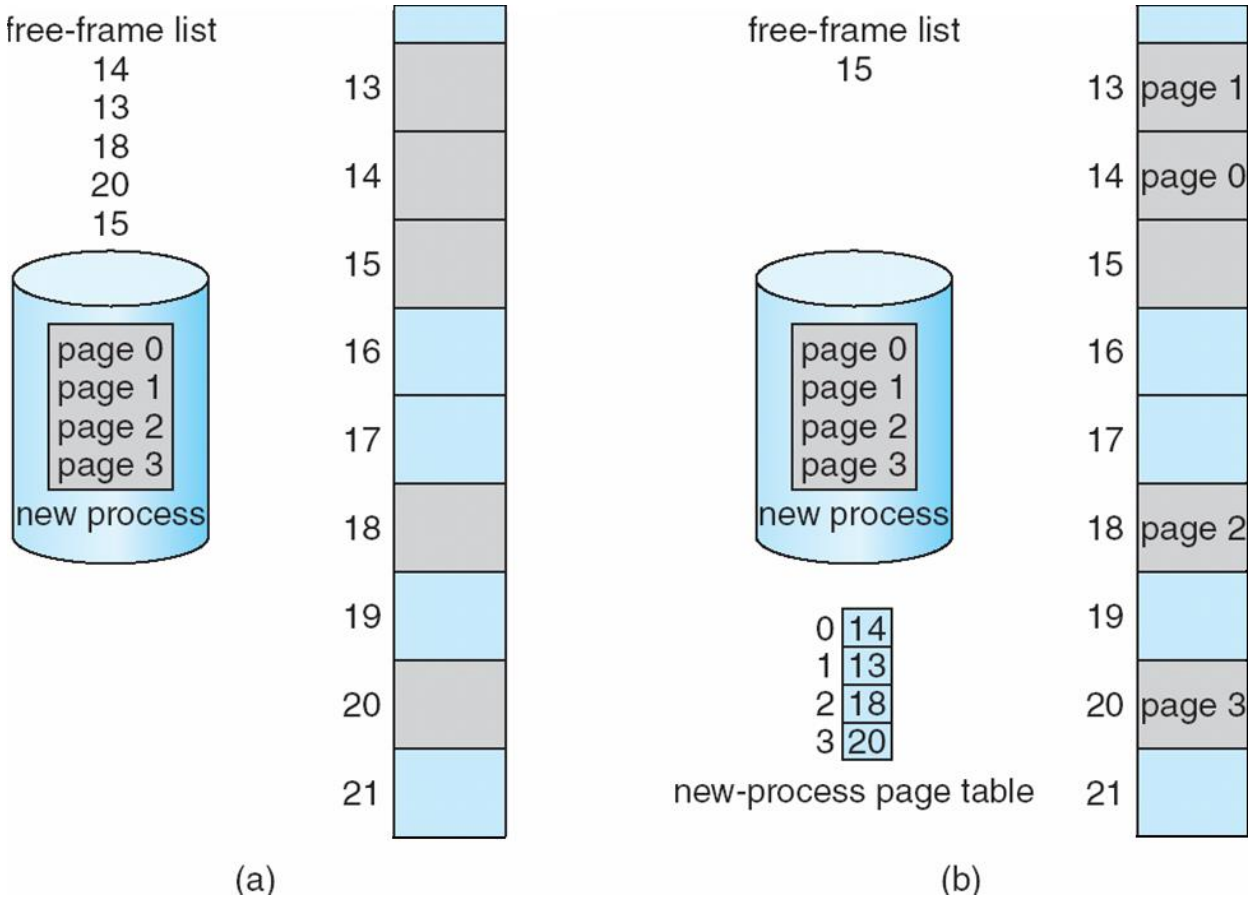
s = 512 KB si e = 4 bytes => $p=\sqrt{2*2^2*2^{19}}=2^{11}=2KB$

- in general, se iau in considerare si alti factori (eg, viteza discului) pt stabilirea dimensiunii optime a paginii





Free Frames



Before allocation

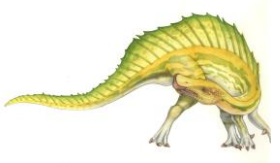
After allocation





Idei principale ale paginarii

- separare clara intre perspectiva programatorului asupra memoriei si memoria fizica
  - programatorul vede memoria ca un spatiu contiguu care contine programul
  - in realitate, programul este “imprastiat” in memoria fizica, paginile sale “amestecandu-se” cu paginile altor programe
- diferenta de perspectiva este gestionata de MMU care pune in corespondenta adrese logice/virtuale cu adrese fizice (pagini -> frame-uri)
- corespondenta este transparenta pentru utilizator (ascunsa de catre kernel)
- tabela de pagini e per proces => acesta nu poate accesa alte pagini decat paginile sale
- SO mentine contabilitatea frame-urilor alocate si respectiv a celor libere (in general se foloseste o *tabela de frame-uri*)





Idei principale ale paginarii (cont'd)

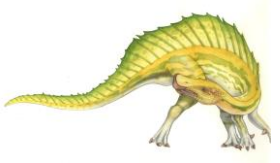
- sistemul de operare mentine cate o tabela de pagini pt fiecare proces ca parte a PCB-ului
- aceasta se incarca in structurile de suport HW pt paginare ale procesorului (MMU) cand procesul este ales pt a rula de catre planificator (scheduler)
- in consecinta, paginarea afecteaza timpul de context-switch





Implementation of Page Table

- Page table is kept in main memory
  - **Page-table base register** (**PTBR**) points to the page table
  - **Page-table length register** (**PTLR**) indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table and one for the data / instruction
- The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called **translation look-aside buffers** (**TLBs**) (also called **associative memory**).





TLB

- cache de mare viteza din MMU care mentine copiile celor mai des folosite intrari din tabela de pagini (PTEs)
- o intrare in TLB contine
  - numarul de pagina si intrarea corespunzatoare din tabela de pagini
  - un bit de validitate care specifica daca intrarea e in uz sau nu
- TLB nu doar ca e mai rapid ca memoria principala, dar difera si ca functionare
  - intrarile TLB sunt referite nu prin adresa ci prin continut
  - fie *pg* numarul paginii asociate de intrarea in TLB cu *pt*, intrarea din tabela de pagini
  - TLB-ul compara **in paralel** *pg* cu toate numerele de pagina din toate intrarile sale
  - daca se gaseste o potrivire, valoarea *pt* corespunzatoare se foloseste de catre MMU pt mapare





TLB miss

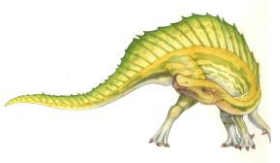
- daca nu se gaseste nici o potrivire in TLB (*TLB miss*):
  - MMU cauta in tabela de pagini din memoria principala o mapare valida
    - daca o gaseste, inlocuieste o intrare din TLB cu noua mapare
  - daca intrarea inlocuita reprezinta eventual o pagina modificata, MMU marcheaza corespunzator intrarea din tabela de pagini
- consecintele multiprogramarii pt operarea TLB-ului
  - la context-switch se schimba tabelele de pagini => maparile din TLB nu mai sunt valide
  - solutii posibile
    - invalidarea intregului TLB, cu consecinte dramatice pt performanta sistemului
    - folosirea PID-ului la indexarea in TLB





Translation Look-Aside Buffer

- Some TLBs store **address-space identifiers** (**ASIDs**) in each TLB entry – uniquely identifies each process to provide address-space protection for that process
  - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
  - Replacement policies must be considered
  - Some entries can be **wired down** for permanent fast access
- in procesoarele moderne, TLB face parte din instruction pipeline
- unele procesoare au TLB separate pt instructiuni si date





Hardware

- Associative memory – parallel search

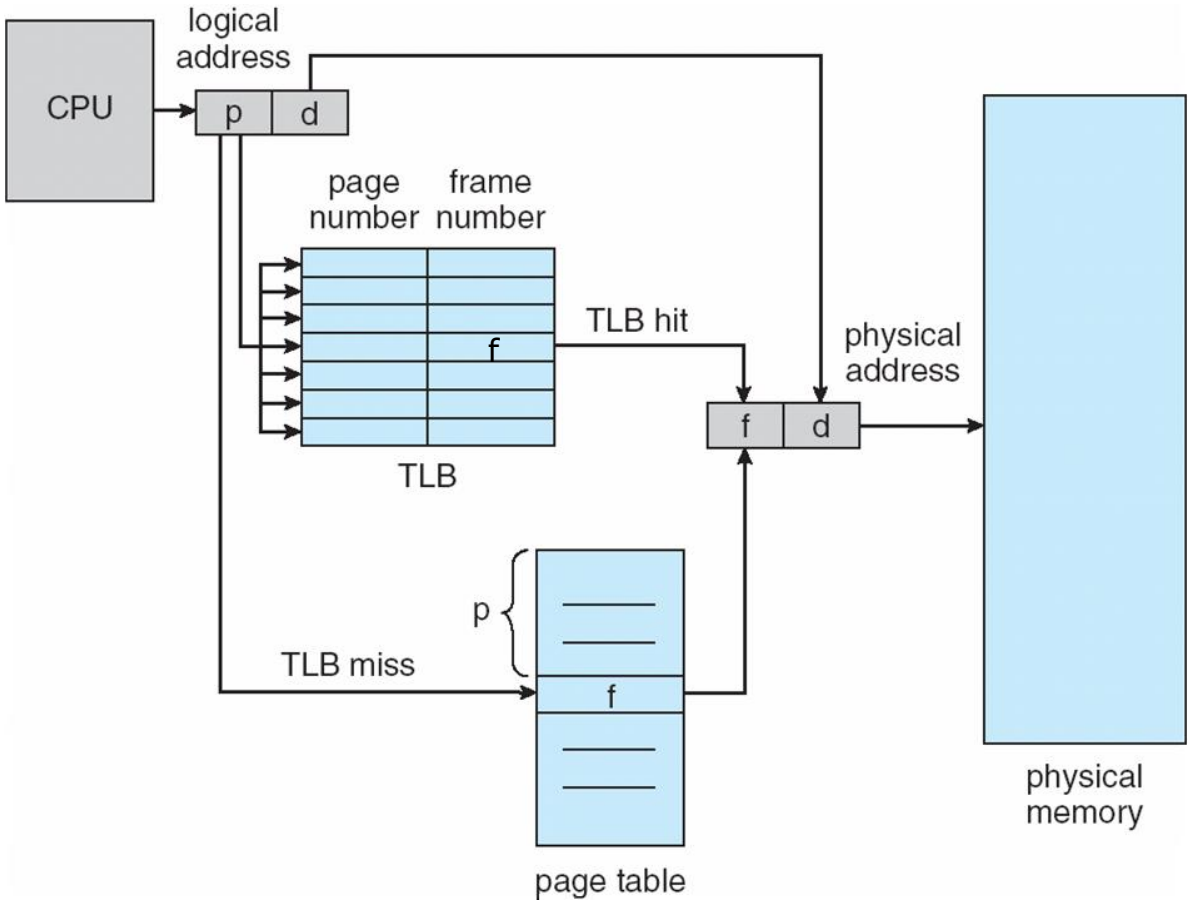
Page #	Frame #

- Address translation (p, d)
  - If p is in associative register, get frame # out
  - Otherwise get frame # from page table in memory





Paging Hardware With TLB







Effective Access Time

- Hit ratio – percentage of times that a page number is found in the TLB
- An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose that 10 nanoseconds to access memory.
  - If we find the desired page in TLB then a mapped-memory access take 10 ns
  - Otherwise we need two memory access so it is 20 ns
- **Effective Access Time** (**EAT**)

  EAT = 0.80 x 10 + 0.20 x 20 = 12 nanoseconds

  implying 20% slowdown in access time
- Consider a more realistic hit ratio of 99%,

  EAT = 0.99 x 10 + 0.01 x 20 = 10.1ns

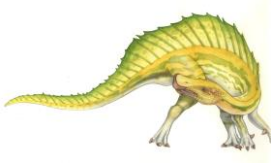
  implying only 1% slowdown in access time.





Informatie aditionala in PTE

- *present bit* P
  - 1 daca frame-ul corespunzator exista in memorie, 0 altfel
  - fiecare acces la memorie generat de proces trece prin MMU
    - P = 1 in PTE-ul corespunzator accesului => acces permis
    - P = 0, MMU genereaza o exceptie numita *page fault*; kernelul trateaza exceptia (aduce pagina corespunzatoare de pe disc si o incarca intr-un frame liber pe care il noteaza in PTE)
  - astfel, doar o parte din proces e in memorie, restul e pe disc
- *referenced bit* R – 1 daca pagina a fost referita (citita/scrisa)
- *modified bit* M - 1 daca pagina a fost scrisa
- *permisiuni* RWX – specifica daca pagina poate fi citita, scrisa, executata
- obs: acesti biti ocupa spatiu in PTE => raman mai putini biti disponibili pt adresare, adica se reduce spatiul de adresare virtuala





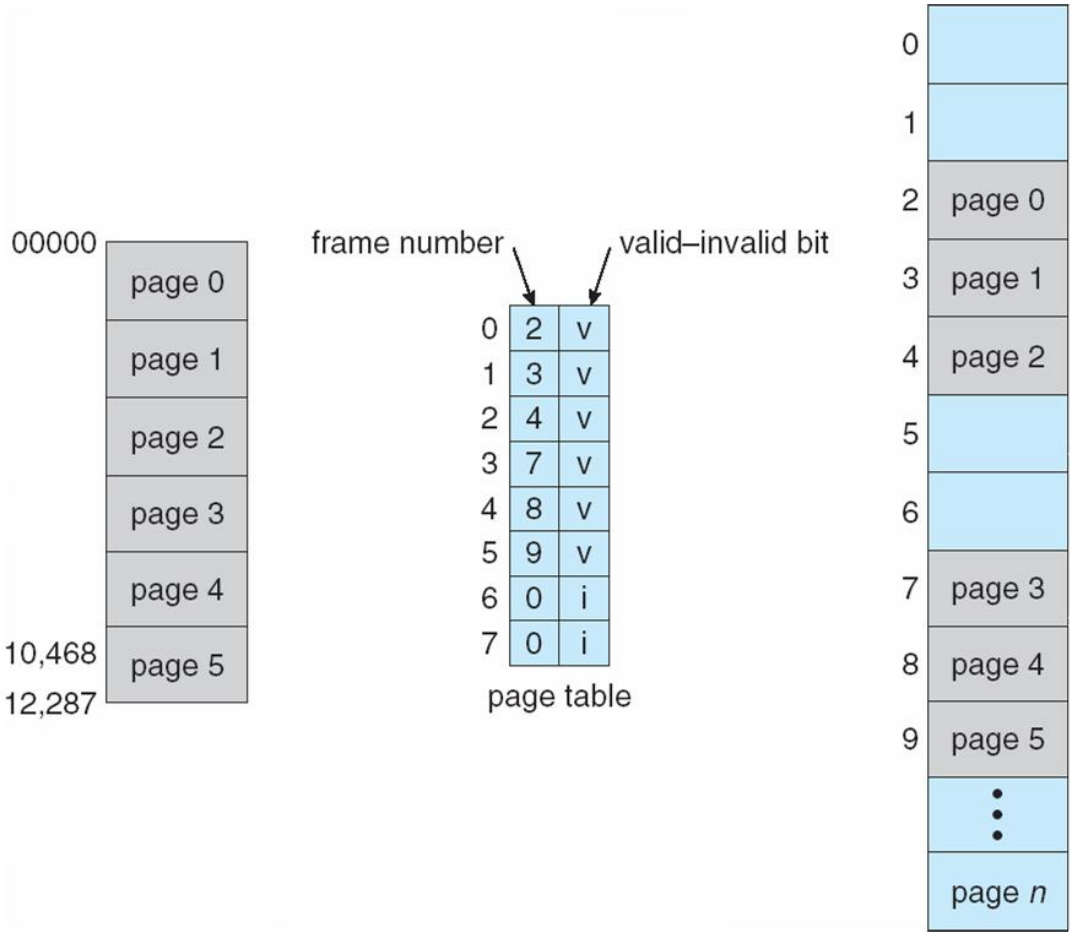
Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
  - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space
  - Or use **page-table length register** (**PTLR**)
- Any violations result in a trap to the kernel





Valid (v) or Invalid (i) Bit In A Page Table







Shared Pages

- **Shared code**
  - One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
  - Similar to multiple threads sharing the same process space
  - Also useful for interprocess communication if sharing of read-write pages is allowed
- **Private code and data**
  - Each process keeps a separate copy of the code and data
  - The pages for the private code and data can appear anywhere in the logical address space





Shared Pages Example

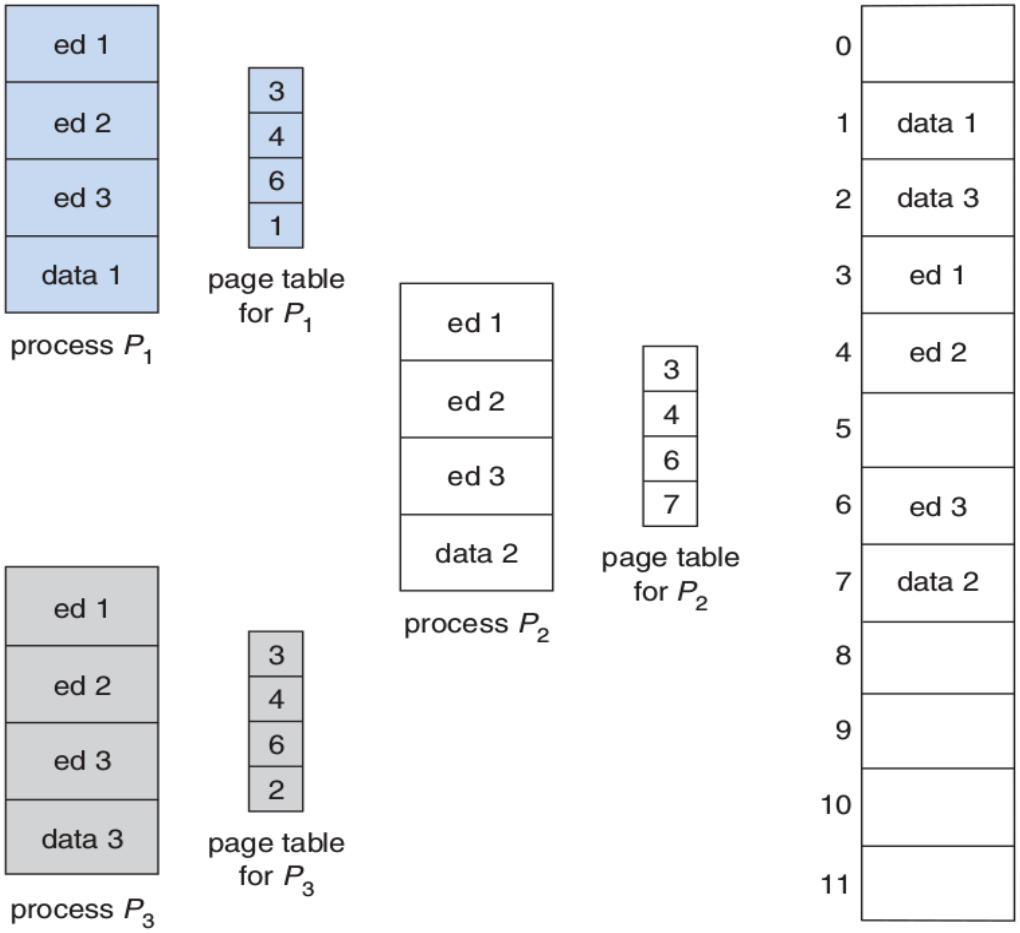


Figure 7.16 Sharing of code in a paging environment.





Structure of the Page Table

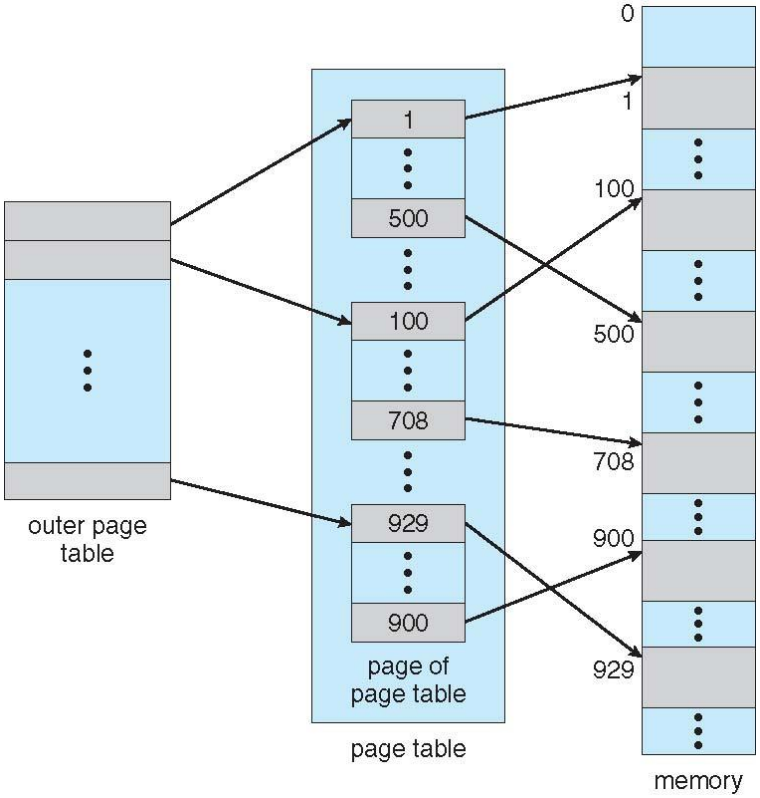
- Memory structures for paging can get huge using straight-forward methods
  - Consider a 32-bit logical address space as on modern computers
  - Page size of 4 KB (2^{12})
  - Page table would have 1 million entries ($2^{32} / 2^{12}$)
  - If each entry is 4 bytes ➔ each process 4 MB of physical address space for the page table alone
    - Don't want to allocate that contiguously in main memory
  - One simple solution is to divide the page table into smaller units
    - Hierarchical Paging
    - Hashed Page Tables
    - Inverted Page Tables

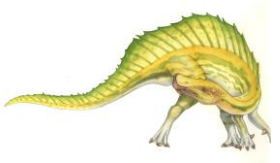




Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table

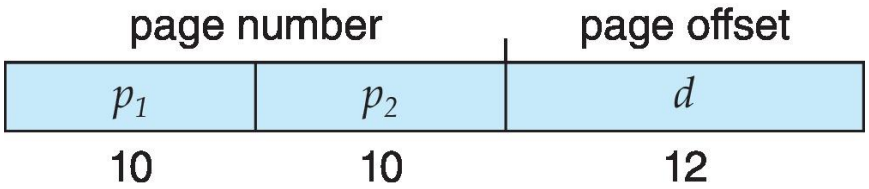




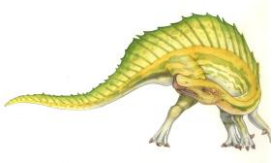


Two-Level Paging Example

- A logical address (on 32-bit machine with 4K page size) is divided into:
  - a page number consisting of 20 bits
  - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
  - a 10-bit page number
  - a 10-bit page offset
- Thus, a logical address is as follows:

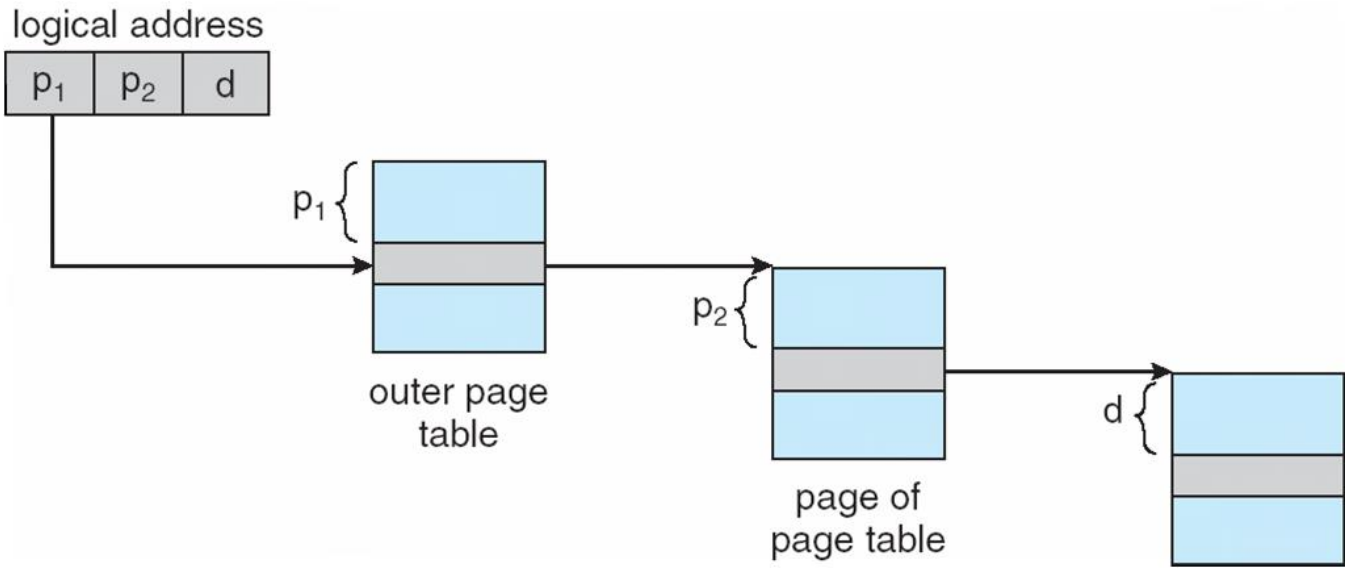


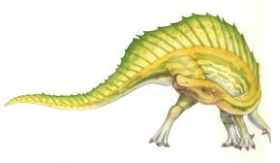
- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**





Address-Translation Scheme

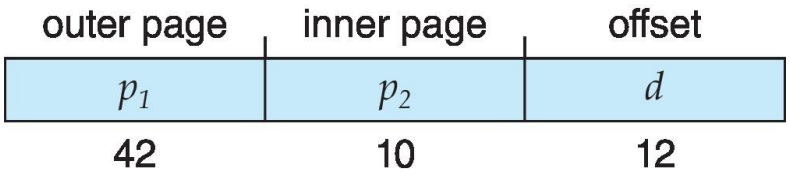






64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
  - Then page table has 2^{52} entries
  - If two level scheme, inner page tables could be 2^{10} 4-byte entries
  - Address would look like

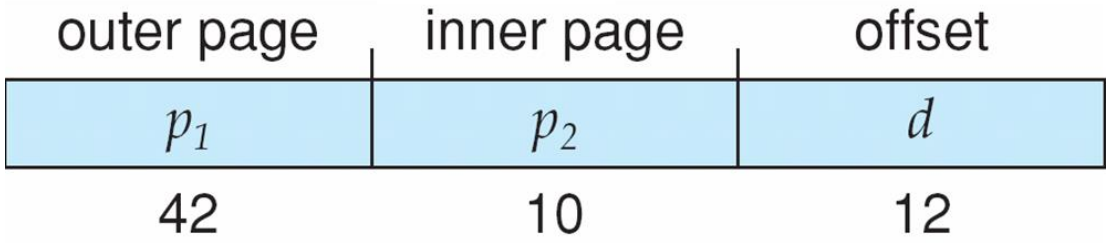


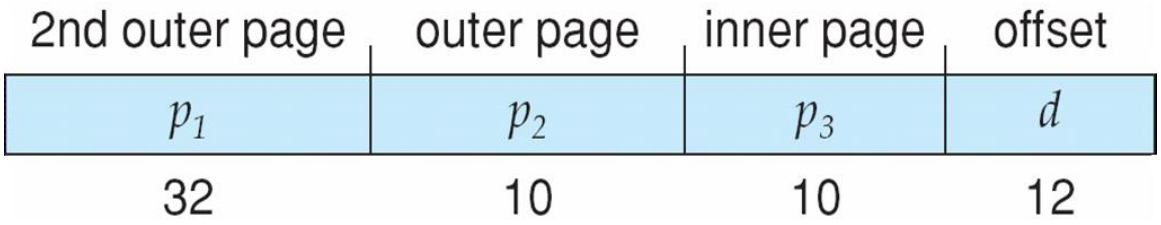
  - Outer page table has 2^{42} entries or 2^{44} bytes
  - One solution is to add a 2nd outer page table
  - But in the following example the 2nd outer page table is still 2^{34} bytes in size
    - And possibly 4 memory access to get to one physical memory location





Three-level Paging Scheme



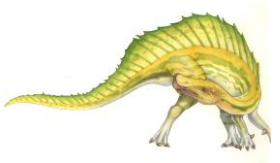






Zero-level paging

- paginare de nivel zero intalnita la unele procesoare RISC (MIPS R3000)
- MIPS R3000 implementeaza doar TLB si nu are HW dedicat pt cautarea in tabelele de pagini
- la TLB miss se genereaza o exceptie si se da controlul SO
- handlerul de exceptie consulta tabelele de pagini din memorie si incarca noua mapare in TLB dupa ce in prealabil a scos o intrare daca nu era spatiu
  - in plus, daca pagina accesata nu era in memorie, executa in prealabil si codul de page fault handler
- motivatia pt acest tip de design
  - simplitatea chip-ului
  - studii de simulare care au calculat TLB miss rate-ul si timpul de incarcare a unei intrari in TLB au aratat ca performanta e acceptabila





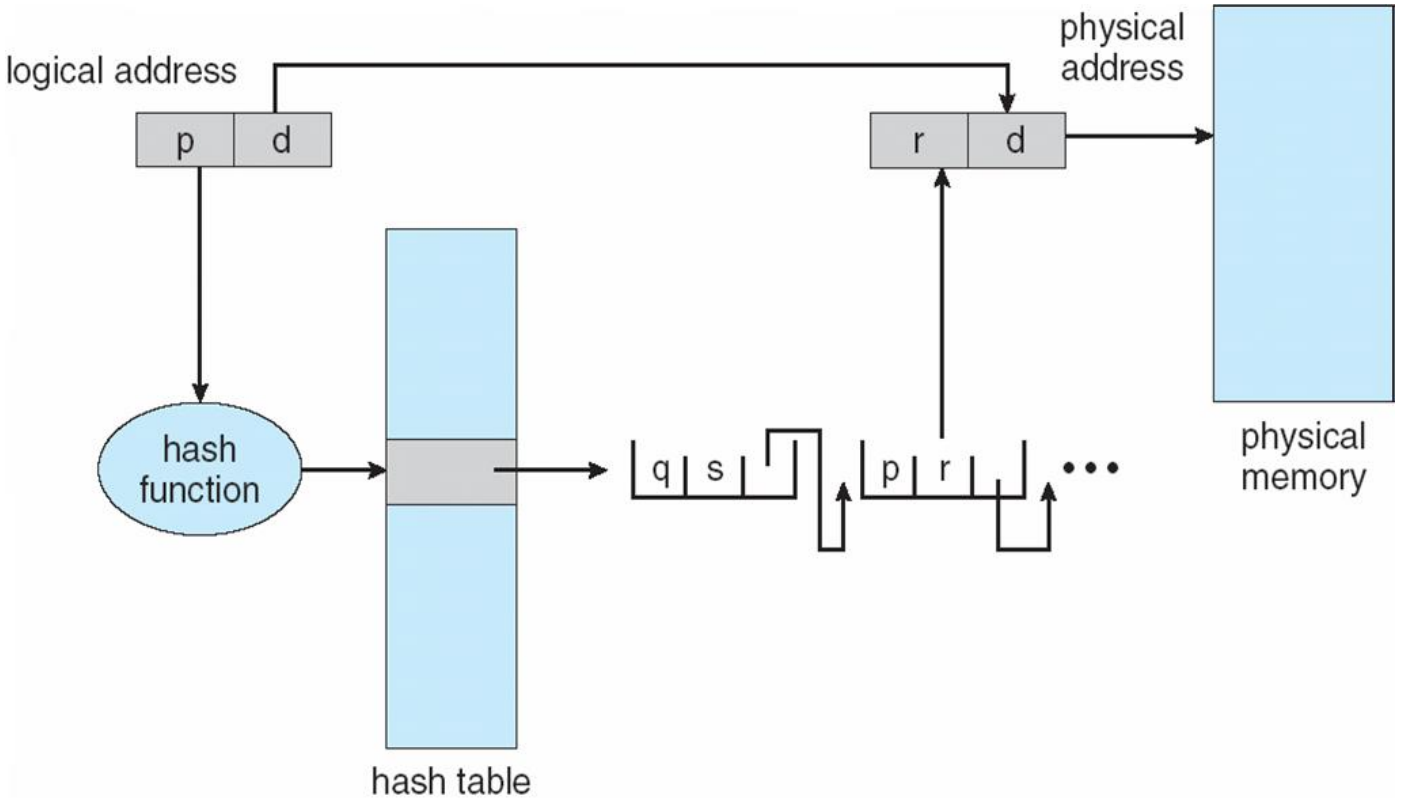
Hashed Page Tables

- Common in address spaces > 32 bits
- The virtual page number is hashed into a page table
  - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
  - If a match is found, the corresponding physical frame is extracted
- Variation for 64-bit addresses is **clustered page tables**
  - Similar to hashed but each entry refers to several pages (such as 16) rather than 1
  - Especially useful for **sparse** address spaces (where memory references are non-contiguous and scattered)





Hashed Page Table







Inverted Page Table

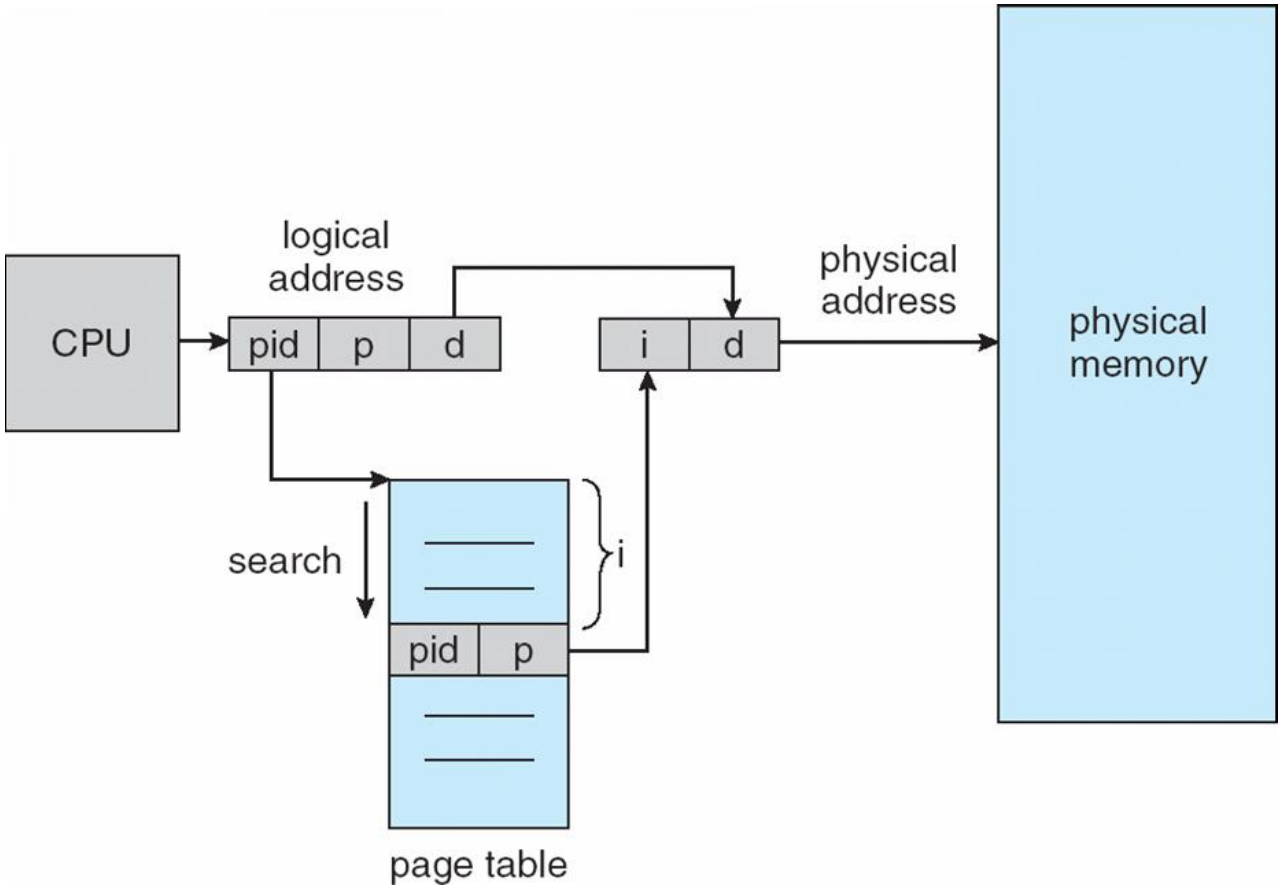
- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
  - TLB can accelerate access
- But how to implement shared memory?
  - One mapping of a virtual address to the shared physical address

=> referintele la adrese virtuale nemapate in tabela inversata rezulta in page-faults





Inverted Page Table Architecture







Segmentarea memoriei

- paginarea ofera un tip de memorie virtuala unidimensionala (i.e., exista un singur spatiu de adrese logice/virtuale)
- vizunea programatorului asupra programului sau nu e liniara
  - programul e o colectie de metode, proceduri, functii, structuri de date (vectori, arbori, tabele hash, etc), etc
  - colectie de segmente de memorie disparate, de dimensiuni variabile
  - elementele unui segment sunt identificate prin offset-ul fata de inceputul segmentului
- programatorul se refera la stiva, biblioteci, etc fara sa considere ce adrese de memorie ocupa aceste componente ale programului sau





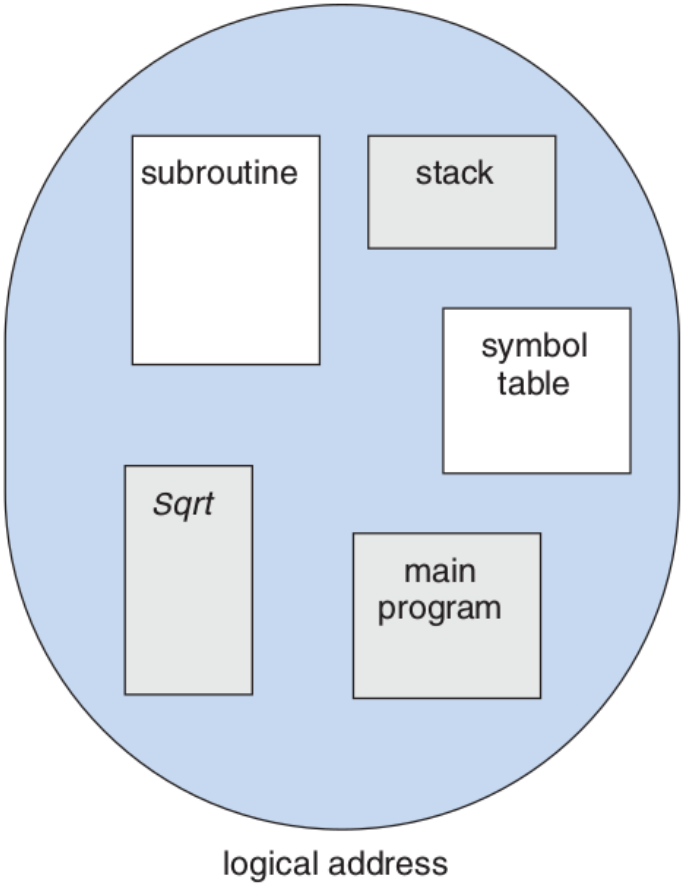
Segmentarea memoriei (cont'd)

- spatiul logic/virtual de adrese devine o colectie de segmente (eg, cod, variabile globale, heap, stive pt thread-uri, libc, etc)
  - fiecare segment are un nume si o lungime
  - adresare: nume segment + offset
  - simplificare: adresa logica/virtuala devine <nr segment, offset>





Segmentare (cont'd)

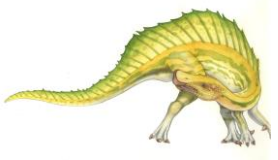






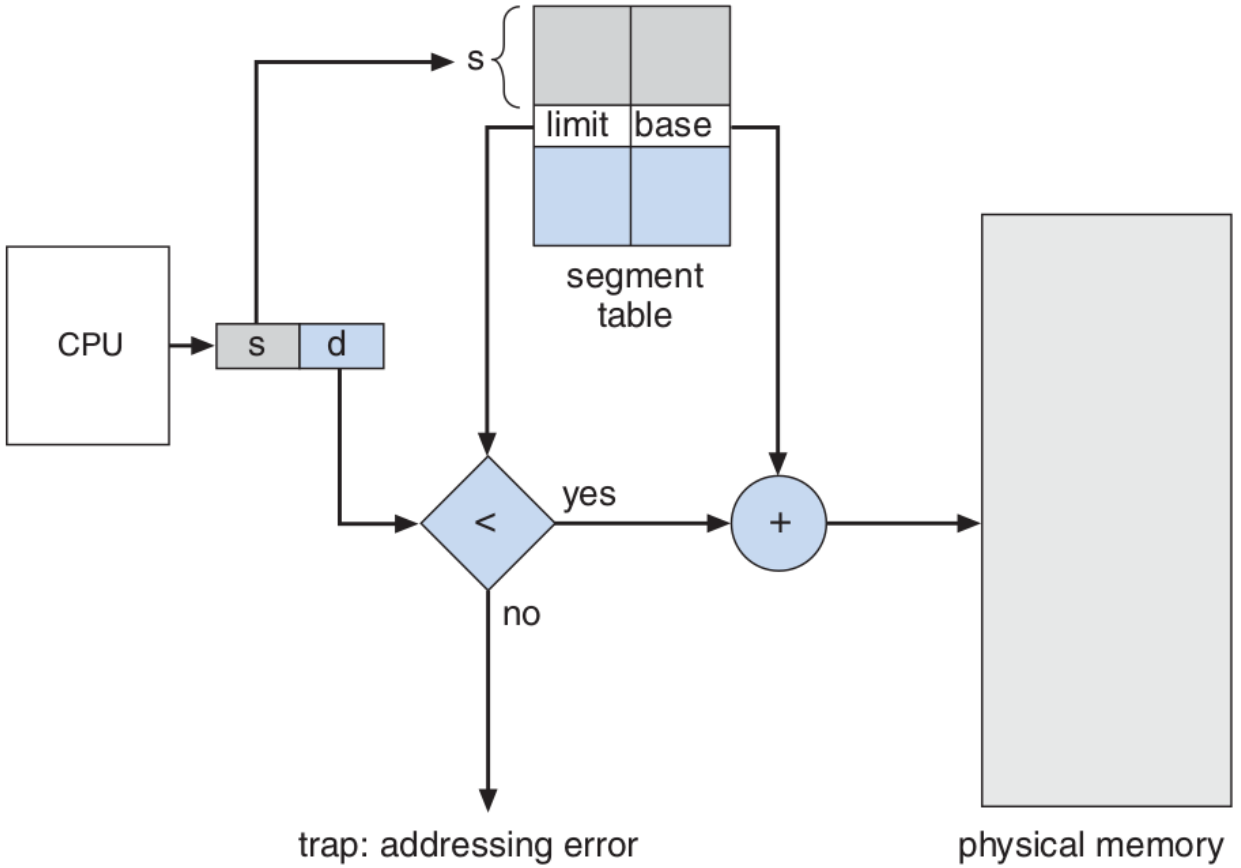
Suport HW pt. segmentare

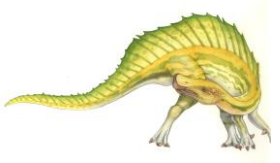
- adresa logica/virtuala bidimensionala, dar adresa fizica liniara
- HW mapeaza adresa logica – fizica folosind o tabela de segmente
- fiecare intrare in tabela are *segment base* si *limit*
- nr de segment indexeaza in tabela, iar offsetul din adresa logica trebuie sa fie intre 0 si limita





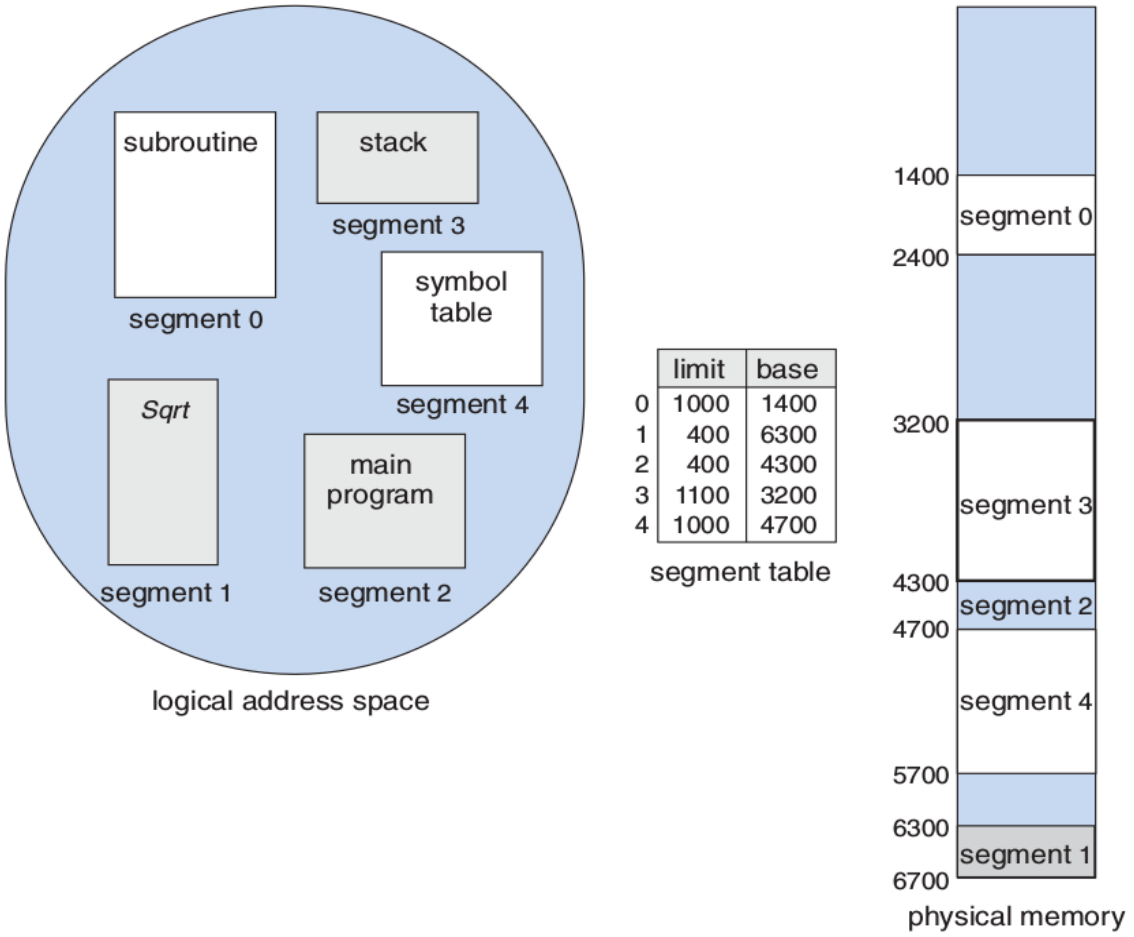
Suport HW pt segmentare







Exemplu segmentare







Avantajele segmentarii

- componentele logice ale programului pot avea dimensiuni arbitrare care se pot chiar modifica dinamic
- spatiile aferente acestor componente logice (segmentele) sunt independente unele de altele

  => nu exista interferente atunci cand dimensiunile lor se modifica dinamic
- existenta structurarii logice in spatii virtuale distincte faciliteaza partajarea de componente intre programe
  - fiecare componente logica a programului poate fi partajata independent de celelalte





Caracteristici ale segmentarii memoriei

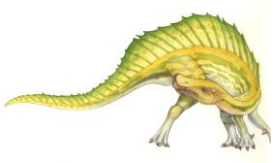
- fiecare spatiu de adrese virtuale independent s.n. *segment* = secventa liniara de adrese de la 0 la o valoare maxima
- segmentele diferite au in mod uzual dimensiuni diferite
- lungimea segmentelor poate varia dinamic
- programatorul e constient de existenta segmentelor si de regula nu le amesteca (eg, cod si date in acelasi segment)
- partajare simplificata a componentelor programului
  - ex: biblioteci partajate (se pun in acelasi segment independent care e partajat de procese)
- linkeditare simplificata a componentelor unui program
  - fiecare procedura are propriul segment, modificarile ulterioare de cod au efect local (nu necesita modificarile altor proceduri)
- segmentele au protectie individuala, ceea ce ajuta la identificarea rapida a erorilor
  - segment de cod RO si X, vector RW dar nu X





Paginare vs. segmentare

- programatorul e constient de existenta segmentelor, dar nu si a paginarii
- paginarea are loc intr-un singur spatiu de adrese virtuale
- in ambele cazuri, spatial de adrese virtual poate depasi dimensiunea memoriei fizice
- paginarea nu distinge intre continutul paginilor (eg, cod vs date) => nu poate asigura protectie specifica
- paginarea nu permite cu usurinta modificarea dinamica a spatiului de adrese virtuale
- paginarea nu faciliteaza partajarea componentelor logice ale programului in nici un fel





Concluzii segmentare

- paginarea e transparenta pt. programator si se foloseste in principal pt a putea incarca si rula programe mai mari decat memoria fizica
- segmentarea ajuta programatorul sa imparta programul in componente logice distincte pe care le aloca in spatii virtuale independente, pe care le poate proteja si partaja cu usurinta
- limitarile segmentarii pure
  - segmentele au dimensiuni variabile, reapar problemele de la multiprogramarea cu partitii variable, gauri, nevoie de compactare, etc
  - daca segmentele sunt prea mari, e posibil sa nu incapa integral in memorie => apare ideea de a combina paginarea cu segmentarea, i.e. *segmentarea cu paginare*





Oracle SPARC Solaris

- Consider modern, 64-bit operating system example with tightly integrated HW
  - Goals are efficiency, low overhead
- Based on hashing, but more complex
- Two hash tables
  - One kernel and one for all user processes
  - Each maps memory addresses from virtual to physical memory
  - Each entry represents a contiguous area of mapped virtual memory,
    - More efficient than having a separate hash-table entry for each page
  - Each entry has base address and span (indicating the number of pages the entry represents)





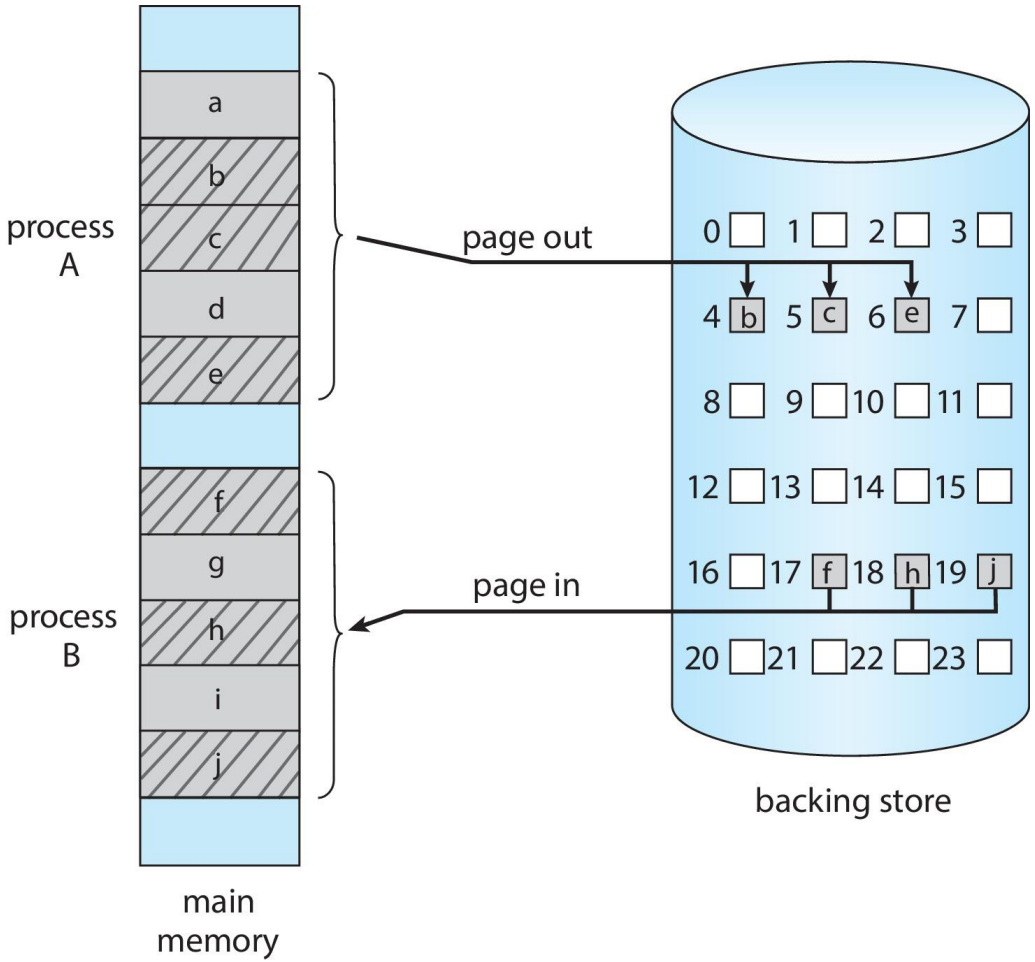
Oracle SPARC Solaris (Cont.)

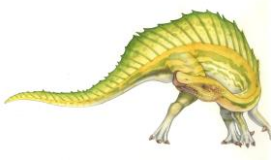
- TLB holds translation table entries (TTEs) for fast hardware lookups
  - A cache of TTEs reside in a translation storage buffer (TSB)
    - Includes an entry per recently accessed page
- Virtual address reference causes TLB search
  - If miss, hardware walks the in-memory TSB looking for the TTE corresponding to the address
    - If match found, the CPU copies the TSB entry into the TLB and translation completes
    - If no match found, kernel interrupted to search the hash table
      - The kernel then creates a TTE from the appropriate hash table and stores it in the TSB, Interrupt handler returns control to the MMU, which completes the address translation.





Swapping with Paging

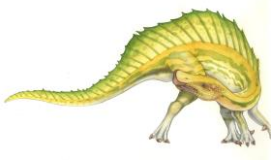






Example: The Intel 32 and 64-bit Architectures

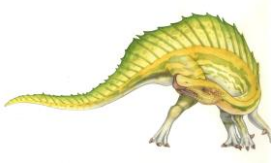
- Dominant industry chips
- Pentium CPUs are 32-bit and called IA-32 architecture
- Current Intel CPUs are 64-bit and called IA-64 architecture
- Many variations in the chips, cover the main ideas here





Example: The Intel IA-32 Architecture

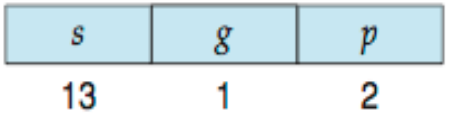
- Supports both segmentation and segmentation with paging
  - Each segment can be 4 GB
  - Up to 16 K segments per process
  - Logical address space divided into two partitions
    - First partition of up to 8 K segments are private to process (kept in **local descriptor table** (**LDT**))
    - Second partition of up to 8K segments shared among all processes (kept in **global descriptor table** (**GDT**))
    - fiecare intrare din LDT/GDT are un descriptor de segment pe 8 bytes (include base + limit)





Example: The Intel IA-32 Architecture (Cont.)

- CPU generates logical address (selector + offset)
  - Selector given to segmentation unit
    - Which produces linear addresses

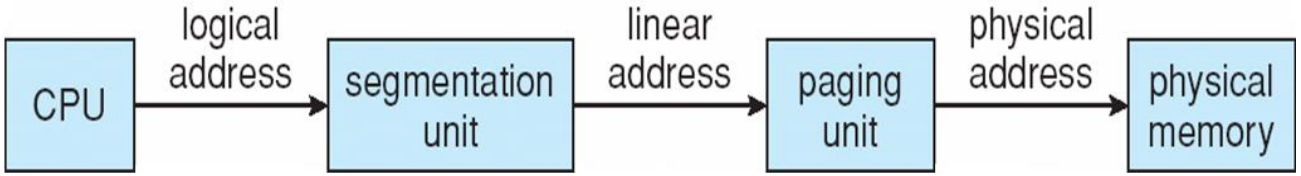


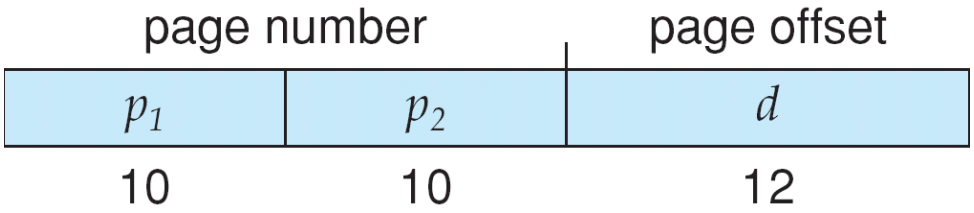
  - Linear address given to paging unit
    - Which generates physical address in main memory
    - Paging units form equivalent of MMU
    - Pages sizes can be 4 KB or 4 MB

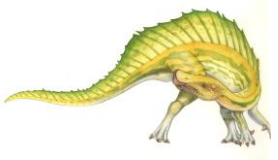




Logical to Physical Address Translation in IA-32

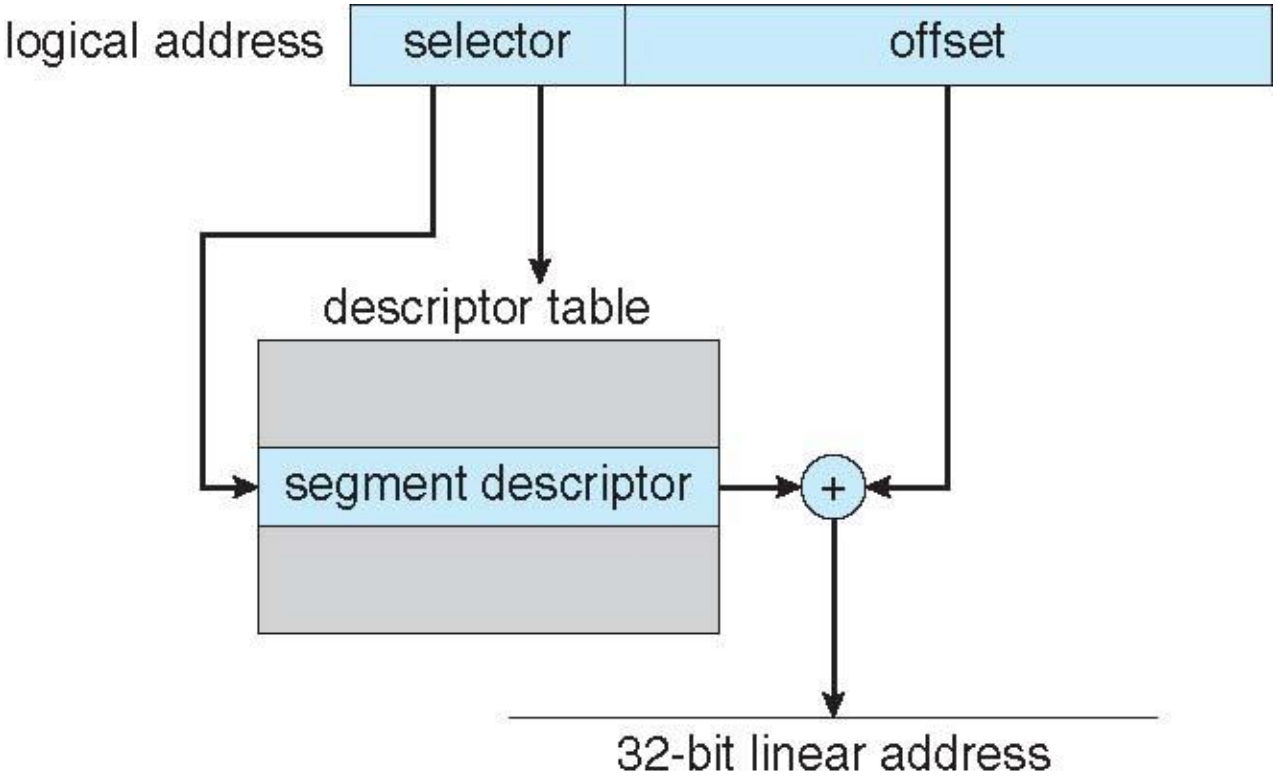








Intel IA-32 Segmentation

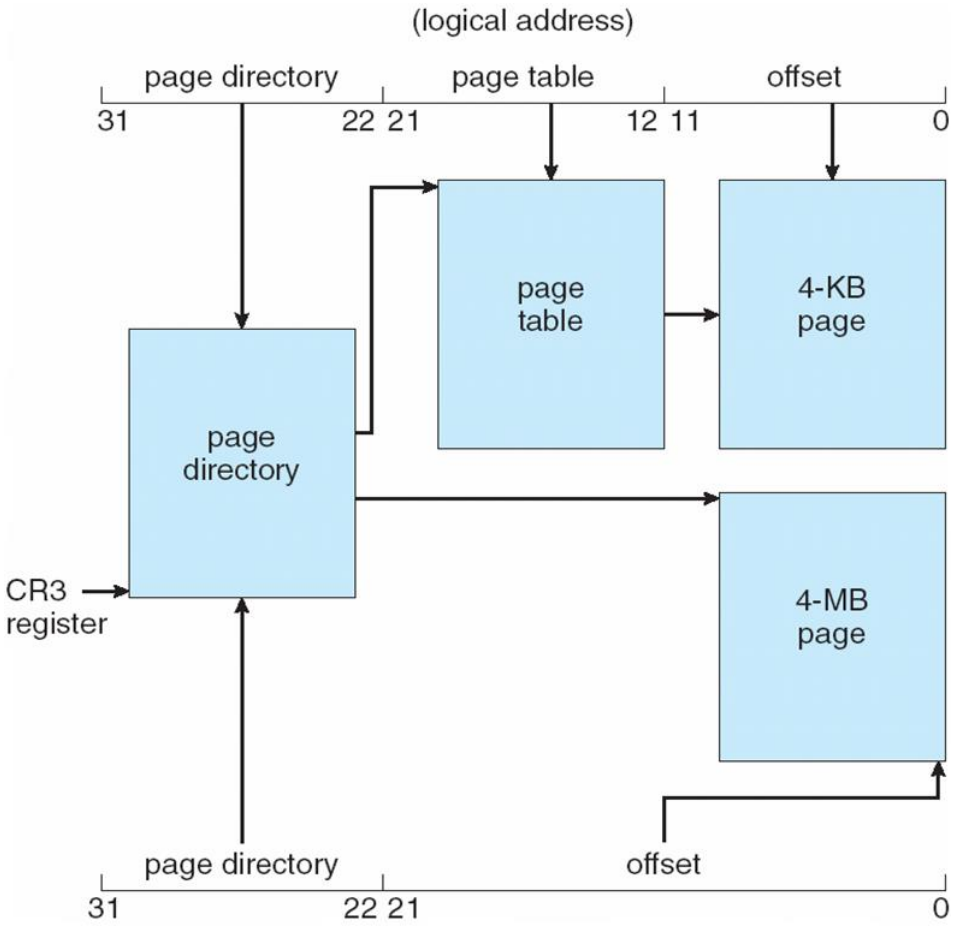


- descriptorul de segment contine registre base + limit folosite in mod uzual pt verificarea adresei virtuale si apoi generarea adresei liniare





Intel IA-32 Paging Architecture

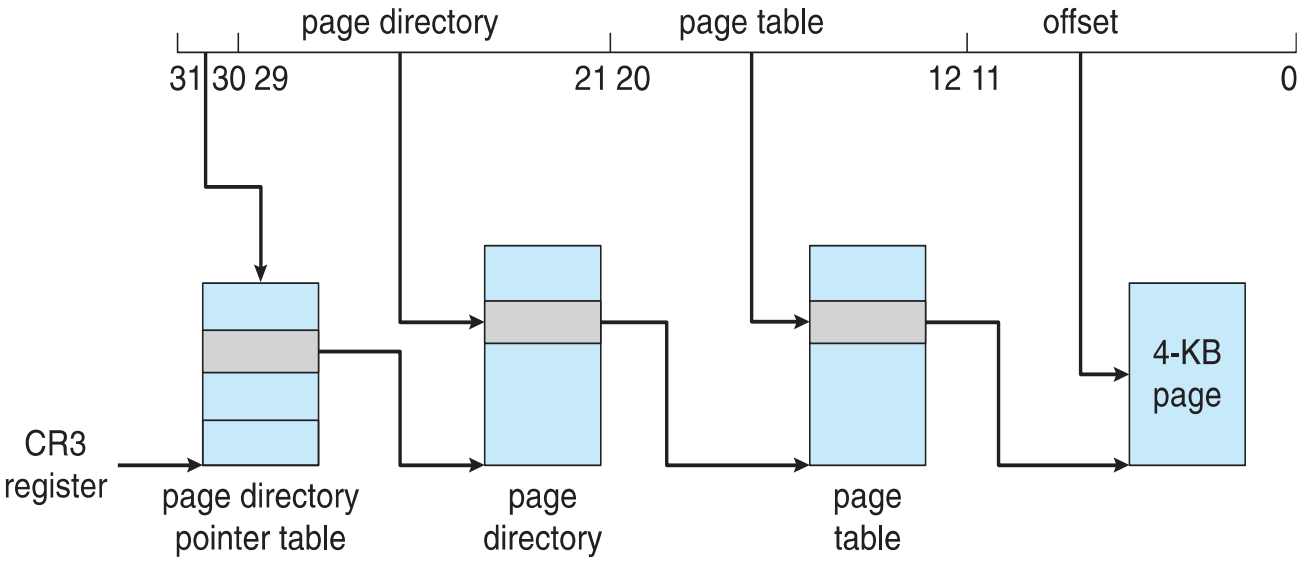






Intel IA-32 Page Address Extensions

- 32-bit address limits led Intel to create **page address extension** (**PAE**), allowing 32-bit apps access to more than 4GB of memory space
  - Paging went to a 3-level scheme
  - Top two bits refer to a **page directory pointer table**
  - Page-directory and page-table entries moved to 64-bits in size
  - Net effect is increasing address space to 36 bits – 64GB of physical memory

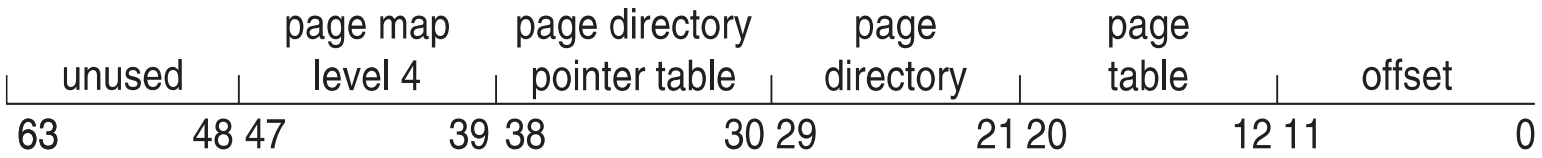






Intel x86-64

- Current generation Intel x86 architecture
- 64 bits is ginormous (> 16 exabytes)
- In practice only implement 48 bit addressing
  - Page sizes of 4 KB, 2 MB, 1 GB
  - Four levels of paging hierarchy
- Can also use PAE so virtual addresses are 48 bits and physical addresses are 52 bits

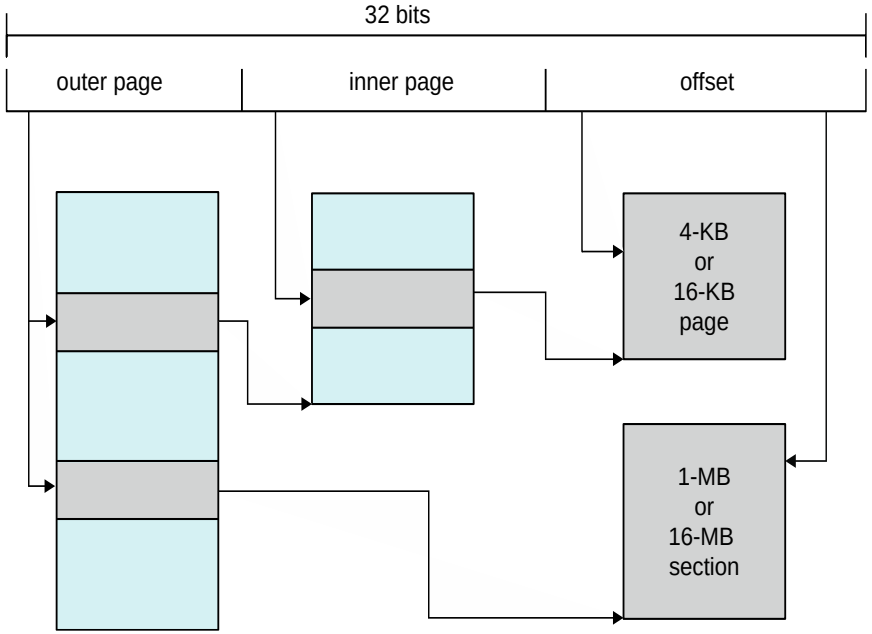


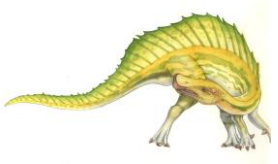




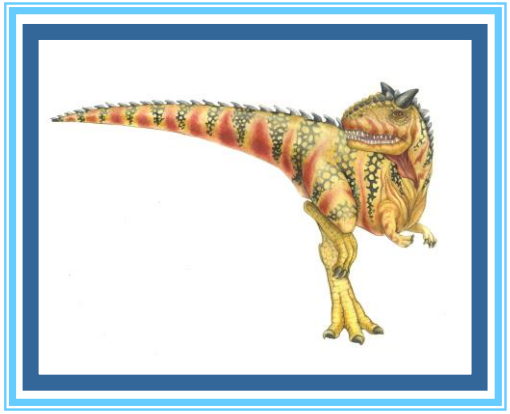
Example: ARM Architecture

- Dominant mobile platform chip (Apple iOS and Google Android devices for example)
- Modern, energy efficient, 32-bit CPU
- 4 KB and 16 KB pages
- 1 MB and 16 MB pages (termed **sections**)
- One-level paging for sections, two-level for smaller pages
- Two levels of TLBs
  - Outer level has two micro TLBs (one data, one instruction)
  - Inner is single main TLB
  - First inner is checked, on miss outers are checked, and on miss page table walk performed by CPU





End of Chapter 9



Silberschatz, Galvin and Gagne ©2018